

Администрирование сервера NGINX

NGINX — это высокопроизводительный сервер, которая реализует функции прокси для веб-серверов и почтовых серверов и потребляет очень мало системных ресурсов. В Сети хватает руководств по ее настройке и примеров конфигураций, но при этом трудно понять, как правильно настроить NGINX для конкретных нужд.

Эта книга расчистит мутные воды конфигурирования NGINX и научит вас настраивать NGINX для решения различных задач. Попутно вы узнаете, что означают некоторые покрытые мраком параметры, и поймете, как разработать конфигурацию, отвечающую именно вашим целям.

Вначале дается краткий обзор процедуры компиляции NGINX и описывается формат конфигурационного файла. Затем автор переходит к модулям и рассказывает о многочисленных настройках, позволяющих использовать NGINX в качестве обратного прокси-сервера. Завершается книга обсуждением поиска и устранения неполадок.

На кого рассчитана эта книга

Если вы опытный системный администратор или инженер, если вы умеете устанавливать и настраивать серверы для определенных целей и хотите освоить NGINX, то эта книга для вас. Наличие опыта работы с NGINX не предполагается.

[PACKT] open source*
PUBLISHING community experience distilled

ДМК
издательство
www.dmk.pf

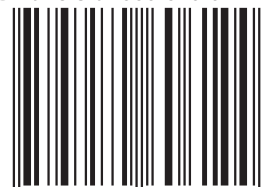
Интернет-магазин www.dmkpress.com
Книга-почтой: orders@aliants-kniga.ru
Оптовая продажа: «Альянс-книга»
(499)782-3889, books@aliants-kniga.ru



Что вы узнаете, прочитав эту книгу

- Как откомпилировать нужный вам сторонний модуль.
- Как написать службу аутентификации, работающую совместно с почтовым прокси-сервером.
- Как подготовить SSL-сертификат для шифрования трафика.
- Как использовать директиву `try_files` для проверки существования файлов.
- Как кэшировать и сжимать ответы для ускорения работы.
- Как интегрироваться с популярными каркасами на PHP с помощью модуля FastCGI.
- Как настроить запись в журналы для получения полезной информации.
- Как искать и устранять причины возникающих проблем.

ISBN 978-5-97060-610-0



9 785970 606100 >

Администрирование сервера NGINX



Администрирование сервера NGINX

Подробное руководство по настройке NGINX
в любой ситуации, с многочисленными примерами
и справочными таблицами для всех директив

Димитрий Айвалиотис

ДМК
издательство

ДМК
издательство

Димитрий Айвалиотис

Администрирование сервера NGINX

Dimitri Aivaliotis

Mastering NGINX

An in-depth guide to configuring NGINX for any situation, including numerous examples and reference tables describing each directive

Димитрий Айвалиотис

Администрирование сервера NGINX

*Подробное руководство по настройке NGINX
в любой ситуации, с многочисленными примерами
и справочными таблицами для всех директив*



Москва, 2018

УДК 004.738.5:004.42Nginx
ББК 32.973.202
A36

Айвалиотис Д.

A36 Администрирование сервера NGINX. – М.: ДМК Пресс, 2018. – 288 с.: ил.

ISBN 978-5-97060-610-0

NGINX – это высокопроизводительный сервер, который реализует функции прокси для веб-серверов и почтовых серверов и потребляет очень мало системных ресурсов. В Интернете хватает руководств по его настройке и примеров конфигураций, но при этом трудно понять, как правильно настроить NGINX для конкретных нужд.

Эта книга расчистит мутные воды конфигурирования NGINX и научит вас настраивать его для решения различных задач. Попутно вы узнаете, что означают некоторые покрытые мраком параметры, и поймете, как разрабатывать конфигурацию, отвечающую именно вашим целям.

Вначале дается краткий обзор процедуры компиляции NGINX и описывается формат конфигурационного файла. Затем автор переходит к модулям и рассказывает о многочисленных настройках, позволяющих использовать NGINX в качестве обратного прокси-сервера. Завершается книга обсуждением поиска и устранения неполадок.

Издание предназначено для системных администраторов или инженеров, имеющих опыт эксплуатации веб-серверов.

УДК 004.738.5:004.42Nginx
ББК 32.973.202

Все права защищены. Любая часть этой книги не может быть воспроизведена в какой бы то ни было форме и какими бы то ни было средствами без письменного разрешения владельцев авторских прав.

Материал, изложенный в данной книге, многократно проверен. Но, поскольку вероятность технических ошибок все равно существует, издательство не может гарантировать абсолютную точность и правильность приводимых сведений. В связи с этим издательство не несет ответственности за возможные ошибки, связанные с использованием книги.

ISBN 978-1-84951-744-7 (анг.)
ISBN 978-5-97060-610-0 (рус.)

Copyright © Packt Publishing
© ДМК Пресс, издание



Содержание

Об авторе	10
О рецензентах	11
Предисловие	14
Глава 1. Установка NGINX и сторонних модулей	19
Установка NGINX с помощью менеджера пакетов	19
CentOS	20
Debian	21
Сборка NGINX из исходного кода	21
Подготовка среды для сборки	22
Компиляция исходного кода	22
Настройка для работы в качестве веб-сервера или почтового сервера	24
Параметры configure для почтового прокси-сервера	24
Параметры configure для определения путей	25
Включение модулей	26
Отключение неиспользуемых модулей	28
Поиск и установка сторонних модулей	30
Полный пример	31
Резюме	32
Глава 2. Руководство по настройке	33
Основы формата конфигурационного файла	33
Глобальные конфигурационные параметры NGINX	34
Включаемые файлы	35
Секция с описанием HTTP-сервера	36
Клиентские директивы	36

Директивы, относящиеся к вводу-выводу	38
Директивы, относящиеся к хэш-таблицам	39
Директивы, относящиеся к сокетам	40
Пример конфигурации	40
Секция с описанием виртуального сервера	41
Местоположения – где, когда и как	45
Секция с описанием почтового сервера	48
Полный пример конфигурации	49
Резюме	50
 Глава 3. Почтовый модуль	51
Простая служба проксирования	51
Служба POP3	53
Служба IMAP	54
Служба SMTP	55
Использование SSL/TLS	56
Полный пример конфигурации почтового модуля	58
Служба аутентификации	60
Использование в связке с memcached	67
Интерпретация журналов	70
Ограничения операционной системы	72
Резюме	73
 Глава 4. NGINX как обратный прокси-сервер	75
Введение в технологию обратного проксирования	76
Модуль проху	77
Унаследованные серверы с куками	81
Модуль upstream	82
Кэширование соединений	83
Алгоритмы балансировки нагрузки	84
Типы проксируемых серверов	85
Единственный проксируемый сервер	85
Несколько проксируемых серверов	86
Проксируемые серверы, работающие по протоколу, отличному от HTTP	87
Проксируемые серверы memcached	88
Проксируемые серверы FastCGI	88
Проксируемые серверы SCGI	89
Проксируемые серверы uWSGI	89

Преобразование конфигурации с «if» в более современную форму	89
Использование документов с описанием ошибок для обработки ошибок проксирования	93
Определение истинного IP-адреса клиента	94
Резюме	95

Глава 5. Обратное проксирование, дополнительные вопросы

Безопасность за счет разделения	98
Шифрование трафика по протоколу SSL	98
Аутентификация клиентов по протоколу SSL	100
Блокирование трафика на основе IP-адреса отправителя	103
Обеспечение масштабируемости за счет изоляции компонентов приложения	105
Оптимизация производительности обратного прокси-сервера	108
Буферизация	108
Кэширование	111
Сохранение	116
Сжатие	117
Резюме	120

Глава 6. NGINX как HTTP-сервер

Архитектура NGINX	121
Базовый модуль HTTP	122
Директива server	123
Протоколирование	124
Поиск файлов	127
Разрешение имен	129
Взаимодействие с клиентами	131
Установка предельных значений для предотвращения недобросовестного использования	133
Ограничение доступа	136
Потоковая передача мультимедийных файлов	140
Предопределенные переменные	141
Использование NGINX совместно с PHP-FPM	143
Пример конфигурации для Drupal	147

Интеграция NGINX и uWSGI	152
Пример конфигурации для Django	153
Резюме	155

Глава 7. NGINX для разработчика

Интеграция с механизмом кэширования	156
Приложения без кэширования	157
Кэширование в базе данных	158
Кэширование в файловой системе	161
Динамическое изменение содержимого	164
Модуль addition	164
Модуль sub	165
Модуль xslt	166
Включение на стороне сервера	167
Принятие решений в NGINX	170
Создание безопасной ссылки	173
Генерация изображений	174
Отслеживание посетителей сайта	178
Предотвращение случайного выполнения кода	179
Резюме	180

Глава 8. Техника устранения неполадок

Анализ журналов	181
Форматы записей в журнале ошибок	181
Примеры записей в журнале ошибок	183
Настройка расширенного протоколирования	186
Отладочное протоколирование	186
Переключение двоичного файла во время выполнения	186
Использование журналов доступа для отладки	193
Типичные ошибки конфигурирования	194
Использование if вместо try_files	195
Использование if для ветвления по имени хоста	196
Неоптимальное использование контекста server	196
Ограничения операционной системы	198
Ограничение на количество файловых дескрипторов	198
Сетевые лимиты	200
Проблемы с производительностью	201
Использование модуля Stub Status	203
Резюме	204

Приложение А. Справочник директив	205
--	-----

Приложение В. Руководство по правилам переписывания	254
Введение в модуль rewrite	254
Создание новых правил переписывания.....	259
Преобразование правил из формата Apache	261
Рекомендация 1: заменить проверки существования каталогов и файлов директивой try_files.....	261
Рекомендация 2: заменить сравнение с REQUEST_URI секцией location.....	262
Рекомендация 3: заменить сравнение с HTTP_HOST секцией server	263
Рекомендация 4: заменить RewriteCond проверкой переменной в директиве if	264
Резюме	265

Приложение С. Сообщество NGINX	266
Список рассылки.....	266
IRC-канал	266
Веб-ресурсы	267
Как правильно составить отчет об ошибке.....	267
Резюме	268

Приложение D. Сохранение сетевых настроек в Solaris	269
--	-----

Предметный указатель	272
-----------------------------------	-----



Об авторе

Димитрий Айвалиотис работает системным архитектором в компании, предоставляющей хостинг в Цюрихе, Швейцария. Начав карьеру с построения вычислительной сети на базе Linux для школы, он затем занимался созданием инфраструктуры высокодоступных двоекцентров обработки данных для банков и онлайн-порталов. Решая проблемы заказчиков (в течение десяти лет), он открыл для себя NGINX и с тех пор использует эту программу в качестве веб-сервера, прокси-сервера и для организации потоковой передачи мультимедийных данных.

Димитрий с отличием закончил бакалавриат физического факультета Политехнического института Ренсселера, а затем получил степень магистра по информационно-управляющим системам в Университете штата Флорида.

Это его первая книга.

Я благодарен Джону Блэкуэллу и Филу Марголису, прочитавшим ранние варианты рукописи. Их советы и критические замечания оказались очень полезны и позволили сделать книгу лучше. Хочу также поблагодарить технических рецензентов за конструктивную критику и указание на допущенные мной ошибки. Все оставшиеся ошибки – целиком моя вина.

Коллектив издательства Packt Publishing немало способствовал претворению этого проекта в жизнь. Их вера в меня как в писателя не давала мне впасть в отчаяние в тяжкие моменты, когда казалось, что все сроки будут сорваны.

Сотрудники компании NGINX, Inc. помогли заполнить пробелы в моем понимании внутренних механизмов работы NGINX. Без них я бы не смог написать эту книгу.

Отдельная благодарность семье. Мои жена и дети вынуждены были мириться с тем, что я тратил немало времени на сочинение книги. Я высоко ценю их терпение на протяжении этого непростого периода.



О рецензентах

Ясир Аднан (Yasir Adnan) живет в столице Бангладеш Дакке. Он изучает информатику и одновременно работает вольнонаемным программистом. Ему доводилось разрабатывать мобильные и веб-приложения, но в настоящее время он занимается в основном мобильными. С ним можно связаться по адресу yasiradnan@outlook.com.

Андрей Алексеев – соучредитель высокотехнологичной компании NGINX, Inc., стоящей за разработкой веб-сервера NGINX. До прихода в NGINX, Inc. в начале 2011 года Андрей работал в интернет-индустрии в отделах информационно-коммуникационных технологий различных предприятий. Андрей получил диплом инженера-электроника в Санкт-Петербургском государственном электротехническом университете и окончил курс по программе MBA для руководителей в школе менеджмента Университета Антверпена.

Антонио П. П. Альмейда (Antonio P. P. Almeida) (@perusio) увлекся NGINX и высокопроизводительными веб-технологиями еще с тех пор, как пытался разрабатывать приложения для Drupal на мало-мощном ноутбуке на базе процессора Centrino с частотой 1,3 ГГц. Из-за прожорливости Apache он был просто вынужден обратиться к NGINX. Он научился выжимать из NGINX максимум возможно в приложениях самых разных типов и, в частности, освоил все тонкости языка настройки NGINX. Антонио живет в Париже. Помимо NGINX, у него есть и другие пристрастия: малоизвестная музыка позднего итальянского средневековья, кино и желание сделать Drupal еще лучше.

Райнер Даффнер (Rainer Duffner) окончил Университет прикладных наук в Констанце, Германия, по специальности «информационные системы» и в настоящее время работает системным инженером в компании EveryWare AG, где помогает заказчикам извлечь максимум пользы из выделенных серверов на платформах FreeBSD, Linux



и Solaris. Он живет в небольшом городке близ Цюриха и в свободное время катается на горном велосипеде в окрестностях Цюриха и по швейцарским горам.

Я благодарен Димитрию за возможность принять участие в рецензировании этой замечательной книги. Ее значение невозможно переоценить.

*Посвящаю своему отцу, который всегда говорил,
что я могу добиться любой цели, которую поставил перед собой.*



Предисловие

NGINX – это высокопроизводительный веб-сервер, потребляющий очень мало системных ресурсов. В Сети немало руководств по его настройке и примеров конфигураций. Задача этой книги – очистить мутные воды конфигурирования NGINX. По ходу дела вы научитесь настраивать NGINX для решения различных задач, узнаете, что означают некоторые покрытые мраком параметры, и поймете, как разработать конфигурацию, отвечающую вашим целям.

Вам больше не потребуется копировать фрагменты найденного где-то конфигурационного скрипта, потому что вы будете знать, как создать файл, делающий в точности то, что нужно. Это умение достигается не сразу, по пути встретится немало ухабов, но благодаря приведенным в этой книге советам вы сможете свободно писать конфигурационные файлы NGINX самостоятельно. А если что-то пойдет наперекос, то сумеете найти причину ошибки сами или, по крайней мере, попросить помощи без чувства вины за то, что не пытались найти ответ своими силами.

Эта книга построена по модульному принципу – так, чтобы максимально облегчить поиск нужной информации. Все главы более-менее независимы. Можете сразу переходить к интересующему вас вопросу. Если складывается ощущение, что пропущено что-то важное, вернитесь назад и прочитайте предшествующие главы. Они организованы так, чтобы можно было строить конфигурационный файл постепенно.

О содержании книги

В главе 1 «Установка NGINX и сторонних модулей» объясняется, как установить NGINX в различных операционных системах и как затем добавить сторонние модули.

В главе 2 «Руководство по настройке» рассказывается о формате конфигурационного файла NGINX. Вы узнаете, для чего предназна-

чены различные контексты, как задавать глобальные параметры и что такое «местоположение».

Глава 3 «Почтовый модуль» посвящена модулю проксирования почты, здесь рассматриваются все аспекты его настройки. Включен пример службы аутентификации.

В главе 4 «NGINX как обратный прокси-сервер» вводится понятие обратного проксирования и описывается использование NGINX в этой роли.

В главе 5 «Обратное проксирование, дополнительные вопросы» более глубоко рассматривается использование NGINX в качестве обратного прокси-сервера для решения проблем масштабирования и производительности.

В главе 6 «NGINX как HTTP-сервер» описывается использование различных модулей, включенных в NGINX для решения типичных задач веб-сервера.

В главе 7 «NGINX для разработчика» показано, как интегрировать NGINX с приложением для ускорения доставки содержимого пользователям.

В главе 8 «Техника устранения неполадок» рассматриваются типичные ошибки при настройке и способы их отладки, а также дают рекомендации по оптимизации производительности.

Приложение А «Справочник директив» содержит удобный справочник по директивам настройки – как описанным в книге, так и ранее не упоминавшимся.

В приложении В «Руководство по правилам переписывания» описано, как работать с модулем переписывания URL в NGINX, и приведено несколько простых шагов преобразования правил переписывания из формата Apache в формат NGINX.

Приложение С «Сообщество NGINX» включает перечень сетевых ресурсов, где можно найти дополнительные сведения.

В приложении D «Сохранение сетевых настроек в Solaris» подробно рассказано о том, что необходимо для сохранения изменений различных сетевых параметров в операционной системе Solaris версии 10 и старше.

Что необходимо для чтения этой книги

В каждой главе, где используются примеры кода, приведены инструкции по установке. По существу, требуется следующее.

- ❑ **Среда сборки:** компилятор, файлы-заголовки и еще кое-что по мелочи.

- ❑ **NGINX:** последняя версия должна подойти.
- ❑ **Ruby:** лучше всего взять дистрибутив с сайта <https://rvm.io>.
- ❑ **Perl:** стандартная версия годится.

На кого рассчитана эта книга

Книга рассчитана на опытных системных администраторов и системных инженеров, знающих, как производятся установка и настройка серверов под конкретные нужды. Предварительного знакомства с NGINX не требуется.

Графические выделения

В этой книге используются различные шрифты для обозначения типа информации. Ниже приведено несколько примеров с пояснениями.

Фрагменты кода внутри абзаца выделяются следующим образом: «NGINX попытается собрать библиотеку статически, если при запуске скрипта `configure` указан параметр `--with-<library>=<path>`».

Кусок кода выглядит так:

```
$ export BUILD_DIR='pwd'
$ export NGINX_INSTALLDIR=/opt/nginx
$ export VAR_DIR=/home/www/tmp
$ export LUAJIT_LIB=/opt/luajit/lib
$ export LUAJIT_INC=/opt/luajit/include/luajit-2.0
```

Чтобы привлечь внимание к участку внутри куска кода, он выделяется полужирным шрифтом:

```
$ export BUILD_DIR='pwd'
$ export NGINX_INSTALLDIR=/opt/nginx
$ export VAR_DIR=/home/www/tmp
$ export LUAJIT_LIB=/opt/luajit/lib
$ export LUAJIT_INC=/opt/luajit/include/luajit-2.0
```

Входная и выходная информация командных утилит выглядит так:

```
$ mkdir $HOME/build
$ cd $HOME/build && tar xzf nginx-<version-number>.tar.gz
```

Новые термины и важные фрагменты выделяются полужирным шрифтом. Например, элементы графического интерфейса в меню

или диалоговых окнах выглядят в книге так: «Нажатие кнопки **Next** приводит к переходу на следующий экран».



Предупреждения и важные примечания выглядят так.



Советы и рекомендации выглядят так.

Отзывы

Мы всегда рады отзывам читателей. Расскажите нам, что вы думаете об этой книге – что вам понравилось или, быть может, не понравилось. Читательские отзывы важны для нас, так как помогают выпускать книги, из которых вы черпаете максимум полезного для себя.

Чтобы отправить обычный отзыв, просто пошлите письмо на адрес feedback@packtpub.com, указав название книги в качестве темы.

Если вы являетесь специалистом в некоторой области и хотели бы стать автором или соавтором книги, познакомьтесь с инструкциями для авторов по адресу www.packtpub.com/authors.

Поддержка клиентов

Счастливым обладателям книг Packt мы можем предложить ряд услуг, которые позволят извлечь из своего приобретения максимум пользы.

Загрузка кода примеров

Вы можете скачать код примеров ко всем книгам издательства Packt, купленным на сайте <http://www.PacktPub.com>. Если книга была куплена в другом месте, зайдите на страницу <http://www.PacktPub.com/support>, зарегистрируйтесь, и мы отправим файлы по электронной почте.

Опечатки

Мы проверяли содержимое книги со всей тщательностью, но какие-то ошибки все же могли проскользнуть. Если вы найдете в нашей книге ошибку в тексте или в коде, пожалуйста, сообщите

нам о ней. Так вы избавите других читателей от разочарования и поможете нам сделать следующие издания книги лучше. При обнаружении опечатки просьба зайти на страницу <http://www.packtpub.com/support>, выбрать книгу, щелкнуть по ссылке **errata submission form** и ввести информацию об опечатке. Проверив ваше сообщение, мы поместим информацию об опечатке на нашем сайте или добавим ее в список замеченных опечаток в разделе Errata для данной книги. Список подтвержденных опечаток можно просмотреть, выбрав название книги на странице <http://www.packtpub.com/support>.

Нарушение авторских прав

Незаконное размещение защищенного авторским правом материала в Интернете – проблема для всех носителей информации. В издательстве Packt мы относимся к защите прав интеллектуальной собственности и лицензированию очень серьезно. Если вы обнаружите незаконные копии наших изданий в любой форме в Интернете, пожалуйста, незамедлительно сообщите нам адрес или название веб-сайта, чтобы мы могли предпринять соответствующие меры.

Просим отправить ссылку на вызывающий подозрение в пиратстве материал по адресу copyright@packtpub.com.

Мы будем признательны за помощь в защите прав наших авторов и содействии в наших стараниях предоставлять читателям полезные сведения.

Вопросы

Если вас смущает что-то в этой книге, вы можете связаться с нами по адресу questions@packtpub.com, и мы сделаем все возможное для решения проблемы.



Глава 1. Установка NGINX и сторонних модулей

Первоначально NGINX задумывался как HTTP-сервер. Он создавался для решения проблемы C10K, описанной Дэниэлом Кегелем (Daniel Kegel) на странице <http://www.kegel.com/c10k.html>, — проектирование веб-сервера, способного обрабатывать одновременно 10 000 соединений. NGINX может это делать за счет основанного на событиях механизма обработки соединений и для достижения цели использует зависящий от ОС механизм событий.

Прежде чем переходить к настройке NGINX, ее необходимо установить. В этой главе описывается, как установить саму NGINX и где взять дополнительные модули. NGINX по природе своей является модульной программой, и существует обширное сообщество разработчиков сторонних модулей, расширяющих функциональность основного сервера. Эти модули прикомпилируются к серверу и устанавливаются вместе с ним.

В этой главе:

- ☐ Установка NGINX с помощью менеджера пакетов.
- ☐ Сборка NGINX из исходного кода.
- ☐ Настройка для работы в качестве веб-сервера или почтового сервера.
- ☐ Включение модулей.
- ☐ Поиск и установка сторонних модулей.
- ☐ Полный пример.

Установка NGINX с помощью менеджера пакетов

Для установки достаточно простой команды менеджера пакетов:

- ☐ Linux (дистрибутивы на основе Debian)

```
sudo apt-get install nginx
```

☐ Linux (дистрибутивы на основе rpm)

```
sudo yum install nginx
```

☐ FreeBSD

```
sudo pkg_install -r nginx
```



Команда `sudo` применяется для получения привилегий суперпользователя ('root'). Если ваша операционная система поддерживает управление доступом на основе ролей (**RBAC – Role-based access control**), то для достижения той же цели следует использовать другую команду, например `pfexec`.

Описанные команды устанавливают NGINX в стандартные места, зависящие от операционной системы. Это предпочтительный метод установки в случае, когда использование пакетов ОС – непреложное требование.

Команда, разрабатывающая ядро NGINX, предлагает также двоичные файлы стабильной версии на странице <http://nginx.org/en/download.html>. Если в дистрибутиве отсутствует пакет `nginx` (как, например, в CentOS), то можно установить заранее откомпилированные и протестированные двоичные файлы, как описано ниже.

CentOS

Добавьте репозиторий NGINX в конфигурацию `yum`, создав следующий файл:

```
sudo vi /etc/yum.repos.d/nginx.repo
```

```
[nginx]
name=nginx repo
baseurl=http://nginx.org/packages/centos/6/$basearch/
gpgcheck=0
enabled=1
```

Затем выполните следующую команду для установки `nginx`:

```
sudo yum install nginx
```

На странице с указанным выше URL имеются также инструкции по установке пакета `nginx-release`.

Debian

Скачайте ключ подписания NGINX — http://nginx.org/keys/nginx_signing.key — и добавьте его в связку ключей apt:

```
sudo apt-key add nginx_signing.key
```

Добавьте репозиторий `nginx.org` в конец файла `/etc/apt/sources.list`:

```
vi /etc/apt/sources.list
deb http://nginx.org/packages/debian/ squeeze nginx
deb-src http://nginx.org/packages/debian/ squeeze nginx
```

Установите `nginx`, выполнив такие команды:

```
sudo apt-get update
sudo apt-get install nginx
```

Если среди пакетов, прилагаемых к операционной системе, нет `nginx` или версия, включенная в пакет, устарела и не умеет делать того, что вам нужно, или пакеты, размещенные на сайте `nginx.org`, не отвечают вашим потребностям, или вы хотите использовать версию NGINX для разработчиков, то остается единственная возможность — собрать NGINX из исходного кода.

Сборка NGINX из исходного кода

Существуют две ветви кода NGINX — стабильная и разрабатываемая. Именно в последней находятся все новые функции, которые интегрируются и тестируются перед включением в стабильную версию. Выпуск разрабатываемой версии сопровождается таким же контролем качества и прогоном функциональных тестов, как и выпуск стабильной версии, поэтому ту и другую можно использовать в производственных системах. Основное различие между ними связано с поддержкой сторонних модулей. В разрабатываемой версии внутренний API может измениться, тогда как в стабильной он фиксирован, поэтому обратная совместимость сторонних модулей гарантируется только для стабильных версий.

Подготовка среды для сборки

Для сборки NGINX из исходного кода система должна отвечать определенным требованиям. Кроме компилятора, понадобятся библиотеки OpenSSL и **PCRE (Perl Compatible Regular Expressions)** и файлы-заголовки для них – если вы хотите иметь поддержку для SSL и модуля переписывания URL соответственно. В некоторых дистрибутивах эти требования удовлетворены изначально. Если же это не так, то вам предстоит найти и установить подходящий пакет либо скачать исходный код, распаковать его в какой-то каталог и сообщить об этом каталоге скрипту конфигурирования NGINX.

NGINX попытается собрать библиотеку статически, если при запуске скрипта `configure` указан параметр `--with-<library>=<path>`. Это бывает полезно, если вы хотите, чтобы NGINX не зависел от того, что установлено в системе, или требуется добиться от двоичного файла `nginx` максимальной производительности. Если используются функции внешних библиотек, доступные лишь начиная с некоторой версии (например, расширение TLS Next Protocol Negotiation – согласование следующего протокола, – появившееся в версии OpenSSL 1.0.1), то указывайте путь к распакованному исходному коду именно этой версии.

Существуют и другие необязательные пакеты для поддержки той или иной функциональности, в том числе алгоритмов хэширования MD5 и SHA-1, библиотеки сжатия `zlib` и библиотеки `libatomic`. Алгоритмы хэширования применяются во многих местах NGINX, в частности для вычисления хэш-кода URI, играющего роль ключа кэша. Библиотека `zlib` используется для сжатия отправляемого клиенту содержимого. Если доступна библиотека `atomic_ops`, то NGINX будет использовать атомарные операции обновления памяти для реализации быстрого алгоритма блокировки памяти.

Компиляция исходного кода

NGINX можно скачать со страницы <http://nginx.org/en/download.html>. Там вы найдете исходный код обеих ветвей в форматах `.tar.gz` и `.zip`. Распакуйте архив во временный каталог:

```
$ mkdir $HOME/build
$ cd $HOME/build && tar xzf nginx-<version-number>.tar.gz
```

Произведите конфигурирование, выполнив команду:

```
$ cd $HOME/build/nginx-<version-number> && ./configure
```

После чего соберите программу:

```
$ make && sudo make install
```

При самостоятельной сборке двоичного файла `nginx` вы можете включить только то, что вам нужно, например указать, от имени какого пользователя должен работать NGINX, или задать подразумеваемые по умолчанию места расположения журналов, чтобы их не нужно было явно прописывать в конфигурационном файле. В таблице ниже приведены параметры `configure`, которые позволяют изменить способ сборки двоичного файла. Эти параметры не зависят от того, какие модули NGINX подключаются.

Общие параметры `configure`

Параметр	Описание
<code>--prefix=<path></code>	Корень дерева установки. Все остальные пути указываются относительно этого корня
<code>--sbin-path=<path></code>	Путь к двоичному файлу <code>nginx</code> . Если не задан, то файл создается в каталоге <code>prefix</code>
<code>--conf-path=<path></code>	Путь к каталогу, в котором <code>nginx</code> ищет свой конфигурационный файл, если тот не указан в командной строке
<code>--error-log-path=<path></code>	Путь к каталогу, в который <code>nginx</code> записывает журнал ошибок, если не задан иным способом
<code>--pid-path=<path></code>	Путь к каталогу, в котором создается <code>pid</code> -файл главного процесса, обычно <code>/var/run</code>
<code>--lock-path=<path></code>	Путь к файлу, управляющему взаимоблокировкой разделяемой памяти
<code>--user=<user></code>	Пользователь, от имени которого запускаются рабочие процессы
<code>--group=<group></code>	Группа, от имени которой запускаются рабочие процессы
<code>--with-file-aio</code>	Включает асинхронный ввод-вывод для FreeBSD 4.3+ и Linux 2.6.22+
<code>--with-debug</code>	Включает отладочное протоколирование. Не рекомендуется для производственных систем

Можно также задать оптимизации, отсутствующие в готовом пакете. Ниже перечислены некоторые особенно полезные параметры.

Параметры `configure`, предназначенные для оптимизации

Параметр	Описание
<code>--with-cc=<path></code>	Если требуется использовать компилятор C, путь к которому не входит в переменную окружения <code>PATH</code>
<code>--with-cpp=<path></code>	Путь к соответствующему препроцессору
<code>--with-cc-opt=<options></code>	Здесь можно задать путь к необходимым включаемым файлам (передается флагу <code>-I<path></code>), уровень оптимизации (<code>-O4</code>) или указать, что нужно компилировать 64-разрядную версию
<code>--with-ld-opt=<options></code>	Компоновщику передается путь к библиотекам (<code>-L<path></code>) и путь поиска во время выполнения (<code>-R<path></code>)
<code>--with-cpu-opt=<cpu></code>	Позволяет задать сборку для конкретного семейства процессоров

Настройка для работы в качестве веб-сервера или почтового сервера

Среди высокопроизводительных веб-серверов NGINX отличается еще и тем, что проектировался для работы в качестве почтового прокси-сервера. В зависимости от поставленной задачи NGINX можно настроить для ускорения работы другого веб-сервера, для работы в качестве веб-сервера, для работы в качестве почтового прокси-сервера или для всего сразу. Иногда удобно иметь один пакет, который можно установить на любой сервер внутри организации, и задать роль NGINX в конфигурационном файле. А иногда – для работы в высокопроизводительных средах, где каждый килобайт на учете, – лучше подготовить урезанный двоичный файл.

Параметры `configure` для почтового прокси-сервера

В таблице ниже перечислены параметры `configure`, имеющие отношение к почтовому модулю.

Параметры `configure`, относящиеся к модулю `mail`

Параметр	Описание
<code>--with-mail</code>	Активируется модуль <code>mail</code> , который по умолчанию выключен
<code>--with-mail_ssl_module</code>	Этот модуль необходим для проксирования почтовых сообщений, в которых используется SSL/TLS

Параметр	Описание
<code>--without-mail_pop3_module</code>	При включенном модуле <code>mail</code> модуль POP3 можно отключить
<code>--without-mail_imap_module</code>	При включенном модуле <code>mail</code> модуль IMAP можно отключить
<code>--without-mail_smtp_module</code>	При включенном модуле <code>mail</code> модуль SMTP можно отключить
<code>--without-http</code>	Этот параметр полностью отключает модуль <code>http</code> ; используйте, только если собираетесь компилировать поддержку <code>mail</code>

Для типичного почтового прокси-сервера я рекомендую такие параметры `configure`:

```
$ ./configure --with-mail --with-mail_ssl_module  
--with-openssl=$ {BUILD_DIR}/openssl-1.0.1c
```

В настоящее время протокол SSL/TLS необходим практически в любой почтовой системе, отказ от него на прокси-сервере лишит пользователей ожидаемой функциональности. Я советую прикомпилировать OpenSSL статически, чтобы не зависеть от наличия библиотеки OpenSSL в операционной системе. Разумеется, переменную `BUILD_DIR`, упоминаемую в приведенной выше команде, необходимо предварительно определить.

Параметры `configure` для определения путей

В таблице ниже перечислены параметры `configure`, относящиеся к модулю `http`, — от активации модуля Perl до задания путей к временным каталогам.

Параметры `configure`, относящиеся к модулю HTTP

Параметр	Описание
<code>--without-http-cache</code>	Если за NGINX стоит другой сервер, то можно настроить NGINX так, чтобы содержимое кэшировалось локально. Этот параметр отключает кэш
<code>--with-http_perl_module</code>	Систему конфигурирования NGINX можно расширить, разрешив писать код на Perl. Этот параметр активирует соответствующий модуль (но при этом снижается производительность)

Параметр	Описание
<code>--with-perl_modules_path=<path></code>	Этот параметр определяет путь к дополнительным Perl-модулям, необходимым встроенному интерпретатору Perl. Его можно задать также в конфигурационном файле
<code>--with-perl=<path></code>	Путь к интерпретатору Perl (версии не ниже 5.6.1), если он отсутствует в списке путей по умолчанию
<code>--http-log-path=<path></code>	Подразумеваемый по умолчанию путь к журналу доступа HTTP
<code>--http-client-body-temp-path=<path></code>	В этом каталоге временно сохраняется тело запроса, полученного от клиента. Если включен модуль WebDAV, рекомендуется размещать этот каталог в той же файловой системе, где содержимое будет храниться в конечном итоге
<code>--http-proxy-temp-path=<path></code>	В этом каталоге хранятся временные файлы, когда NGINX используется в качестве прокси-сервера
<code>--http-fastcgi-temp-path=<path></code>	Каталог для временных файлов FastCGI
<code>--http-uwsgi-temp-path=<path></code>	Каталог для временных файлов uWSGI
<code>--http-scgi-temp-path=<path></code>	Каталог для временных файлов SCGI

Включение модулей

Помимо модулей `http` и `mail`, в дистрибутив NGINX включен еще целый ряд модулей. По умолчанию они не активированы, но это можно сделать, указав при запуске `configure` параметр `--with-<имя-модуля>_module`.

Параметры `configure`, относящиеся к прочим модулям

Параметр	Описание
<code>--with-http_ssl_module</code>	Если требуется шифровать HTTP-трафик, то есть обрабатывать URL-адреса, начинающиеся с <code>https</code> , то необходимо задать этот параметр (при этом необходима библиотека OpenSSL)
<code>--with-http_realip_module</code>	Если NGINX расположен за балансировщиком нагрузки L7 или иным устройством, которое передает IP-адрес клиента в HTTP-заголовке, то необходимо включить этот модуль. Он позволяет корректно обрабатывать ситуации, когда разные клиенты по видимости имеют один и тот же IP-адрес

Параметр	Описание
<code>--with-http_addition_module</code>	Этот модуль производит преобразование XML-ответов, основываясь на одной или нескольких таблицах стилей XSLT (при этом необходимы библиотеки <code>libxml2</code> и <code>libxslt</code>)
<code>--with-http_image_filter_module</code>	Этот модуль работает как фильтр изображений, обрабатывая их до передачи клиенту (при этом необходима библиотека <code>libgd</code>)
<code>--with-http_geoip_module</code>	При наличии этого модуля в конфигурационных блоках можно устанавливать различные переменные, которые позволяют принимать решения в зависимости от местоположения клиента, определяемого по его IP-адресу (при этом необходимы библиотека <code>MaxMind GeoIP</code> и соответствующие откомпилированные файлы базы данных)
<code>--with-http_sub_module</code>	Этот модуль реализует подстановочный фильтр, который заменяет одну присутствующую в ответе строку другой
<code>--with-http_dav_module</code>	Если этот модуль включен, то распознаются директивы в конфигурационном файле, относящиеся к использованию WebDAV. Включайте этот модуль только при необходимости, поскольку его неправильная настройка создает угрозы безопасности
<code>--with-http_flv_module</code>	Этот модуль обеспечивает псевдопотокковую передачу видеофайлов в формате Flash
<code>--with-http_mp4_module</code>	Этот модуль обеспечивает псевдопотокковую передачу видеофайлов в формате H.264/AAC
<code>--with-http_gzip_static_module</code>	Этот модуль поддерживает передачу предварительно сжатых статических файлов для ресурсов, URL-адреса которых задаются без указания суффикса <code>.gz</code>
<code>--with-http_gunzip_static_module</code>	Этот модуль реализует распаковку сжатого содержимого для клиентов, которые не поддерживают сжатия в формате <code>gzip</code>
<code>--with-http_random_index_module</code>	Этот модуль следует включать, если вы хотите возвращать индексный файл, случайно выбранный среди всех файлов в некотором каталоге
<code>--with-http_secure_link_module</code>	Этот модуль реализует механизм хэширования ссылки на URL-адрес, так что вычислить реальную ссылку сможет только клиент, знающий правильный пароль
<code>--with-http_stub_status_module</code>	Этот модуль позволит собирать статистику от самого NGINX. Полученную информацию можно представить в графическом виде с помощью пакета <code>RRDtool</code> или аналогичной программы

Как видите, все эти модули дополняют функциональность модуля `http`. Само их включение на этапе компиляции никак не отражается на производительности во время выполнения. Снижение возможно, только если эти модули упомянуты в конфигурационном файле.

Для ускорения и проксирования веб-сервера я рекомендую задавать такие параметры `configure`:

```
$ ./configure --with-http_ssl_module --with-http_realip_module --with-http_geoip_module --with-http_stub_status_module --with-openssl=${BUILDDIR}/openssl-1.0.1c
```

А для работы в качестве веб-сервера – такие:

```
$ ./configure --with-http_stub_status_module
```

Различие в том, «каким боком» NGINX обращен к клиенту. В роли веб-ускорителя NGINX является также конечной точкой для SSL-запросов и обрабатывает запросы от клиентов, находящихся за прокси-сервером, определяя, откуда в действительности поступил запрос и принимая соответствующие решения. В роли веб-сервера необходима только стандартная функциональность обслуживания запросов на файлы.

Я рекомендую всегда включать модуль `stub_status`, поскольку он позволяет получать статистические данные о работе NGINX.

Отключение неиспользуемых модулей

Существует также ряд относящихся к `http` модулей, которые по умолчанию включены, но могут быть деактивированы путем задания параметра `--without-<имя-модуля>_module`. Если вам эти модули не нужны, можете со спокойной душой отключить их.

Параметры `configure`, относящиеся к модулю HTTP

Параметр	Описание
<code>--without-http_charset_module</code>	Модуль <code>charset</code> отвечает за установку заголовка <code>Content-Type</code> , а также за преобразование из одной кодировки в другую
<code>--without-http_gzip_module</code>	Модуль <code>gzip</code> работает как выходной фильтр, сжимая отправляемые клиенту данные
<code>--without-http_ssi_module</code>	Этот модуль обрабатывает запросы с включением на стороне сервера (Server Side Include). Если включен модуль <code>Perl</code> , то становится доступна дополнительная команда <code>SSI</code> – <code>perl</code>

Параметр	Описание
--without-http_userid_module	Модуль <code>userid</code> позволяет NGINX устанавливать куки, служащие для идентификации клиента. Впоследствии для отслеживания действий клиента можно использовать переменные <code>\$uid_set</code> и <code>\$uid_got</code>
--without-http_access_module	Этот модуль контролирует доступ к ресурсам, основываясь на IP-адресе клиента
--without-http_auth_basic_module	Этот модуль ограничивает доступ с помощью простой схемы аутентификации (HTTP Basic Authentication)
--without-http_autoindex_module	Модуль <code>autoindex</code> наделяет NGINX способностью генерировать страницу с содержимым каталогов, в которых нет индексного файла
--without-http_geo_module	Этот модуль позволяет устанавливать значения конфигурационных переменных, исходя из IP-адреса клиента, а затем принимать на этой основе те или иные решения
--without-http_map_module	Модуль <code>map</code> позволяет сопоставлять одной переменной другую
--without-http_split_clients_module	Этот модуль создает переменные, которые можно использовать для проведения A/B-тестирования
--without-http_referer_module	Этот модуль позволяет NGINX блокировать запросы, исходя из HTTP-заголовка <code>Referer</code>
--without-http_rewrite_module	Модуль <code>rewrite</code> позволяет заменять URI, исходя из различных условий
--without-http_proxy_module	Модуль <code>proxy</code> позволяет NGINX передавать запросы другому серверу или группе серверов
--without-http_fastcgi_module	Этот модуль позволяет NGINX передавать запросы серверу FastCGI
--without-http_uwsgi_module	Этот модуль позволяет NGINX передавать запросы серверу uWSGI
--without-http_scgi_module	Этот модуль позволяет NGINX передавать запросы серверу SCGI
--without-http_memcached_module	Этот модуль позволяет NGINX взаимодействовать с сервером memcached, сохраняя ответы на запросы в переменной
--without-http_limit_conn_module	Этот модуль позволяет NGINX устанавливать ограничения на количество соединений, основываясь на каких-то условиях, чаще всего на IP-адресе
--without-http_limit_req_module	Этот модуль позволяет NGINX ограничить частоту запросов, ассоциированных с каким-то условием

Параметр	Описание
<code>--without-http_empty_gif_module</code>	Этот модуль порождает прозрачное изображение в формате GIF размером 1×1 пиксель
<code>--without-http_browser_module</code>	Этот модуль позволяет определять настройки в зависимости от заголовка User-Agent в запросе. В результате анализа номера версии в этом заголовке устанавливаются определенные переменные
<code>--without-http_upstream_ip_hash_module</code>	Этот модуль определяет группу серверов, которые можно использовать в сочетании с различными модулями проксирования

Поиск и установка сторонних модулей

Как и во многих проектах с открытым исходным кодом, вокруг NGINX сформировалось активное сообщество разработчиков. Благодаря модульной структуре NGINX сообщество имеет возможность разрабатывать и публиковать модули, обеспечивающие дополнительную функциональность. Таких модулей немало, поэтому, прежде чем приступать к разработке собственного, имеет смысл познакомиться с тем, что есть.

Процедура установки стороннего модуля очень проста.

1. Найдите интересующий вас модуль (на сайте <https://github.com> или <http://wiki.nginx.org/3rdPartyModules>).
2. Скачайте модуль.
3. Распакуйте архив с исходным кодом.
4. Прочитайте файл README, если он имеется. Проверьте, нет ли дополнительных зависимостей.
5. Сконфигурируйте NGINX так, чтобы она использовала модуль `./configure --add-module=<path>`.

В результате получится двоичный файл `nginx`, в который включена функциональность добавленного модуля.

Помните, что многие сторонние модули носят экспериментальный характер. Перед тем как использовать модуль в производственной системе, протестируйте его. И не забывайте, что в разрабатываемые версии NGINX могут быть внесены такие изменения API, что сторонние модули перестанут работать.

Особо отметим модуль `ngx_lua`, который позволяет использовать в качестве скриптового языка конфигурирования Lua вместо Perl. По сравнению с `perl`, у этого модуля есть важное достоинство: он не

блокирует выполнение программы и тесно интегрирован с другими сторонними модулями. Полная инструкция по установке приведена на странице <http://wiki.nginx.org/HttpLuaModule#Installation>. В следующем разделе мы на примере этого модуля продемонстрируем процедуру установки стороннего модуля.

Полный пример

Составив представление о том, что можно сконфигурировать, вы можете построить двоичный файл, точно отвечающий вашим потребностям. В примере ниже задаются префикс путей, пользователь, группа, некоторые пути, отключаются одни модули, включаются другие и добавляются два сторонних модуля.

```
$ export BUILD_DIR='pwd'
$ export NGINX_INSTALLDIR=/opt/nginx
$ export VAR_DIR=/home/www/tmp
$ export LUAJIT_LIB=/opt/luajit/lib
$ export LUAJIT_INC=/opt/luajit/include/luajit-2.0

$ ./configure \
    --prefix=${NGINX_INSTALLDIR} \
    --user=www \
    --group=www \
    --http-client-body-temp-path=${VAR_DIR}/client_body_temp \
    --http-proxy-temp-path=${VAR_DIR}/proxy_temp \
    --http-fastcgi-temp-path=${VAR_DIR}/fastcgi_temp \
    --without-http_uwsgi_module \
    --without-http_scgi_module \
    --without-http_browser_module \
    --with-openssl=${BUILD_DIR}/../openssl-1.0.1c \
    --with-pcre=${BUILD_DIR}/../pcre-8.32 \
    --with-http_ssl_module \
    --with-http_realip_module \
    --with-http_sub_module \
    --with-http_flv_module \
    --with-http_gzip_static_module \
    --with-http_gunzip_module \
    --with-http_secure_link_module \
    --with-http_stub_status_module \
    --add-module=${BUILD_DIR}/ngx_devel_kit-0.2.17 \
    --add-module=${BUILD_DIR}/ngx_lua-0.7.9
```

После многочисленных сообщений о том, что скрипт `configure`шел в вашей системе, печатается такая сводка:

Configuration summary

- + using PCRE library: /home/builder/build/pcre-8.32
- + using OpenSSL library: /home/builder/build/openssl-1.0.1c
- + md5: using OpenSSL library
- + sha1: using OpenSSL library
- + using system zlib library

```
nginx path prefix: "/opt/nginx"
nginx binary file: "/opt/nginx/sbin/nginx"
nginx configuration prefix: "/opt/nginx/conf"
nginx configuration file: "/opt/nginx/conf/nginx.conf"
nginx pid file: "/opt/nginx/logs/nginx.pid"
nginx error log file: "/opt/nginx/logs/error.log"
nginx http access log file: "/opt/nginx/logs/access.log"
nginx http client request body temporary files: "/home/www/tmp/client_body_temp"
nginx http proxy temporary files: "/home/www/tmp/proxy_temp"
nginx http fastcgi temporary files: "/home/www/tmp/fastcgi_temp"
```

Как видим, `configure` нашел все, что мы просили, и подтвердил указанные пути. Теперь можно собрать и установить `nginx`, как объяснялось в начале этой главы.

Резюме

В этой главе мы познакомились с различными модулями NGINX. Собирая двоичный файл самостоятельно, вы можете включить только ту функциональность, которая вам необходима. Сборка и установка программ – дело, вам знакомое, поэтому вы не потратите много времени на подготовку среды сборки и предварительную установку зависимостей. Подгоняя NGINX под свои нужды, не стесняйтесь включать или выключать те или иные модули.

Далее мы дадим общий обзор настройки NGINX, чтобы вы поняли, как подходить к этой задаче.



Глава 2. Руководство по настройке

Формат конфигурационного файла NGINX прост и логичен. Понимание его устройства и назначения отдельных секций – одно из условий самостоятельного составления конфигурационных файлов. В этой главе мы постараемся достичь данной цели, рассмотрев следующие вопросы.

- ❑ Основы формата конфигурационного файла.
- ❑ Глобальные конфигурационные параметры NGINX.
- ❑ Включаемые файлы.
- ❑ Секция с описанием HTTP-сервера.
- ❑ Секция с описанием виртуального сервера.
- ❑ Местоположения – где, когда и как.
- ❑ Секция с описанием почтового сервера.
- ❑ Полный пример конфигурации.

Основы формата конфигурационного файла

Конфигурационный файл NGINX состоит из секций. Секции устроены следующим образом:

```
<секция> {  
  
    <директива> <параметры>;  
  
}
```

Обратите внимание, что строки, содержащие директивы, оканчиваются точкой с запятой (;). Это признак конца строки. Фигурные скобки вводят новый конфигурационный контекст, но мы будем называть такие контексты просто «секциями».

Глобальные конфигурационные параметры NGINX

В глобальной секции задаются параметры, оказывающие влияние на сервер в целом. Его формат отличается от описанного выше. Глобальная секция может включать как конфигурационные директивы, например `user` и `worker_processes`, так и секции, например `events`. Глобальная секция не заключается в фигурные скобки.

В таблице ниже приведены наиболее распространенные директивы, задаваемые в глобальном контексте.

Глобальные конфигурационные директивы

Директива	Описание
<code>user</code>	Пользователь и группа, от имени которых исполняются рабочие процессы. Если группа опущена, то подразумевается группа, имя которой совпадает с именем пользователя
<code>worker_processes</code>	Количество рабочих процессов, создаваемых сразу после запуска. Эти процессы обрабатывают запросы на соединения со стороны клиентов. Сколько процессов задать, зависит от сервера и в первую очередь от дисковой подсистемы и сетевой инфраструктуры. Если решаются в основном счетные задачи, то рекомендуется задавать этот параметр равным количеству процессорных ядер, а если задачи, требующие интенсивного ввода-вывода, – то умножать это количество на коэффициент от 1,5 до 2
<code>error_log</code>	Это файл, в который записываются сообщения об ошибках. Если ни в каком другом контексте директивы <code>error_log</code> нет, то в этом файле будут регистрироваться вообще все ошибки. Вторым параметром директивы обозначается уровень сообщений, попадающих в журнал (<code>debug</code> , <code>info</code> , <code>notice</code> , <code>warn</code> , <code>error</code> , <code>crit</code> , <code>alert</code> , <code>emerg</code>). Сообщения уровня <code>debug</code> выводятся, только если программа была сконфигурирована с параметром <code>--with-debug</code>
<code>pid</code>	Файл, в котором хранится идентификатор главного процесса. Переопределяет значение, заданное на этапе конфигурирования и компиляции
<code>use</code>	Определяет метод обработки соединения. Переопределяет значение, заданное при компиляции, и если используется, то должна содержаться в контексте <code>events</code> . Обычно не нуждается в переопределении, разве что в случае, когда значение по умолчанию приводит к ошибкам

Директива	Описание
<code>worker_connections</code>	Эта директива задает максимальное число соединений, одновременно открытых в одном рабочем процессе. Сюда входят, в частности, соединения с клиентами и с проксируемыми серверами (но не только). Особенно важно это для обратных прокси-серверов – чтобы достичь указанного количества одновременно открытых соединений, может понадобиться настройка на уровне операционной системы

Ниже приведен простой пример задания описанных директив.

```
# мы хотим, чтобы nginx работала от имени пользователя 'www'
user www;

# рабочая нагрузка счетная и имеется 12 процессорных ядер
worker_processes 12;

# явно задаем путь к обязательному журналу ошибок
error_log /var/log/nginx/error.log;

# явно задаем путь к pid-файлу
pid /var/run/nginx.pid;

# создаем конфигурационный контекст для модуля 'events'
events {

    # мы работаем в системе Solaris и обнаружили, что при использовании
    # подразумеваемого по умолчанию механизма обработки соединений nginx
    # со временем перестает отвечать на запросы, поэтому переходим на
    # следующий по качеству механизм
    use /dev/poll;

    # произведение этого числа и значения worker_processes
    # показывает, сколько может быть одновременно открыто соединений
    # для одной пары IP:порт
    worker_connections 2048;
}
```

Глобальная секция должна находиться в начале конфигурационного файла `nginx.conf`.

Включаемые файлы

Включать файлы можно в любое место конфигурационного файла. Их цель – сделать файл более удобным для восприятия и обеспечить повторное использование некоторых частей. Включаемые файлы

должны быть синтаксически корректны с точки зрения записи директив и блоков. Для включения файла нужно указать путь к нему:

```
include /opt/local/etc/nginx/mime.types;
```

Метасимволы в пути позволяют включить сразу несколько файлов:

```
include /opt/local/etc/nginx/vhost/*.conf;
```

Если указан неполный путь, то NGINX считает, что путь задан относительно главного местоположения конфигурационного файла.

Проверить правильность конфигурационного файла можно, запустив NGINX следующим образом:

```
nginx -t -c <path-to-nginx.conf>
```

При этом на наличие ошибок проверяются и все включаемые файлы.

Секция с описанием HTTP-сервера

Секция (или конфигурационный контекст), описывающая HTTP-сервер, доступна, только если при конфигурировании NGINX не был задан параметр `--without-http`, отключающий модуль HTTP. В этой секции описываются все аспекты работы с модулем HTTP – именно с ним вы чаще всего будете иметь дело.

Поскольку конфигурационных директив, относящихся к работе с HTTP-соединениями, много, мы разобьем их на несколько категорий и будем рассматривать каждую категорию в отдельности.

Клиентские директивы

Директивы из этой категории относятся к самому соединению с клиентом, а также описывают некоторые аспекты поведения для клиентов разных типов.

Клиентские директивы модуля HTTP

Директива	Описание
<code>chunked_transfer_encoding</code>	Позволяет отключить специфицированный в стандарте HTTP/1.1 механизм поблочной передачи данных (chunked transfer encoding) в ответе клиенту

Директива	Описание
<code>client_body_buffer_size</code>	Задаёт размер буфера для чтения тела запроса клиента. По умолчанию для буфера выделяются две страницы памяти. Увеличение размера позволяет предотвратить запись во временный файл на диске
<code>client_body_in_file_only</code>	Используется для отладки или последующей обработки тела запроса клиента. Если значение равно <code>on</code> , то тело запроса принудительно записывается в файл
<code>client_body_in_single_buffer</code>	Заставляет NGINX сохранить все тело запроса клиента в одном буфере, чтобы уменьшить количество операций копирования
<code>client_body_temp_path</code>	Определяет путь к каталогу для сохранения файлов с телами запросов клиентов
<code>client_body_timeout</code>	Задаёт время между последовательными операциями чтения тела запроса клиента
<code>client_header_buffer_size</code>	Задаёт размер буфера для чтения заголовка запроса клиента, если он превышает подразумеваемую по умолчанию величину 1 КБ
<code>client_header_timeout</code>	Время, отведенное на чтение всего заголовка запроса
<code>client_max_body_size</code>	Максимальный размер тела запроса клиента. В случае превышения отправляется ответ 413 (Request Entity Too Large)
<code>keepalive_disable</code>	Запрещает соединения типа <code>keep-alive</code> для некоторых браузеров
<code>keepalive_requests</code>	Определяет, сколько запросов можно принять по одному соединению типа <code>keep-alive</code> , прежде чем закрывать его
<code>keepalive_timeout</code>	Определяет, сколько времени соединение типа <code>keep-alive</code> может оставаться открытым. Можно задать второй параметр, используемый для формирования заголовка ответа «Keep-Alive»
<code>large_client_header_buffers</code>	Задаёт максимальное число и размер буферов для чтения большого заголовка запроса клиента
<code>msie_padding</code>	Разрешает или запрещает добавлять комментарии в ответы со статусом больше 400 для увеличения размера ответа до 512 байт при работе с MSIE
<code>msie_refresh</code>	Разрешает или запрещает отправлять MSIE-клиентам ответ <code>Refresh</code> вместо перенаправления

Директивы, относящиеся к вводу-выводу

Эти директивы управляют отправкой статических файлов и порядком работы с файловыми дескрипторами.

Директивы модуля HTTP, относящиеся к вводу-выводу

Директива	Описание
<code>aio</code>	Разрешает использование асинхронного файлового ввода-вывода. Это возможно во всех современных версиях FreeBSD и дистрибутивах Linux. В FreeBSD директиву <code>aio</code> можно использовать для предварительной загрузки данных для <code>sendfile</code> . В Linux требуется директива <code>directio</code> , которая автоматически отключает <code>sendfile</code>
<code>directio</code>	Разрешает использовать зависящий от операционной системы флаг при чтении файлов, размер которых больше или равен указанному. Обязательна при использовании директивы <code>aio</code> в Linux
<code>directio_alignment</code>	Устанавливает выравнивание для <code>directio</code> . Обычно подразумеваемого по умолчанию значения 512 достаточно, но при использовании XFS в Linux рекомендуется увеличить до 4 К
<code>open_file_cache</code>	Настраивает кэш, в котором могут храниться дескрипторы открытых файлов, информация о существовании каталогов и информация об ошибках поиска файлов
<code>open_file_cache_errors</code>	Разрешает или запрещает кэширование ошибок поиска файлов в кэше <code>open_file_cache</code>
<code>open_file_cache_min_uses</code>	Задаёт минимальное число обращений к файлу в течение времени, заданного параметром <code>inactive</code> директивы <code>open_file_cache</code> , необходимое для того, чтобы дескриптор файла оставался в кэше открытых дескрипторов
<code>open_file_cache_valid</code>	Задаёт время между последовательными проверками актуальности данных, хранящихся в кэше <code>open_file_cache</code>
<code>postpone_output</code>	Задаёт минимальный размер порции данных, отправляемых клиенту. Если возможно, данные не будут отправляться, пока не накопится указанное количество
<code>read_ahead</code>	Если возможно, ядро будет сразу считывать из файла столько байтов, сколько указано в параметре <code>size</code> . Поддерживается в текущих версиях FreeBSD и Linux (в Linux параметр <code>size</code> игнорируется)

Директива	Описание
sendfile	Разрешает использовать системный вызов <code>sendfile (2)</code> для прямого копирования из одного файлового дескриптора в другой
sendfile_max_chunk	Задаёт максимальный размер данных, который можно скопировать за один вызов <code>sendfile (2)</code> . Без этого ограничения одно быстрое соединение может целиком захватить рабочий процесс

Директивы, относящиеся к хэш-таблицам

Директивы из этой категории управляют выделением статической памяти для определенных переменных. NGINX вычисляет минимально необходимый размер при запуске и после изменения конфигурации. Как правило, достаточно настроить только один из параметров `*_hash_max_size` с помощью соответствующей директивы. NGINX выдает предупреждение при попытке задать сразу несколько таких параметров. Переменным вида `*_hash_bucket_size` по умолчанию присваивается значение, кратное размеру строки кэша процессора, чтобы минимизировать количество обращений к кэшу, необходимое для чтения записи. Поэтому изменять их не рекомендуется. Дополнительные сведения см. на странице <http://nginx.org/en/docs/hash.html>.

Директивы модуля HTTP, относящиеся к хэш-таблицам

Директива	Описание
server_names_hash_bucket_size	Задаёт размер кластера в хэш-таблицах имен серверов
server_names_hash_max_size	Задаёт максимальный размер хэш-таблиц имен серверов
types_hash_bucket_size	Задаёт размер кластера в хэш-таблицах типов
types_names_hash_max_size	Задаёт максимальный размер хэш-таблиц типов
variables_hash_bucket_size	Задаёт размер кластера в хэш-таблицах прочих переменных
variables_names_hash_max_size	Задаёт максимальный размер хэш-таблиц прочих переменных

Директивы, относящиеся к сокетам

Эти директивы описывают установку различных параметров TCP-сокетов, создаваемых NGINX.

Директивы модуля HTTP, относящиеся к сокетам

Директива	Описание
<code>lingering_close</code>	Определяет, следует ли оставлять соединение открытым в ожидании дополнительных данных от клиента
<code>lingering_time</code>	Связана с директивой <code>lingering_close</code> и определяет, сколько времени держать сокет открытым для обработки дополнительных данных
<code>lingering_timeout</code>	Также связана с директивой <code>lingering_close</code> и определяет, сколько времени держать соединение открытым в ожидании дополнительных данных
<code>reset_timedout_connection</code>	Если значение этой директивы равно <code>on</code> , то сокеты, для которых истек тайм-аут, сбрасываются немедленно, в результате чего освобождается выделенная для них память. По умолчанию сокет остается в состоянии <code>FIN_WAIT1</code> . На соединения типа <code>keep-alive</code> эта директива не распространяется, они всегда закрываются обычным образом
<code>send_lowat</code>	Если значение отлично от нуля, то NGINX пытается минимизировать количество операций отправки данных через клиентские сокеты. В Linux, Solaris и Windows эта директива игнорируется
<code>send_timeout</code>	Задаёт тайм-аут между двумя последовательными операциями записи при передаче ответа клиенту
<code>tcp_nodelay</code>	Разрешает или запрещает использование параметра <code>TCP_NODELAY</code> для соединений типа <code>keep-alive</code>
<code>tcp_nopush</code>	Учитывается только при использовании директивы <code>sendfile</code> . Разрешает NGINX отправлять заголовки ответа одним пакетом, а также передавать файл полными пакетами

Пример конфигурации

Ниже приведен пример конфигурационной секции модуля HTTP:

```
http {  
    include      /opt/local/etc/nginx/mime.types;  
    default_type application/octet-stream;
```

```
sendfile on;
tcp_nopush on;
tcp_nodelay on;
keepalive_timeout 65;
server_names_hash_max_size 1024;
}
```

Этот контекстный блок должен располагаться после всех глобальных директив в файле `nginx.conf`.

Секция с описанием виртуального сервера

По соглашению в контексте, начинающемся ключевым словом `server`, находится описание «виртуального сервера». Так называется логический набор ресурсов, сопоставленный со значением директивы `server_name`. Виртуальные серверы отвечают на запросы по протоколу HTTP и потому входят в состав секции `http`.

Виртуальный сервер определяется сочетанием директив `listen` и `server_name`. Директива `listen` задает комбинацию IP-адреса и номера порта либо путь к сокету в домене UNIX:

```
listen address[:port];
listen port;
listen unix:path;
```

Директива `listen` однозначно описывает привязку сокетa в NGINX. Дополнительно в ней могут присутствовать следующие параметры.

Параметры директивы `listen`

Параметр	Описание	Примечание
<code>default_server</code>	Означает, что данный сервер будет сервером по умолчанию для указанной пары адрес:порт	
<code>setfib</code>	Задаёт таблицу маршрутизации FIB для прослушивающего сокета	Поддерживается только в ОС FreeBSD. Игнорируется для сокетов в домене UNIX
<code>backlog</code>	Задаёт параметр <code>backlog</code> в системном вызове <code>listen()</code>	По умолчанию равен –1 в FreeBSD и 511 на всех остальных платформах
<code>rcvbuf</code>	Задаёт параметр <code>SO_RCVBUF</code> для прослушивающего сокета	

Параметр	Описание	Примечание
sndbuf	Задаёт параметр <code>SO_SNDBUF</code> для прослушивающего сокета	
accept_filter	Задаёт имя фильтра приема: <code>dataready</code> или <code>httpready</code>	Поддерживается только в ОС FreeBSD
deferred	Задаёт параметр <code>TCP_DEFER_ACCEPT</code> , означающий, что требуется отложить вызов <code>accept()</code>	Поддерживается только в ОС Linux
bind	Означает, что для данной пары адрес:порт нужен отдельный вызов <code>bind()</code>	Отдельный вызов <code>bind()</code> производится без специального указания, если для сокета заданы какие-то другие специальные параметры
ipv6only	Устанавливает значение параметра <code>IPV6_V6ONLY</code>	Можно установить только один раз при запуске. Игнорируется для сокетов в домене UNIX
ssl	Означает, что этот порт предназначен только для соединений по протоколу HTTPS	Позволяет построить более компактную конфигурацию
so_keepalive	Для прослушивающего сокета задается режим TCP keepalive	

Несмотря на свою простоту, директива `server_name` позволяет решить целый ряд задач конфигурирования. По умолчанию ее значение равно "", то есть секция `server` без директивы `server_name` сопоставляется с запросом, в котором отсутствует заголовок `Host`. Этим можно воспользоваться, например, для отбрасывания запросов без этого заголовка:

```
server {
    listen 80;
    return 444;
}
```

Нестандартный код ответа HTTP 444, использованный в этом примере, заставляет NGINX немедленно закрыть соединение.

Помимо обычной строки, NGINX допускает использование в директиве `server_name` метасимвола `*`:

- ❑ метасимвол можно указывать вместо поддомена: *.example.com;
- ❑ метасимвол можно указывать вместо домена верхнего уровня: www.example.*;
- ❑ существует особая форма, которая соответствует поддомену или самому домену: .example.com (соответствует как *.example.com, так и example.com).

Параметр директивы `server_name` может быть и регулярным выражением, для этого нужно лишь предпослать имени знак тильды (~):

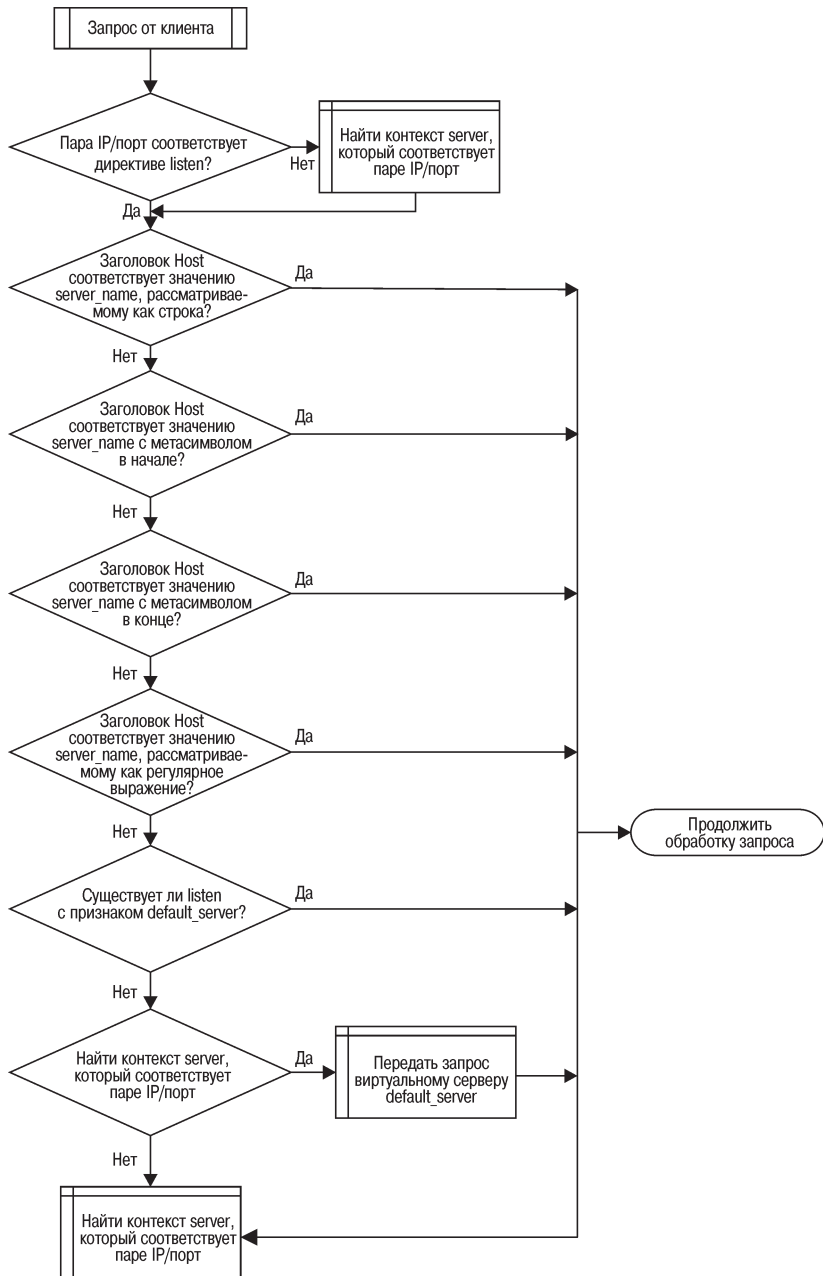
```
server_name ~^www\.example\.com$;  
server_name ~^www(\d+)\.example\.(com)$;
```

Вторая форма применяется для запоминания подвыражений, на которые затем можно сослаться (по номеру \$1, \$2 и т. д.) в последующих директивах.

Чтобы определить, какой виртуальный сервер должен обслуживать запрос, NGINX применяется следующий алгоритм.

1. Сопоставить IP-адрес и порт с указанными в директиве `listen`.
2. Сопоставить заголовок `Host` со значением директивы `server_name`, рассматриваемым как строка.
3. Сопоставить заголовок `Host` со значением директивы `server_name`, рассматриваемым как строка, считая, что в начале находится метасимвол `*`.
4. Сопоставить заголовок `Host` со значением директивы `server_name`, рассматриваемым как строка, считая, что в конце находится метасимвол `*`.
5. Сопоставить заголовок `Host` со значением директивы `server_name`, рассматриваемым как регулярное выражение.
6. Если все попытки сопоставления заголовка `Host` закончились неудачей, использовать ту директиву `listen`, в которой имеется признак `default_server`.
7. Если все попытки сопоставления заголовка `Host` закончились неудачей и директивы `listen` с признаком `default_server` не существует, использовать первый сервер, в котором директива `listen` удовлетворяет условию шага 1.

Эта логика изображена на следующей блок-схеме:



Признак `default_server` позволяет обработать запросы, которые иначе остались бы необработанными. Поэтому рекомендуется всегда задавать этот признак явно, чтобы не гадать потом, почему запросы обрабатываются странным образом.

Кроме того, признак `default_server` полезен, когда требуется сконфигурировать несколько виртуальных серверов с одной и той же директивой `listen`. Все описанные выше директивы будут одинаковы для всех подходящих блоков `server`.

Местоположения — где, когда и как

Директива `location` может встречаться в секции с описанием виртуального сервера и содержит в качестве параметра URI-адрес, поступивший от клиента или в результате внутренней переадресации. Местоположения могут быть вложенными (с несколькими исключениями). Их назначение — определить максимально специализированную конфигурацию для обработки запроса.

Местоположение задается следующим образом:

```
location [модификатор] uri {...}
```

Можно задавать также именованные местоположения:

```
location @name {...}
```

Именованное местоположение доступно только при внутренней переадресации и может быть задано лишь на уровне контекста сервера. При этом сохраняется тот URI, который был перед входом в блок `location`.

Модификаторы изменяют обработку местоположения следующим образом:

Модификаторы местоположения

Модификатор	Обработка
=	Сравнить буквально и завершить поиск
~	Сопоставление с регулярным выражением с учетом регистра
~*	Сопоставление с регулярным выражением без учета регистра
^^	Прекратить обработку до сопоставления этого местоположения с регулярным выражением, если это совпадение с самым длинным префиксом. Отметим, что это не сопоставление с регулярным выражением, задача данного модификатора — как раз предотвратить такое сопоставление

Когда приходит запрос, для указанного в нем URI ищется самое подходящее местоположение. Происходит это следующим образом.

- ❑ Среди местоположений без регулярного выражения ищется самое специфичное (с самым длинным совпадающим префиксом), порядок следования местоположений при этом не учитывается.
- ❑ Сопоставление с регулярными выражениями производится в том порядке, в каком они следуют в конфигурационном файле. Поиск прекращается при обнаружении первого совпадения. Далее для обработки запроса используется самое подходящее местоположение.

Описанное выше сопоставление производится после декодирования URI; например, строка "%20" в URI совпадает с пробелом " " в местоположении.

Именованные местоположения можно использовать только при внутренней переадресации запросов.

В таблице ниже перечислены директивы, которые могут встречаться только внутри местоположения.

Директивы, употребляемые внутри секции location

Директива	Описание
alias	Определяет путь в файловой системе, соответствующий имени местоположения. Если местоположение задано с помощью регулярного выражения, то значение <code>alias</code> должно ссылаться на запомненные подвыражения. <code>alias</code> подставляется вместо части URI, сопоставившейся с местоположением; оставшаяся часть URI не изменяется и становится частью пути в файловой системе. Применение директивы <code>alias</code> чревато ошибками в случае перемещения участков конфигурационного файла, поэтому лучше использовать директиву <code>root</code> , если только нет необходимости модифицировать URI для поиска нужного файла
internal	Означает, что данное местоположение можно использовать только для внутренних запросов (переадресации, определенной в других директивах, переписывании запросов, для страниц ошибок и т. д.)
limit_except	Ограничивает применение данного местоположения только указанными глаголами HTTP (<code>GET</code> включает также <code>HEAD</code>)

Кроме того, некоторые директивы, относящиеся к секции `http`, могут употребляться и в секции `location`. Полный перечень см. в приложении А «Справочник по директивам».

Директива `try_files` заслуживает особого упоминания. Хотя ее можно использовать в контексте `server`, чаще она встречается в описании местоположения. Ее задача — поискать перечисленные в параметрах файлы в указанном порядке; как только будет найден первый файл, поиск прекращается. Обычно этот механизм применяется, чтобы сопоставить потенциальные файлы с некоторой переменной, а затем передать обработку именованному местоположению, как показано в следующем примере:

```
location / {
    try_files $uri $uri/ @mongrel;
}

location @mongrel {
    proxy_pass http://appserver;
}
```

Здесь если переданному URI не соответствует файл, то неявно производится попытка найти каталог, а затем обработка перепоручается серверу `appserver` с помощью прокси. О том, как использовать секции `location`, директивы `try_files` и `proxy_pass` для решения различных задач, мы будем не раз говорить на страницах этой книги.

Местоположения могут быть вложенными, за исключением следующих случаев:

- ❑ префикс равен `"="`;
- ❑ местоположение именованное.

Рекомендуется вкладывать местоположения, определяемые регулярными выражениями, в местоположения с простой строкой в качестве URI, например:

```
# сначала входим через root
location / {

    # затем ищем самую длинную подстроку
    # отметим, что сопоставление с регулярным выражением не производится
    location ~^ /css {

        # а затем сопоставляем с этим регулярным выражением
        location ~* /css/.*\..css$ {
        }
    }
}
```

Секция с описанием почтового сервера

Секция с описанием почтового сервера, или конфигурационный контекст почты, доступна, только если при конфигурировании NGINX был задан параметр `--with-mail`. В этой секции описываются все аспекты работы с почтовым модулем.

Почтовый модуль содержит директивы, определяющие поведение проксирования соединений с почтовыми серверами. В серверном контексте могут употребляться те же директивы `listen` и `server_name`, которые мы рассматривали в секции `http`.

NGINX умеет проксировать протоколы IMAP, POP3 и SMTP. В таблице ниже перечислены директивы, относящиеся к этому модулю.

Директивы для модуля mail

Директива	Описание
<code>auth_http</code>	Задаёт сервер, который служит для аутентификации пользователя в протоколах POP3 и IMAP. Функциональность этого сервера подробно рассматривается в главе 3
<code>imap_capabilities</code>	Определяет, какие возможности протокола IMAP4 поддерживает проксируемый сервер
<code>pop3_capabilities</code>	Определяет, какие возможности протокола POP3 поддерживает проксируемый сервер
<code>protocol</code>	Определяет, какой протокол поддерживает данный контекст виртуального сервера
<code>proxy</code>	Разрешает или запрещает проксирование почты
<code>proxy_buffer</code>	Позволяет задать размер буфера, используемого для проксирования. По умолчанию размер буфера равен размеру страницы
<code>proxy_pass_error_message</code>	Полезна, если процедура аутентификации на проксируемом сервере возвращает осмысленное сообщение клиенту
<code>proxy_timeout</code>	Используется, если тайм-аут должен быть больше значения по умолчанию – 24 часа
<code>xclient</code>	В протоколе SMTP предусмотрена проверка на основе параметров IP/HELO/LOGIN, которые передаются в команде XCLIENT. Данная директива позволяет NGINX передать эту информацию серверу

Если NGINX собирался с поддержкой SSL (с параметром `--with-mail_ssl_module`), то, помимо вышеописанных, доступны следующие директивы.

Директивы для модуля mail с поддержкой SSL

Директива	Описание
ssl	Включает или выключает поддержку SSL-транзакций в этом контексте
ssl_certificate	Определяет путь к сертификату (или сертификатам) SSL в формате PEM для данного виртуального сервера
ssl_certificate_key	Определяет путь к секретному ключу SSL в формате PEM для данного виртуального сервера
ssl_ciphers	Определяет, какие шифры нужно поддерживать в этом контексте (в формате OpenSSL)
ssl_prefer_server_ciphers	Указывает, что при использовании протоколов SSLv3 и TLSv1 серверные шифры более приоритетны, чем клиентские
ssl_protocols	Определяет, какие протоколы SSL разрешить
ssl_session_cache	Определяет параметры кэша сеансов SSL, а также разрешает или запрещает разделение кэша всеми рабочими процессами
ssl_session_timeout	Определяет, сколько времени клиент может пользоваться одними и теми же параметрами SSL при условии, что они хранятся в кэше

Полный пример конфигурации

Ниже приведен пример конфигурационного файла, содержащего секции, рассмотренные в этой главе. Подчеркнем, что его не следует копировать и использовать «как есть». Скорее всего, он вам не подойдет, а включен лишь для того, чтобы дать представление о том, как выглядит полный конфигурационный файл.

```

user www;
worker_processes 12;
error_log /var/log/nginx/error.log;
pid /var/run/nginx.pid;

events {
    use /dev/poll;
    worker_connections 2048;
}

http {
    include /opt/local/etc/nginx/mime.types;
    default_type application/octet-stream;
    sendfile on;
    tcp_nopush on;
    tcp_nodelay on;

```

```
keepalive_timeout 65;
server_names_hash_max_size 1024;

server {
    listen 80;
    return 444;
}

server {
    listen 80;
    server_name www.example.com;

    location / {
        try_files $uri $uri/ @mongrel;
    }

    location @mongrel {
        proxy_pass http://127.0.0.1:8080;
    }
}
```

Резюме

В этой главе мы рассмотрели устройство конфигурационного файла NGINX. Его модульная структура в какой-то мере отражает структуру самого NGINX. Глобальный конфигурационный блок описывает все аспекты поведения NGINX в целом. Для каждого протокола, поддерживаемого NGINX, имеется отдельная конфигурационная секция. Порядок обработки запросов можно уточнить, описав виртуальные серверы внутри контекста протокола (`http` или `mail`) таким образом, что запрос будет направляться серверу, обслуживающему конкретную пару IP-адрес/порт. В контексте `http` можно затем определить местоположения, сопоставляемые с URI запроса. Местоположения могут быть вложенными или упорядоченными иным способом, это позволяет направить запрос в нужное место файловой системы или серверу приложений.

Но пока мы ничего не сказали о конфигурировании других модулей, включаемых в двоичный файл `nginx` на этапе компиляции. Мы будем обсуждать эти дополнительные директивы при рассмотрении использования модуля для решения конкретной задачи. Мы также не объяснили, какие переменные NGINX предоставляет для конфигурирования и как ими пользоваться. Об этом пойдет речь далее. Данная глава посвящена лишь основам настройки NGINX.

В следующей главе мы рассмотрим настройку почтового модуля NGINX, который осуществляет проксирование электронной почты.



Глава 3. Почтовый модуль

NGINX проектировался не только для обслуживания веб-запросов, но и как средство проксирования почтовых служб. В этой главе мы научимся настраивать NGINX в качестве почтового прокси-сервера для протоколов POP3, IMAP и SMTP. Мы рассмотрим следующие вопросы.

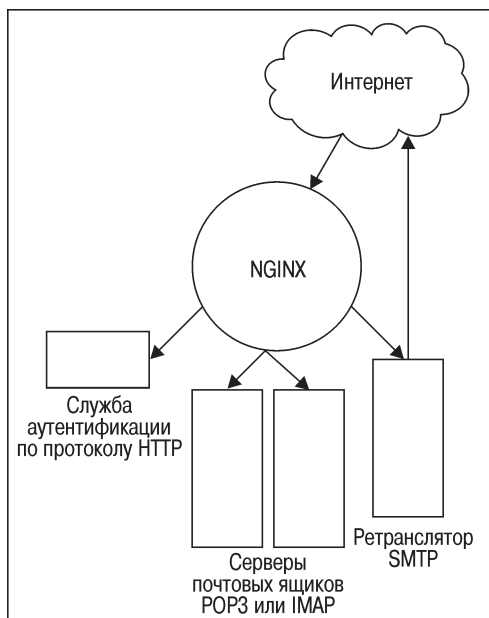
- ☐ Простая служба проксирования.
- ☐ Служба аутентификации.
- ☐ Использование в связке с memcached.
- ☐ Интерпретация журналов.
- ☐ Ограничения операционной системы.

Простая служба проксирования

Модуль почтового прокси-сервера в NGINX первоначально был разработан для компании FastMail. Она хотела предоставлять своим пользователям единственную оконечную точку IMAP, а размещать реальную почтовую учетную запись на одном из нескольких почтовых серверов заднего плана. В то время для проксирования использовалась классическая модель Unix, основанная на системном вызове `fork()`, то есть для каждого нового соединения порождался новый процесс. В IMAP соединения существуют очень долго, поэтому и процессы оказывались долгожителями. В результате прокси-серверы работали крайне медленно, поскольку им приходилось управлять многочисленными процессами, созданными для обслуживания соединений. Событийная модель процессов NGINX гораздо лучше отвечала таким требованиям. В качестве почтового прокси-сервера NGINX может перенаправлять трафик на сколько угодно почтовых серверов заднего плана, на которых размещаются учетные записи. Это открывает возможность предоставить клиентам единственную оконечную точку, а количество реальных серверов масштабировать с увеличением числа пользователей. На базе данной модели постро-

ены как коммерческие, так и открытые решения для электронной почты, в том числе Atmail и Zimbra.

На рисунке ниже графически представлена описанная схема.



Поступивший запрос обрабатывается в соответствии с протоколом. Почтовый модуль можно настроить по-разному для протоколов POP3, IMAP и SMTP. Для каждого протокола NGINX обращается к службе аутентификации, передавая ей имя пользователя и пароль. В случае успешной аутентификации соединение проксируется почтовому серверу, указанному в ответе, полученном от службы аутентификации. В противном случае соединение с клиентом разрывается. Таким образом, служба аутентификации определяет, какую из служб POP3/IMAP/SMTP должен использовать клиент и на каком сервере. Количество почтовых серверов не ограничено, и NGINX может выступать в роли прокси-сервера для всех, предоставляя клиентам единый центральный шлюз.

Прокси-сервер действует от чьего-то имени. В данном случае NGINX действует от имени почтового клиента, закрывая одно соединение и открывая другое с проксируемым сервером. Это

означает, что прямого соединения между почтовым клиентом и настоящим сервером почтовых ящиков или ретранслятором SMTP нет.



Правила, основанные на информации, содержащейся в соединении с клиентом, работать не будут, если только почтовая программа не поддерживает расширений, например XCLIENT для SMTP.

При проектировании архитектуры с участием прокси-сервера нужно иметь в виду важный момент – компьютер, на котором размещен прокси-сервер, должен поддерживать больше соединений, чем типичный проксируемый сервер. Такая же вычислительная мощность и память, как у сервера почтовых ящиков, ему не нужна, но количество одновременных соединений следует принимать во внимание.

Служба POP3

Протокол **Post Office Protocol** – это стандартный протокол Интернета для доступа к сообщениям, хранящимся на сервере почтовых ящиков. Его текущая версия имеет номер 3, отсюда и название **POP3**. Обычно почтовый клиент запрашивает у сервера почтовых ящиков все новые сообщения в одном сеансе, а затем закрывает соединение, после чего сервер почтовых ящиков удаляет все сообщения, помеченные как прочитанные.

Чтобы использовать NGINX в роли прокси для POP3, необходимо включить несколько простых директив:

```
mail {  
    auth_http localhost:9000/auth;  
  
    server {  
        listen 110;  
        protocol pop3;  
        proxy on;  
    }  
}
```

В этом фрагменте мы включаем почтовый модуль и настраиваем его для работы со службой POP3, которая запущена на порту 9000 на той же машине. NGINX будет прослушивать порт 110 для всех локальных IP-адресов, играя роль прокси-сервера для POP3. Обратите внимание, что сами почтовые серверы мы здесь не настраи-

ваем – задача известить NGINX о том, с каким сервером соединить конкретного клиента, возлагается на службу аутентификации.

Если почтовый сервер поддерживает не все возможности (или вы хотите явно ограничить их перечень), то NGINX поможет в этом:

```
mail {  
    pop3_capabilities TOP USER;  
}
```

Возможности (capabilities) – это способ объявить о поддержке необязательных команд. В случае POP3 клиент может запросить список поддерживаемых возможностей до или после аутентификации, поэтому важно правильно настроить их в NGINX.

Можно также задать перечень поддерживаемых методов аутентификации:

```
mail {  
    pop3_auth apop cram-md5;  
}
```

Если поддерживается метод аутентификации APOP, то служба аутентификации должна сообщить NGINX пароль пользователя в открытом виде, чтобы та смогла сгенерировать MD5-свертку.

Служба IMAP

Протокол **Internet Message Access Protocol** также является стандартным протоколом Интернета для доступа к сообщениям, хранящимся на сервере почтовых ящиков. Он существенно расширяет функциональность более раннего протокола POP. Как правило, все сообщения остаются на сервере, поэтому к одному почтовому ящику может обращаться несколько клиентов. Это также означает, что с IMAP-сервером заднего плана может быть установлено гораздо больше долгоживущих соединений, чем с POP3-сервером.

Проксирование соединений с IMAP-сервером настраивается примерно так же, как с POP3-сервером:

```
mail {  
    auth_http localhost:9000/auth;  
    imap_capabilities IMAP4rev1 UIDPLUS QUOTA;  
    imap_auth login cram-md5;  
  
    server {
```

```
listen 143;
protocol imap;
proxy on;
}
}
```

Отметим, что протокол задавать необязательно, потому что `imap` подразумевается по умолчанию. Мы включили его лишь для большей ясности.

Директивы `imap_capabilities` и `imap_auth` работают так же, как их аналоги для POP3.

Служба SMTP

Simple Mail Transport Protocol – стандартный протокол Интернета для передачи почтовых сообщений от одного сервера другому или от клиента серверу. Первоначально в нем не была предусмотрена аутентификация, но теперь определено расширение SMTP-AUTH.

Выше мы уже познакомились с логикой настройки почтового модуля. Она несложна и применима также к проксированию SMTP:

```
mail {
    auth_http localhost:9000/auth;
    smtp_capabilities PIPELINING 8BITMIME DSN;
    smtp_auth login cram-md5;

    server {
        listen 25;
        protocol smtp;
        proxy on;
    }
}
```

Наш прокси-сервер объявляет о поддержке лишь тех возможностей, которые перечислены в директиве `smtp_capabilities`, в противном случае он выдаст только список поддерживаемых механизмов аутентификации, потому что список расширений отправляется клиенту в ответ на команду `HELO/EHLO`. Это может быть полезно при проксировании на несколько SMTP-серверов, поддерживающих различные возможности. Можно настроить NGINX, так что он будет объявлять только о возможностях, поддерживаемых всеми серверами. Важно указать в списке лишь расширения, которые поддерживает сам SMTP-сервер.

Поскольку SMTP-AUTH – расширение SMTP, которое необязательно поддерживается в любой конфигурации, NGINX умеет проксировать SMTP-соединения, которые вообще не аутентифицируются. В таком случае службе аутентификации доступны только команды протокола HELO, MAIL FROM и RCPT TO, на основании которых она должна определить, какому серверу передать соединение с данным клиентом. В этой ситуации значением директивы `smtp_auth` должно быть `none`.

Использование SSL/TLS

Если организация требует шифровать почтовый трафик или вы сами хотите обеспечить повышенную защиту, то можете настроить NGINX так, чтобы использовался протокол TLS, реализовав тем самым спецификации POP3 поверх SSL, IMAP поверх SSL или SMTP поверх SSL. Для включения поддержки TLS нужно добавить либо директиву `starttls on` для поддержки команд STLS/STARTTLS, либо директиву `ssl on` для поддержки чистого протокола SSL/TLS. Кроме того, необходимо добавить директивы `ssl_*`, адаптированные для вашего вычислительного центра:

```
mail {
    # разрешить команду STLS для POP3 и STARTTLS для IMAP и SMTP
    starttls on;

    # отдать приоритет списку шифров сервера, чтобы мы сами определяли
    # уровень защиты
    ssl_prefer_server_ciphers on;

    # использовать только эти протоколы
    ssl_protocols TLSv1 SSLv3;

    # использовать только стойкие наборы шифров,
    # запретив анонимные DH и MD5, и отсортировать по стойкости
    ssl_ciphers HIGH:!ADH:!MD5:@STRENGTH;

    # использовать разделяемый кэш сеансов SSL, общий для всех
    # рабочих процессов
    ssl_session_cache shared:MAIL:10m;

    # сертификат и ключ для этого компьютера
    ssl_certificate /usr/local/etc/nginx/mail.example.com.crt;
    ssl_certificate_key /usr/local/etc/nginx/mail.example.com.key;
}
```

О различиях между соединением с чистым SSL/TLS и переходом от открытого соединения к зашифрованному с помощью SSL/TLS см. статью на странице https://www.fastmail.fm/help/technology_ssl_vs_tls_starttls.html.



Генерация SSL-сертификата с помощью OpenSSL

Для тех, кто никогда раньше не генерировал SSL-сертификаты, ниже приведена последовательность шагов.

Выполнить запрос на создание сертификата:

```
$ openssl req -newkey rsa:2048 -nodes -out mail.example.com.csr
-keyout mail.example.com.key
```

В ответ должен быть напечатан такой текст:

```
.....
.....
....+++
.....+++
writing new private key to 'mail.example.com.key'
-----
You are about to be asked to enter information that will
be incorporated
into your certificate request.
What you are about to enter is what is called a
Distinguished Name or a DN.
There are quite a few fields but you can leave some
blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [AU]:CH
State or Province Name (full name) [Some-State]:Zurich
Locality Name (eg, city) []:ZH
Organization Name (eg, company) [Internet Widgits Pty
Ltd]:Example Company
Organizational Unit Name (eg, section) []:
Common Name (e.g. server FQDN or YOUR name) []:mail.
example.com
Email Address []:
Please enter the following 'extra' attributes
to be sent with your certificate request
A challenge password []:
An optional company name []:
```

Этот запрос на подписание сертификата (Certificate Signing Request) (файл mail.example.com.csr) можно отправить для подписания в удостоверяющий центр, например Verisign или GoDaddy, либо подписать самостоятельно.

```
$ openssl x509 -req -days 365 -in mail.example.com.csr
-signkey mail.example.com.key -out mail.example.com.crt
```

В ответ будет напечатано:

```
Signature ok
subject=/C=CH/ST=Zurich/L=ZH/O=Example Company/CN=mail.
example.com
Getting Private key
```

Подписанный сертификат показан на рисунке ниже.

Отметим, что при использовании самоподписанного сертификата клиент, подключающийся к серверу, получит сообщение об ошибке. На производственный сервер следует устанавливать только сертификаты, подписанные признанным удостоверяющим центром.

```
-----BEGIN CERTIFICATE-----
MIIDPCCAiQCCQDdPKFcY1X35jANBgkqhkiG9w0BAQUFADBGMQswCQYDVQGEWJD
SDEPMAOGA1UECAwGwnVyawNoMQswCQYDVQQHDAJaSDEYMBYGA1UECgwPRXhhbXBs
ZSBDb21wYW55MRkwFwYDVQQDDbBtYWlsLmV4YW1wbGUuYy29tMB4XDTEyMDg0
MjczMloXDTEyMDgzMTEOMjczMlowYDELMAkGA1UEBhMCQ0gxZDANBgNVBAGMBGp1
cmlljaDELMAkGA1UEBwwCwGxGDAwBgNVBAoMD0V4YW1wbGUgQ29tcGFueTEZMBcG
A1UEAwwQbW9wY29tcGFueG9wY29tcGFueG9wY29tcGFueG9wY29tcGFueG9wY29tc
AQAQcGgEBANBwUGZQIKR+iuTxlPko/zSR+DbjDYqbMo4PdNvEN46nTFMkktv0SiK
1kfkl9L2jzVcmUUSZayLp3woDgxRpkpQ5eRpb7yefszWpJLxfVPTgftXtQkkftPvn
uz0Mf70gd2Xt8uI6n0At0DAR8+CxebIPRwIwZBXPrwFFjQvy4/qD7EXs33+xS08
9CMxkGo2FpQCSYE39jN3JtI29YibnZh01NALHRvnqyw3mdzR340mu5WNfjL/NElp
MOYFL7+SwzI4ktgmAo+Mic6JnXC0bSjrLlxZjWfn/STQiYQVzUi4jd1CswWtChw
tv67TRQ3edgvssvzfZlm7QfBbdYgjkUCAwEAATANBgkqhkiG9w0BAQUFAAOCAQEA
TdfdnGMRk2w/1KCGbxrg9bVmfKXUSIfpwyt0hG02EtLx83TzajqwtOKhmPh9Q/lc
GZdF1PGscdJ2Bc0eJBUGyt6mevEi2Dg4h727yVvnacnViQvzyLxQgmeC5rDej4EC
yDzzi4n0I/rddjPeQ0+cMFHz26scsKY0Remzp0yHT8JhK8AF2i0i0LzwaMqxClL
U7lkinHdTAg6nT4WpH05HtSBno8Xco/ujY6xIrShiP0na0d/B4TRCMB96KYhyMdd
Ayr0ZgLqsskKeAlnmusJA/7zbp1LwHarvUVFpzKed73554lfJ5kpy0ciHrIfyfj/2
dM/tjsDVjpE2B/meYBx8Kg==
-----END CERTIFICATE-----
```

Полный пример конфигурации почтового модуля

Доступ к почтовым службам часто осуществляется через один шлюз. В следующем примере конфигурации NGINX разрешено обслуживать трафик по протоколам POP3, IMAP и SMTP (в том числе зашифрованный) через одну службу аутентификации, предоставляя также клиентам возможность использовать команды STLS/STARTTLS на незашифрованных портах:

```
events {
    worker_connections 1024;
}

mail {
    server_name mail.example.com;
    auth_http localhost:9000/auth;

    proxy on;

    ssl_prefer_server_ciphers on;
    ssl_protocols TLSv1 SSLv3;
    ssl_ciphers HIGH:!ADH:!MD5:@STRENGTH;
    ssl_session_cache shared:MAIL:10m;
    ssl_certificate /usr/local/etc/nginx/mail.example.com.crt;
    ssl_certificate_key /usr/local/etc/nginx/mail.example.com.key;

    pop3_capabilities TOP USER;
    imap_capabilities IMAP4rev1 UIDPLUS QUOTA;
    smtp_capabilities PIPELINING 8BITMIME DSN;

    pop3_auth apop cram-md5;
    imap_auth login cram-md5;
    smtp_auth login cram-md5;

    server {
        listen 25;
        protocol smtp;
        timeout 120000;
    }

    server {
        listen 465;
        protocol smtp;
        ssl on;
    }

    server {
        listen 587;
        protocol smtp;
        starttls on;
    }

    server {
        listen 110;
        protocol pop3;
        starttls on;
    }

    server {
```

```
listen 995;
protocol pop3;
ssl on;
}

server {
    listen 143;
    protocol imap;
    starttls on;
}

server {
    listen 993;
    protocol imap;
    ssl on;
}
}
```

Как видите, в начале контекста `mail` мы объявили имя сервера. Это сделано для того, чтобы ко всем нашим почтовым службам можно было обращаться по имени `mail.example.com`. Даже если истинное имя машины, на которой работает NGINX, другое и у каждого почтового сервера есть собственное имя хоста, мы хотим, чтобы этот прокси-сервер был единственной точкой, известной пользователям. Именно это имя будет использоваться всюду, где NGINX должен представиться, например в начальном приветствии по протоколу SMTP.

Директива `timeout` в контексте сервера SMTP нужна для того, чтобы удвоить значение по умолчанию, поскольку нам известно, что этот конкретный проксируемый ретранслятор SMTP добавляет искусственную задержку, чтобы воспрепятствовать рассылке через него почтового спама.

Служба аутентификации

В предыдущем разделе мы несколько раз упомянули службу аутентификации, но не сказали, что конкретно она делает. Когда NGINX получает от пользователя запрос по протоколу POP3, IMAP или SMTP, то первым делом необходимо аутентифицировать соединение. NGINX не выполняет аутентификацию самостоятельно, а отправляет запрос службе аутентификации. Ответ, полученный от этой службы, NGINX использует, чтобы установить соединение с проксируемым почтовым сервером.

Служба аутентификации может быть написана на любом языке, лишь бы она была согласована с протоколом аутентификации, установленным NGINX. Этот протокол похож на HTTP, поэтому написать свою службу аутентификации несложно.

В запросе NGINX службе аутентификации присутствуют следующие заголовки:

- ☐ Host
- ☐ Auth-Method
- ☐ Auth-User
- ☐ Auth-Pass
- ☐ Auth-Salt
- ☐ Auth-Protocol
- ☐ Auth-Login-Attempt
- ☐ Client-IP
- ☐ Client-Host
- ☐ Auth-SMTP-Helo
- ☐ Auth-SMTP-From
- ☐ Auth-SMTP-To

Смысл их не нуждается в объяснении, и не все заголовки обязательны. Подробнее мы рассмотрим их, когда будем писать собственную службу аутентификации.

Для реализации службы аутентификации мы выбрали язык Ruby. Если его нет на вашей машине, не торопитесь устанавливать. Написанные на Ruby программы очень легко читать, поэтому просто смотрите на код и комментарии к нему. Адаптация этой программы к вашей среде и порядок ее запуска выходят за рамки этой книги. Данный пример приведен лишь в качестве отправной точки для написания необходимой вам службы аутентификации.



О том, как установить Ruby, подробно написано на сайте <https://rvm.io>.

Начнем с рассмотрения запроса.

Сначала мы извлекаем из полученных от NGINX заголовков интересные нас значения.

```
# механизм аутентификации
meth = @env['HTTP_AUTH_METHOD']
# имя пользователя (login)
user = @env['HTTP_AUTH_USER']
# пароль, открытый или зашифрованный в зависимости от используемого
```

```
# механизма аутентификации
pass = @env['HTTP_AUTH_PASS']
# в некоторых механизмах аутентификации для шифрования открытого пароля
# необходима затравка, но не в нашем примере
salt = @env['HTTP_AUTH_SALT']
# проксируемый протокол
proto = @env['HTTP_AUTH_PROTOCOL']
# количество попыток должно быть целым числом
attempt = @env['HTTP_AUTH_LOGIN_ATTEMPT'].to_i
# в нашей реализации не используется, приведено только для справки
client = @env['HTTP_CLIENT_IP']
host = @env['HTTP_CLIENT_HOST']
```



Что означает символ @?

Символ @ в Ruby обозначает переменную класса. Мы используем такие переменные, чтобы было проще передавать значения из одного места программы в другое. В коде выше переменная `@env` ссылается на окружение, передаваемое в Rack-запрос. Помимо нужных нам HTTP-заголовков, окружение содержит дополнительную информацию о том, как запущена служба.

Разобравшись с тем, как обработать пришедшие от NGINX заголовки, мы теперь должны что-то сделать с содержащейся в них информацией и отправить NGINX ответ. Ожидается, что ответ от службы аутентификации будет содержать следующие заголовки:

- ☐ `Auth-Status`: любое значение, кроме OK, считается ошибкой.
- ☐ `Auth-Server`: IP-адрес, на который проксируется соединение.
- ☐ `Auth-Port`: порт, на который проксируется соединение.
- ☐ `Auth-User`: имя пользователя, которое должно использоваться для аутентификации на почтовом сервере.
- ☐ `Auth-Pass`: пароль в открытом виде для команды APOP.
- ☐ `Auth-Wait`: сколько секунд ждать перед повторной попыткой аутентификации.
- ☐ `Auth-Error-Code`: альтернативный код ошибки для возврата клиенту.

Чаще всего используются заголовки `Auth-Status`, `Auth-Server` и `Auth-Port`. Их присутствия в ответе достаточно, чтобы аутентификация считалась состоявшейся.

Как видно из показанного ниже фрагмента кода, в зависимости от ситуации могут включаться и другие заголовки. Сам ответ содержит только заголовки с подставленными в них значениями.

Сначала проверим, не исчерпано ли количество попыток:

```
# вернуть ошибку, если превышено максимальное количество попыток входа
if attempt > @max_attempts
    @res[«Auth-Status»] = «Maximum login attempts exceeded»
    return
end
```

Если это не так, возвращаем подходящие заголовки, записывая в них значения, полученные от нашего механизма аутентификации:

```
@res[«Auth-Status»] = «OK»
@res[«Auth-Server»] = @mailhost
# возвращаем порт, соответствующий этому протоколу
@res[«Auth-Port»] = MailAuth::Port[proto]
# если используется APOP, то пароль нужно вернуть в открытом виде
if meth == 'apop' && proto == 'pop3'
    @res[«Auth-User»] = user
    @res[«Auth-Pass»] = pass
end
```

Если попытка аутентификации не удалась, мы должны сообщить об этом NGINX.

```
# в случае ошибки аутентификации возвращаем соответствующий ответ
@res[«Auth-Status»] = «Invalid login or password»
# и устанавливаем время в секундах, которое клиент должен
# ждать перед следующей попыткой
@res[«Auth-Wait»] = «3»
# Можно также установить код ошибки, который следует вернуть SMTP-клиенту
@res[«Auth-Error-Code»] = «535 5.7.8»
```

Не все заголовки обязательны; какие именно включать, зависит от состояния запроса на аутентификацию и (или) возникших ошибок.



Интересно, что в заголовке `Auth-User` можно вернуть не то имя пользователя, которое указано в запросе. Это может оказаться полезным, например, в случае, когда производится переход от старого проксируемого сервера, который принимал имя пользователя без домена, к новому, который требует указания домена. Благодаря указанной возможности NGINX может передавать новому проксируемому серверу модифицированное имя.

База данных для аутентификации может быть произвольной: плоский текстовый файл, каталог LDAP, реляционная СУБД и т. д. Не-

обязательно использовать то же хранилище, к которому обращается сама почтовая служба, однако данные в обоих хранилищах должны быть синхронизированы во избежание ошибок из-за их расхождения.

В нашем примере данные для аутентификации хранятся в простом хэше:

```
@auths = { "test:1234" => '127.0.1.1' }
```

Для проверки пользователя нужно просто произвести поиск в этом хэше:

```
# возвращается значение, соответствующее ключу 'user:pass'
if @auths.key?("#{user}:#{pass}")
  @mailhost = @auths["#{user}:#{pass}"]
  return true
# если такого ключа нет, метод возвращает false
else
  return false
end
```

Объединяя все три части, получаем полный код службы аутентификации:

```
#!/usr/bin/env rackup
# Это простой HTTP-сервер, согласованный с протоколом аутентификации,
# который определен почтовым модулем NGINX.
#
require 'logger'
require 'rack'

module MailAuth

  # настраиваем соответствие протоколов и портов
  Port = {
    'smtp' => '25',
    'pop3' => '110',
    'imap' => '143'
  }

  class Handler

    def initialize
      # настраиваем протоколирование, как для почтовой службы
      @log = Logger.new("| logger -p mail.info")
      # заменяем обычную временную метку именем службы и ее pid'ом
      @log.datetime_format = "nginx_mail_proxy_auth pid: "
      # заголовок "Auth-Server" должен содержать IP-адрес
```

```
@mailhost = '127.0.0.1'
# задаем максимальное число попыток входа
@max_attempts = 3
# Для этого примера в качестве 'базы данных' аутентификации мы используем
# фиксированный хэш.
# В реальной программе нужно будем заменить его методом, который
# подключается к каталогу LDAP или к базе данных
@auths = { "test:1234" => '127.0.1.1' }
end
```

После описанной выше инициализации модуля мы сообщаем Rack, какие запросы хотим обрабатывать, и определяем метод `get`, который будет отвечать на запросы от NGINX.

```
def call(env)
  # интересные нас заголовки находятся в окружении
  @env = env
  # настраиваем объекты запроса и ответа
  @req = Rack::Request.new(env)
  @res = Rack::Response.new
  # передаем управление методу, называемому так же, как HTTP-метод,
  # которым был отправлен запрос
  self.send(@req.request_method.downcase)
  # возвращаемся сюда, чтобы завершить ответ
  @res.finish
end

def get
  # механизм аутентификации
  meth = @env['HTTP_AUTH_METHOD']
  # имя пользователя (login)
  user = @env['HTTP_AUTH_USER']
  # пароль, открытый или зашифрованный в зависимости от используемого
  # механизма аутентификации
  pass = @env['HTTP_AUTH_PASS']
  # в некоторых механизмах аутентификации для шифрования открытого пароля
  # необходима затравка, но не в нашем примере
  salt = @env['HTTP_AUTH_SALT']
  # проксируемый протокол
  proto = @env['HTTP_AUTH_PROTOCOL']
  # количество попыток должно быть целым числом
  attempt = @env['HTTP_AUTH_LOGIN_ATTEMPT'].to_i
  # в нашей реализации не используется, приведено только для справки
  client = @env['HTTP_CLIENT_IP']
  host = @env['HTTP_CLIENT_HOST']

  # вернуть ошибку, если превышено максимальное количество попыток входа
  if attempt > @max_attempts
    @res["Auth-Status"] = "Maximum login attempts exceeded"
```

```

    return
end

# для особого случая smtp-транзакций, не требующих аутентификации,
# в файле nginx.conf содержится такой фрагмент:
#     smtp_auth none;
# возможно, понадобится справочная таблица для переадресации
# некоторых отправителей на определенные SMTP-серверы
if meth == 'none' && proto == 'smtp'
    helo = @env['HTTP_AUTH_SMTP_HELO']
    # мы хотим выделить адреса из следующих двух заголовков
    from = @env['HTTP_AUTH_SMTP_FROM'].split(/: /)[1]
    to = @env['HTTP_AUTH_SMTP_TO'].split(/: /)[1]
    @res[«Auth-Status»] = «OK»
    @res[«Auth-Server»] = @mailhost
    # возвращаем порт, соответствующий этому протоколу
    @res[«Auth-Port»] = MailAuth::Port[proto]
    @log.info(«a mail from #{from} on #{helo} for #{to}»)
# пытаемся выполнить аутентификацию с помощью информации
# из полученных заголовков
elsif auth(user, pass)
    @res[«Auth-Status»] = «OK»
    @res[«Auth-Server»] = @mailhost
    # возвращаем порт, соответствующий этому протоколу
    @res[«Auth-Port»] = MailAuth::Port[proto]
    # если используется APOP, то пароль нужно вернуть в открытом виде
    if meth == 'apop' && proto == 'pop3'
        @res[«Auth-User»] = user
        @res[«Auth-Pass»] = pass
    end
    @log.info(«+ #{user} from #{client}»)
# попытка аутентификации не удалась
else
    # в случае ошибки аутентификации возвращаем соответствующий ответ
    @res[«Auth-Status»] = «Invalid login or password»
    # и устанавливаем время в секундах, которое клиент должен
    # ждать перед следующей попыткой
    @res[«Auth-Wait»] = «3»
    # Можно также установить код ошибки, который следует вернуть
    # SMTP-клиенту
    @res[«Auth-Error-Code»] = «535 5.7.8»
    @log.info(«! #{user} from #{client}»)
end
end

```

end

Следующая секция объявлена закрытой (*private*), поскольку находящиеся в ней методы может вызывать только сам содержащий их класс. Метод `auth` – рабочая лошадка службы аутентификации, он проверяет корректность имени пользователя и пароля. Метод

`method_missing` служит для обработки недопустимых HTTP-методов и возвращает ошибку Not Found (Не найдено):

```
private

# наш метод аутентификации; измените в соответствии со своим окружением
def auth(user, pass)
  # просто возвращает значение, соответствующее ключу 'user:pass'
  if @auths.key?("#{user}:#{pass}")
    @mailhost = @auths["#{user}:#{pass}"]
    return @mailhost
  # если такого ключа нет, метод возвращает false
  else
    return false
  end
end

# на случай, если какой-нибудь другой процесс попытается обратиться к
# этой службе и отправит запрос методом, отличным от GET
def method_missing(env)
  @res.status = 404
end

end # класс MailAuthHandler
end # модуль MailAuth
```

И в последней секции настраиваем сервер, так чтобы запросы к URI `/auth` маршрутизировались правильному обработчику:

```
# настраиваем каркас Rack
use Rack::ShowStatus
# сопоставляем URI /auth наш обработчик аутентификации
map «/auth» do
  run MailAuth::Handler.new
end
```

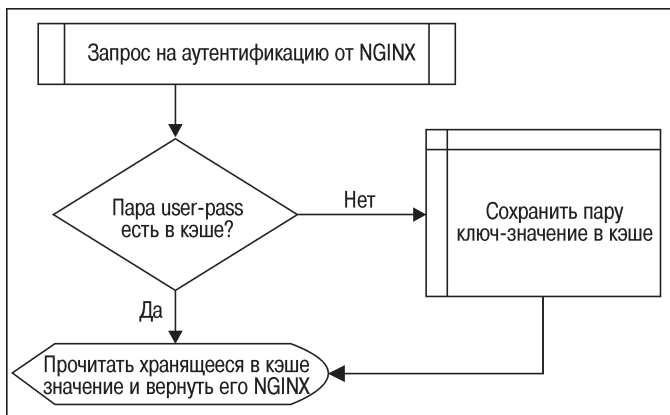
Этот код нужно сохранить в файле `nginx_mail_proxy_auth.ru` и вызывать с флагом `-p <port>`, который говорит, какой порт он должен прослушивать. Дополнительные сведения о каркасе Rack, реализующем интерфейс к веб-серверу, смотрите на сайте <http://rack.github.com>.

Использование в связке с memcached

Если клиенты часто обращаются к почтовым службам через ваш прокси-сервер или службе аутентификации доступно не слишком много ресурсов, то имеет смысл включить в инфраструктуру слой кэ-

ширования. Для этой цели мы воспользуемся программой `memcached`, которая будет кэшировать результаты аутентификации в памяти.

NGINX может искать ключ в `memcached`, но только в контексте определенного местоположения в модуле `http`. Поэтому мы должны реализовать собственный слой кэширования вне NGINX.



Как видно из блок-схемы, мы сначала проверяем, есть ли данная комбинация имени пользователя и пароля в кэше. Если нет, мы запрашиваем информацию у службы аутентификации и помещаем пару ключ-значение в кэш. В противном случае эти данные можно взять прямо из кэша.



Компания Zimbra разработала модуль `memcached` для NGINX, который делает все то же самое непосредственно в контексте NGINX. Но пока этот код не включен в официальный дистрибутив NGINX.

Показанный ниже код дополняет нашу исходную службу аутентификации слоем кэширования (в данном случае это, пожалуй, перебор, но этот код может служить основой для работы с базой данных для аутентификации, размещенной в сети).

```
# gem install memcached (зависит от библиотек libssl2 и gettext)
require 'memcached'
```

```
# здесь задаются IP-адрес и порт, на котором работает memcached
@cache = Memcached.new(«localhost:11211»)
```

```
def get_cache_value(user, pass)
  resp = ''
  begin
    # сначала смотрим, есть ли ключ в кэше
    resp = @cache.get(«#{user}:#{pass}»)
  rescue Memcached::NotFound
    # если нет, вызываем метод auth
    resp = auth(user, pass)
    # и сохраняем полученный результат в кэше с ключом 'user:pass'
    @cache.set(«#{user}:#{pass}», resp)
  end
  # явно возвращаем ответ вызывающей программе
  return resp
end
```

Чтобы использовать этот код, необходимо, конечно, установить и запустить memcached. Для вашей операционной системы, скорее всего, имеется готовый пакет:

❑ Linux (на основе Debian)

```
sudo apt-get install memcached
```

❑ Linux (на основе rpm)

```
sudo yum install memcached
```

❑ FreeBSD

```
sudo pkg_add -r memcached
```

Для настройки memcached нужно просто указать параметры при запуске. Никакого конфигурационного файла, читаемого непосредственно memcached, не существует, хотя операционная система может включать файл, который служит для упрощения передачи параметров.

Ниже перечислены наиболее важные параметры memcached:

- ❑ -l: задает адрес (или адреса), прослушиваемый memcached (по умолчанию любой). Важно отметить, что в целях безопасности memcached не должен прослушивать адрес, доступный из Интернета, потому что никакой аутентификации не предусмотрено;
- ❑ -m: задаем объем оперативной памяти, отводимой под кэш (в мегабайтах);
- ❑ -c: задает максимальное количество одновременных соединений (по умолчанию 1024);
- ❑ -p: задает порт, прослушиваемый memcached (по умолчанию 11211).

Задав разумные значения этих параметров, остается запустить memcached.

Теперь, заменив строку `elseif auth(user, pass)` строкой `elseif get_cache_value(user, pass)` в файле `nginx_mail_proxy_auth.ru`, мы получим службу аутентификации со слоем кэширования, который ускоряет обработку запросов.

Интерпретация журналов

Файлы журналов – лучшее средство понять, что происходит, когда система работает не так, как ожидается. Объем записываемой в журнал информации зависит от указанного в конфигурационном файле уровня подробности и от того, был ли NGINX собран с поддержкой отладки (параметр `--enable-debug`).

В каждой строке журнала указывается один из уровней серьезности, сконфигурированных в директиве `error_log`. Определены следующие уровни (в порядке возрастания серьезности): `debug`, `info`, `notice`, `warn`, `error`, `crit`, `alert` и `emerg`. Если задан некоторый уровень, то в журнал помещаются сообщения этого и более высоких уровней. По умолчанию предполагается уровень `error`.

Для модуля `mail` обычно задается уровень протоколирования `info`, чтобы можно было получать столько информации, сколько возможно без вывода отладочных сообщений. Отладочное протоколирование в данном случае дало бы только сведения о точках входа в функции и паролх, указанных для соединений.



Поскольку почта сильно зависит от правильности функционирования DNS, многие ошибки могут быть связаны с некорректными записями DNS или устареванием информации в кэше. Если вы подозреваете, что ошибка вызвана именно неправильным разрешением имен, то можете узнать у NGINX, какой IP-адрес был сопоставлен конкретному доменному имени, включив отладочное протоколирование. К сожалению, для этого понадобится перекомпилировать двоичный файл `nginx`, если он изначально не был скомпилирован с поддержкой отладки.

Ниже описана типичная последовательность записей журнала, относящихся к сеансу по протоколу POP3.

Сначала клиент устанавливает соединение с прокси-сервером:

```
<timestamp> [info] <worker pid>#0: *<connection id> client <ip address>  
connected to 0.0.0.0:110
```

После успешного входа выводится сообщение, содержащее все сведения о соединении:

```
<timestamp> [info] <worker pid>#0: *<connection id> client logged in, client:
<ip address>, server: 0.0.0.0:110, login: «<username>», upstream: <upstream
ip>:<upstream port>, [<client ip>:<client port>- <local ip>:110] <=> [<local
ip>:<high port>-<upstream ip>:<upstream port>]
```

Отметим, что часть перед двусторонней стрелкой <=> относится к стороне клиент–прокси, а часть после стрелки – к стороне прокси–почтовый сервер. Эта информация выводится еще раз при завершении сеанса.

```
<timestamp> [info] <worker pid>#0: *<connection id> proxied session done,
client: <ip address>, server: 0.0.0.0:110, login: «<username>», upstream:
<upstream ip>:<upstream port>, [<client ip>:<client port>- <local ip>:110] <=>
[<local ip>:<high port>-<upstream ip>:<upstream port>]
```

Таким образом, мы видим, какие порты использованы на всех сторонах соединения, что помогает отлаживать потенциальные ошибки и при необходимости коррелировать записи в этом журнале и в журнале брандмауэра.

Остальные записи журнала на уровне info относятся к тайм-аутам и некорректным командам либо ответам, полученным от клиента или от проксируемого сервера.

На уровне warn обычно выводятся сообщения об ошибках конфигурации:

```
<timestamp> [warn] <worker pid>#0: *<connection id> «starttls» directive
conflicts with «ssl on»
```

Многие сообщения, выводимые на уровне error, свидетельствуют об ошибках при работе со службой аутентификации. Из следующих сообщений понятно, в каком состоянии находилось соединение в момент ошибки:

```
<timestamp> [error] <worker pid>#0: *<connection id> auth http server
127.0.0.1:9000 timed out while in http auth state, client: <client ip>, server:
0.0.0.0:25
<timestamp> [error] <worker pid>#0: *<connection id> auth http server
127.0.0.1:9000 sent invalid response while in http auth state, client: <client
ip>, server: 0.0.0.0:25
```

Если по какой-либо причине на запрос об аутентификации не получен ответ, соединение разрывается. NGINX не знает, какому серверу передать запрос клиента, поэтому может только закрыть соединение с сообщением `Internal server error` и кодом ответа, зависящим от протокола.

В журнал помещается запись, которая может содержать или не содержать имя пользователя. Ниже показано сообщение для SMTP-соединения, требующего аутентификации:

```
<timestamp> [error] <worker pid>#0: *<connection id> auth http server
127.0.0.1:9000 did not send server or port while in http auth state, client:
<client ip>, server: 0.0.0.0:25, login: «<login>»
```

Отметим, что в двух предыдущих сообщениях информация об имени пользователя (`login`) отсутствовала.

Сообщение уровня `alert` означает, что NGINX не смог установить ожидаемое значение параметра, но в остальном работает нормально.

Напротив, сообщения уровня `emerg` означают, что NGINX не смог запуститься; необходимо либо устранить причину ошибки, либо изменить конфигурацию. Если NGINX уже запущен, то он не станет перезапускать рабочие процессы, пока не будет произведено изменение:

```
<timestamp> [error] <worker pid>#0: *<connection id> no «http_auth» is defined
for server in /opt/nginx/conf/nginx.conf:32
```

В данном случае мы должны определить службу аутентификации с помощью директивы `http_auth`.

Ограничения операционной системы

Вы можете оказаться в ситуации, когда NGINX работает не так, как ожидается, – то соединения рвутся, то в журнале появляются предупреждения. В этих случаях необходимо хорошо понимать, какие ограничения налагает на NGINX операционная система и как настроить ее для оптимальной работы сервера.

У почтового прокси-сервера проблемы чаще всего связаны с ограничением количества соединений. Чтобы понять, что это значит, следует разобраться, как NGINX обрабатывает запросы на установление соединения от клиентов. Главный процесс NGINX запускает

несколько отдельных рабочих процессов. Каждый процесс способен обработать фиксированное число соединений, определяемое директивой `worker_connections`. Для каждого проксируемого соединения NGINX открывает новое соединение с почтовым сервером. Для любого соединения необходим файловый дескриптор и по одному TCP-порту из диапазона эфемерных портов для каждой пары (IP-адрес, порт почтового сервера) (см. объяснение ниже).

Количество открытых файловых дескрипторов в зависимости от операционной системы настраивается в файле ресурсов или путем отправки сигнала демону, управляющему ресурсами. Чтобы узнать текущее значение, введите следующую команду оболочки:

```
ulimit -n
```

Если ваши расчеты показывают, что этот порог слишком мал или в журнале ошибок появляется сообщение `worker_connections exceed open file resource limit`, значит, значение необходимо увеличить. Сначала настройте максимальное число открытых файловых дескрипторов на уровне операционной системы – только для пользователя, от имени которого работает NGINX, или глобально. Затем в директиве `worker_rlimit_nofile` в главном контексте файла `nginx.conf` укажите новое значение. Чтобы измененное значение вступило в силу, достаточно послать `nginx` сигнал перезагрузки конфигурации (HUP); перезапускать главный процесс при этом не нужно.

Если вы столкнулись с нехваткой TCP-портов, то следует расширить диапазон эфемерных портов. Из этого диапазона операционная система выбирает порты для исходящих соединений. По умолчанию может быть всего 5000 портов, но обычно их бывает 16 384. Хорошее описание процедуры расширения этого диапазона в различных операционных системах приведено на странице http://www.ncftp.com/ncftpd/doc/misc/ephemeral_ports.html.

Резюме

В этой главе мы показали, как настраивать NGINX для проксирования соединений по протоколам POP3, IMAP и SMTP. Каждый протокол можно настроить по отдельности, объявив о поддержке тех или иных возможностей проксируемым сервером. Можно шифровать трафик по протоколу TLS при наличии у сервера подходящего SSL-сертификата.

Для функционирования почтового модуля необходима служба аутентификации, поскольку без нее никакое проксирование невозможно. Мы привели подробный пример такой службы, описав, что ожидается в запросе и как следует формировать ответ. Опираясь на этот фундамент, вы сможете написать службу аутентификации, отвечающую вашим потребностям.

Умение интерпретировать журналы – один из самых важных навыков администратора. NGINX формирует достаточно детальные, хотя и не сразу понятные, журналы. Но зная, в какие моменты жизни соединения выводятся различные сообщения и в каком состоянии находится NGINX в каждый такой момент, нетрудно понять, что происходит.

NGINX, как и любая другая программа, работает в контексте операционной системы. Поэтому крайне важно знать, как ослабить ограничения, которые ОС налагает на NGINX. Если дальнейшее ослабление невозможно, то следует искать другое архитектурное решение – увеличить количество серверов, на которых работает NGINX, или применить какую-то другую технику, позволяющую уменьшить количество соединений, обрабатываемых одним экземпляром.

В следующей главе мы узнаем, как настроить NGINX для проксирования HTTP-соединений.



Глава 4. NGINX как обратный прокси-сервер

Обратным прокси-сервером называется веб-сервер, который является конечной точкой соединения с клиентом и открывает новое соединение с проксируемым сервером от имени клиента. **Проксируемый сервер** определяется как сервер, с которым NGINX устанавливает соединение для выполнения запроса клиента. Проксируемые серверы могут принимать разные формы, и для каждой NGINX можно настроить по-разному.

Настройка NGINX, которую мы как раз и изучаем, не всегда бывает простой. Для решения внешне похожих задач применяются различные директивы. Некоторые параметры вообще не следует использовать, поскольку это может привести к неожиданным результатам.

Иногда проксируемый сервер не может выполнить запрос. NGINX умеет доставлять клиенту сообщения об ошибках – непосредственно от проксируемого сервера, со своего локального диска или путем переадресации на страницу, размещенную совсем на другом сервере.

В силу самой природы обратного прокси-сервера проксируемый сервер не получает информацию непосредственно от клиента. Часть этой информации, например истинный IP-адрес клиента, важна для отладки и для отслеживания. Ее можно передать проксируемому серверу в виде заголовков.

Мы рассмотрим эти вопросы, а также дадим обзор некоторых директив из модуля прокси-сервера в следующих разделах.

- ☐ Введение в технологию обратного проксирования.
- ☐ Типы проксируемых серверов.
- ☐ Преобразование конфигурации с «if» в более современную форму.

- ❑ Использование документов с описанием ошибок для обработки ошибок проксирования.
- ❑ Определение истинного IP-адреса клиента.

Введение в технологию обратного проксирования

NGINX может выступать в роли обратного прокси-сервера, который является конечной точкой соединения с клиентом и открывает новое соединение с проксируемым сервером. По ходу дела запрос можно проанализировать, исходя из URI-адреса, параметров клиента или иной логики, чтобы лучше обслужить его. Любая часть исходного URL-запроса при прохождении через обратный прокси-сервер может быть преобразована.

С точки зрения проксирования, наиболее важна директива `proxy_pass`. Она принимает один параметр – URI-адрес, на который следует передать запрос. Использование `proxy_pass` с указанием URI заменяет `request_uri` этой частью. Так, в следующем примере `/uri` будет заменено на `/newuri` при передаче запроса проксируемому серверу:

```
location /uri {  
    proxy_pass http://localhost:8080/newuri;  
}
```

Но из этого правила есть два исключения. Во-первых, если местоположение определено с помощью регулярного выражения, то преобразование URI не производится. В примере ниже URI `/local` будет передан далее без изменения, а не заменен на `/foreign`, как, вероятно, хотел автор:

```
location ~ ^/local {  
    proxy_pass http://localhost:8080/foreign;  
}
```

Второе исключение состоит в том, что если внутри местоположения есть правило переписывания, которое изменяет URI, и затем NGINX использует этот URI для обработки запроса, то преобразование не производится. В примере ниже проксируемому серверу передается URI `/index.php?page=<match>`, где `<match>` – запомненное подвыражение в скобках, а не `/index`, как указано в части URI директивы `proxy_pass`:

```
location / {
    rewrite /(.*$ /index.php?page=$1 break;
    proxy_pass http://localhost:8080/index;
}
```



Здесь флаг `break` используется для того, чтобы немедленно прекратить обработку директив модуля `rewrite`.

В обоих случаях часть URI после доменного имени в директиве `proxy_pass` несущественна, поэтому ее можно опустить без ущерба для полноты конфигурации:

```
location ~ ^/local {
    proxy_pass http://localhost:8080;
}

location / {
    rewrite /(.*$ /index.php?page=$1 break;
    proxy_pass http://localhost:8080;
}
```

Модуль proxy

В следующей таблице перечислены некоторые наиболее употребительные директивы модуля `proxy`.

Директивы модуля proxy

Директива	Описание
<code>proxy_connect_timeout</code>	Максимальное время ожидания соединения с проксируемым сервером
<code>proxy_cookie_domain</code>	Подменяет атрибут <code>domain</code> в заголовке <code>Set-Cookie</code> от проксируемого сервера; можно указать строку, регулярное выражение или ссылку на переменную
<code>proxy_cookie_path</code>	Подменяет атрибут <code>path</code> в заголовке <code>Set-Cookie</code> от проксируемого сервера; можно указать строку, регулярное выражение или ссылку на переменную
<code>proxy_headers_hash_bucket_size</code>	Максимальный размер имен заголовков
<code>proxy_headers_hash_max_size</code>	Общий размер заголовков, полученных от проксируемого сервера
<code>proxy_hide_header</code>	Список заголовков, которые не следует передавать клиенту

Директива	Описание
<code>proxy_http_version</code>	Версия протокола HTTP, которой следует придерживаться при взаимодействии с проксируемым сервером (для соединений типа <code>keepalive</code> должна быть 1.1)
<code>proxy_ignore_client_abort</code>	Если значение равно <code>on</code> , то NGINX не станет разрывать соединение с проксируемым сервером, в случае когда клиент разрывает свое соединение
<code>proxy_ignore_headers</code>	Какие заголовки можно игнорировать при обработке ответа от проксируемого сервера
<code>proxy_intercept_errors</code>	Если значение равно <code>on</code> , то NGINX будет отображать страницу, заданную директивой <code>error_page</code> , вместо ответа, полученного от проксируемого сервера
<code>proxy_max_temp_file_size</code>	Максимальный размер временного файла, в который записывается часть ответа, в случае когда он не помещается целиком в буферах памяти
<code>proxy_pass</code>	Задаёт URL проксируемого сервера, которому передается запрос
<code>proxy_pass_header</code>	Отменяет сокрытие заголовков, определенных в директиве <code>proxy_hide_header</code> , разрешая передавать их клиенту
<code>proxy_pass_request_body</code>	Если значение равно <code>off</code> , то тело запроса не передается проксируемому серверу
<code>proxy_pass_request_headers</code>	Если значение равно <code>off</code> , то заголовки запроса не передаются проксируемому серверу
<code>proxy_read_timeout</code>	Сколько времени может пройти между двумя последовательными операциями чтения данных от проксируемого сервера, прежде чем соединение будет закрыто. Значение следует увеличить, если проксируемый сервер обрабатывает запросы медленно
<code>proxy_redirect</code>	Перезаписывает заголовки <code>Location</code> и <code>Refresh</code> , полученные от проксируемого сервера; полезно для обхода допущений, принятых каркасом разработки приложений
<code>proxy_send_timeout</code>	Сколько времени может пройти между двумя последовательными операциями записи данных на проксируемый сервер, прежде чем соединение будет закрыто
<code>proxy_set_body</code>	Эта директива позволяет изменить тело запроса, отправляемое проксируемому серверу

Директива	Описание
proxy_set_header	Перезаписывает заголовки, отправляемые проксируемому серверу. Может также применяться для подавления некоторых заголовков (если в качестве значения указать пустую строку)
proxy_temp_file_write_size	Ограничивает размер данных в одной операции записи во временный файл, чтобы NGINX не слишком долго блокировала исполнение программы при обработке одного запроса
proxy_temp_path	Каталог для хранения временных файлов, получаемых от проксируемого сервера. Может быть многоуровневым

Ниже многие из перечисленных выше директив собраны воедино. Этот фрагмент можно вставить в конфигурационный файл в то место, где находится директива `proxy_pass`.

Содержимое файла `proxy.conf`:

```
proxy_redirect off;
proxy_set_header Host $host;
proxy_set_header X-Real-IP $remote_addr;
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
client_max_body_size 10m;
client_body_buffer_size 128k;
proxy_connect_timeout 30;
proxy_send_timeout 15;
proxy_read_timeout 15;
proxy_send_lowat 12000;
proxy_buffer_size 4k;
proxy_buffers 4 32k;
proxy_busy_buffers_size 64k;
proxy_temp_file_write_size 64k;
```

Мы задали в этих директивах значения, которые, на наш взгляд, полезны для настройки обратного прокси-сервера.

- ❑ В директиве `proxy_redirect` задано значение `off`, потому что переписывать заголовок `Location` в большинстве случаев не нужно.
- ❑ Заголовок `Host` устанавливается так, чтобы проксируемый сервер мог сопоставить запрос с виртуальным сервером или как-то иначе использовать часть переданного клиентом URL-адреса, содержащую имя хоста.

- ❑ Заголовки X-Real-IP и X-Forwarded-For служат аналогичным целям – передать проксируемому серверу информацию об IP-адресе подключившегося клиента.
 - Переменная `$remote_addr`, подставляемая в заголовок X-Real-IP, содержит IP-адрес клиента, определенный NGINX.
 - Переменная `$proxy_add_x_forwarded_for` содержит значение заголовка X-Forwarded-For из запроса клиента, за которым следует значение переменной `$remote_addr`.
- ❑ Директива `client_max_body_size`, строго говоря, не является директивой модуля `proxy`, но упомянута здесь, поскольку имеет прямое отношение к конфигурации прокси-сервера. Если ее значение слишком мало, то загруженные клиентом файлы не попадут проксируемому серверу. Задавая значение этой переменной, помните, что размер файла, загружаемого с помощью веб-формы, обычно оказывается больше, чем видно в файловой системе.
- ❑ Директива `proxy_connect_timeout` определяет, как долго NGINX будет ждать установления соединения с проксируемым сервером.
- ❑ Директивы `proxy_read_timeout` и `proxy_send_timeout` задают максимальное время между последовательными операциями чтения от проксируемого сервера и записи на него.
- ❑ Директива `proxy_send_lowat` применима только в ОС FreeBSD и определяет, сколько байт должно скопиться в буфере отправки сокета, чтобы данные были переданы протоколу.
- ❑ Директивы `proxy_buffer_size`, `proxy_buffers` и `proxy_busy_buffers_size` будут подробно рассмотрены в следующей главе. Пока скажем лишь, что они определяют видимую быстроту реакции NGINX на запросы пользователя.
- ❑ Директива `proxy_temp_file_write_size` определяет, сколько времени рабочий процесс может блокировать исполнение при записи данных во временный файл: чем больше значение, тем дольше процесс блокируется.

Эти директивы можно включить в конфигурационный файл, как показано ниже, и использовать многократно:

```
location / {  
    include proxy.conf;  
    proxy_pass http://localhost:8080;  
}
```

Если какая-то директива в конкретном случае должна иметь другое значение, то его можно переопределить после включения файла:

```
location /uploads {
    include proxy.conf;
    client_max_body_size500m;
    proxy_connect_timeout 75;
    proxy_send_timeout 90;
    proxy_read_timeout 90;
    proxy_pass http://localhost:8080;
}
```



В данном случае порядок важен. Если одна и та же директива встречается в конфигурационном файле (с учетом включаемых файлов) несколько раз, то NGINX возьмет последнее значение.

Унаследованные серверы с куками

Бывает, что несколько унаследованных приложений необходимо разместить за одной общей окончечной точкой. Унаследованные приложения написаны в предположении, что они взаимодействуют непосредственно с клиентом. Они устанавливают в куках собственное доменное имя и считают, что к ним всегда возможен доступ по адресу /. Если поместить перед этими серверами новую окончечную точку, то данные предположения окажутся неверными. В следующем фрагменте конфигурационного файла атрибуты `domain` и `path` в куках переписываются с учетом новой окончечной точки:

```
server {
    server_name app.example.com;

    location /legacy1 {
        proxy_cookie_domain legacy1.example.com app.example.com;
        proxy_cookie_path $uri /legacy1$uri;
        proxy_redirect default;
        proxy_pass http://legacy1.example.com/;
    }
}
```



Значение переменной `$uri` уже включает начальную косую черту (/), так что дублировать ее не нужно.

```
location /legacy2 {
    proxy_cookie_domain legacy2.example.org app.example.com;
    proxy_cookie_path $uri /legacy2$uri;
```

```
proxy_redirect default;
proxy_pass http://legacy2.example.org/;
}

location / {
    proxy_pass http://localhost:8080;
}
}
```

Модуль upstream

С модулем proxy тесно связан модуль upstream. Директива upstream начинает новый контекст, в котором определяется группа проксируемых серверов. Этим серверам можно приписать разные веса (чем выше вес, тем больше соединений будет передано конкретному проксируемому серверу), они могут иметь разные типы (TCP или в домене UNIX). Их даже можно пометить как остановленные на техническое обслуживание.

В следующей таблице перечислены директивы, допустимые в контексте upstream.

Директивы модуля upstream

Директива	Описание
ip_hash	Обеспечивает равномерное распределение клиентских соединений по всем серверам за счет хэширования IP-адреса по его сети класса C
keepalive	Количество соединений с проксируемыми серверами, кэшируемых в одном рабочем процессе. При использовании с HTTP-соединениями значение proxy_http_version должно быть равно 1.1, а значение proxy_set_header — Connection ""
least_conn	Активирует алгоритм балансировки нагрузки, согласно которому очередной запрос передается серверу с наименьшим числом активных соединений
server	Определяет адрес (доменное имя, или IP-адрес с необязательным номером TCP-порта, или путь к сокету в домене UNIX) и необязательные параметры проксируемого сервера, а именно: • weight: относительный вес сервера; • max_fails: максимальное число неудачных попыток установления связи с сервером в течение времени fail_timeout, после которого сервер помечается как неработоспособный; • fail_timeout: время, в течение которого сервер должен ответить на запрос, и время, в течение которого сервер считается неработоспособным; • backup: такой сервер получает запрос, только если все остальные неработоспособны; • down: помечает сервер как непригодный для обработки запросов

Кэширование соединений

Директива `keepalive` заслуживает особого упоминания. Она говорит, сколько соединений с проксируемым сервером NGINX должен держать открытыми в каждом рабочем процессе. Кэш соединений полезен в случае, когда NGINX должен постоянно держать некоторое количество открытых соединений с проксируемым сервером. Если проксируемый сервер работает по протоколу HTTP, то NGINX может задействовать механизм постоянных соединений, специфицированный в версии HTTP/1.1, для поддержания таких открытых соединений.

```
upstream apache {
    server 127.0.0.1:8080;
    keepalive 32;
}

location / {
    proxy_http_version 1.1;
    proxy_set_header Connection "";
    proxy_pass http://apache;
}
```

Здесь мы говорим, что хотели бы держать 32 открытых соединения с Apache, работающим на порту 8080 сервера `localhost`. NGINX должен будет выполнить процедуру кэширования TCP только для первых 32 соединений в каждом рабочем процессе, а затем может оставить эти соединения открытыми, не посылая заголовок `Connection` со значением `close`. В директиве `proxy_http_version` мы указываем, что хотели бы работать с проксируемым сервером по протоколу HTTP/1.1. Мы также очищаем значение заголовка `Connection` с помощью директивы `proxy_set_header`, чтобы не проксировать соединения с клиентом самостоятельно.

Если для обслуживания запросов потребуется больше 32 соединений, то NGINX их, конечно, откроет. Но по достижении порогового значения NGINX будет закрывать редко используемые соединения, чтобы общее число открытых соединений оставалось равным 32, как указано в директиве `keepalive`.

Этот механизм применим и для проксирования соединений по протоколам, отличным от HTTP. В примере ниже говорится, что NGINX должен поддерживать 64 соединения с двумя экземплярами `memcached`:

```
upstream memcaches {  
    server 10.0.100.10:11211;  
    server 10.0.100.20:11211;  
    keepalive 64;  
}
```

Изменение алгоритма балансировки нагрузки с подразумеваемого по умолчанию циклического (`round-robin`) на `ip_hash` или `least_conn` должно быть задано до директивы `keepalive`:

```
upstream apaches {  
    least_conn;  
    server 10.0.200.10:80;  
    server 10.0.200.20:80;  
    keepalive 32;  
}
```

Алгоритмы балансировки нагрузки

Для выбора проксируемого сервера, которому передается очередной запрос, модуль `upstream` может применять один из трех алгоритмов балансировки нагрузки: циклический, по хэш-коду IP-адреса или с наименьшим количеством соединений. По умолчанию подразумевается **циклический** (`round-robin`) алгоритм, для его активации никакой директивы не нужно. В этом случае выбирается сервер, следующий за тем, который был выбран для обслуживания предыдущего запроса, — с учетом следования серверов в конфигурационном блоке и их весов. Циклический алгоритм пытается обеспечить справедливое распределение трафика, основываясь на понятии очередности.

Алгоритм **хэширования IP-адреса**, активируемый директивой `ip_hash`, основан на предположении, что запросы от клиентов с некоторыми IP-адресами должны попадать одному и тому же проксируемому серверу. В качестве ключа хэширования NGINX берет первые три октета IPv4-адреса или весь IPv6-адрес. Таким образом, множеству близких IP-адресов всегда сопоставляется один и тот же проксируемый сервер. Цель этого механизма — обеспечить не справедливое распределение, а постоянство связи между клиентом и обслуживающим его сервером.

Третий из поддерживаемых модулем `upstream` алгоритмов балансировки нагрузки, **с наименьшим количеством соединений**, выбирается директивой `least_conn`. Он ставит целью равномерное распределе-

ние нагрузки между проксируемыми серверами путем выбора того, у которого количество активных соединений наименьшее. Различия в вычислительной мощности проксируемых серверов можно учесть с помощью параметра `weight` директивы `server`. При выборе сервера с наименьшим количеством соединений алгоритм принимает во внимание вес.

Типы проксируемых серверов

Проксируемым называется сервер, которому NGINX передает запрос на соединение. Он может находиться на другой физической или виртуальной машине, но это необязательно. Проксируемый сервер может быть демоном, прослушивающим сокет в домене UNIX на локальной машине, или одним из многих демонов, прослушивающих порты TCP на другой машине. Это может быть сервер Apache с модулями для обработки запросов различных типов или сервер промежуточного уровня Rack, предоставляющий HTTP-интерфейс к приложениям, написанным на Ruby. В любом случае NGINX можно настроить как прокси-сервер.

Единственный проксируемый сервер

Веб-сервер Apache часто применяется для обслуживания как статических файлов, так и интерпретируемых скриптов разных типов. Наличие в Сети обширной документации и пособий помогает пользователям быстро настроить свою любимую CMS (систему управления содержимым). К сожалению, из-за ограничений на ресурсы Apache в типичной конфигурации не может обслужить много одновременных запросов. NGINX, напротив, спроектирован для обслуживания именно такого трафика и потребляет очень мало ресурсов. Поскольку большинство CMS заранее сконфигурированы для работы с Apache и опираются на использование файла `.htaccess` для дополнительной настройки, то задействовать сильные стороны NGINX проще всего, используя его в качестве прокси-сервера к экземпляру Apache:

```
server {  
    location / {  
        proxy_pass http://localhost:8080;  
    }  
}
```

Это самая простая из всех возможных конфигураций прокси-сервера. NGINX служит конечной точкой для всех клиентских соединений и проксирует запросы на порт 8080 на локальном компьютере. Предполагается, что Apache настроен на прослушивание порта localhost:8080.

Подобная конфигурация, как правило, дополняется директивами, позволяющими NGINX самостоятельно обслуживать запросы на статические файлы, а остальные передавать Apache:

```
server {
    location / {
        try_files $uri @apache;
    }

    location @apache {
        proxy_pass http://127.0.0.1:8080;
    }
}
```

Директива `try_files` (относящая к базовому модулю `http`) по очереди проверяет файлы, пока не найдет совпадение. Так, в показанном выше примере NGINX сам доставит файлы, которые соответствуют URI в запросе клиента и находятся в его корневом каталоге. Если файл не найден, то NGINX передаст запрос Apache для дальнейшей обработки. Мы воспользовались именованным местоположением, чтобы проксировать запрос после неудачной попытки найти файл локально.

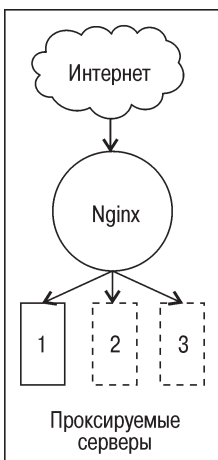
Несколько проксируемых серверов

Есть также возможность настроить NGINX так, чтобы передавать запросы нескольким проксируемым серверам. Для этого следует объявить контекст `upstream`, определить в нем несколько серверов и сослаться на этот контекст в директиве `proxy_pass`:

```
upstream app {
    server 127.0.0.1:9000;
    server 127.0.0.1:9001;
    server 127.0.0.1:9002;
}

server {
    location / {
        proxy_pass http://app;
    }
}
```

При такой конфигурации NGINX будет передавать поступающие запросы циклически трем проксируемым серверам. Это полезно, когда приложение способно в каждый момент времени обрабатывать только один запрос и мы хотим поручить NGINX взаимодействие с клиентами, чтобы ни один из серверов приложений не был перегружен. Подобная конфигурация изображена на следующем рисунке:



В разделе «Алгоритмы балансировки нагрузки» выше в этой главе описаны и другие алгоритмы балансировки нагрузки. Какой из них выбрать, зависит от конкретных обстоятельств.

Если некоторый клиент должен всегда попадать на один и тот же проксируемый сервер, то есть требуется обеспечить некое слабое подобие липких сеансов, то следует воспользоваться директивой `ip_hash`. В случае когда распределение запросов характеризуется широким разбросом времени обработки одного запроса, то лучше выбрать алгоритм `least_conn`. Подразумеваемый по умолчанию циклический алгоритм хорош для общего случая, когда не требуется учитывать особенности клиента или проксируемого сервера.

Проксируемые серверы, работающие по протоколу, отличному от HTTP

До сих пор мы говорили только о проксируемых серверах, работающих по протоколу HTTP. Для них используется директива `proxy_pass`. Но, как отмечалось выше в разделе «Кэширование соеди-

нений», NGINX может проксировать запросы серверам различных типов. Каждому типу соответствует своя директива `*_pass`.

Проксируемые серверы *memcached*

Модуль NGINX `memcached` (по умолчанию включенный) отвечает за взаимодействие с демоном `memcached`. При этом клиент не взаимодействует с `memcached` напрямую, то есть NGINX выступает в роли обратного прокси-сервера. Модуль `memcached` наделяет NGINX способностью общаться по протоколу `memcached`, то есть производить поиск ключа до передачи запроса серверу приложений.

```
upstream memcachedes {
    server 10.0.100.10:11211;
    server 10.0.100.20:11211;
}

server {
    location / {
        set $memcached_key «$uri?$args»;
        memcached_pass memcachedes;
        error_page 404 = @appserver;
    }

    location @appserver {
        proxy_pass http://127.0.0.1:8080;
    }
}
```

В директиве `memcached_pass` переменная `$memcached_key` используется для формирования ключа поиска. Если соответствующего значения не найдено (`error_page 404`), то запрос передается компьютеру `localhost`, на котором предположительно работает сервер, способный обработать этот запрос и вставить пару ключ-значение в кэш, обслуживаемый экземпляром `memcached`.

Проксируемые серверы *FastCGI*

Использование сервера `FastCGI` – широко распространенный способ запуска PHP-приложений, работающих за сервером NGINX. Модуль `fastcgi` компилируется по умолчанию и активируется директивой `fastcgi_pass`. В результате NGINX получает возможность взаимодействовать по протоколу `FastCGI` с одним или несколькими проксируемыми серверами. Набор таких серверов определяется следующим образом:

```
upstream fastcgis {  
    server 10.0.200.10:9000;  
    server 10.0.200.20:9000;  
    server 10.0.200.30:9000;  
}
```

А вот как им передаются запросы к корню:

```
location / {  
    fastcgi_pass fastcgis;  
}
```

Это очень примитивная конфигурация, но принцип использования FastCGI она демонстрирует. Модуль `fastcgi` располагает многочисленными директивами и возможностями настройки, которые мы будем обсуждать в главе 6.

Проксируемые серверы SCGI

NGINX поддерживает также протокол SCGI за счет встроенного модуля `scgi`. Принцип такой же, как для модуля `fastcgi`. NGINX передает запросы проксируемому серверу, указанному в директиве `scgi_pass`.

Проксируемые серверы uWSGI

Протокол uWSGI когда-то был очень популярен в среде разработчиков на языке Python. NGINX поддерживает подключение к проксируемому серверу с приложениями на Python с помощью модуля `uwsgi`. Конфигурируется он так же, как модуль `fastcgi`, только для указания проксируемого сервера служит директива `uwsgi_pass`. Пример приведен в главе 6.

Преобразование конфигурации с «if» в более современную форму

Применение директивы `if` внутри секции `location` считается оправданным только в некоторых ситуациях. Ее можно использовать в сочетании с директивами `return` и `rewrite` с флагом `last` или `break`, но в остальных случаях лучше избегать. Объясняется это возможностью получить неожиданные результаты. Рассмотрим такой пример:

```
location / {
    try_files /img /static @imageserver;
    if ($request_uri ~ «/blog») {
        proxy_pass http://127.0.0.1:9000;
        break;
    }

    if ($request_uri ~ «/tickets») {
        proxy_pass http://tickets.example.com;
        break;
    }
}

location @imageserver {
    proxy_pass http://127.0.0.1:8080;
}
```

Здесь мы пытаемся определить, какому проксируемому серверу передать запрос, основываясь на значении переменной `$request_uri`. На первый взгляд, конфигурация кажется естественной и разумной, потому что в простых тестах работает правильно. Однако изображения не обслуживаются ни из местоположения `/img` в файловой системе, ни из местоположения `/static` там же, ни из именованного местоположения `@imageserver`. Директива `try_files` попросту не работает, когда в том же самом местоположении присутствует директива `if`. Директива `if` создает собственное неявное местоположение со своим обработчиком содержимого, в данном случае — модулем `proxy`. Таким образом, внешний обработчик содержимого, в котором находится директива `try_files`, вообще никогда не вызывается. Чтобы добиться желаемого, эту конфигурацию следует переписать иначе.

Подумаем, как NGINX обрабатывает запрос. Найдя соответствие IP-адресу и порту, NGINX сначала выбирает виртуальный сервер, основываясь на заголовке `Host`. Затем он просматривает все определенные для него местоположения в поисках подходящего URI. Поэтому, чтобы сконфигурировать селектор на основе URI, лучше просто определить несколько местоположений, как показано в примере ниже:

```
location /blog {
    proxy_pass http://127.0.0.1:9000;
}

location /tickets {
    proxy_pass http://tickets.example.com;
```

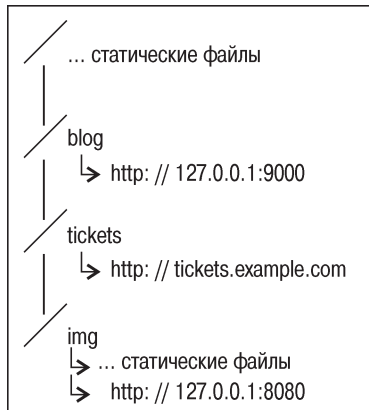
```
}

location /img {
    try_files /static @imageserver;
}

location / {
    root /static;
}

location @imageserver {
    proxy_pass http://127.0.0.1:8080;
}
```

Эта конфигурация иллюстрируется на рисунке ниже.



Вот еще один пример конфигурации с директивами if:

```
server {

    server_name marketing.example.com communication.example.com
    marketing.example.org communication.example.org marketing.example.net
    communication.example.net;

    if ($host ~* (marketing\.example\.com|marketing\.example\.
    org|marketing\.example\.net)) {

        rewrite ^/$ http://www.example.com/marketing/application.do redirect;
    }

    if ($host ~* (communication\.example\.com|communication\.example\.
```

```
org|communication\.example\.net)) {  
  
    rewrite ^/$ http://www.example.com/comms/index.cgi redirect;  
}  
  
if ($host ~* (www\.example\.org|www\.example\.net)) {  
    rewrite ^/(.*)$ http://www.example.com/$1 redirect;  
}  
}
```

Здесь имеется ряд директив `if`, сравнивающих значение заголовка `Host` (или, если его нет, значение `server_name`) с различными регулярными выражениями. После обнаружения соответствия URI переписывается с целью перехода на соответствующий компонент приложения. Мало того что сопоставление каждого URI с несколькими регулярными выражениями крайне неэффективно, так еще и нарушается правило «не должно быть `if` внутри `location`».

Такую конфигурацию лучше переписать в виде нескольких последовательных контекстов `server`, в каждом из которых URL подменяется адресом соответствующего компонента:

```
server {  
    server_name marketing.example.com marketing.example.org  
    marketing.example.net;  
  
    rewrite ^ http://www.example.com/marketing/application.do permanent;  
}  
  
server {  
    server_name communication.example.com communication.example.org  
    communication.example.net;  
  
    rewrite ^ http://www.example.com/comms/index.cgi permanent;  
}  
  
server {  
    server_name www.example.org www.example.net;  
  
    rewrite ^ http://www.example.com$request_uri permanent;  
}
```

В каждом блоке мы оставили только те имена серверов, которые относятся к соответствующей директиве `rewrite`, поэтому директивы `if` не понадобились. Во всех правилах `rewrite` мы заменили флаг `redirect` флагом `permanent`, чтобы сообщить браузеру, что это полный URL-адрес, который нужно запомнить и автоматически использо-

вать при следующем обращении к этому домену. В последнем правиле `rewrite` мы также заменили сопоставление с регулярным выражением `(~/(.*)$)` сравнением с готовой переменной `$request_uri`, которая содержит ту же информацию, но избавляет от необходимости сопоставлять строку с регулярным выражением и запоминать подвыражение в переменной.

Использование документов с описанием ошибок для обработки ошибок проксирования

Бывает, что проксируемый сервер не может обработать запрос. В таких случаях NGINX можно настроить так, чтобы возвращался документ, хранящийся на локальном диске.

```
server {  
    error_page 500 502 503 504 /50x.html;  
  
    location = /50x.html {  
        root share/examples/nginx/html;  
    }  
}
```

Или на внешнем сайте:

```
server {  
    error_page 500 http://www.example.com/maintenance.html;  
}
```

При проксировании на несколько серверов можно определить дополнительный сервер «последней надежды» (fallback), который обрабатывает запросы, которые оказались не по зубам другим серверам. Это полезно, когда сервер последней надежды может вернуть ответ, зависящий от запрошенного URI:

```
upstream app {  
    server 127.0.0.1:9000;  
    server 127.0.0.1:9001;  
    server 127.0.0.1:9002;  
}  
  
server {  
    location / {
```

```
error_page 500 502 503 504 = @fallback;  
proxy_pass http://app;  
}  
  
location @fallback {  
    proxy_pass http://127.0.0.1:8080;  
}  
}
```



Знак “=” в строке `error_page` означает, что мы хотим вернуть код состояния, полученный от последнего параметра, в данном случае от местоположения `@fallback`.

В этих примерах рассматриваются случаи, когда код ошибки равен 500 или больше. NGINX может также возвращать `error_page` для кодов ошибки 400 или больше, если значение директивы `proxy_intercept_errors` равно `on`, как в следующем примере:

```
server {  
    proxy_intercept_errors on;  
    error_page 400 403 404 /40x.html;  
  
    location = /40x.html {  
        root share/examples/nginx/html;  
    }  
}
```



Если в директиве `error_page` присутствует код ошибки 401, то аутентификация окажется незавершенной. Это можно делать, например, в случае, когда сервер аутентификации выведен из эксплуатации, но в общем случае не рекомендуется.

Определение истинного IP-адреса клиента

При использовании прокси-сервера у клиента нет прямого соединения с проксируемым сервером. Поэтому проксируемый сервер не может получать информацию непосредственно от клиента. Любую информацию, в том числе истинный IP-адрес клиента, следует передавать в заголовках. Для этой цели NGINX предоставляет директиву `proxy_set_header`:

```
proxy_set_header X-Real-IP $remote_addr;  
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
```

В результате IP-адрес клиента передается в каждом из заголовков X-Real-IP и X-Forwarded-For. Во втором случае принимается во внимание заголовок из запроса клиента. Если в запросе присутствует заголовок X-Forwarded-For, то IP-адрес клиента будет добавлен в конец этого заголовка, через запятую. В зависимости от конфигурации проксируемого сервера необходима одна из этих двух директив. Например, чтобы настроить Apache, так чтобы в его журналах в качестве IP-адреса клиента использовался заголовок X-Forwarded-For, нужно воспользоваться спецификатором форматирования %{<имя-заголовка>}i.

Ниже показано, как изменить подразумеваемый по умолчанию формат «объединенного» (combined) журнала Apache:

```
LogFormat "%{X-Forwarded-For}i %l %u %t \"%r\" \"%s %b \"%{Referer}i\"  
\"%{User-Agent}i\" combined
```

С другой стороны, если проксируемый сервер требует нестандартного заголовка, например Client-IP, то и эту задачу легко решить следующим образом:

```
proxy_set_header Client-IP $remote_addr;
```

Точно так же проксируемым серверам можно передавать и другую информацию, например заголовок Host:

```
proxy_set_header Host $host;
```

Резюме

Мы рассмотрели вопрос об использовании NGINX в качестве обратного прокси-сервера. Его эффективная модель обработки соединений идеальна для прямого взаимодействия с клиентами. Получив запрос, NGINX может затем открыть новое соединение с одним из нескольких проксируемых серверов, приняв во внимание сильные и слабые стороны каждого. Использование директивы if внутри секции location считается допустимым только в определенных ситуациях. Поняв, как NGINX обрабатывает запрос, мы можем подготовить конфигурацию, в большей степени отвечающую решаемой задаче. Если у NGINX по какой-то причине не получается установить соединение с проксируемым сервером, то он может вернуть альтернативную страницу. Поскольку NGINX служит конечной точкой для



клиентских запросов, проксируемые серверы могут получать информацию о клиенте только с помощью заголовков запроса, проксированного NGINX. Описанные соображения помогут при написании конфигурационного файла NGINX, идеально отвечающего вашим потребностям.

В следующей главе мы рассмотрим более сложные приемы обратного проксирования.



Глава 5. Обратное проксирование, дополнительные вопросы

В предыдущей главе мы видели, что обратный прокси-сервер устанавливает соединения с проксируемыми серверами от имени клиентов. Сами проксируемые серверы не имеют прямой связи с клиентами. Так сделано по разным причинам, в том числе ради безопасности, масштабируемости и повышения производительности.

Обратный прокси-сервер обеспечивает безопасность, потому что если бы злоумышленник захотел попасть напрямую на проксируемый сервер, ему нужно было бы сначала взломать обратный прокси-сервер. Соединения с клиентом можно шифровать по протоколу HTTPS. Оконечной точкой таких SSL-соединений может быть обратный прокси-сервер, если проксируемый сервер не может или не должен предоставлять средства шифрования самостоятельно. NGINX может выступать в роли терминатора SSL-соединений, а также реализовывать дополнительные списки управления доступом и ограничения, зависящие от различных свойств клиента.

Масштабируемость достигается за счет того, что обратный прокси-сервер устанавливает параллельные соединения с несколькими проксируемыми серверами, которые клиентам представляются одним сервером. Если приложению нужна дополнительная вычислительная мощность, то пул серверов, расположенных за единственным обратным прокси-сервером, можно расширить.

Обратный прокси-сервер может повысить производительность приложения несколькими способами. Во-первых, он может кэшировать и сжимать содержимое перед отправкой его клиенту. В роли обратного прокси-сервера NGINX способен обрабатывать больше одновременных клиентских соединений, чем типичный сервер приложений. Иногда NGINX конфигурируют так, что он обслуживает

запросы на статическое содержимое из локального дискового кэша, а проксируемому серверу передает только динамические запросы. Клиенты могут оставлять соединения с NGINX открытыми, тогда как сам NGINX закрывает соединения с проксируемыми серверами немедленно, освобождая тем самым их ресурсы.

Мы обсудим все эти вопросы, а также оставшиеся директивы модуля `proxy` в следующих разделах.

- ❑ Безопасность за счет разделения.
- ❑ Обеспечение масштабируемости за счет изоляции компонентов приложения.
- ❑ Оптимизация производительности обратного прокси-сервера.

Безопасность за счет разделения

Мы можем повысить безопасность, отделив точку, в которой клиенты подключаются к приложению. Это одна из основных причин использования обратного прокси-сервера в архитектуре системы. Клиент напрямую подключается только к машине, на которой работает прокси-сервер. Эту машину необходимо как следует защитить, чтобы противник не мог найти точку проникновения в систему.

Безопасность – столь обширная тема, что мы лишь вкратце отметим, на что обратить внимание.

- ❑ Устанавливайте брандмауэр перед обратным прокси-сервером, открывая доступ только к порту 80 (и 443, если предполагаются также HTTPS-соединения).
- ❑ Запускайте NGINX от имени непривилегированного пользователя (в разных операционных системах он называется `www`, `webservd` или `www-data`).
- ❑ Шифруйте трафик для предотвращения перехвата.

О последней рекомендации мы подробнее поговорим в следующем разделе.

Шифрование трафика по протоколу SSL

NGINX часто используют как терминатор SSL-соединений – потому что проксируемый сервер не поддерживает SSL или с целью освободить его от накладных расходов, связанных с обработкой SSL-соединений. Для этого необходимо откомпилировать двоичный файл `nginx` с поддержкой SSL (параметр `--with_http_ssl_module`) и установить сертификат и ключ SSL.



Подробнее о том, как самостоятельно сгенерировать сертификат SSL, см. врезку «Генерация SSL-сертификата с помощью OpenSSL» в главе 3 «Почтовый модуль».

Ниже показан пример конфигурации для разрешения HTTPS-соединений с сервером `www.example.com`:

```
server {
    listen 443 default ssl;

    server_name www.example.com;

    ssl_prefer_server_ciphers on;
    ssl_protocols TLSv1 SSLv3;
    ssl_ciphers RC4:HIGH:!aNULL:!MD5:@STRENGTH;
    ssl_session_cache shared:WEB:10m;
    ssl_certificate /usr/local/etc/nginx/www.example.com.crt;
    ssl_certificate_key /usr/local/etc/nginx/www.example.com.key;

    location / {
        proxy_set_header X-FORWARDED-PROTO https;
        proxy_pass http://upstream;
    }
}
```

Здесь мы сначала активируем модуль `ssl`, указав параметр `ssl` в директиве `listen`. Затем мы говорим, что хотим отдать предпочтение серверным, а не клиентским шифрам, чтобы иметь возможность использовать только шифры, признанные наиболее безопасными. Это не позволит клиенту предложить нерекомендуемый шифр. В директиве `ssl_session_cache` мы задаем режим `shared`, чтобы все рабочие процессы могли воспользоваться плодами дорогостоящего начального согласования параметров SSL, уже один раз выполненного для некоторого клиента. Несколько виртуальных серверов могут пользоваться одной и той же директивой `ssl_session_cache`, если все они сконфигурированы с одним и тем же именем или эта директива находится в контексте `http`. Вторая и третья части значения – имя кэша и его размер соответственно. Далее заданы сертификат и ключ для данного хоста. Отметим, что для файла ключа следует установить права доступа так, чтобы читать его мог только главный процесс. Для заголовка `X-FORWARDED-PROTO` мы задали значение `https`, чтобы приложение, работающее на проксируемом сервере, знало о том, что исходный запрос был отправлен по протоколу HTTPS.



Шифры SSL

Шифры в показанной выше конфигурации выбираются из набора, подразумеваемого в NGINX по умолчанию. В него не входят шифры вообще без аутентификации (aNULL) и шифры, в которых используется алгоритм хэширования MD5. Шифр RC4 помещен в начало набора, поэтому предпочтение отдается шифрам, не подверженным атаке BEAST, которая описана в документе CVE-2011-3389. Строка @STRENGTH в конце значения говорит о том, что список шифров следует упорядочить по длине ключа алгоритма шифрования.

Выше мы описали шифрование трафика между клиентом и обратным прокси-сервером. Но можно также шифровать трафик между прокси-сервером и проксируемым сервером:

```
server {  
    ...  
  
    proxy_pass https://upstream;  
}
```

Обычно это применяется только в системах, где даже внутренняя сеть считается небезопасной.

Аутентификация клиентов по протоколу SSL

В некоторых приложениях используется информация, полученная из предъявленного клиентом сертификата SSL, но в системе с обратным прокси-сервером она непосредственно недоступна. Чтобы передать ее приложению, необходимо, чтобы NGINX добавил заголовок:

```
location /ssl {  
    proxy_set_header ssl_client_cert $ssl_client_cert;  
    proxy_pass http://upstream;  
}
```

В переменной `$ssl_client_cert` хранится сертификат клиента в формате PEM. Мы передаем его проксируемому серверу в одноименном заголовке. Далее приложение может использовать эту информацию, как ему угодно.

Вместо того чтобы передавать проксируемому серверу весь клиентский сертификат, NGINX может самостоятельно проделать часть

работы и убедиться, что сертификат хотя бы корректен, то есть подписан известным удостоверяющим центром (УЦ), не отозван и дата окончания срока действия еще не наступила.

```
server {
    ...

    ssl_client_certificate /usr/local/etc/nginx/ClientCertCAs.pem;
    ssl_crl /usr/local/etc/nginx/ClientCertCRLs.crl;
    ssl_verify_client on;
    ssl_verify_depth 3;

    error_page 495 = @noverify;
    error_page 496 = @nocert;

    location @noverify {
        proxy_pass http://insecure?status=notverified;
    }

    location @nocert {
        proxy_pass http://insecure?status=nocert;
    }

    location / {
        if ($ssl_client_verify = FAILED) {
            return 495;
        }
        proxy_pass http://secured;
    }
}
```

Показанный выше фрагмент, в котором предварительно проверяется корректность клиентского сертификата SSL, состоит из следующих частей.

- ❑ В аргументе директивы `ssl_client_certificate` указывается путь к представленному в формате PEM списку сертификатов корневых УЦ, которые могут подписывать клиентские сертификаты.
- ❑ Аргумент директивы `ssl_crl` определяет путь к списку отозванных сертификатов, который публикуется удостоверяющим центром, подписывающим клиентские сертификаты. Этот список необходимо периодически загружать с помощью отдельной процедуры.
- ❑ Директива `ssl_verify_client` говорит, что NGINX должна проверять корректность SSL-сертификатов, предъявленных клиентами.

- ❑ Директива `ssl_verify_depth` определяет максимальное количество сторон, подписавших сертификат; если их больше, сертификат признается недействительным. SSL-сертификат может быть подписан одним или несколькими промежуточными УЦ. Сертификат промежуточного УЦ или сертификат подписавшего его корневого УЦ должен присутствовать в пути, указанном в директиве `ssl_client_certificate`, иначе NGINX сочтет его недействительным.
- ❑ В случае ошибки при проверке клиентского сертификата NGINX возвращает нестандартный код ошибки 495. Мы определили для этого кода страницу ошибки `error_page`, которая переадресует запрос в именованное местоположение, где он будет обработан отдельным проксируемым сервером. Мы также включили проверку переменной `$ssl_client_verify` внутри корневого местоположения, чтобы для некорректного сертификата возвращался такой же код.
- ❑ Если сертификат некорректен, то NGINX возвращает нестандартный код ошибки 496, который мы также перехватываем с помощью директивы `error_page`, указывающей на именованное местоположение, где запрос передается отдельному обработчику ошибок.

Только в том случае, когда клиент предъявил корректный SSL-сертификат, NGINX передаст запрос проксируемому серверу. При таком подходе мы можем быть уверены, что проксируемый сервер получает запросы только от аутентифицированных пользователей. Это важная часть механизма защиты, реализуемого обратным прокси-сервером.



Начиная с версии 1.3.7, NGINX поддерживает протокол OCSP для проверки сертификатов клиентов. О том, как активировать эту функцию, см. описание директив `ssl_stapling*` и `ssl_trusted_certificate` в приложении А.

Если приложению все-таки нужна какая-то информация из клиентского сертификата, например чтобы авторизовать клиента, то NGINX может передать ее в заголовке:

```
location / {  
    proxy_set_header X-HTTP-AUTH $ssl_client_s_dn;  
    proxy_pass http://secured;  
}
```

Теперь приложение, работающее на защищенном проксируемом сервере, может проверить значение в заголовке X-HTTP-AUTH и предоставить клиенту доступ к тем или иным возможностям. Переменная `$ssl_client_s_dn` содержит отличительное имя (DN) субъекта, хранящееся в клиентском сертификате. Зная его, приложение может справиться с базой данных или поискать пользователя в LDAP-каталоге.

Блокирование трафика на основе IP-адреса отправителя

Поскольку обратный прокси-сервер является окончательной точкой для клиентских соединений, то можно ограничить количество клиентов, исходя из их IP-адресов. Это полезно в случае обнаружения попытки недобросовестного использования, когда очень много недопустимых обращений исходят от клиентов с IP-адресами из некоторого ограниченного множества. Как и в языке Perl, решить задачу можно несколькими способами. Ниже мы обсудим применение для этой цели модуля GeoIP.

Двоичный файл `nginx` должен быть откомпилирован с поддержкой модуля GeoIP (параметр `--with-http_geoip_module`), а в системе должна быть установлена библиотека MaxMind GeoIP. Путь к откомпилированной базе данных указывается в директиве `geoip_country` в контексте `http`. Это самый эффективный способ запретить или разрешить IP-адреса по коду страны:

```
geoip_country /usr/local/etc/geo/GeoIP.dat;
```

Если IP-адрес клиента присутствует в базе данных, то в переменной `$geoip_country_code` будет находиться двухбуквенный код соответствующей страны по классификатору ISO.

Мы воспользуемся данными, полученными от модуля GeoIP, применив также модуль с похожим названием `geo`. Модуль `geo` предоставляет очень простой интерфейс для установки переменных на основе IP-адреса клиента. Он создает именованный контекст, в котором первый параметр – проверяемый IP-адрес, а второй – значение, присваиваемое переменной при обнаружении соответствия. Сочетание обоих модулей позволит нам блокировать IP-адреса из некоторых стран, разрешив в то же время доступ с некоторых IP-адресов.

Допустим, мы представляем услугу швейцарским банкам. Мы хотим, чтобы Google мог индексировать открытые разделы сайта, но

при этом разрешить доступ к ним только с IP-адресов в Швейцарии. Кроме того, мы хотим, чтобы локальная сторожевая служба могла получить доступ к сайту и удостовериться в его работоспособности. Мы определяем переменную `$exclusions`, по умолчанию равную 0. Если какое-нибудь из сформулированных выше условий выполняется, то переменная получит значение 1, означающее, что доступ к сайту разрешен:

```
http {
    # путь к базе данных GeoIP
    geoip_country /usr/local/etc/geo/GeoIP.dat;

    # определяем переменную $exclusions и перечисляем все IP-адреса,
    # которым разрешен доступ, задавая для них значение 1
    geo $exclusions {
        default 0;
        127.0.0.1 1;
        216.239.32.0/19 1;
        64.233.160.0/19 1;
        66.249.80.0/20 1;
        72.14.192.0/18 1;
        209.85.128.0/17 1;
        66.102.0.0/20 1;
        74.125.0.0/16 1;
        64.18.0.0/20 1;
        207.126.144.0/20 1;
        173.194.0.0/16 1;
    }

    server {
        # страна, которой доступ разрешен, - Швейцария, с кодом «CH»
        if ($geoip_country_code = «CH») {
            set $exclusions 1;
        }

        location / {
            # если IP-адрес не принадлежит Швейцарии и не находится в нашем
            # списке, то переменная $exclusions равна "0", и мы возвращаем
            # HTTP-код "Forbidden"
            if ($exclusions = "0") {
                return 403;
            }

            # всем остальным клиентам разрешен доступ к проксируемому серверу
            proxy_pass http://upstream;
        }
    }
}
```

Это лишь один из способов решить задачу блокирования доступа к сайту на основе IP-адреса клиента. Можно также сохранять IP-адрес в каком-нибудь хранилище ключей и значений, увеличивать счетчик при каждом запросе и блокировать доступ, если количество запросов в течение определенного периода превысило пороговое значение.

Обеспечение масштабируемости за счет изоляции компонентов приложения

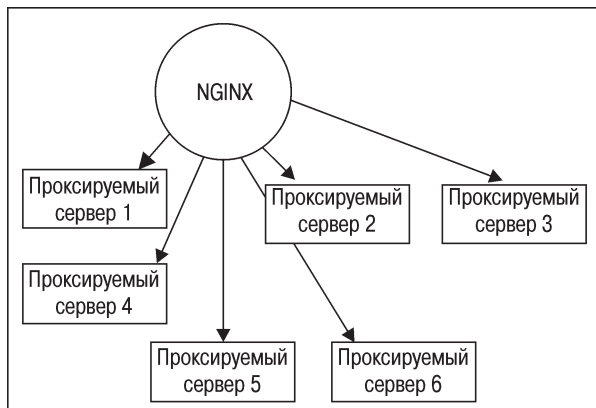
Масштабирование приложение бывает двух видов: горизонтальное и вертикальное. Под вертикальным масштабированием понимается наращивание ресурсов компьютера с целью обслужить больше клиентских запросов. Горизонтальное масштабирование – это расширение пула доступных машин, обслуживающих клиентов, чтобы ни одна из них не была перегружена запросами. Этот подход зачастую является более рентабельным вне зависимости от того, что представляют собой эти машины – виртуализированные экземпляры, работающие в облаке, или физические компьютеры, размещенные в центре обработки данных. Именно в такой инфраструктуре NGINX находит свое место в качестве обратного прокси-сервера.

Благодаря чрезвычайно низкому ресурсопотреблению NGINX идеален в роли брокера клиентских приложений. NGINX обрабатывает соединения с клиентами и при этом способен одновременно обслуживать несколько запросов. В зависимости от конфигурации NGINX либо возвращает файл из своего локального кэша, либо передает запрос проксируемому серверу для дальнейшей обработки. В роли проксируемого может выступать любой сервер, поддерживающий протокол HTTP. При таком подходе удастся обслужить больше запросов, чем если бы проксируемый сервер отвечал напрямую:

```
upstream app {  
    server 10.0.40.10;  
    server 10.0.40.20;  
    server 10.0.40.30;  
}
```

Со временем начальный набор проксируемых серверов можно расширять. Допустим, что количество обращений к сайту настоль-

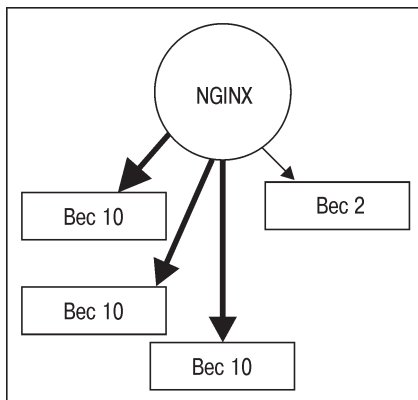
ко увеличилось, что текущий набор перестал с ними справляться. Если использовать NGINX в качестве обратного прокси-сервера, то с этой бедой легко справиться, добавив дополнительные проксируемые серверы.



Добавление проксируемых серверов производится следующим образом:

```
upstream app {  
    server 10.0.40.10;  
    server 10.0.40.20;  
    server 10.0.40.30;  
    server 10.0.40.40;  
    server 10.0.40.50;  
    server 10.0.40.60;  
}
```

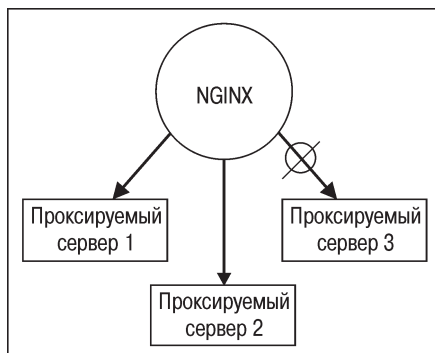
Быть может, настало время переписать приложение или перенести его на сервер с другим прикладным стеком. Прежде чем перенести все приложение, мы можем поместить один сервер в активный пул для тестирования в условиях реальной нагрузки с реальными клиентами. Этому серверу можно передавать меньше запросов, чтобы смягчить негативную реакцию пользователей в случае возникновения ошибок.



Это делается с помощью такой конфигурации:

```
upstream app {  
    server 10.0.40.10 weight 10;  
    server 10.0.40.20 weight 10;  
    server 10.0.40.30 weight 10;  
    server 10.0.40.100 weight 2;  
}
```

С другой стороны, пусть требуется вывести проксируемый сервер из эксплуатации для проведения планового обслуживания. Если пометить его признаком `down` в конфигурационном файле, то NGINX перестанет передавать ему запросы, и мы сможем заняться обслуживанием:



Ниже показано, как это делается:

```
upstream app {  
    server 10.0.40.10;  
    server 10.0.40.20;  
    server 10.0.40.30 down;  
}
```

Система должна быстро реагировать в случае, когда сервер перестает отвечать на запросы. Для некоторых приложений в директивах тайм-аута можно задать очень малое время:

```
location / {  
    proxy_connect_timeout 5;  
    proxy_read_timeout 10;  
    proxy_send_timeout 10;  
}
```

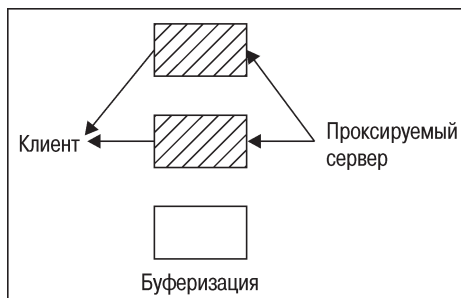
Но будьте осторожны: если ни один проксируемый сервер не сможет ответить в отведенное время, то NGINX вернет ошибку **504 Gateway Timeout Error**.

Оптимизация производительности обратного прокси-сервера

NGINX можно по-разному настраивать, чтобы выжать максимум возможного из приложения, для которого оно служит обратным прокси-сервером. За счет буферизации, кэширования и сжатия NGINX может существенно ускорить время реакции приложения на действия клиента.

Буферизация

Смысл буферизации изображен на следующем рисунке:



С точки зрения производительности проксирования, самым важным фактором является буферизация. По умолчанию NGINX пытается прочитать от проксируемого сервера столько данных, сколько возможно, и так быстро, как возможно, прежде чем вернуть ответ клиенту. Он локально буферизует ответ, чтобы можно было отправить его клиенту одним разом. Если часть запроса от клиента или ответа от проксируемого сервера записывается на диск, то производительность может снизиться. Это компромисс между оперативной и дисковой памятью. Поэтому, настраивая NGINX для работы в качестве обратного прокси-сервера, так важно уделять внимание описанным ниже директивам.

Директивы модуля проху для управления буферизацией

Директива	Описание
proxy_buffer_size	Размер буфера, используемого для первой части ответа от проксируемого сервера, в которой находятся заголовки
proxy_buffering	Включает или выключает буферизацию проксированного содержимого. Если буферизация выключена, то ответы отправляются клиенту синхронно сразу после получения при условии, что параметр <code>proxy_max_temp_file_size</code> равен 0. Если этот параметр равен 0 и <code>proxy_buffering</code> равно on, то буферизация включена, но диск при этом не используется
proxy_buffers	Количество и размер буферов для хранения ответов от проксируемых серверов
proxy_busy_buffers_size	Суммарный размер буферов, которые могут быть заняты для отправки ответа клиенту, пока ответ ещё не прочитан целиком. Обычно устанавливается в два раза больше, чем размер, указанный в директиве <code>proxy_buffers</code>

Помимо рассмотренных выше директив, на буферизацию может оказать влияние проксируемый сервер – путем установки заголовка `X-Accel-Buffering`. По умолчанию значение в этом заголовке равно `yes`, то есть ответы буферизуются. Задание значения `no` полезно для приложений, основанных на модели `Comet`, и в случае потоковой передачи данных по протоколу HTTP, когда буферизовать ответ не нужно.

Измеряя средний размер запросов и ответов, проходящих через обратный прокси-сервер, можно оптимально настроить размеры буферов. Каждая из директив буферизации относится к одному соединению. Принимая во внимание также накладные расходы ОС

на установление соединения, мы можем вычислить, сколько одновременных соединений с клиентами можно поддерживать при имеющемся объеме оперативной памяти.

Подразумеваемое по умолчанию значение директивы `proxy_buffers` (8 4k или 8 8k в зависимости от операционной системы) рассчитано на большое число одновременных соединений. Посмотрим, на сколько именно. На типичной машине, где запущен NGINX, объем памяти составляет 1 ГБ, и большая ее часть может быть отдана NGINX. Сколько-то памяти необходимо отвести под кэш файловой системы и другие нужды, поэтому примем осторожную оценку: 768 МБ.

Восемь буферов по 4 КБ займут 32 768 байтов ($8 \times 4 \times 1024$) на каждое активное соединение.

Мы отвели под NGINX 768 МБ, то есть 805 306 368 байтов ($768 \times 1024 \times 1024$).

Поделив одно на другое, получаем $805\,306\,368 / 32\,768 = 24\,576$ активных соединений.

Таким образом, в конфигурации по умолчанию NGINX сможет одновременно обслужить чуть меньше 25 000 соединений в предположении, что буферы постоянно заполнены. Нужно учитывать и другие факторы, например кэширование содержимого и простаивание соединений, но грубую оценку мы получили.

Если теперь принять показанные ниже средние размеры запроса и ответа, то окажется, что восьми буферов по 4 КБ недостаточно для обработки типичного запроса. Мы хотим, чтобы NGINX буферизовал столько полученных от проксируемого сервера данных, сколько возможно; конечная цель – передать пользователю сразу весь ответ.

- ❑ Средний размер запроса: 800 байтов.
- ❑ Средний размер ответа: 900 КБ.



В примерах настройки, приведенных ниже, используется больше памяти ценой уменьшения количества одновременных активных соединений. Это оптимизации, которые не следует рассматривать как рекомендации в общем случае. NGINX уже оптимизирован для обслуживания большого числа медленных клиентов и нескольких быстрых проксируемых серверов. В настоящее время наблюдается смещение в сторону мобильных пользователей, так что клиентское соединение оказывается существенно медленнее, чем при широкополосном доступе. Поэтому, прежде чем приступать к оптимизации, нужно хорошо изучить, как именно пользователи подключаются.

Для этого нужно увеличить размеры буферов, чтобы ответ уместился в них целиком:

```
http {  
    proxy_buffers 30 32k;  
}
```

Это, конечно, означает, что количество одновременно обслуживаемых пользователей резко уменьшится.

Тридцать буферов по 32 КБ займут 983 040 байтов ($30 \times 32 \times 1024$) на каждое активное соединение.

Мы отвели под NGINX 768 МБ, то есть 805 306 368 байтов ($768 \times 1024 \times 1024$).

Поделив одно на другое, получаем $805\,306\,368 / 983\,040 = 819,2$ активного соединения.

Совсем не много. Уменьшим количество буферов, так чтобы NGINX начал передавать данные клиенту, продолжая читать ответ в оставшиеся буферы.

```
http {  
    proxy_buffers 4 32k;  
    proxy_busy_buffers_size 64k;  
}
```

Четыре буфера по 32 КБ займут 131 072 байта ($4 \times 32 \times 1024$) на каждое активное соединение.

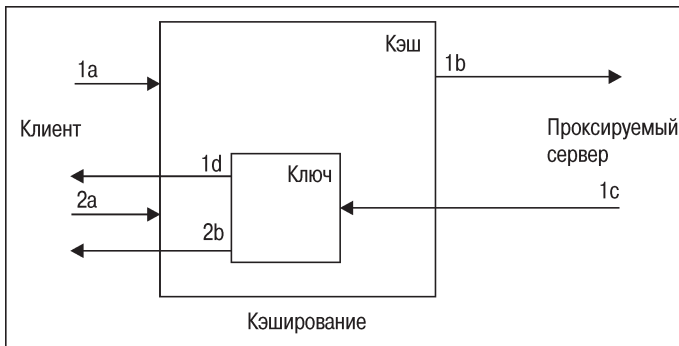
Мы отвели под NGINX 768 МБ, то есть 805 306 368 байтов ($768 \times 1024 \times 1024$).

Поделив одно на другое, получаем $805\,306\,368 / 131\,072 = 6144$ активных соединения.

Что касается самой машины, где работает обратный прокси-сервер, то мы можем масштабировать ее либо вертикально, добавив память (при объеме ОЗУ 6 ГБ мы получим приблизительно 37 000 соединений), либо горизонтально, поставив за балансировщиком нагрузки столько дополнительных машин с ОЗУ объемом 1 ГБ, сколько необходимо для обслуживания ожидаемого количества одновременных пользователей.

Кэширование

Идея кэширования представлена на рисунке ниже.



NGINX может кэшировать ответы от проксируемого сервера, так что при повторном поступлении того же запроса не нужно будет снова обращаться к серверу. На рисунке показана следующая последовательность операций.

- ❑ **1a:** Клиент отправляет запрос.
- ❑ **1b:** Соответствующий запросу ключ не найден в кэше, поэтому NGINX обращается к проксируемому серверу.
- ❑ **1c:** Проксируемый сервер отвечает, и NGINX помещает ответ в кэш.
- ❑ **1d:** Ответ отправляется клиенту.
- ❑ **2a:** Другой клиент отправляет запрос с таким же ключом кэширования.
- ❑ **2b:** NGINX обслуживает запрос непосредственно из кэша, не обращая за ответом к проксируемому серверу.

Директивы модуля проху для управления кэшированием

Директива	Описание
proxy_cache	Определяет размер зоны разделяемой памяти, отведенной под кэш
proxy_cache_bypass	Одна или несколько строковых переменных. Если хотя бы одна из них непуста и не содержит нуль, то ответ будет запрошен у проксируемого сервера, а не взят из кэша
proxy_cache_key	Строка, которая используется как ключ для поиска значения в кэше. Можно использовать переменные, но следует внимательно следить за тем, чтобы не кэшировалось несколько копий одного и того же содержимого

Директива	Описание
proxy_cache_lock	Если эта директива принимает значение on, то предотвращается отправка нескольких запросов проксируемому серверу (или серверам) в случае отсутствия в кэше. NGINX дожидается результата первого запроса, поместит его в кэш и только потом приступит к обслуживанию остальных. Эта блокировка действует в пределах одного рабочего процесса
proxy_cache_lock_timeout	Сколько времени запрос может ждать появления записи в кэше или освобождения блокировки proxy_cache_lock
proxy_cache_min_uses	Сколько запросов с данным ключом должно поступить, прежде чем ответ будет помещен в кэш
proxy_cache_path	Каталог, в котором хранятся кэшированные ответы, и зона разделяемой памяти (keys_zone=name:size) для хранения активных ключей и метаданных ответа. Необязательные параметры: - levels: список разделенных двоеточиями длин имен подкаталогов на каждом уровне (1 или 2); допускается не более трех уровней вложенности; - inactive: максимальное время нахождения неактивного запроса в кэше, по прошествии которого он вытесняется; - max_size: максимальный размер кэша; по его достижении процесс-диспетчер удаляет элементы, к которым дольше всего не было обращений; - loader_files: максимальное количество кэшированных файлов, метаданные которых загружаются в кэш на одной итерации процесса-загрузчика; - loader_sleep: число миллисекунд между итерациями процесса-загрузчика кэша; - loader_threshold: максимальное время, отведенное на одну итерацию процесса-загрузчика кэша
proxy_cache_use_stale	В каких случаях допустимо использовать устаревшие кэшированные данные, если при доступе к проксируемому серверу произошла ошибка. Параметр updating разрешает использовать кэшированный ответ, если как раз в данный момент загружаются более свежие данные
proxy_cache_valid	Сколько времени считать действительным кэшированный ответ с кодом 200, 301 или 302. Если перед параметром time указан необязательный код ответа, то заданное время относится только к ответу с таким кодом. Специальный параметр any означает, что в течение заданного времени следует кэшировать ответ с любым кодом

Показанная ниже конфигурация предназначена для кэширования всех ответов в течение шести часов при общем размере кэша 1 ГБ. Актуальные элементы, то есть такие, к которым были обращения на протяжении шестичасового периода тайм-аута, остаются действительными сутки. По истечении этого времени у проксируемого сервера будет запрошен новый ответ. Если проксируемый сервер не отвечает из-за ошибки, произошел тайм-аут, получен недопустимый заголовок или кэшированный элемент обновляется, то разрешается использовать устаревший ответ из кэша. Размер области разделяемой памяти **CACHE** задан равным 10 МБ, ссылка на него производится из местоположения, для которого поддерживается кэширование.

```
http {
    # размещается в той же файловой системе, что и proxy_cache_path
    proxy_temp_path /var/spool/nginx;

    # из соображений безопасности этим каталогом владеет тот же
    # пользователь, который определен в директиве user (от имени
    # которого запущены рабочие процессы)
    proxy_cache_path /var/spool/nginx keys_zone=CACHE:10m levels=1:2
        inactive=6h max_size=1g;

    server {
        location / {
            # используем include, чтобы загрузить файл с общими параметрами
            include proxy.conf;

            # ссылаемся на определенную выше область разделяемой памяти
            proxy_cache CACHE;
            proxy_cache_valid any 1d;
            proxy_cache_use_stale error timeout invalid_header updating
                http_500 http_502 http_503 http_504;
            proxy_pass http://upstream;
        }
    }
}
```

При такой конфигурации NGINX создаст ряд подкаталогов в каталоге `/var/spool/nginx`, которые распределяют кэшированные данные сначала по последнему символу MD5-свертки URI-адреса, а затем по следующим двум, считая от конца. Например, ответ на запрос к URI `“/ this-is-a-typical-url”` будет храниться в файле:

```
/var/spool/nginx/3/f1/614c16873c96c9db2090134be91cbf13
```

Помимо директивы `proxy_cache_valid`, кэшированием ответов в NGINX управляют еще некоторые заголовки. Значение заголовка имеет больший приоритет, чем директива.

- ❑ Проксируемый сервер может установить заголовок `X-Accel-Expires`, управляющий поведением кэширования:
 - целочисленное значение в нем указывает, сколько секунд хранить ответ в кэше;
 - если значение равно 0, то кэширование этого ответа запрещено;
 - значение, начинающееся символом @, интерпретируется как время в секундах с начала «эпохи». Ответ хранится до наступления этого момента времени.
- ❑ У заголовков `Expires` и `Cache-Control` одинаковый приоритет.
- ❑ Если значение в заголовке `Expires` определяет момент в будущем, то ответ кэшируется до наступления этого момента.
- ❑ Заголовок `Cache-Control` может принимать следующие значения:
 - `no-cache`
 - `no-store`
 - `private`
 - `max-age`
- ❑ Ответ кэшируется, только если задано значение `max-age`, которое должно быть положительным числом, то есть `max-age=x`, где $x > 0$.
- ❑ Если присутствует заголовок `Set-Cookie`, то ответ не кэшируется. Это правило можно отменить, воспользовавшись директивой `proxy_ignore_headers`:
`proxy_ignore_headers Set-Cookie;`
- ❑ Но при этом не забудьте сделать значение куков частью ключа `proxy_cache_key`:
`proxy_cache_key «$host$request_uri $cookie_user»;`

Однако, поступая так, следует позаботиться о том, чтобы для одного и того же URI-адреса не кэшировалось несколько ответов. Это может случиться, если для открытого содержимого случайно установлен заголовок `Set-Cookie`, который становится частью ключа доступа к данным. Один из способов обеспечить эффективное использование кэша – назначить открытому содержимому отдельное местоположение. Например, изображения можно хранить в месте положении `/img`, для которого определен другой ключ `proxy_cache_key`:

```
server {
    proxy_ignore_headers Set-Cookie;

    location /img {
        proxy_cache_key «$host$request_uri»;
        proxy_pass http://upstream;
    }

    location / {
        proxy_cache_key «$host$request_uri $cookie_user»;
        proxy_pass http://upstream;
    }
}
```

Сохранение

С концепцией кэша тесно связано понятие **сохранения**. В случае когда сервер возвращает большие статические файлы, которые никогда не изменяются, то есть нет никаких причин вытеснять записи из кэша, то NGINX предлагает механизм «сохранения» (store), чтобы ускорить обслуживание таких запросов. NGINX сохраняет локальные копии заданных вами файлов. Эти файлы остаются на диске и более не запрашиваются у проксируемого сервера. Если такой файл изменится на проксируемом сервере, то его сохраненную копию должен будет удалить какой-то внешний процесс, иначе NGINX продолжит возвращать ее. Поэтому для небольших статических файлов использование кэша предпочтительнее.

В следующем примере демонстрируются директивы, применяемые для сохранения копий файлов:

```
http {
    proxy_temp_path /var/www/tmp;

    server {
        root /var/www/data;

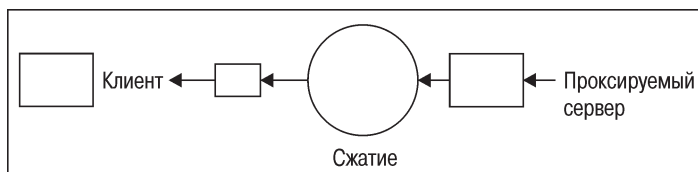
        location /img {
            error_page 404 = @store;
        }

        location @store {
            internal;
            proxy_store on;
            proxy_store_access group:r all:r;
            proxy_pass http://upstream;
        }
    }
}
```

В этой конфигурации определена секция `server`, в которой корень `root` размещен в той же файловой системе, что и путь `proxy_temp_path`. Местоположение `/img` наследует `root`, обслуживая файлы, расположенные в подкаталоге `/var/www/data`. Если файл не найден (код ошибки 404), то производится переход к именованному местоположению `@store` для запроса файла у проксируемого сервера. Директива `proxy_store` означает, что мы хотим сохранять файлы в унаследованном от `root` каталоге с правами 0644 (для владельца подразумеваются права `user:rw`, а для группы (`group`) и всех прочих (`all`) задаются в директиве `proxy_store_access`). Это все, что требуется от NGINX для сохранения локальных копий статических файлов, полученных от проксируемого сервера.

Сжатие

Идея сжатия представлена на рисунке ниже.



За счет снижения объема трафика можно уменьшить время передачи ответа. NGINX умеет сжимать ответ, полученный от проксируемого сервера, до передачи его клиенту. Модуль `gzip`, по умолчанию включенный, нередко используется обратным прокси-сервером для сжатия содержимого, когда это имеет смысл. Для некоторых типов файлов сжатие не дает ощутимого эффекта. С другой стороны, некоторые клиенты плохо реагируют на сжатое содержимое. То и другое можно учесть при настройке:

```
http {
    gzip on;
    gzip_http_version 1.0;
    gzip_comp_level 2;
    gzip_types text/plain text/css application/x-javascript text/xml
        application/xml application/xml+rss text/javascript
        application/javascript application/json;
    gzip_disable msie6;
}
```

Здесь мы говорим, что хотим сжимать содержимое с указанными типами MIME алгоритмом `gzip` с уровнем сжатия 2, если запрос поступил по протоколу HTTP версии не ниже 1.0, но только в том случае, когда пользовательский агент – не Internet Explorer старой версии. Мы поместили этот фрагмент конфигурации в контекст `http`, чтобы он распространялся на все определяемые серверы.

В таблице ниже перечислены директивы, относящиеся к модулю `gzip`.

Директивы модуля `gzip`

Директива	Описание
<code>gzip</code>	Разрешает или запрещает сжатие ответов
<code>gzip_buffers</code>	Определяет количество и размеры буферов для сжатия ответа
<code>gzip_comp_level</code>	Уровень сжатия <code>gzip</code> (1–9)
<code>gzip_disable</code>	Регулярное выражение, описывающее те пользовательские агенты, которые не должны получать сжатое содержимое. Специальное значение <code>msie6</code> является сокращением выражения <code>MSIE [4-6]\.</code> , которое исключает MSIE 6.0 ... SV1
<code>gzip_min_length</code>	Минимальная длина ответа (определяемая заголовком <code>Content-Length</code>), до которой вопрос о сжатии вообще не рассматривается
<code>gzip_proxied</code>	Разрешает или запрещает сжатие ответа, уже прошедшего через прокси-сервер. Параметр может принимать одно или несколько значений из следующего списка: <code>off</code>: запретить сжатие; <code>expired</code>: разрешить сжатие, если ответ не должен кэшироваться в соответствии с заголовком <code>Expires</code>; <code>no-cache</code>: разрешить сжатие, если заголовок <code>Cache-Control</code> содержит значение <code>no-cache</code>; <code>no-store</code>: разрешить сжатие, если заголовок <code>Cache-Control</code> содержит значение <code>no-store</code>; <code>private</code>: разрешить сжатие, если заголовок <code>Cache-Control</code> содержит значение <code>private</code>; <code>no-last-modified</code>: разрешить сжатие, если ответ не содержит заголовка <code>Last-Modified</code>; <code>no-etag</code>: разрешить сжатие, если ответ не содержит заголовка <code>ETag</code>; <code>auth</code>: разрешить сжатие, если ответ содержит заголовок <code>Authorization</code>; <code>any</code>: разрешить сжатие для любого ответа, если запрос содержит заголовок <code>Via</code>
<code>gzip_types</code>	Типы MIME (в дополнение к <code>text/html</code>), которые следует сжимать
<code>gzip_vary</code>	Разрешает или запрещает включение в ответ заголовка <code>Vary: Accept-Encoding</code> , если директива <code>gzip</code> активна

Если при включенном сжатии большие файлы обрезаются, то причина, вероятно, в директиве `gzip_buffers`. По умолчанию она принимает значение 32 4к или 16 8к (в зависимости от платформы), то есть суммарный объем буферов равен 128 КБ. Это означает, что NGINX не может сжать файл длиннее 128 КБ. Если на страницу загружается большая несжатая библиотека JavaScript, то вы можете столкнуться с этим пределом. В таком случае просто увеличьте количество буферов, чтобы их общий размер был достаточен для размещения всего файла.

```
http {
    gzip on;
    gzip_min_length 1024;
    gzip_buffers 40 4k;
    gzip_comp_level 5;
    gzip_types text/plain application/x-javascript application/json;
}
```

В примере выше сжимать можно файлы размером до $40 \times 4 \times 1024 = 163\,840$ байтов (160 КБ). Кроме того, мы включили директиву `gzip_min_length`, которая разрешает сжимать только файлы, размер которых больше 1 КБ. Уровень сжатия (`gzip_comp_level`) 4 или 5 обычно дает хороший компромисс между скоростью сжатия и размером результирующего файла. Проведение измерений на собственном оборудовании – лучший способ определить оптимальное значение.

Помимо сжатия ответов на лету, NGINX умеет доставлять предварительно сжатые файлы – благодаря модулю `gzip_static`. По умолчанию этот модуль не компилируется, но его можно включить с помощью параметра `--with-http_gzip_static_module` скрипта `configure`. У этого модуля имеется единственная собственная директива `gzip_static`, но он также понимает следующие директивы модуля `gzip`, определяющие, когда проверять предварительно сжатые файлы:

- ☐ `gzip_http_version`
- ☐ `gzip_proxied`
- ☐ `gzip_disable`
- ☐ `gzip_vary`

В следующем примере конфигурации мы разрешаем доставку предварительно сжатых файлов, если запрос содержит заголовок `Authorization` и в ответе присутствует заголовок `Expires` или `Cache-Control`, запрещающий кэширование:

```
http {  
    gzip_static on;  
    gzip_proxied expired no-cache no-store private auth;  
}
```

Резюме

В этой главе мы видели, как эффективно использовать NGINX в качестве обратного прокси-сервера. Он может решать три задачи, вместе или по отдельности: повышение безопасности, масштабируемости и (или) производительности. Безопасность повышается за счет отделения приложения от конечного пользователя. Для обеспечения масштабируемости NGINX можно использовать в сочетании с несколькими проксируемыми серверами. А производительность приложения непосредственно связана с тем, как быстро оно отвечает на запросы пользователей. Мы рассмотрели различные способы повысить быстроту реакции приложения. Чем меньше время ответа, тем лучше пользователям.

В следующей главе мы займемся вопросом об использовании NGINX в качестве HTTP-сервера. До сих пор мы говорили только о работе NGINX в роли обратного прокси-сервера, но он умеет гораздо больше.



Глава 6. NGINX как HTTP-сервер

HTTP-сервер – это программа, основная задача которой состоит в доставке веб-страниц клиентам в ответ на их запросы. Происхождение веб-страницы может быть каким угодно – от простого HTML-файла на диске до многокомпонентного каркаса, генерирующего зависящее от пользователя содержимое, которое динамически обновляется с помощью AJAX или WebSocket. NGINX – модульная программа, способная обслуживать HTTP-запросы любым необходимым способом.

В этой главе мы рассмотрим различные модули, которые в совокупности превращают NGINX в масштабируемый HTTP-сервер. Обсуждаются следующие вопросы.

- ☐ Архитектура NGINX.
- ☐ Базовый модуль HTTP.
- ☐ Установка предельных значений для предотвращения недобросовестного использования.
- ☐ Ограничение доступа.
- ☐ Поточковая передача мультимедийных файлов.
- ☐ Предопределенные переменные.
- ☐ Использование NGINX совместно с PHP-FPM.
- ☐ Интеграция NGINX и uWSGI.

Архитектура NGINX

NGINX состоит из одного главного и нескольких рабочих процессов. Все они однопоточные и способны одновременно обслуживать тысячи соединений. Большая часть работы производится рабочим процессом, поскольку именно он обрабатывает запросы клиентов. Чтобы как можно быстрее реагировать на запросы, NGINX использует встроенный в операционную систему механизм событий.

Главный процесс NGINX отвечает за чтение конфигурационного файла, работу с сокетами, запуск рабочих процессов, открытие файлов журналов и компиляции встроенных скриптов на языке Perl.

Он же реагирует на административные команды, передаваемые с помощью сигналов.

Рабочий процесс NGINX исполняет цикл событий, в котором обрабатываются входящие соединения. Все модули NGINX исполняются в рабочем процессе, то есть там производятся обработка запросов, фильтрация, обработка соединений с прокси-сервером и многое другое. Такая модель позволяет операционной системе отделять рабочие процессы друг от друга и оптимально планировать их выполнение на различных процессорных ядрах. Если какие-то операции, например дискового ввода-вывода, блокируют один рабочий процесс, то нагрузка перемещается на рабочие процессы, исполняемые на других ядрах.

Главный процесс NGINX запускает еще несколько вспомогательных процессов для выполнения специализированных задач. Среди них **загрузчик кэша** и **диспетчер кэша**. Загрузчик кэша отвечает за подготовку метаданных, необходимых рабочим процессам для использования кэша, а диспетчер кэша – за проверку записей кэша и удаление тех, для которых истек срок хранения.

NGINX имеет модульную структуру. Главный процесс предоставляет основу для работы всех модулей. Протоколы и обработчики реализованы в виде отдельных модулей. Из отдельных модулей выстраивается конвейер обработки запросов. Поступивший запрос передается по цепочке фильтров, которые его обрабатывают. Один из таких фильтров предназначен для обработки подзапросов, этот механизм – одна из самых интересных возможностей NGINX.

С помощью подзапросов NGINX может вернуть ответ на запрос, URI которого отличается от указанного клиентом. Подзапросы могут быть вложенными и вызывать другие подзапросы. Фильтры позволяют собрать ответы на несколько подзапросов и составить от них ответ, отправляемый клиенту. По ходу дела в игру вступают различные модули. Подробное описание внутренней структуры NGINX см. на странице <http://www.aosabook.org/en/nginx.html>.

Далее в этой главе мы рассмотрим модуль http и несколько вспомогательных модулей.

Базовый модуль HTTP

Модуль http является центральным в NGINX, он отвечает за взаимодействие с клиентами по протоколу HTTP. В главе 2 «Руководство по настройке» мы уже обсуждали следующие аспекты этого модуля:

- ☐ клиентские директивы;
- ☐ директивы файлового ввода-вывода;
- ☐ директивы хэширования;
- ☐ директивы работы с сокетами;
- ☐ директиву `listen`;
- ☐ сопоставление запроса с директивами `server_name` и `location`.

В этой главе мы рассмотрим оставшиеся директивы, тоже разбив их на категории.

Директива `server`

Директива `server` открывает новый контекст. Мы уже видели примеры ее использования на страницах этой книги. Но пока не уделили достаточного внимания понятию **сервера по умолчанию**.

В NGINX сервером по умолчанию называется первый из множества серверов, прослушивающих один и тот же IP-адрес и порт, указанные в директиве `listen`. Сервер по умолчанию можно также назначить с помощью параметра `default_server` в директиве `listen`.

Сервер по умолчанию полезен, когда нужно определить набор директив, общих для всех серверов, прослушивающих один и тот же IP-адрес и порт:

```
server {  
    listen 127.0.0.1:80;  
    server_name default.example.com;  
    server_name_in_redirect on;  
}  
  
server {  
    listen 127.0.0.1:80;  
    server_name www.example.com;  
}
```

В этом примере для сервера `www.example.com` будет действовать та же директива `server_name_in_redirect` с параметром `on`, что и для сервера `default.example.com`. Это было бы так, даже если бы в описаниях обоих серверов не было директивы `listen`, потому что тогда они бы все равно прослушивали один и тот же IP-адрес и порт (а именно `*:80`, подразумеваемые в директиве `listen` по умолчанию). Однако наследование не гарантируется. Существует лишь несколько наследуемых директив, и какие именно, меняется от версии к версии.

У сервера по умолчанию есть лучшее применение – обработка запросов, поступающих на данный IP-адрес и порт, но не имеющих

заголовка `Host`. Если вы не хотите, чтобы запросы без заголовка `Host` обрабатывал именно сервер по умолчанию, то можете задать в какой-нибудь секции `server` пустую директиву `server_name`. Тогда такие запросы будут поступать именно этому серверу.

```
server {  
    server_name "";  
}
```

В таблице ниже перечислены директивы, употребляемые в секции `server`.

Директивы HTTP-сервера

Директива	Описание
<code>port_in_redirect</code>	Определяет, нужно ли указывать данный порт при переадресации средствами NGINX
<code>server</code>	Создает новый конфигурационный контекст, определяющий виртуальный хост. Директива <code>listen</code> определяет один или несколько IP-адресов и портов, а директива <code>server_name</code> – значения заголовка <code>Host</code> , которым этот контекст соответствует
<code>server_name</code>	Задаёт имя виртуального хоста
<code>server_name_in_redirect</code>	Разрешает использовать первое из указанных в директиве <code>server_name</code> значений при любой переадресации, произведенной NGINX в данном контексте
<code>server_tokens</code>	Разрешает или запрещает включение номера версии NGINX в отправляемые сообщения об ошибках и заголовок <code>Server</code> (по умолчанию принимает значение <code>on</code>)

Протоколирование

В NGINX реализована очень гибкая модель протоколирования. На каждом уровне конфигурации может быть свой журнал доступа. Кроме того, на одном уровне можно задавать несколько журналов доступа с разными форматами с помощью директивы `log_format`. Эта директива, которая должна находиться в секции `http`, позволяет точно указать, что протоколировать.

Поскольку путь к файлу журнала может содержать переменные, то есть возможность задавать конфигурацию динамически. В примере ниже показано, как это может выглядеть на практике.

```

http {
    log_format vhost '$host $remote_addr - $remote_user [$time_local] '
        '"$request" $status $body_bytes_sent '
        '"$http_referer" "$http_user_agent"';
    log_format downloads '$time_iso8601 $host $remote_addr '
        '"$request" $status $body_bytes_sent $request_time';
    open_log_file_cache max=1000 inactive=60s;
    access_log logs/access.log;

    server {
        server_name ~^(www\.)?(.+)$;

        access_log logs/combined.log vhost;
        access_log logs/$2/access.log;

        location /downloads {
            access_log logs/downloads.log downloads;
        }
    }
}

```

В таблице ниже описаны использованные в этом фрагменте директивы.

Директивы протоколирования из модуля HTTP

Директива	Описание
access_log	Определяет, куда и как записывать журналы доступа. Первый параметр – путь к файлу журнала. Путь может содержать переменные. Специальное значение <code>off</code> отключает протоколирование. Необязательный второй параметр определяет директиву <code>log_format</code> , описывающую формат журнала. Если этот параметр не задан, используется предопределенный формат. Необязательный третий параметр задает размер буфера в случае, если запись в журнал буферизуется. При использовании буферизации этот размер не должен превосходить длину операции атомарной записи на диск для конкретной файловой системы. Если третий параметр равен <code>gzip</code> , то журнал буферизуется и на лету сжимается при условии, что двоичный файл <code>nginx</code> собирался с библиотекой <code>zlib</code> . Последний параметр <code>flush</code> определяет, сколько времени данные могут оставаться в буфере в памяти перед сбросом на диск
log_format	Определяет состав и формат полей в журнале. Список переменных, относящихся к журналу, приведен в следующей таблице

Директива	Описание
log_not_found	Поддавляет запись в журнал сообщений об ошибке 404 (по умолчанию принимает значение on)
log_subrequest	Разрешает или запрещает протоколирование подзапросов в журнале доступа (по умолчанию off)
open_log_file_cache	Кэширует дескрипторы тех открытых файлов, упоминаемых в директивах <code>access_log</code> , в путях к которым встречаются переменные. Допустимы следующие параметры: • <code>max</code>: максимальное число дескрипторов в кэше; • <code>inactive</code>: сколько времени ждать записи в этот журнал, перед тем как закрыть его; • <code>min_uses</code>: сколько раз этот дескриптор должен быть использован в течение времени <code>inactive</code>, чтобы оставаться открытым; • <code>valid</code>: NGINX будет часто проверять, по-прежнему ли этот дескриптор соответствует файлу с именем, по которому был открыт; • <code>off</code>: отключает кэширование

В примере ниже журнал сжимается согласно алгоритму `gzip` с уровнем 4. Буфер имеет подразумеваемый по умолчанию размер 64 КБ и сбрасывается на диск не реже одного раза в минуту.

```
access_log /var/log/nginx/access.log.gz combined gzip=4 flush=1m;
```

Отметим, что при задании сжатия (`gzip`) параметр `log_format` также должен быть задан.

Подразумеваемая по умолчанию директива `log_format` с именем `combined` выглядит следующим образом:

```
log_format combined '$remote_addr - $remote_user [$time_local] '
                    '»$request» $status $body_bytes_sent '
                    '»$http_referer» «$http_user_agent»';
```

Как видите, для большей наглядности директиву можно разбить на строки. На самом форматировании это никак не сказывается. В описании формата разрешается использовать любые переменные. В таблице ниже переменные, помеченные звездочкой, можно использовать только в директиве `log_format`, а остальные – также в других местах конфигурационного файла.

Переменные, используемые при описании формата журнала

Переменная	Значение
\$body_bytes_sent	Количество отправленных клиенту байтов, исключая заголовки
\$bytes_sent	Общее количество отправленных клиенту байтов
\$connection	Порядковый номер, уникально идентифицирующий соединение
\$connection_requests	Количество запросов, поступивших по данному соединению
\$msec	Время в секундах с точностью до миллисекунды
\$pipe *	Был ли запрос обработан конвейером (p) или нет (.)
\$request_length *	Длина запроса, включая HTTP-метод, URI-адрес, версию протокола HTTP, заголовки и тело
\$request_time	Время обработки запроса с точностью до миллисекунды с момента получения от клиента первого байта запроса и до момента отправки клиенту последнего байта ответа
\$status	Код состояния в ответе
\$time_iso8601 *	Местное время в формате ISO8601
\$time_local *	Местное время в общепринятом формате журнала (%d/%b/%Y:%H:%M:%S %z)

В этом разделе мы говорили исключительно о настройке журнала доступа (`access_log`). Но NGINX может также протоколировать ошибки. Директива `error_log` описана в главе 8 «Техника устранения неполадок».

Поиск файлов

Чтобы ответить на запрос, NGINX передает его обработчику содержимого, определяемому директивой `location`. Сначала апробируются безусловные обработчики: `perl`, `proxy_pass`, `flv`, `mp4` и т. д. Если ни один не подходит, то запрос передается одному из следующих обработчиков в указанном порядке: `random index`, `index`, `autoindex`, `gzip_static`, `static`. Запросы, в которых URI завершается знаком косой черты, поступают одному из индексных обработчиков. Если модуль `gzip` не активирован, то запрос обрабатывается модулем `static`. Как эти модули находят файл или каталог в файловой системе, определяется сочетанием нескольких директив. Директиву `root` лучше всего определять внутри сервера по умолчанию или, по крайней мере, вне директив `location`, чтобы она относилась ко всему серверу:

```
server {  
    root /home/customer/html;  
  
    location / {  
        index index.html index.htm;  
    }  
  
    location /downloads {  
        autoindex on;  
    }  
}
```

В этом примере возвращаемые файлы следует искать в каталоге `/home/customer/html`, который считается корневым. Если клиент указал только имя домена, NGINX попытается найти файл `index.html`. Если такого файла нет, то NGINX будет искать файл `index.htm`. Если пользователь укажет в браузере URI-адрес `/downloads`, то получит список файлов в этом каталоге в формате HTML. Это упрощает доступ к сайтам, на которых размещается интересное пользователям программное обеспечение. NGINX автоматически переписывает URI каталога, добавляя в конец знак косой черты, а затем выполняет переадресацию по протоколу HTTP. NGINX дописывает указанный URI в конец корневого каталога, чтобы получить путь к запрошенному клиентом файлу. Если такого файла не существует, клиенту возвращается сообщение об ошибке **404 Not Found**. Если вы не хотите, чтобы клиент получал такое сообщение, можете попробовать доставить файл из другого места в файловой системе и вернуться к универсальной странице, только если все остальные возможности исчерпаны. Для этой цели можно использовать директиву `try_files`:

```
location / {  
    try_files $uri $uri/ backups/$uri /generic-not-found.html;  
}
```

Для пущей безопасности NGINX может проверить путь к файлу, который собирается доставить, и, если где-то в нем встречается символическая ссылка, вернуть клиенту сообщение об ошибке:

```
server {  
    root /home/customer/html;  
    disable_symlinks if_not_owner from=$document_root;  
}
```

Здесь NGINX вернет сообщение «Permission Denied», если в части пути после `/home/customer/html` найдена символическая ссылка и файл, на который она указывает, не принадлежит пользователю с таким же идентификатором.

Описанные только что директивы перечислены в следующей таблице.

Директивы HTTP-сервера, относящиеся к путям в файловой системе

Директива	Описание
<code>disable_symlinks</code>	Определяет, должна ли NGINX проверять наличие символических ссылок в пути к файлу перед его отправкой клиенту. Распознаются следующие параметры: • <code>off</code>: отключает проверку символических ссылок (по умолчанию); • <code>on</code>: если какая-нибудь часть пути является символической ссылкой, доступ запрещается; • <code>if_not_owner</code>: если какая-нибудь часть пути является символической ссылкой, которая ведет на файл, принадлежащий другому пользователю, то доступ запрещается; • <code>from=part</code>: часть пути до <code>part</code> включительно не проверяется на символические ссылки, а остаток проверяется в соответствии с режимом, заданным параметром <code>on</code> или <code>if_not_owner</code>
<code>root</code>	Задаёт путь к «корню документов». При поиске файлов заданный в запросе URI добавляется в конец этого пути
<code>try_files</code>	Проверяет существование файлов, указанных в параметрах. Если ни один файл, кроме последнего, не найден, то последний параметр считается «последней надеждой», поэтому позаботьтесь о том, чтобы такой файл или именованное местоположение существовали, либо задайте возвращаемый код состояния в формате <code>=<status code></code>

Разрешение имен

Если вместо IP-адресов проксируемых серверов в директивах `upstream` или `*_pass` используются доменные имена, то NGINX по умолчанию обращается к механизму разрешения имен, встроенному в операционную систему, за IP-адресом, который необходим для установления соединения с сервером. Это происходит только один раз, при первом обращении к проксируемому серверу, и не происходит вовсе, если в директиве `*_pass` встречается переменная. Однако можно сконфигурировать для NGINX отдельный разреше-

тель. В этом случае можно переопределить время жизни (TTL), возвращаемое DNS-сервером, и использовать в директивах `*_pass` переменные.

```
server {  
    resolver 192.168.100.2 valid=300s;  
}
```

Директивы разрешения имен

Директива	Описание
resolver	Задаёт имена одного или нескольких серверов, используемых для получения IP-адресов по доменным именам. Необязательный параметр <code>valid</code> переопределяет время TTL, заданное в записи о доменном имени

Чтобы заставить NGINX каждый раз получать IP-адрес заново, поместите логическое имя в переменную. Разрешая эту переменную, NGINX неявно производит поиск IP-адреса в DNS. Но чтобы это заработало, необходимо включить директиву `resolver`:

```
server {  
    resolver 192.168.100.2;  
  
    location / {  
        set $backend upstream.example.com;  
        proxy_pass http://$backend;  
    }  
}
```

Разумеется, используя DNS для поиска проксируемого сервера, мы предполагаем, что разрешитель всегда доступен. Если это не так, произойдет ошибка шлюза. Чтобы сократить время ожидания ответа клиентом, следует присвоить небольшое значение параметру `resolver_timeout`. Затем ошибку шлюза можно перехватить в директиве `error_page`, предназначенной специально для этой цели.

```
server {  
    resolver 192.168.100.2;  
    resolver_timeout 3s;  
    error_page 504 /gateway-timeout.html;  
  
    location / {  
        proxy_pass http://upstream.example.com;  
    }  
}
```

Взаимодействие с клиентами

NGINX может взаимодействовать с клиентами разными способами. Настраиваются как атрибуты самого соединения (IP-адрес, тайм-ауты, свойство keepalive и т. д.), так и заголовки для согласования содержимого. В следующей таблице описаны директивы для конфигурирования различных заголовков и кодов ответа, которые служат клиенту указанием либо запросить страницу, либо взять ее из собственного кэша.

Директивы модуля HTTP для взаимодействия с клиентами

Директива	Описание
default_type	Определяет подразумеваемый по умолчанию MIME-тип ответа. Используется в случае, когда MIME-тип файла не удается сопоставить ни с одним из определенных в директиве <code>types</code>
error_page	Определяет URL-адрес страницы, которую нужно вернуть, если код ответа попадает в диапазон ошибок. Необязательный параметр, следующий за знаком <code>=</code> , позволяет изменить код ответа. Если после знака равенства не указан код ответа, то он берется из URI-адреса, при этом соответствующая страница должна возвращаться каким-то проксируемым сервером
etag	Отключает автоматическую генерацию заголовка ответа ETag для статических ресурсов (по умолчанию <code>on</code>)
if_modified_since	Управляет порядком сравнения времени модификации ответа со значением в заголовке запроса If-Modified-Since. • <code>off</code>: заголовок If-Modified-Since игнорируется. • <code>exact</code>: точное соответствие (по умолчанию). • <code>before</code>: время модификации ответа меньше или равно значению в заголовке If-Modified-Since
ignore_invalid_headers	Разрешает или запрещает игнорировать заголовки с недопустимыми именами (по умолчанию <code>on</code>). Допустимыми считаются имена, содержащие буквы в кодировке ASCII, цифры, знак минус и, возможно, знак подчеркивания (определяется директивой <code>underscores_in_headers</code>)
merge_slashes	Разрешает или запрещает удаление идущих подряд знаков косой черты. Подразумеваемое по умолчанию значение <code>on</code> означает, что NGINX будет заменять несколько соседних знаков / одним
recursive_error_pages	Разрешает производить несколько переадресаций с помощью директивы <code>error_page</code> (по умолчанию <code>off</code>)

Директива	Описание
types	Определяет соответствие между MIME-типами и расширениями имен файлов. В дистрибутив NGINX входит файл <code>conf/mime.types</code> , содержащий большинство соответствий. Как правило, достаточно просто включить этот файл директивой <code>include</code>
underscores_in_headers	Разрешает или запрещает использование символа подчеркивания в заголовках запросов от клиентов. Если оставлено подразумеваемое по умолчанию значение <code>off</code> , то анализ таких заголовков производится согласно режиму, заданному в директиве <code>ignore_invalid_headers</code>

Директива `error_page` — одна из самых гибких в NGINX. С ее помощью можно вернуть любую страницу в случае возникновения ошибки. Эта страница может находиться на локальной машине, а может динамически генерироваться сервером приложений или даже размещаться совсем на другом сайте.

```
http {
    # обобщенная страница, возвращаемая при любой ошибке самого сервера
    error_page 500 501 502 503 504 share/examples/nginx/50x.html;

    server {
        server_name www.example.com;
        root /home/customer/html;

        # если файл не найден, то возвращается содержимое файла
        # /home/customer/html/404.html
        error_page 404 /404.html;

        location / {
            # ошибки сервера для этого виртуального хоста переадресуются
            # специальному обработчику в приложении
            error_page 500 501 502 503 504 = @error_handler;
        }

        location /microsite {
            # если не существует файл в области /microsite,
            # то клиенту демонстрируется страница с другого сервера
            error_page 404 http://microsite.example.com/404.html;
        }

        # именованное местоположение, содержащее специальный
        # обработчик ошибок
        location @error_handler {
            # мы задаем здесь тип по умолчанию, чтобы браузер
```

```
# корректно отобразил страницу ошибки
default_type text/html;
proxy_pass http://127.0.0.1:8080;
}
}
}
```

Установка предельных значений для предотвращения недобросовестного использования

Мы создаем сайты, чтобы пользователи на них заходили. Мы хотим, чтобы наши сайты всегда были открыты для добросовестного доступа. Но это означает, что необходимо принимать меры для ограничения доступа недобросовестным пользователям. Под «недобросовестностью» можно понимать разные вещи, например отправку более одного запроса в секунду одним пользователем или чрезмерно большое количество соединений, открытых с одного и того же IP-адреса. Недобросовестность может также принимать форму **DDOS-атаки** (распределенная атака типа «отказ от обслуживания»), когда роботы с различных компьютеров, разбросанных по всему миру, одновременно пытаются обращаться к сайту с максимальной частотой. В этом разделе мы рассмотрим, как противостоять таким недобросовестным действиям и обеспечить доступность сайта.

Сначала приведем конфигурационные директивы, предназначенные для этой цели.

Директивы модуля HTTP для задания предельных значений

Директива	Описание
limit_conn	Определяет зону разделяемой памяти (настраиваемую с помощью директивы <code>limit_conn_zone</code>) и максимальное количество соединений с одинаковым значением ключа
limit_conn_log_level	Если NGINX ограничивает соединения согласно директиве <code>limit_conn</code> , то эта директива определяет уровень протоколирования для сообщения о достижении пороговой величины
limit_conn_zone	В первом параметре задается ключ, к которому относятся ограничения, указанные в директиве <code>limit_conn</code> . Вторым параметром задается имя зоны разделяемой памяти, в которой хранится не более указанного числа соединений для каждого ключа, а также размер этой зоны (<code>name=size</code>)

Директива	Описание
<code>limit_rate</code>	Ограничивает скорость (в байтах/с) отдачи содержимого клиентам. Ограничение действует на уровне соединения, то есть один клиент может повысить свою пропускную способность, открыв несколько соединений
<code>limit_rate_after</code>	Начинает применять ограничение <code>limit_rate</code> после того, как передано указанное количество байтов
<code>limit_req</code>	Задаёт ограничение при резком увеличении (всплеске) количества запросов для указанного ключа в зоне разделяемой памяти (заданной в директиве <code>limit_req_zone</code>). Всплеск описывается вторым параметром. Если до возникновения всплеска задерживать запросы не нужно, следует включить третий параметр <code>nodelay</code>
<code>limit_req_log_level</code>	Если NGINX ограничивает количество запросов согласно директиве <code>limit_req</code> , то эта директива определяет уровень протоколирования для сообщения о достижении пороговой величины. Сообщение о применении задержки имеет уровень, на единицу меньший указанного в этой директиве
<code>limit_req_zone</code>	В первом параметре задается ключ, к которому относятся ограничения, указанные в директиве <code>limit_req</code> . Второй параметр задает имя зоны разделяемой памяти, в которой хранится не более указанного числа запросов для каждого ключа, а также размер этой зоны (<code>name:size</code>). Третий параметр определяет количество запросов в секунду (<code>r/s</code>) или в минуту (<code>r/m</code>), при превышении которого начинается применение ограничения
<code>max_ranges</code>	Задаёт максимальное количество диапазонов, допустимых в запросе с указанием диапазонов байтов. Если задано значение 0, то поддержка диапазонов байтов отключается

В примере ниже мы разрешаем не более 10 соединений с одного IP-адреса. Для нормального доступа к сайту этого должно быть достаточно, потому что современные браузеры открывают только два или три соединения с одним сервером. Однако не забывайте, что для всех пользователей, находящихся за прокси-сервером, сервер видит один и тот же IP-адрес. Поэтому следите за появлением в журналах сообщений об ошибке 503 (Service Unavailable), означающей применение этого ограничения:

```
http {
    limit_conn_zone $binary_remote_addr zone=connections:10m;
    limit_conn_log_level notice;

    server {
```

```
    limit_conn connections 10;
}
}
```

Ограничение доступа на основе частоты запросов выглядит почти так же, но работает несколько иначе. Когда мы ограничиваем количество страниц, запрашиваемых пользователем в единицу времени, NGINX вставляет задержку после запроса первой страницы и до тех пор, пока число избыточных запросов не превысит размер всплеска. Возможно, вы этого не хотите, поэтому NGINX предоставляет возможность подавить задержку с помощью параметра `nodelay`:

```
http {
    limit_req_zone $binary_remote_addr zone=requests:10m rate=1r/s;
    limit_req_log_level warn;

    server {
        limit_req zone=requests burst=10 nodelay;
    }
}
```



Использование переменной `$binary_remote_addr`

В примере выше мы воспользовались переменной `$binary_remote_addr`, чтобы узнать, сколько места в памяти отводится для хранения одного IP-адреса. На 32-разрядных платформах это 32 байта, на 64-разрядных – 64 байта. Поэтому сконфигурированная зона размером 10m на 32-разрядных платформах способна хранить 320 000 состояний, а на 64-разрядных – 160 000.

Можно также ограничить потребление полосы пропускания одним клиентом, чтобы несколько недобросовестных пользователей не заняли всю доступную полосу. Но имейте в виду одну тонкость: директива `limit_rate` работает на уровне соединений. Если одному клиенту разрешено открывать несколько соединений, то он легко сможет обойти это ограничение:

```
location /downloads {
    limit_rate 500k;
}
```

Можно вместо этого разрешить частые запросы при загрузке небольших файлов, но ввести ограничение для файлов большего размера:

```
location /downloads {
    limit_rate_after 1m;
    limit_rate 500k;
}
```

Комбинируя различные ограничения на частоту, можно создать конфигурацию с очень гибкими ограничениями на действия клиентов:

```
http {
    limit_conn_zone $binary_remote_addr zone=ips:10m;
    limit_conn_zone $server_name zone=servers:10m;
    limit_req_zone $binary_remote_addr zone=requests:10m rate=1r/s;
    limit_conn_log_level notice;
    limit_req_log_level warn;
    reset_timedout_connection on;

    server {
        # эти ограничения применяются ко всему виртуальному серверу
        limit_conn ips 10;

        # не более 1000 одновременных соединений с одним server_name
        limit_conn servers 1000;

        location /search {
            # здесь мы ограничиваем частоту запросов только для URL /search
            limit_req zone=requests burst=3 nodelay;
        }

        location /downloads {
            # используем limit_conn, чтобы ограничить потребление полосы
            # пропускания одним клиентом, не позволяя обойти ограничение
            limit_conn connections 1;
            limit_rate_after 1m;
            limit_rate 500k;
        }
    }
}
```

Ограничение доступа

В предыдущем разделе мы рассмотрели различные способы поставить преграды на пути недобросовестного посетителя сайта, работающего под управлением NGINX. Теперь посмотрим, как ограничить доступ ко всему сайту или его частям. Есть два способа ограничения доступа: с некоторых IP-адресов или некоторым пользователям.

Комбинируя эти способы, мы можем разрешить пользователям доступ к сайту либо с определенных IP-адресов, либо при условии ввода корректного имени и пароля.

В этом нам помогут следующие директивы.

Директивы модуля HTTP для управления доступом

Директива	Описание
allow	Разрешает доступ с указанного IP-адреса, из указанной сети или отовсюду (<i>all</i>)
auth_basic	Разрешает аутентификацию по схеме HTTP Basic Authentication. Параметр задает имя области (<i>realm</i>). Специальное значение <i>off</i> говорит о том, что значение <i>auth_basic</i> , заданное на родительском уровне, отменяется
auth_basic_user_file	Определяет, где находится файл, содержащий тройки <i>username:password:comment</i> , который используется для аутентификации клиентов. Поле <i>password</i> должно быть зашифровано алгоритмом <i>crypt</i> . Поле <i>comment</i> необязательно
deny	Запрещает доступ с указанного IP-адреса, из указанной сети или отовсюду (<i>all</i>)
satisfy	Разрешает доступ, если все (<i>all</i>) или хотя бы одна (<i>any</i>) из предыдущих директив разрешает доступ. Подразумеваемое по умолчанию значение <i>all</i> говорит, что пользователь должен находиться в определенной сети и ввести правильный пароль

Чтобы разрешить доступ только клиентам, которые заходят с определенных IP-адресов, используйте директивы *allow* и *deny* следующим образом:

```
location /stats {  
    allow 127.0.0.1;  
    deny all;  
}
```

Здесь мы разрешаем доступ к URI */stats* только с локального компьютера *localhost*.

Чтобы разрешить доступ только аутентифицированным пользователям, воспользуйтесь директивами *auth_basic* и *auth_basic_user_file*:

```
server {  
    server_name restricted.example.com;  
    auth_basic «restricted»;  
    auth_basic_user_file conf/htpasswd;  
}
```

Пользователь, желающий зайти на сайт `restricted.example.com`, должен будет ввести имя и пароль, присутствующие в файле `htpasswd` в подкаталоге `conf` корневого каталога NGINX. Записи в `htpasswd` можно сгенерировать любым инструментом, поддерживающим стандартную для UNIX функцию `crypt()`. Так, приведенный ниже скрипт на Ruby генерирует файл нужного формата.

```
#!/usr/bin/env ruby
# разбираем флаги в командной строке
require 'optparse'

OptionParser.new do |o|
  o.on('-f FILE') { |file| $file = file }
  o.on('-u', '--username USER») { |u| $user = u }
  o.on('-p', '--password PASS») { |p| $pass = p }
  o.on('-c', '--comment COMM (optional)») { |c| $comm = c }
  o.on('-h') { puts o; exit }
  o.parse!

  if $user.nil? or $pass.nil?
    puts o; exit
  end
end

# инициализируем массив ASCII-символов, используемый в качестве затравки
ascii = ('a'..'z').to_a + ('A'..'Z').to_a + ('0'..'9').to_a + [ ".", "/" ]

$lines = []
begin
  # читаем аутентификационные данные из файла
  File.open($file) do |f|
    f.lines.each { |l| $lines << l }
  end
rescue Errno::ENOENT
  # если файл не найден (при первом использовании), инициализируем массив
  $lines = ["#{ $user }:#{ $pass }\n"]
end

# удаляем редактируемую запись о пользователе из текущего списка
$lines.map! do |line|
  unless line =~ /^#{ $user }:/
    line
  end
end

# шифруем пароль с помощью crypt()
pass = $pass.crypt(ascii[rand(64)] + ascii[rand(64)])

# если есть комментарий, вставляем его
```

```
if $comm
  $lines << "#{user}:#{pass}:#{$comm}\n"
else
  $lines << "#{user}:#{pass}\n"
end

# записываем новый файл, при необходимости создав его
File.open($file, File::RDWR|File::CREAT) do |f|
  $lines.each { |l| f << l}
end
```

Сохраните этот код в файле `http_auth_basic.rb` и выполните его, указав имя файла (`-f`), имя пользователя (`-u`) и пароль (`-p`). Будет сгенерирована запись в формате, который понимает директива NGINX `auth_basic_user_file`:

```
$ ./http_auth_basic.rb -f htpasswd -u testuser -p 123456
```

Для случая, когда имя и пароль нужно вводить, только если вход осуществляется не с одного из predetermined IP-адресов, в NGINX предусмотрена директива `satisfy`. В примере ниже задан параметр `any`, подходящий для сценария или-или:

```
server {
  server_name intranet.example.com;

  location / {
    auth_basic «intranet: please login»;
    auth_basic_user_file conf/htpasswd-intranet;
    allow 192.168.40.0/24;
    allow 192.168.50.0/24;
    deny all;
    satisfy any;
  }
}
```

Если, напротив, требуется, чтобы пользователь мог заходить только с определенных IP-адресов и при этом аутентифицировался, то следует воспользоваться параметром `all`, который и так подразумевается по умолчанию. Поэтому директиву `satisfy` можно опустить, оставив только директивы `allow`, `deny`, `auth_basic` и `auth_basic_user_file`:

```
server {
  server_name stage.example.com;

  location / {
    auth_basic «staging server»;
```

```
auth_basic_user_file conf/htpasswd-stage;
allow 192.168.40.0/24;
allow 192.168.50.0/24;
deny all;
}
```

Потоковая передача мультимедийных файлов

NGINX умеет обслуживать запросы на некоторые типы видео-файлов. Модули `flv` и `mp4`, включенные в базовый дистрибутив, выполняют так называемую **псевдопотокową передачу**. Это означает, что NGINX начнет передачу с того места видеофайла, которое указано в параметре запроса `start`.

Для поддержки псевдопотокowej передачи на этапе конфигурирования сборки необходимо указать флаги `--with-http_flv_module` (для Flash Video – FLV) и (или) `--with-http_mp4_module` (для H.264/AAC). Тогда в конфигурационном файле можно будет использовать следующие директивы.

Директивы модуля HTTP для управления потоковой передачей

Директива	Описание
<code>flv</code>	Активирует модуль <code>flv</code> для данного местоположения
<code>mp4</code>	Активирует модуль <code>mp4</code> для данного местоположения
<code>mp4_buffer_size</code>	Задаёт начальный размер буфера для доставки MP4-файлов
<code>mp4_max_buffer_size</code>	Задаёт максимальный размер буфера для обработки метаданных MP4

Активация псевдопотокowej передачи FLV для местоположения сводится к добавлению директивы `flv`:

```
location /videos {
    flv;
}
```

У псевдопотокowej передачи MP4 параметров больше, так как в формате H.264 имеются метаданные, которые необходимо разбирать. Поиск в файле доступен, если проигрыватель обнаружил атом "moov atom". Поэтому для повышения производительности метаданные должны находиться в начале файла. Если в журнале появляется сообщение

Имя переменной	Значение
\$cookie_name	Значение куки с именем name
\$document_root	Значение директивы root или alias для текущего запроса
\$document_uri	Псевдоним \$uri
\$host	Значение заголовка запроса Host, если таковой присутствует. В противном случае это значение, заданное в директиве server_name, отвечающей запросу
\$hostname	Имя хоста, на котором работает NGINX
\$http_name	Значение заголовка запроса с именем name. Если имя заголовка содержит дефисы, они преобразуются в знаки подчеркивания. Прописные буквы преобразуются в строчные
\$https	Для соединений, шифруемых по протоколу SSL, принимает значение on. В противном случае пустая строка
\$is_args	Если в запросе есть аргументы, принимает значение ?. В противном случае пустая строка
\$limit_rate	Значение, заданное в директиве limit_rate. Если значение не установлено, то с помощью этой переменной можно регулировать скорость отдачи содержимого клиентам
\$nginx_version	Номер версии работающей программы nginx
\$pid	Идентификатор рабочего процесса
\$query_string	Псевдоним \$args
\$realpath_root	Значение директивы root или alias для текущего запроса после разрешения всех символических ссылок
\$remote_addr	IP-адрес клиента
\$remote_port	Номер порта клиента
\$remote_user	При использовании простой схемы аутентификации по HTTP эта переменная содержит имя пользователя
\$request	Полный запрос, полученный от клиента, включая метод HTTP, URI-адрес, версию протокола HTTP, заголовки и тело
\$request_body	Тело запроса для использования в местоположениях, обрабатываемых директивой *_pass
\$request_body_file	Путь к временному файлу, в котором сохранено тело запроса. Для гарантированного создания этого файла директива client_body_in_file_only должна принимать значение on
\$request_completion	Если запрос получен полностью, принимает значение OK, иначе пустая строка
\$request_filename	Путь к файлу для текущего запроса, формируемый на основе значения директивы root или alias и URI-адреса
\$request_method	HTTP-метод текущего запроса
\$request_uri	Полный URI, указанный в запросе, включая аргументы
\$scheme	Схема текущего запроса: HTTP или HTTPS
\$sent_http_name	Значение заголовка ответа с именем name. Если имя заголовка содержит дефисы, они преобразуются в знаки подчеркивания. Прописные буквы преобразуются в строчные

Имя переменной	Значение
\$server_addr	Адрес сервера, принявшего запрос
\$server_name	Значение директивы <code>server_name</code> виртуального сервера, принявшего запрос
\$server_port	Номер порта сервера, принявшего запрос
\$server_protocol	Версия протокола HTTP для текущего запроса
\$status	Код состояния ответа
\$tcpinfo_rtt \$tcpinfo_rttvar \$tcpinfo_snd_cwnd \$tcpinfo_rcv_space	Если система поддерживает опцию сокета <code>TCP_INFO</code> , то в эти переменные заносится соответствующая информация
\$uri	Нормализованный URI текущего запроса

Использование NGINX совместно с PHP-FPM

Долгое время Apache считался единственным веб-сервером для сайтов, написанных на PHP, поскольку его модуль `mod_php` позволяет без труда интегрировать PHP непосредственно с веб-сервером. Но после включения **PHP-FPM** в ядро PHP появилась альтернатива. PHP-FPM – это технология исполнения PHP-скриптов под управлением FastCGI-сервера. Главный процесс PHP-FPM берет на себя заботу о запуске рабочих процессов с учетом нагрузки на сайт; при необходимости дочерние процессы перезапускаются. Главный процесс взаимодействует с другими службами по протоколу FastCGI. Подробнее о PHP-FPM можно прочитать на странице <http://php.net/manual/en/install.fpm.php>.

В NGINX имеется модуль `fastcgi`, который может взаимодействовать не только с PHP-FPM, но и с любым сервером, совместимым с протоколом FastCGI. Он включен по умолчанию, поэтому для использования NGINX совместно с FastCGI-серверами ничего специально делать не нужно.

Директивы модуля FastCGI

Директива	Описание
<code>fastcgi_buffer_size</code>	Размер буфера для первой части ответа от FastCGI-сервера, в которой находятся заголовки
<code>fastcgi_buffers</code>	Количество и размер буферов для получения ответа от FastCGI-сервера в расчете на одно соединение

Директива	Описание
<code>fastcgi_busy_buffers_size</code>	Суммарный размер буферов, которые могут быть заняты для отправки ответа клиенту, пока ответ от FastCGI-сервера ещё не прочитан целиком. Обычно устанавливается вдвое большим, чем размер, указанный в директиве <code>fastcgi_buffers</code>
<code>fastcgi_cache</code>	Определяет зону разделяемой памяти, используемую для кэширования
<code>fastcgi_cache_bypass</code>	Одна или несколько строковых переменных. Если хотя бы одна из них непуста и не содержит ноль, то ответ будет запрошен у FastCGI-сервера, а не взят из кэша
<code>fastcgi_cache_key</code>	Строка, которая используется как ключ для поиска значения в кэше
<code>fastcgi_cache_lock</code>	Если включено, то предотвращается отправка нескольких запросов FastCGI-серверу в случае отсутствия в кэше
<code>fastcgi_cache_lock_timeout</code>	Сколько времени запрос может ждать появления записи в кэше или освобождения блокировки <code>fastcgi_cache_lock</code>
<code>fastcgi_cache_min_uses</code>	Сколько запросов с данным ключом должно поступить, прежде чем ответ будет помещен в кэш
<code>fastcgi_cache_path</code>	Каталог, в котором хранятся кэшированные ответы, и зона разделяемой памяти (<code>keys_zone=name:size</code>) для хранения активных ключей и метаданных ответа. Необязательные параметры: <code>levels</code>: список разделенных двоеточиями длин имен подкаталогов на каждом уровне (1 или 2); допускается не более трех уровней вложенности; <code>inactive</code>: максимальное время нахождения неактивного запроса в кэше, по прошествии которого он вытесняется; <code>max_size</code>: максимальный размер кэша; по его достижении процесс-диспетчер удаляет элементы, к которым дольше всего не было обращений; <code>loader_files</code>: максимальное количество кэшированных файлов, метаданные которых загружаются в кэш на одной итерации процесса-загрузчика; <code>loader_sleep</code>: число миллисекунд между итерациями процесса-загрузчика кэша; <code>loader_threshold</code>: максимальное время, отведенное на одну итерацию процесса-загрузчика кэша
<code>fastcgi_cache_use_stale</code>	В каких случаях допустимо использовать устаревшие кэшированные данные, если при доступе к FastCGI-серверу произошла ошибка. Параметр <code>updating</code> разрешает использовать кэшированный ответ, если как раз в данный момент загружаются более свежие данные

Директива	Описание
<code>fastcgi_cache_valid</code>	Сколько времени считать действительным кэшированный ответ с кодом 200, 301 или 302. Если перед параметром <code>time</code> указан необязательный код ответа, то заданное время относится только к ответу с таким кодом. Специальный параметр <code>any</code> означает, что в течение заданного времени следует кэшировать ответ с любым кодом
<code>fastcgi_connect_timeout</code>	Максимальное время, в течение которого NGINX ожидает установления соединения при отправке запроса FastCGI-серверу
<code>fastcgi_hide_header</code>	Список заголовков, которые не следует передавать клиенту
<code>fastcgi_ignore_client_abort</code>	Если значение равно <code>on</code> , то NGINX не станет разрывать соединение с FastCGI-сервером в случае, когда клиент разрывает свое соединение
<code>fastcgi_ignore_headers</code>	Какие заголовки можно игнорировать при обработке ответа от FastCGI-сервера
<code>fastcgi_index</code>	Задаёт имя файла, дописываемое в конец строки <code>\$fastcgi_script_name</code> после знака косой черты
<code>fastcgi_intercept_errors</code>	Если значение равно <code>on</code> , то NGINX будет отображать страницу, заданную директивой <code>error_page</code> , вместо ответа, полученного от FastCGI-сервера
<code>fastcgi_keep_conn</code>	Разрешает соединения типа <code>keepalive</code> с FastCGI-сервером, индексируя его не закрывать соединение немедленно
<code>fastcgi_max_temp_file_size</code>	Максимальный размер временного файла, в который записывается часть ответа в случае, когда он не умещается целиком в буферах памяти
<code>fastcgi_next_upstream</code>	Определяет условия, при которых для ответа будет выбран следующий FastCGI-сервер. Это не происходит, если клиент уже что-то отправил. Условия задаются с помощью следующих параметров: • <code>error</code>: произошла ошибка при взаимодействии с FastCGI-сервером; • <code>timeout</code>: произошел тайм-аут при взаимодействии с FastCGI-сервером; • <code>invalid_header</code>: FastCGI-сервер вернул пустой или недопустимый ответ; • <code>http_500</code>: FastCGI-сервер вернул ответ с кодом ошибки 500; • <code>http_503</code>: FastCGI-сервер вернул ответ с кодом ошибки 503; • <code>http_404</code>: FastCGI-сервер вернул ответ с кодом ошибки 404; • <code>off</code>: запретить передачу запроса следующему FastCGI-серверу в случае ошибки

Директива	Описание
<code>fastcgi_no_cache</code>	Одна или несколько строковых переменных. Если хотя бы одна из них непуста и не содержит ноль, то NGINX не будет помещать в кэш ответ, полученный от FastCGI-сервера
<code>fastcgi_param</code>	Задаёт имя и значение параметра, передаваемого FastCGI-серверу. Если следует передавать только параметры с непустыми значениями, то необходимо задать дополнительный параметр <code>if_not_empty</code>
<code>fastcgi_pass</code>	Определяет FastCGI-сервер, которому передается запрос, в виде пары <code>address:port</code> или <code>unix:path</code> для сокета в домене UNIX
<code>fastcgi_pass_header</code>	Отменяет сокрытие заголовков, определенных в директиве <code>fastcgi_hide_header</code> , разрешая передавать их клиенту
<code>fastcgi_read_timeout</code>	Сколько времени может пройти между двумя последовательными операциями чтения данных от FastCGI-сервера, прежде чем соединение будет закрыто
<code>fastcgi_send_timeout</code>	Сколько времени может пройти между двумя последовательными операциями записи данных на FastCGI-сервер, прежде чем соединение будет закрыто
<code>fastcgi_split_path_info</code>	Задаёт регулярное выражение с двумя запоминаемыми подвыражениями. Первая запомненная строка становится значением переменной <code>\$fastcgi_script_name</code> , вторая – значением переменной <code>\$fastcgi_path_info</code> . Необходимо только для приложений, в которых используется переменная <code>PATH_INFO</code>
<code>fastcgi_store</code>	Разрешает сохранение полученных от FastCGI-сервера ответов в файлах на диске. Если параметр равен <code>on</code> , то в качестве пути к каталогу с сохраняемыми файлами используется значение, заданное в директиве <code>alias</code> или <code>root</code> . Можно вместо этого задать строку, определяющую другой каталог для хранения файлов
<code>fastcgi_store_access</code>	Какие права доступа следует задать для новых файлов, создаваемых в соответствии с директивой <code>fastcgi_store</code>
<code>fastcgi_temp_file_write_size</code>	Ограничивает размер данных в одной операции записи во временный файл, чтобы NGINX не слишком долго блокировала исполнение программы при обработке одного запроса
<code>fastcgi_temp_path</code>	Каталог для хранения временных файлов, получаемых от FastCGI-сервера. Может быть многоуровневым

Пример конфигурации для Drupal

Drupal (<http://drupal.org>) – это популярная платформа с открытым исходным кодом для управления содержимым. Она установлена на многих известных сайтах. Как и большинство веб-каркасов на основе PHP, Drupal обычно работает под Apache с модулем `mod_php`. Мы покажем, как настроить NGINX для запуска Drupal.

На сайте <https://github.com/perusio/drupal-with-nginx> имеется очень подробная инструкция по настройке Drupal для работы с NGINX. Она куда детальнее того, что мы можем позволить себе в этой книге, но некоторые вещи мы все же отметим и остановимся на различиях между Drupal 6 и Drupal 7:

```
## Определяем переменную $no_slash_uri variable для Drupal 6.
map $uri $no_slash_uri {
    ~(?<no_slash>.*)$ $no_slash;
}

server {
    server_name www.example.com;
    root /home/customer/html;
    index index.php;

    # не закрывать соединения с FastCGI-сервером (используется в сочетании с
    # директивой "keepalive" в секции upstream)
    fastcgi_keep_conn on;

    # Местоположение по умолчанию.
    location / {
        ## (Drupal 6) Использовать index.html, если нет index.php.
        location = / {
            error_page 404 =200 /index.html;
        }

        # Обслуживание обычных частных файлов (тех, что обрабатываются Drupal).
        location ~ /system/files/ {
            include fastcgi_private_files.conf;
            fastcgi_pass 127.0.0.1:9000;

            # Добавить следующую строку, чтобы в журнал ошибок не попадало
            # сообщение об ошибке 404 при доступе к каталогу system/files.
            # Ошибка 404 в данном случае – ожидаемое поведение.
            log_not_found off;
        }

        # Попытка прямого доступа к частным файлам возвращает ошибку 404.
        location ~ /sites/default/files/private/ {
```

```

    internal;
}

## (Drupal 6) При доступе к изображению, сгенерированному imagescache,
## пытаемся вернуть его, если оно имеется, иначе переправляем запрос
## Drupal для генерации или (перегенерации).
location ~* /imagescache/ {
    access_log off;
    expires 30d;
    try_files $uri /index.php?q=$no_slash_uri&$args;
}

# Обработка изображений в Drupal 7, когда imagescache - часть ядра
location ~* /files/styles/ {
    access_log off;
    expires 30d;
    try_files $uri @drupal;
}

```

Следующие далее настройки модуля Advanced Aggregation отличаются только значением location. Вот как выглядит конфигурация этого модуля для CSS:

```

# Поддержка CSS модулем Advanced Aggregation.
location ~* /sites/default/files/advagg_css/ {
    location ~* /sites/default/files/advagg_css/css_ [[:alnum:]]+\.css$ {

```

А так для JavaScript:

```

# Поддержка JS модулем Advanced Aggregation.
location ~* /sites/default/files/advagg_js/ {
    location ~* /sites/default/files/advagg_js/js_ [[:alnum:]]+\.js$ {

```

Далее приведены строки, общие для обеих секций:

```

    access_log off;
    add_header Pragma '';
    add_header Cache-Control 'public, max-age=946080000';
    add_header Accept-Ranges '';

    # Это для Drupal 7
    try_files $uri @drupal;

    ## Это для Drupal 6 (оставить что-то одно)
    try_files $uri /index.php?q=$no_slash_uri&$args;
}
}

# Все статические файлы обслуживаются непосредственно.

```

```
location ~* ^.+\.?(?:css|cur|js|jpe?g|gif|htc|ico|png|html|x ml)$ {
    access_log off;
    expires 30d;

    # Посылать все сразу.
    tcp_nodelay off;

    # Настроить файловый кэш операционной системы
    open_file_cache max=3000 inactive=120s;
    open_file_cache_valid 45s;
    open_file_cache_min_uses 2;
    open_file_cache_errors off;
}

# Обработка файлов PDF и powerpoint.
location ~* ^.+\.?(?:pdf|pptx?)$ {
    expires 30d;
    # Посылать все сразу.
    tcp_nodelay off;
}
```

На примере звуковых файлов демонстрируется использование асинхронного ввода-вывода. Местоположение MP3 выглядит так:

```
# MP3-файлы обслуживаются с использованием AIO, если ОС поддерживает.
location ~ /sites/default/files/audio/mp3 {
    location ~* ~/sites/default/files/audio/mp3/.*\..mp3$ {
```

А местоположение для файлов Ogg/Vorbis – так:

```
# Ogg/Vorbis-файлы обслуживаются с использованием AIO, если
# ОС поддерживает.
location ~ /sites/default/files/audio/ogg {
    location ~* ~/sites/default/files/audio/ogg/.*\..ogg$ {
```

Следующие строки являются общими для обоих местоположений:

```
    directio 4k; # for XFS
    tcp_nopush off;
    aio on;
    output_buffers 1 2M;
}

# Псевдопотоковая передача FLV-файлов
location ~ /sites/default/files/video/flv {
    location ~* ~/sites/default/files/video/flv/.*\..flv$ {
        flv;
    }
}
```

Следующие две секции также похожи. Псевдопоточковая передача для H264-файлов настраивается так:

```
# Псевдопотоковая передача H264-файлов.
location ~ /sites/default/files/video/mp4 {
    location ~* ~/sites/default/files/video/mp4/.*/. (?:(mp4|mov))$ { }
```

А псевдопотокковая передача для AAC-файлов – так:

```
# Псевдопоточковая передача AAC-файлов.
location ~ /sites/default/files/video/m4a {
    location ~* ~/sites/default/files/video/m4a/.*/.m4a$ {
```

Следующие строки являются общими для обеих секций:

```

mp4;
mp4_buffer_size 1M;
mp4_max_buffer_size 5M;
}
}

# Модуль Advanced Help делает доступным файлы README,
# предоставленные всеми модулями.
location ~ ^/help/ {
    location ~* ^/help/[~/*]/README\.txt$ {
        include fastcgi_private_files.conf;
        fastcgi_pass 127.0.0.1:9000;
    }
}

# Копируем директиву Apache <FilesMatch> для стандартного для Drupal файла
# .htaccess.
# Запрещаем доступ ко всем файлам, содержащим код.
# Возвращаем ошибку 404, чтобы не раскрывать информацию.
# Также скрываем текстовые файлы.
location ~* ^(?:.+\.?(?:htaccess|make|txt|engine|inc|info|
install|module|profile|po|sh|.*sql|test|theme|tpl(?:\.(php)?|xhtml)|
code-style|.pl|Entries.*|Repository|Root|/Tag|Template)$ {
    return 404;
}

# Сначала пробуем URI и перебрасываем на /index.php?q=$uri&$args,
# если не найдено.
try_files $uri @drupal;

## (Drupal 6) Сначала пробуем URI и перебрасываем на
# /index.php?q=$no_slash_uri&$args, если не найдено
# (оставить что-то одно).
try_files $uri /index.php?q=$no_slash_uri&$args;

```

```

} # конец местоположения по умолчанию

# Ограничиваем доступ к обязательным файлам PHP. Сужаем
# область воздействия эксплойтов. Обработка PHP-кода и
# цикла событий Drupal.
location @drupal {
    # Включаем конфигурацию FastCGI.
    include fastcgi_drupal.conf;
    fastcgi_pass 127.0.0.1:9000;
}

location @drupal-no-args {
    include fastcgi_private_files.conf;
    fastcgi_pass 127.0.0.1:9000;
}

## (Drupal 6)
## Ограничиваем доступ к обязательным файлам PHP. Сужаем
## область воздействия эксплойтов. Обработка PHP-кода и
## цикла событий Drupal.
## (оставить что-то одно)
location = /index.php {

    # Помечен internal в качестве профилактической меры защиты.
    # Прямой доступ к файлу index.php запрещен; доступ возможен только
    # со стороны NGINX из других местоположений или в результате
    # внутренней переадресации.
    internal;
    fastcgi_pass 127.0.0.1:9000;
}

```

Все последующие местоположения возвращают 404, чтобы запретить доступ:

```

# Запретить доступ к каталогу .git: возвращаем 404, чтобы не
# раскрывать информацию.
location ~ /\.git { return 404; }

# Запретить доступ к каталогу patches.
location ~ /patches { return 404; }

# Запретить доступ к каталогу backup.
location ~ /backup { return 404; }

# Запретить протоколирование обращений к файлу robots.txt в журналах
# доступа.
location = /robots.txt {
    access_log off;
}

```

```
# Поддержка RSS-каналов.
location = /rss.xml {
    try_files $uri @drupal-no-args;
    ## (Drupal 6: оставить что-то одно)
    try_files $uri /index.php?q=$uri;
}

# Поддержка карты сайта в формате XML.
location = /sitemap.xml {
    try_files $uri @drupal-no-args;
    ## (Drupal 6: оставить что-то одно)
    try_files $uri /index.php?q=$uri;
}

# Поддержка favicon. Вернуть прозрачный GIF размером 1x1,
# если не существует.
location = /favicon.ico {
    expires 30d;
    try_files /favicon.ico @empty;
}

# Возвращает хранящийся в памяти прозрачный GIF размером 1x1.
location @empty {
    expires 30d;
    empty_gif;
}

# Все прочие попытки обратиться к PHP-файлам возвращают 404.
location ~* ^.+\.php$ {
    return 404;
}
} # конец серверного контекста
```

Включаемые директивой `include` файлы здесь для краткости не приводятся. Их можно найти в репозитории `perusio` на сайте GitHub, упомянутом в начале этого раздела.

Интеграция NGINX и uWSGI

Python **WSGI (Web Server Gateway Interface)** – спецификация интерфейса, формализованная в документе PEP-3333 (<http://www.python.org/dev/peps/pep-3333/>). Ее цель – описать «стандартный интерфейс между веб-серверами и веб-приложениями или каркасами на языке Python, с тем чтобы повысить переносимость приложения с одного веб-сервера на другой». Вследствие популярности в сообществе пользователей Python разработаны реализации спецификации WSGI на другие языки. Сервер uWSGI, хотя и не был написан спе-

циально для Python, позволяет запускать приложения, отвечающие этой спецификации. Протокол взаимодействия с сервером uWSGI называется uwsgi. Дополнительные сведения о сервере uWSGI, в том числе инструкции по установке, примеры конфигурационных файлов и список поддерживаемых языков, можно найти на страницах <http://projects.unbit.it/uwsgi/> и <https://github.com/unbit/uwsgi-docs>.

Модуль NGINX uwsgi можно настроить для взаимодействия с этим сервером с помощью директив, аналогичных рассмотренным выше директивам fastcgi_*. Большая их часть обладает той же семантикой, что в случае FastCGI, с тем очевидным различием, что их имена начинаются префиксом uwsgi_ вместо fastcgi_. Однако есть и несколько исключений – директивы uwsgi_modifier1, uwsgi_modifier2 и uwsgi_string. Первые две устанавливают, соответственно, первый или второй модификатор в заголовке пакета uwsgi. А директива uwsgi_string позволяет NGINX передать произвольную строку серверу uWSGI или любому другому серверу, совместимому с протоколом uwsgi и поддерживающему модификатор eval. Модификаторы – это специфическая особенность протокола uwsgi. Таблица допустимых модификаторов с описанием их назначения приведена на странице <http://uwsgi-docs.readthedocs.org/en/latest/Protocol.html>.

Пример конфигурации для Django

Django (<https://www.djangoproject.com/>) – это написанный на Python каркас для быстрой разработки высокопроизводительных веб-приложений. Он получил широкое распространение, и теперь для него существуют веб-приложения самых разных видов.

Ниже приведен пример конфигурационного файла, демонстрирующего подключение NGINX к нескольким приложениям для Django, работающим под управлением сервера uWSGI в режиме Emperor с включенным модулем FastRouter. Дополнительные сведения о такой эксплуатации uWSGI – см. ссылки в комментариях.

```
http {
    # запустить сервер uWSGI, к которому мы будем подключаться
    # uwsgi --master --emperor /etc/djangoapps --fastrouter 127.0.0.1:3017
    # --fastrouter-subscription-server 127.0.0.1:3032
    # см. http://uwsgi-docs.readthedocs.org/en/latest/Emperor.html
    # и http://projects.unbit.it/uwsgi/wiki/Example
    upstream emperor {
        server 127.0.0.1:3017;
    }
```

```
server {
    # при задании корня документов используется переменная, чтобы
    # можно было обслуживать несколько сайтов, - отметим, что все
    # статические файлы должны находиться в подкаталоге "static",
    # а все закачанные пользователями файлы - в подкаталоге "media"
    # см. https://docs.djangoproject.com/en/dev/howto/static-files/
    root /home/www/sites/$host;

    location / {
        # CSS-файлы находятся в подкаталоге "styles"
        location ~* ^.\. $ {
            root /home/www/sites/$host/static/styles;
            expires 30d;
        }

        # если путь не найден в корне документов, то запрос
        # передается Django, работающему под управлением uWSGI
        try_files $uri @django;
    }

    location @django {
        # переменная $document_root должна указывать на код приложения
        root /home/www/apps/$host;

        # файл uwsgi_params входит в состав дистрибутива nginx
        include uwsgi_params;

        # ссылаемся на определенную выше секцию upstream
        # сервер uWSGI работает в режиме Emperor с модулем FastRouter
        uwsgi_param UWSGI_FASTROUTER_KEY $host;
        uwsgi_pass emperor;
    }

    # файл robots.txt находится в подкаталоге "static"
    # задание точного соответствия ускоряет поиск
    location = /robots.txt {
        root /home/www/sites/$host/static;
        access_log off;
    }

    # еще одно точное соответствие
    location = /favicon.ico {
        error_page 404 = @empty;
        root /home/www/sites/$host/static;
        access_log off;
        expires 30d;
    }

    # генерируем пустое изображение, на которое есть ссылка выше
    location @empty {
```

```
        empty_gif;
    }

    # попытка напрямую обратиться к файлу с расширением '.py'
    # приводит к ошибке 404 (Not Found)
    location ~* ^.+\.py$ {
        return 404;
    }
}
```

При такой конфигурации можно динамически обслуживать несколько сайтов, ничего не меняя в настройках.

Резюме

В этой главе мы рассмотрели директивы NGINX, описывающие порядок обслуживания запросов по протоколу HTTP. Эту функциональность предоставляет не только сам модуль http, но и ряд вспомогательных модулей, необходимых для нормальной работы NGINX. По умолчанию эти модули включены. Комбинирование директив, принадлежащих разным модулям, позволяет построить конфигурацию, отвечающую нашим требованиям. Мы показали, как NGINX ищет файлы, соответствующие запрошенному URI. Мы обсудили различные директивы, управляющие взаимодействием HTTP-сервера с клиентами, и узнали о различных способах применения директивы `error_page`. Мы убедились, что NGINX позволяет ограничивать доступ, основываясь на потреблении полосы пропускания, скорости отдачи содержимого и количестве соединений.

Мы также видели, как разрешить доступ только с определенных IP-адресов или пользователям, прошедшим аутентификацию. Мы рассмотрели средства протоколирования NGINX и разобрались, как включать в журналы лишь интересующую нас информацию. Мы также бегло остановились на псевдопоточковой передаче. NGINX предоставляет ряд переменных, которые можно использовать в конфигурационном файле. Была описана возможность использования модуля `fastcgi` для подключения к PHP-FPM-приложениям и модуля `uwsgi` для взаимодействия с сервером uWSGI. В примерах было продемонстрировано применение директив, обсуждавшихся как в этой, так и в других главах.

В следующей главе мы познакомимся с модулями, которые помогают интегрировать NGINX с собственным приложением.



Глава 7. NGINX для разработчика

До сих пор мы рассматривали настройку NGINX для решения различных задач. Но пока ни слова не было сказано о том, что NGINX предлагает разработчикам приложений. Существует несколько способов интегрировать NGINX с приложением. Мы обсудим их в следующих разделах.

- ☐ Интеграция с механизмом кэширования.
- ☐ Динамическое изменение содержимого.
- ☐ Включение на стороне сервера.
- ☐ Принятие решений в NGINX.
- ☐ Создание безопасной ссылки.
- ☐ Генерация изображений.
- ☐ Отслеживание посетителей сайта.
- ☐ Предотвращение случайного выполнения кода.

Интеграция с механизмом кэширования

NGINX великолепно справляется с обслуживанием статического содержимого. Он спроектирован для поддержки свыше 100 000 одновременных соединений при минимальном потреблении системных ресурсов. Интеграция динамического веб-приложения с таким замечательным сервером может резко повысить его производительность. Возможно, мы не сумеем поддержать так много одновременных соединений, но уменьшить время реакции системы на действия пользователя мы все-таки сможем.

О кэшировании мы рассказывали в главе 5 «Обратное проксирование, дополнительные вопросы». В этом разделе мы более подробно остановимся на интеграции механизма кэширования NGINX с веб-приложением. Возможно, ваше приложение уже умеет что-то кэшировать. Быть может, оно записывает заранее сформированные

страницы в базу данных, чтобы не выполнять накладную процедуру построения страницы снова и снова. Или, что еще лучше, сохраняет сформированные страницы в файловой системе, позволяя NGINX проявить свои поразительные способности в деле обслуживания статических файлов. Какой бы механизм кэширования ни использовался в вашем приложении (даже если его нет вовсе), NGINX позволит интегрировать его с сервером.

Приложения без кэширования

Даже если приложение вообще ничего не кэширует, NGINX все равно может ускорить время реакции. Поскольку модули `proxy` и `fastcgi` пользуются механизмом кэширования, то вам остается настроить кэширование для своего приложения с помощью директив `proxy_cache_*` или `fastcgi_cache_*`. Директивы `proxy_cache_*` описаны в разделе «Кэширование» главы 5, а директивы `fastcgi_cache_*` — в главе 6.

Здесь мы расскажем о том, как ваше приложение может попросить NGINX кэшировать отдельные страницы. Для этого нужно отправить NGINX определенные заголовки. Можно использовать как стандартные заголовки `Expires` или `Cache-Control`, так и специальный заголовок `X-Accel-Expires`, который NGINX интерпретирует самостоятельно и не передает клиенту. Этот заголовок позволяет приложению точно управлять временем кэширования файла и, в частности, прекратить кэширование объектов, которые обычно находятся в кэше очень долго.

Предположим, что имеется новостное приложение, страницы которого загружаются слишком медленно. Причин может быть много, но анализ показал, что каждая страница всякий раз строится заново, что сопровождается чтением содержимого из базы данных. Поэтому, когда заходит пользователь, приложение должно подключиться к базе данных, выполнить несколько SQL-запросов, разобрать результаты и только потом отправить сформированную страницу. Из-за многочисленных связей с системой, стоящей за этим веб-приложением, изменить архитектуру и реализовать более разумную стратегию построения страниц не так-то просто.

С учетом имеющихся ограничений вы решаете остановиться на следующей стратегии кэширования.

- ❑ Лицевая страница должна кэшироваться одну минуту, поскольку содержит часто изменяющийся список ссылок на статьи.
- ❑ Каждая статья кэшируется один день, поскольку, будучи опубликована, она уже не изменяется, но вместе с тем мы не хотим

забивать кэш старыми данными, которые придется удалять ввиду нехватки места.

- ❑ Изображения кэшируются столько времени, сколько возможно, поскольку они хранятся в базе данных и извлекать их оттуда накладно.

Для поддержки этой стратегии мы сконфигурируем NGINX следующим образом:

```
http {
    # настраиваем две зоны памяти для ключей и метаданных и пути к
    # каталогам, в которых будут храниться сами кэшированные объекты
    proxy_cache_path /var/spool/nginx/articles keys_zone=ARTICLES:16m
    levels=1:2 inactive=1d;
    proxy_cache_path /var/spool/nginx/images keys_zone=IMAGES:128m levels=1:2
    inactive=30d;

    # но оба каталога находятся в той же файловой системе,
    # что и proxy_temp_path
    proxy_temp_path /var/spool/nginx;

    server {
        location / {
            # здесь находится список статей
            proxy_cache_valid 1m;
        }

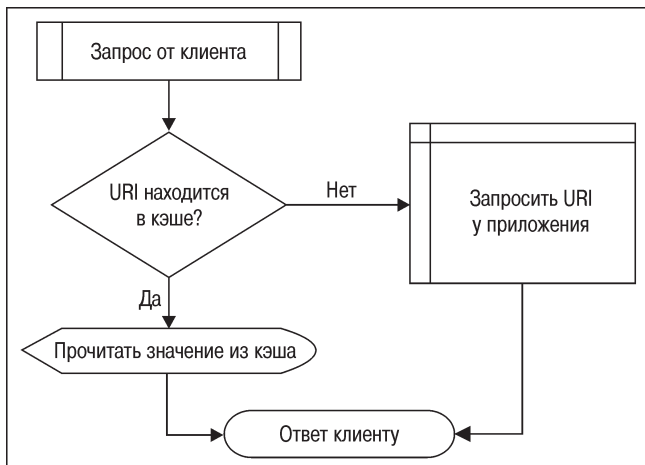
        location /articles {
            # URL каждой статьи начинается с «/articles»
            proxy_cache_valid 1d;
        }

        location /img {
            # URL любого изображения начинается с «/img»
            proxy_cache_valid 10y;
        }
    }
}
```

Тем самым наши требования удовлетворены. Мы реализовали кэширование для унаследованной системы, которая кэширования не поддерживала.

Кэширование в базе данных

Если приложение кэширует сформированные страницы в базе данных, то, скорее всего, будет нетрудно хранить их в кэше memcached. NGINX может обслуживать запросы непосредственно из этого кэша. Такая логика показана на рисунке ниже.



Интерфейс очень прост, но может быть настолько гибким, насколько необходимо. NGINX ищет ключ в хранилище. Если находит, то возвращает значение клиенту. Построение правильного ключа – задача настройки, которую мы обсудим ниже. Сохранение ассоциированного с ключом значения – задача не NGINX, а приложения.

Решить, какой ключ использовать, не слишком сложно. Для неперсонализированных ресурсов лучше всего использовать сам URI. Ключ записывается в переменную `$memcached_key`:

```
location / {
    set $memcached_key $uri;
    memcached_pass 127.0.0.1:11211;
}
```

Если для построения страницы приложение читает аргументы запроса, то их нужно включать и в ключ `$memcached_key`:

```
location / {
    set $memcached_key «$uri?$args»;
    memcached_pass 127.0.0.1:11211;
}
```

Если ключ не найден, то NGINX нужны какие-то средства для запроса страницы у приложения. Хочется надеяться, что приложение затем поместит пару ключ-значение в кэш memcached, чтобы следующий запрос можно было обслужить из памяти. NGINX вернет ошибку «Not Found», если ключ отсутствует в memcached, поэтому самый правильный способ переадресовать запрос приложению –

воспользоваться директивой `error_page` и завести специальное место-положение для обработки запроса. Мы должны включить сюда же коды ошибок «Bad Gateway» и «Gateway Timeout» на случай, если `memcached` не отвечает на запрос о наличии ключа:

```
server {  
    location / {  
        set $memcached_key "$uri?$args";  
        memcached_pass 127.0.0.1:11211;  
        error_page 404 502 504 = @app;  
    }  
  
    location @app {  
        proxy_pass 127.0.0.1:8080;  
    }  
}
```

Напомним, что, увидев знак равенства (=) в аргументах директивы `error_page`, NGINX подставит код, возвращенный страницей, которая указана в последнем аргументе. Это позволяет преобразовать ошибку в нормальный ответ.

В таблице ниже описаны директивы, относящиеся к модулю `memcached`, который включается в `nginx` по умолчанию:

Директивы модуля `memcached`

Директива	Описание
<code>memcached_buffer_size</code>	Размер буфера для ответа от <code>memcached</code> . Этот ответ синхронно отправляется клиенту
<code>memcached_connect_timeout</code>	Максимальное время, в течение которого NGINX ожидает установления соединения при отправке запроса серверу <code>memcached</code>
<code>memcached_next_upstream</code>	Определяет условия, при которых для ответа будет выбран следующий сервер <code>memcached</code> . Условия задаются с помощью следующих параметров: • <code>error</code>: произошла ошибка при взаимодействии с сервером <code>memcached</code>; • <code>timeout</code>: произошел тайм-аут при взаимодействии с сервером <code>memcached</code>; • <code>invalid_response</code>: сервер <code>memcached</code> вернул пустой или недопустимый ответ; • <code>not_found</code>: ключ отсутствует в кэше <code>memcached</code>; • <code>off</code>: запретить передачу запроса следующему серверу <code>memcached</code>
<code>memcached_pass</code>	Определяет имя или адрес сервера <code>memcached</code> и его порт. Можно также указывать группу серверов, объявленную в контексте <code>upstream</code>

Директива	Описание
<code>memcached_read_timeout</code>	Сколько времени может пройти между двумя последовательными операциями чтения данных от сервера <code>memcached</code> , прежде чем соединение будет закрыто
<code>memcached_send_timeout</code>	Сколько времени может пройти между двумя последовательными операциями записи данных на сервер <code>memcached</code> , прежде чем соединение будет закрыто

Кэширование в файловой системе

Предположим, что приложение записывает сформированные страницы в файлы. Поскольку вы знаете, сколько времени файл остается актуальным, то можете настроить NGINX так, чтобы он отправлял клиенту вместе с файлом заголовки, содержащие инструкции самому клиенту и прокси-серверам на пути к нему о том, как долго хранить файл в кэше. Там самым вы разрешите использовать локальный кэш клиента, не изменив ни единой строки кода в приложении.

Для этого следует настроить заголовки `Expires` и `Cache-Control`. Это стандартные HTTP-заголовки, которые понимают клиенты и прокси-серверы. В приложение никаких изменений вносить не нужно, достаточно прописать эти заголовки в конфигурационном блоке NGINX для соответствующего местоположения. NGINX упрощает эту задачу, предоставляя директивы `expires` и `add_header`.

Директивы для модификации заголовков

Директива	Описание
<code>add_header</code>	Добавляет заголовок в ответ с кодом состояния 200, 204, 206, 301, 302, 303, 304 или 307
<code>expires</code>	Добавляет или изменяет заголовки <code>Expires</code> и <code>Cache-Control</code> . Может быть задан необязательный параметр <code>modified</code> , за которым следует время (параметр <code>time</code>) или одно из ключевых слов: <code>epoch</code> , <code>max</code> либо <code>off</code> . Если присутствует только <code>time</code> , то значением в заголовке <code>Expires</code> будет текущее время плюс время, указанное в параметре <code>time</code> . Значение в заголовке <code>Cache-Control</code> будет равно <code>max-age=t</code> , где <code>t</code> – время в секундах, указанное в качестве аргумента. Если параметру <code>time</code> предшествует параметр <code>modified</code> , то в заголовок <code>Expires</code> записывается время модификации файла плюс время, указанное в параметре <code>time</code> . Если параметр <code>time</code> содержит знак @, то указанное время интерпретируется как время суток; например, <code>@12h</code> – это полдень. Параметр <code>epoch</code> соответствует абсолютному времени <code>Thu, 01 Jan 1970 00:00:01 GMT</code> . Если задан параметр <code>max</code> , то в заголовок <code>Expires</code> записывается значение <code>Thu, 31 Dec 2037 23:55:55 GMT</code> , а <code>Cache-Control</code> устанавливается на 10 лет. Отрицательное значение времени приводит к записи в <code>Cache-Control</code> значения <code>no-cache</code>

Зная, как поступать с файлами, которые генерирует ваше приложение, вы можете правильно настроить эти заголовки. Возьмем в качестве примера приложение, для которого главная страница должна кэшироваться 5 минут, все JavaScript и CSS-файлы – 24 часа, все HTML-страницы – 3 дня, а изображения – так долго, как возможно.

```
server {
    root /home/www;

    location / {
        # index.html сопоставляется явно, так что регулярное выражение
        # *.html ниже не будет соответствовать главной странице
        location = /index.html {
            expires 5m;
        }

        # для файлов с расширениями .js или .css (Javascript и CSS)
        location ~* /\.*(js|css)$ {
            expires 24h;
        }

        # все страницы с расширением .html
        location ~* /\.*.html$ {
            expires 3d;
        }
    }

    # все изображения находятся в отдельном местоположении (/img)
    location /img {
        expires max;
    }
}
```

Чтобы понять, как при такой конфигурации устанавливаются заголовки, посмотрим, как выглядит каждое местоположение в браузере. В любом современном браузере имеется встроенное или подключаемое дополнительно средство для просмотра заголовков запроса и ответа. На снимках экрана ниже показано, как заголовки ответов для разных местоположений отображаются в Chrome.

- ❑ **Главная страница (index.html):** в заголовке Expires установлено время, которое на 5 минут позже того, что указано в заголовке Date. Параметр max-age в заголовке Cache-Control равен 300 секунд.

```
▼ Response Headers view parsed
HTTP/1.1 200 OK
Server: nginx/1.2.2
Date: Sat, 15 Dec 2012 19:01:33 GMT
Content-Type: text/html
Content-Length: 170
Last-Modified: Sat, 15 Dec 2012 18:31:41 GMT
Connection: keep-alive
Expires: Sat, 15 Dec 2012 19:06:33 GMT
Cache-Control: max-age=300
Accept-Ranges: bytes
```

- ❑ **CSS-файл:** в заголовке Expires установлено время, которое на 24 часа позже того, что указано в заголовке Date. Параметр max-age в заголовке Cache-Control равен 86 400 секунд.

```
▼ Response Headers view parsed
HTTP/1.1 200 OK
Server: nginx/1.2.2
Date: Sat, 15 Dec 2012 19:07:43 GMT
Content-Type: text/plain
Content-Length: 69
Last-Modified: Sat, 15 Dec 2012 18:31:33 GMT
Connection: keep-alive
Expires: Sun, 16 Dec 2012 19:07:43 GMT
Cache-Control: max-age=86400
Accept-Ranges: bytes
```

- ❑ **HTML-файл:** в заголовке Expires установлено время, которое на 3 дня позже того, что указано в заголовке Date. Параметр max-age в заголовке Cache-Control равен 259 200 секунд.

```
▼ Response Headers view parsed
HTTP/1.1 200 OK
Server: nginx/1.2.2
Date: Sat, 15 Dec 2012 19:10:16 GMT
Content-Type: text/html
Content-Length: 170
Last-Modified: Sat, 15 Dec 2012 18:39:12 GMT
Connection: keep-alive
Expires: Tue, 18 Dec 2012 19:10:16 GMT
Cache-Control: max-age=259200
Accept-Ranges: bytes
```

- ❑ **Изображение:** в заголовке Expires установлено время Thu, 31 Dec 2037 23:55:55 GMT. Параметр max-age в заголовке Cache-Control равен 315 360 000 секунд.

```
▼ Response Headers    view parsed
HTTP/1.1 200 OK
Server: nginx/1.2.2
Date: Sat, 15 Dec 2012 19:07:43 GMT
Content-Type: image/jpeg
Content-Length: 26246
Last-Modified: Sat, 15 Dec 2012 18:28:41 GMT
Connection: keep-alive
Expires: Thu, 31 Dec 2037 23:55:55 GMT
Cache-Control: max-age=315360000
Accept-Ranges: bytes
```

Поместив всего лишь одну директиву expires в нужное место, мы можем настроить кэширование файлов в течение необходимого времени.

Динамическое изменение содержимого

Иногда бывает полезно подвергнуть дополнительной обработке данные, поступившие от приложения. Быть может, мы хотели бы добавить на страницу строку, показывающую, какой фронтальный сервер доставил эту страницу клиенту. Или каким-либо образом преобразовать уже сформированную страницу. NGINX предлагает три модуля, которые могут быть в этом смысле полезны: addition, sub и xslt.

Модуль addition

Модуль addition работает как фильтр, который добавляет текст до и (или) после ответа. По умолчанию он не компилируется, поэтому если вам нужна эта функция, то ее надо включить на этапе конфигурирования с помощью параметра `--with-http_addition_module`.

Этот фильтр ссылается на подзапрос, результат которого добавляется в начало или в конец ответа:

```
server {
    root /home/www;

    location / {
        add_before_body /header;
```

```
    add_after_body /footer;
}

location /header {
    proxy_pass http://127.0.0.1:8080/header;
}

location /footer {
    proxy_pass http://127.0.0.1:8080/footer;
}
}
```

Директивы модуля `addition` перечислены в таблице ниже.

Директивы модуля `addition`

Директива	Описание
<code>add_before_body</code>	Поместить результат обработки подзапроса перед телом ответа
<code>add_after_body</code>	Поместить результат обработки подзапроса после тела ответа
<code>addition_types</code>	Указывается список MIME-типов ответа (в дополнение к <code>text/html</code>), для которых производится добавление. Звездочка (*) означает все MIME-типы

Модуль `sub`

Модуль `sub` работает как фильтр, который заменяет один текст другим. По умолчанию он не компилируется, поэтому если вам нужна эта функция, то ее надо включить на этапе конфигурирования с помощью параметра `--with-http_sub_module`.

Работать с ним несложно. В директиве `sub_filter` задается заменяемая и заменяющая строка, а фильтр ищет первую строку без учета регистра и подставляет вместо нее вторую:

```
location / {
    sub_filter </head> '<meta name=>frontend< meta name=>web3<';
}
```

Здесь мы «на проходе» через NGINX добавили новый тег `meta` в заголовок страницы.

Можно также искать все совпадения. Для этого добавьте директиву `sub_filter_once`, указав в ней параметр `off`. Это может быть, например, полезно для замены всех относительных ссылок на странице абсолютными:

```
location / {
    sub_filter_once off;
    sub_filter '<img src=>img/' '<img src=>img/';
}
```

Если искомая строка содержит пробелы или кавычки, то ее необходимо заключить в кавычки, чтобы NGINX распознал ее всю как первый параметр.

NGINX автоматически применяет директиву `sub_filter` к любому HTML-файлу. Если же требуется выполнить подстановку в файлах других типов, например JavaScript или CSS, то следует указать соответствующий MIME-тип с помощью директивы `sub_filter_types`.

```
location / {
    sub_filter_types text/css;
    sub_filter url(img/ 'url(/img/';
}
```



Поскольку тип `text/html` подразумевается по умолчанию, добавлять его не нужно – он не пропадет, если указать дополнительные MIME-типы. Это справедливо для всех директив задания MIME-типов, встречающихся в NGINX.

Все вышеупомянутые директивы сведены в следующую таблицу.

Директивы модуля `sub`

Директива	Описание
<code>sub_filter</code>	Задаёт заменяемую строку (без учета регистра) и заменяющую строку. Заменяющая строка может содержать переменные
<code>sub_filter_once</code>	Если параметр равен <code>off</code> , то директива <code>sub_filter</code> применяется ко всем вхождениям заменяемой строки
<code>sub_filter_types</code>	Указывается список MIME-типов ответа (в дополнение к <code>text/html</code>), для которых производится замена. Звездочка (*) означает все MIME-типы

Модуль `xslt`

Модуль `xslt` работает как фильтр, который преобразует XML-документ с помощью таблиц стилей XSLT. По умолчанию он не компилируется, поэтому если вам нужна эта функция, то необходимо установить библиотеки `libxml2` и `libxslt` и включить модуль на этапе конфигурирования с помощью параметра `--with-http_xslt_module`.

Для использования модуля `xslt` следует определить DTD-схему, в которой объявлены знаковые сущности. После этого задается одна или несколько таблиц стилей XSLT для обработки XML-документа вместе с параметрами:

```
location / {
    xml_entities /usr/local/share/dtd/entities.dtd;
    xsl_stylesheet /usr/local/share/xslt/style1.xslt;
    xsl_stylesheet /usr/local/share/xslt/style2.xslt theme=blue;
}
```

Директивы модуля `xslt` перечислены в следующей таблице.

Директивы модуля `xslt`

Директива	Описание
<code>xml_entities</code>	Путь к файлу DTD-схемы, где объявлены знаковые сущности, на которые есть ссылки в обрабатываемом XML-файле
<code>xslt_param</code>	Параметры, передаваемые в таблицы стилей; значениями являются выражения XPath
<code>xslt_string_param</code>	Параметры, передаваемые в таблицы стилей; значениями являются строки
<code>xslt_stylesheet</code>	Путь к таблице стилей XSLT, применяемой для преобразования XML-ответа. Можно передать параметры в виде списка пар ключ-значение
<code>xslt_types</code>	Указывается список MIME-типов ответа (в дополнение к <code>text/xml</code>), для которых производится замена. Звездочка (*) означает все MIME-типы. Если результатом преобразования является HTML-документ, то его MIME-тип будет заменен на <code>text/html</code>

Включение на стороне сервера

Модуль `ssi` также является фильтром, причем одним из самых гибких. Он разрешает использовать технологию Server Side Includes для встраивания логики обработки в веб-страницу. Поведение модуля управляется следующими директивами.

Директивы модуля `ssi`

Директива	Описание
<code>ssi</code>	Разрешает обработку SSI-файлов
<code>ssi_silent_errors</code>	Подавляет сообщение, которое обычно выводится в случае ошибки во время обработки SSI
<code>ssi_types</code>	Указывается список MIME-типов ответа (в дополнение к <code>text/html</code>), для которых обрабатываются команды SSI. Звездочка (*) означает все MIME-типы

Команды SSI, поддерживаемые NGINX, приведены в таблице ниже. Все они устроены по одному образцу:

```
<!--# команда параметр1=значение1 параметр2=значение2 ... -->
```

Команды SSI

Команда	Аргумент	Пояснение
block		Определяет секцию, на которую можно сослаться в команде include. Заканчивается командой <code><!--# endblock --></code>
	name	Имя блока
config		Устанавливает глобальные параметры, используемые во время обработки SSI
	errmsg	Задаёт строку, которая используется как сообщение об ошибке обработки SSI. По умолчанию [an error occurred while processing the directive]
	timefmt	Строка, передаваемая функции <code>strftime()</code> для форматирования временных меток в других командах. По умолчанию %A, %d-%b-%Y %H:%M:%S %Z
echo		Выводит значение переменной
	var	Имя выводимой переменной
	encoding	Метод кодирования переменной. Допустимы значения none, url и entity. По умолчанию entity
	default	Значение, которое выводится, если переменная не определена. Если не задано, по умолчанию выводится none
if		Вычисляет условие. Если true, то следующий далее блок включается. Глубина вложенности цепочки if, elsif, else и endif не более единицы
	Expr	Вычисляемое логическое выражение: • существование переменной (<code>expr=»\$var»</code>); • сравнение (<code>expr=»\$var = text»</code> или <code>expr=»\$var != text»</code>); • сопоставление с регулярным выражением (<code>expr=»\$var = /regex/»</code> или <code>expr=»\$var != /regex/»</code>)
include		Выводит результат подзапроса
	file	Имя включаемого файла
	virtual	URI подзапроса, результат которого включается
	stub	Блок, включаемый вместо пустого тела или в случае ошибки при обработке
	wait	Если этот параметр присутствует, то при наличии на одной странице нескольких команд include они будут обрабатываться последовательно
	set	Если подзапрос, указанный в параметре virtual, ссылается на местоположение с директивой proxy_pass или memcached_pass, то его результат можно сохранить в переменной, имя которой указано в параметре set

Команда	Аргумент	Пояснение
set		Создает переменную и присваивает ей значение
	var	Имя переменной
	value	Значение переменной

SSI-файл – это не что иное, как обычный HTML-файл с командами, помещенными внутрь комментариев. Таким образом, если для какого-то местоположения, содержащего такой файл, модуль ssi не активирован, то HTML-часть тем не менее будет отрисована, хотя страница окажется неполной.

Ниже приведен пример SSI-файла, в котором подзапросы используются для формирования верхнего и нижнего колонтитулов, а также меню:

```
<html>
  <head>
    <title>*** Тестовая страница SSI ***</title>
    <link rel="stylesheet" href="/css/layout.css" type="text/css"/>
    <!--# block name="boilerplate" -->
    <p>...</p>
    <!--# endblock -->
  </head>
  <body>
    <div id="header">
      <!--# include virtual="/render/header?page=$uri" stub="boilerplate" -->
    </div>
    <div id="menu">
      <!--# include virtual="/render/menu?page=$uri" stub="boilerplate" -->
    </div>
    <div id="content">
      <p>Здесь находится содержимое страницы.</p>
    </div>
    <div id="footer">
      <!--# include virtual="/render/footer?page=$uri" stub="boilerplate" -->
    </div>
  </body>
</html>
```

Команда stub применяется для отрисовки содержимого по умолчанию в случае, когда при обработке подзапроса произошла ошибка.

Если этих команд недостаточно для реализации задуманной логики, то можно воспользоваться модулем perl, который предоставляет встроенный интерпретатор языка Perl, – его должно хватить для решения практически любой задачи обработки или конфигурирования.

Принятие решений в NGINX

Иногда мы ловим себя на попытке использовать конфигурационные директивы NGINX совершенно непредусмотренными способами. С этим часто приходится сталкиваться в конфигурациях, где производится много проверок в попытке эмулировать некую логическую цепочку. В этом случае лучше воспользоваться встроенным в NGINX модулем `perl`. Он предоставляет в ваше распоряжение всю гибкость языка Perl для создания нужной конфигурации.

Модуль `perl` по умолчанию не компилируется, поэтому для его включения необходимо указать на этапе конфигурирования параметр `--with-http_perl_module`. Кроме того, сам интерпретатор Perl должен быть откомпилирован с флагами `-Dusemultiplicity=yes` (или `-Dusethreads=yes`) и `-Dusymalloc=no`. Когда NGINX перезагружает конфигурационный файл, происходит некоторая утечка памяти в модуле `perl`, и последний параметр в какой-то мере сглаживает эту проблему.

После включения в `nginx` встроенного интерпретатора Perl становятся доступны следующие директивы.

Директивы модуля `perl`

Директива	Описание
<code>perl</code>	Активирует обработчик Perl в данном местоположении. В аргументе указывается имя обработчика или строка, содержащая полный код подпрограммы
<code>perl_modules</code>	Определяет дополнительные пути поиска Perl-модулей
<code>perl_require</code>	Задаёт имя Perl-модуля, который должен загружаться при каждом изменении конфигурационного файла NGINX. Может встречаться несколько раз, если требуется указать несколько модулей
<code>perl_set</code>	Устанавливает Perl-обработчик, который присваивает значение переменной. В аргументе указывается имя обработчика или строка, содержащая полный код подпрограммы

В Perl-скриптах, используемых в конфигурационном файле NGINX, доступен объект `$r`, представляющий запрос. У этого объекта имеются следующие методы:

- ❑ `$r->args`: аргументы запроса;
- ❑ `$r->filename`: имя файла, на который ссылается URI;
- ❑ `$r->has_request_body(handler)`: этот обработчик вызывается, если у запроса имеется тело;
- ❑ `$r->allow_ranges`: разрешает использование в ответе диапазонов байтов;

- ❑ `$r->discard_request_body`: отбрасывает тело запроса;
- ❑ `$r->header_in(header)`: значение в указанном заголовке запроса;
- ❑ `$r->header_only`: говорит NGINX, что клиенту нужно вернуть только заголовки;
- ❑ `$r->header_out(header, value)`: записывает в указанный заголовок ответа указанное значение;
- ❑ `$r->internal_redirect(uri)`: производит внутреннюю переадресацию на указанный URI, после того как Perl-обработчик закончит выполнение;
- ❑ `$r->print(text)`: помещает в ответ клиенту указанный текст;
- ❑ `$r->request_body`: тело запроса, если оно умещается в памяти;
- ❑ `$r->request_body_file`: тело запроса, если оно сохранено во временном файле;
- ❑ `$r->request_method`: HTTP-метод запроса;
- ❑ `$r->remote_addr`: IP-адрес клиента;
- ❑ `$r->flush`: немедленно отправляет данные клиенту;
- ❑ `$r->sendfile(name[, offset[, length]])`: отправляет клиенту указанный файл, после того как Perl-обработчик закончит выполнение (можно задать необязательное смещение от начала и длину);
- ❑ `$r->send_http_header([type])`: отправляет клиенту заголовки ответа (можно задать необязательный тип содержимого);
- ❑ `$r->status(code)`: устанавливает в ответе HTTP-код состояния;
- ❑ `$r->sleep(millisecons, handler)`: взводит таймер на указанное количество миллисекунд. После срабатывания таймера NGINX выполнит обработчик, а до этого времени продолжает обработку других запросов;
- ❑ `$r->unescape(text)`: декодирует URI-кодированный текст;
- ❑ `$r->uri`: URI, указанный в запросе;
- ❑ `$r->variable(name[, value])`: либо возвращает именованную, локальную для запроса переменную, либо присваивает ей указанное значение.

Модуль `perl` можно использовать также в сочетании с SSI. В команде SSI можно следующим образом указать код, написанный на Perl:

```
<!--# perl sub=>module::function» arg=>parameter1» arg=>parameter2» ... -->
```

Рассмотрим пример работы с модулем `perl`. Наша цель – передать запрос проксируемому серверу, определяемому первой буквой

указанного в запросе URI-адреса. Это можно было бы сделать, определив несколько местоположений в NGINX, но при использовании Perl-обработчика конфигурация получается короче.

Первый шаг – определить действия в Perl-обработчике:

```
# upstreammapper.pm

# имя пакета
package upstreammapper;

# включаем определения методов объекта запроса в nginx
use nginx;

# эта подпрограмма вызывается из nginx
sub handler {
    my $r = shift;
    my @alpha = («a»..»z»);
    my %upstreams = ();

    # не мудрствуя лукаво, описываем соответствие между буквой и
    # IP-адресом, последний октет которого изменяется от 10 до 35
    foreach my $idx (0..$#alpha) {
        $upstreams{ $alpha[$idx] } = $idx + 10;
    }

    # создаем массив, содержащий все символы URI-адреса
    my @uri = split(/,,$r->uri);

    # чтобы можно было использовать первую букву в качестве ключа
    my $ip = «10.100.0.» . $upstreams{ $uri[1] };
    return $ip;
}

1;

__END__
```

Теперь настраиваем NGINX так, чтобы этот модуль использовался для определения подходящего сервера:

```
http {
    # путь относительно главного конфигурационного файла
    perl_modules perl/lib;
    perl_require upstreammapper.pm;

    # результат, возвращенный обработчиком, сохраняется в переменной $upstream
    perl_set $upstream upstreammapper::handler;
}
```

Передаем запрос нужному проксируемому серверу:

```
location / {  
    include proxy.conf;  
    proxy_pass http://$upstream;  
}  
}
```

Это очень простой пример реализации логики конфигурирования в Perl-обработчике. Но точно так же можно решить и другие подобные задачи.



Perl-обработчик должен быть максимально сфокусирован на решении конкретной задачи. Пока NGINX ожидает его завершения, весь рабочий процесс, в котором обрабатывается данный запрос, блокирован. Поэтому ввод-вывод и обращения к DNS следует выполнять вне Perl-обработчика.

Создание безопасной ссылки

Иногда требуется защитить часть содержимого сайта, но подключать для этого механизм полноценной аутентификации не хочется. Один из способов решить задачу – воспользоваться модулем `secure_link`. Для его включения в NGINX необходимо на этапе конфигурирования задать параметр `--with-http_secure_link`. В результате становятся доступными директива `secure_link_secret` и переменная `$secure_link`.

Принцип работы модуля `secure_link` заключается в вычислении MD5-свертки ссылки, конкатенированной с секретным словом. Если свертка совпадает с указанной в URI, то в переменную `$secure_link` записывается часть URI после свертки. В противном случае переменная `$secure_link` будет содержать пустую строку.

Одно из возможных применений – сгенерировать страницу со ссылками на скачиваемые файлы, используя секретное слово. Это слово записывается в конфигурационный файл NGINX и позволяет проверить, разрешен ли доступ к ссылкам. И слово, и сама страница время от времени меняются, чтобы воспрепятствовать доступу по сохраненной ссылке в будущем. Этот сценарий иллюстрируется в примере ниже.

Сначала выбираем секретное слово – `supersecret`. Затем генерируем MD5-свертку интересующих нас ссылок:

```
$ echo -n «alphabet_soup.pdfsupersecret» | md5sum
8082202b04066a49a1ae8da9ec4feba1 -
$ echo -n «time_again.pdfsupersecret» | md5sum
5b77faadb4f5886c2ffb81900a6b3a43 -
```

Теперь можно поместить ссылки в HTML-разметку:

```
<a href=»/downloads/8082202b04066a49a1ae8da9ec4feba1/alphabet_soup.
pdf»>alphabet soup</a>
<a href=»/downloads/5b77faadb4f5886c2ffb81900a6b3a43/time_again.pdf»>time
again</a>
```

Эти ссылки действительны, только если в директиве `secure_link_secret` в конфигурационном файле указано то же секретное слово, которое использовалось для генерации сверток:

```
# доступ к ресурсам, URI-адреса которых начинаются с /downloads/, защищен
location /downloads/ {
    # это строка, с помощью которой генерировались свертки
    secure_link_secret supersecret;

    # запретить доступ, если свертки не совпадают
    if ($secure_link = «») {
        return 403;
    }
    try_files /downloads/$secure_link =404;
}
```

Чтобы убедиться, что ссылка без свертки не работает, добавим на HTML-страницу еще одну ссылку:

```
<a href=»/downloads/bare_link.pdf»>bare link</a>
```

Попытка перейти по ней приводит к ошибке «403 Forbidden», как и ожидалось.



Описанная выше техника с применением модуля `secure_link` – лишь один из способов решения поставленной задачи. Даже в самой NGINX имеется альтернативный подход, описанный на странице <http://wiki.nginx.org/HttpSecureLinkModule>.

Генерация изображений

Вместо того чтобы самостоятельно писать модуль манипулирования изображениями, вы можете поручить NGINX некоторые простые преобразования: поворот, изменение размера или обрезание.

Чтобы воспользоваться этой функциональностью, необходимо установить библиотеку `libgd` и откомпилировать модуль `image_filter`, задав на этапе конфигурирования параметр `--with-http_image_filter_module`. В результате становятся доступными директивы, перечисленные в таблице ниже.



Библиотека GD (`libgd`) предназначена для манипулирования изображениями и написана на C. Она нередко используется в сочетании с такими языками программирования, как PHP или Perl, с целью генерации изображений для веб-сайтов. В NGINX модуль `image_filter` обращается к `libgd` для создания простого механизма изменения размера изображений, который будет рассмотрен в примере ниже.

Директивы модуля `image_filter`

Директива	Описание
<code>empty_gif</code>	Порождает прозрачный GIF размером 1×1 для данного местоположения
<code>image_filter</code>	Преобразует изображение в соответствии с одним из следующих параметров: • <code>off</code>: отключить преобразование изображений; • <code>test</code>: проверяет, что ответ представлен в формате GIF, JPEG или PNG. В противном случае возвращается ошибка 415 (Unsupported Media Type); • <code>size</code>: выводит информацию об изображении в формате JSON; • <code>rotate</code>: поворачивает изображение против часовой стрелки на угол 90, 180 или 270 градусов; • <code>resize</code>: пропорционально уменьшает изображение до указанных размеров. Если требуется уменьшить только по одному измерению, то в качестве второго можно указать «—». При использовании в сочетании с <code>rotate</code> поворот производится сначала. В случае ошибки возвращается код 415 (Unsupported Media Type); • <code>crop</code>: уменьшает изображение до размера большей из двух заданных сторон и обрезает лишние края по другой стороне. Если требуется уменьшить только по одному измерению, то в качестве второго можно указать “-”. При использовании в сочетании с <code>rotate</code> поворот производится сначала. В случае ошибки возвращается код 415 (Unsupported Media Type)
<code>image_filter_buffer</code>	Размер буфера для обработки изображений. Если потребуется больше памяти, сервер вернет ошибку 415 (Unsupported Media Type)

Директива	Описание
<code>image_filter_jpeg_quality</code>	Качество результирующего изображения в формате JPEG. Не рекомендуется указывать значение больше 95
<code>image_filter_sharpen</code>	Повышает резкость результирующего изображения на указанный процент
<code>image_filter_transparency</code>	Определяет, сохранять ли прозрачность при обработке изображений в форматах PNG и GIF. Подразумеваемое по умолчанию значение <code>on</code> означает сохранение прозрачности

Отметим, что директива `empty_gif` не является частью модуля `image_filter`, а включается в NGINX по умолчанию.

С помощью этих директив мы можем следующим образом построить механизм изменения размера изображений:

```
location /img {
    try_files $uri /resize/$uri;
}

location ~* /resize/(?.<name>.*)(?<width>[[:digit:]]*)
x(?<height>[[:digit:]]*)\.(?<extension>gif|jpe?g|png)$ {
    error_page 404 = /resizer/$name.$extension?width=$width&height=$ height;
}

location /resizer {
    image_filter resize $arg_width $arg_height;
}
```

Здесь мы сначала пытаемся вернуть изображение, запрошенное в URI. Если файла с таким именем нет, то мы переходим в местоположение `/resize`, где определено регулярное выражение, выделяющее из имени файла требуемый размер изображения. Обратите внимание на именованные запоминаемые группы, позволяющие создавать переменные с осмысленными именами. Затем выделенные размеры передаются в местоположение `/resizer`, так что мы имеем исходное имя файла в URI, а ширину и высоту – в виде именованных аргументов.

Теперь все это можно объединить с директивой `proxy_store` или `proxy_cache`, чтобы сохранить изображение с новыми размерами. Тогда при следующем обращении к тому же URI «дергать» модуль `image_filter` не понадобится:

```
server {
    root /home/www;

    location /img {
```

```
    try_files $uri /resize/$uri;
}

location /resize {
    error_page 404 = @resizer;
}

location @resizer {
    internal;
    proxy_pass http://localhost:8080$uri;
    proxy_store /home/www/img$request_uri;
    proxy_temp_path /home/www/tmp/proxy_temp;
}
}

server {
    listen 8080;
    root /home/www/img;

    location ~* /resize/(?<name>.*)(?<width>[[:digit:]]*)
x(?<height>[[:digit:]]*)\.(?<extension>gif|jpe?g|png)$ {
        error_page 404 = /resizer/$name.$extension?width=$width&height=$height;
    }

    location /resizer {
        image_filter resize $arg_width $arg_height;
    }
}
```

Из таблицы директив модуля `image_filter` видно, что любая ошибка приводит к возврату кода 415. Мы можем перехватить этот код и вернуть в этом случае пустой GIF, так что пользователь все-таки получит изображение, а не сообщение об ошибке:

```
location /thumbnail {
    image_filter resize 90 90;
    error_page 415 = @empty;
}

location = @empty {
    access_log off;
    empty_gif;
}
```

На параметре `size` модуля `image_filter` стоит остановиться особо. Если в некотором местоположении указан этот параметр, то возвращается не само изображение, а сведения о нем. Это может быть полезно, когда приложению нужно получить метаданные изображения, перед тем как производить изменение размера или обрезание:

```
location /img {
    image_filter size;
}
```

Результатом является JSON-объект такого вида:

```
{ «img» : { «width»: 150, «height»: 200, «type»: «png» } }
```

Отслеживание посетителей сайта

Модуль `userid` предлагает ненавязчивый способ отслеживать посетителей сайта. Он устанавливает куки, идентифицирующие уникальных клиентов. Значение куки сохраняется в переменной `$uid_set`. Когда пользователь снова зайдет на сайт со все еще действительными куки, значение куки можно будет получить из переменной `$uid_got`. Ниже приведен пример использования.

```
http {
    log_format useridget ' $remote_addr - $uid_got [$time_local] '
                        '"$request" $status $body_bytes_sent '
                        '"$http_referer" "$http_user_agent"';

    server {
        server_name .example.com;
        access_log logs/example.com-access.log useridget;

        userid on;
        userid_name uid;
        userid_domain example.com;
        userid_path /;
        userid_expires 365d;
        userid_p3p 'policyref="/w3c/p3p.xml", CP="CUR ADM OUR NOR STA NID"';
    }
}
```

Все эти директивы перечислены в таблице ниже.

Директивы модуля `userid`

Директива	Описание
<code>userid</code>	Активирует модуль в соответствии со следующими параметрами: • <code>on</code>: устанавливает куки версии 2 и записывает в журнал полученные куки; • <code>v1</code>: устанавливает куки версии 1 и записывает в журнал полученные куки; • <code>log</code>: отключает установку куков, но продолжает записывать их в журнал; • <code>off</code>: отключает как установку, так и протоколирование куков

Директива	Описание
userid_domain	Настраивает записываемый в куки домен
userid_expires	Задаёт срок хранения куки. Если указано значение max, то устанавливается дата 31 Dec 2037 23:55:55 GMT
userid_name	Задаёт имя куки (по умолчанию uid)
userid_p3p	Настраивает заголовок P3P для сайтов, которые объявляют свою политику конфиденциальности, следуя протоколу Platform for Privacy Preferences Project
userid_path	Задаёт путь, указываемый в куки
userid_service	Идентификатор службы, которая устанавливает куки. Например, для куки версии 2 по умолчанию подразумевается IP-адрес сервера, установившего куки

Предотвращение случайного выполнения кода

Иногда при разработке конфигурационного файла можно случайно включить не то, что предполагалось. Рассмотрим, к примеру, такой конфигурационный блок:

```
location ~* \.php {
    include fastcgi_params;
    fastcgi_pass 127.0.0.1:9000;
}
```

На первый взгляд, мы здесь просто передаем все запросы к РНР-файлам FastCGI-серверу, который отвечает за их разработку. И все было бы хорошо, если бы РНР обрабатывала только файлы, являющиеся частью приложения. Однако из-за различий в способе компиляции и настройки РНР так бывает не всегда. Это может оказаться проблемой, если загружаемые пользователями файлы попадают в ту же часть файловой системы, где находятся РНР-файлы.

Можно запретить пользователям загружать файлы с расширением .php, разрешив только расширения .jpg, .png и .gif. Но злонамеренный пользователь мог бы загрузить графический файл с внедренным PHP-кодом и заставить FastCGI-сервер выполнить этот код, передав URI, в котором указано имя загруженного файла.

Чтобы избежать этого, нужно либо присвоить параметру `RNP.cgi.fix_pathinfo` значение 0, либо модифицировать конфигурацию следующим образом:

```
location ~* \.php {  
    try_files $uri =404;  
    include fastcgi_params;  
    fastcgi_pass 127.0.0.1:9000;  
}
```

Здесь мы с помощью директивы `try_files` проверяем существование файла, перед тем как передавать запрос FastCGI-серверу для обработки PHP-приложением.



Не забывайте анализировать конфигурацию на предмет соответствия поставленным целям. Если имеется всего несколько файлов, то, быть может, лучше явно перечислить, какие PHP-файлы разрешено исполнять, чем описывать секцию `location` регулярным выражением и добавлять директивы `try_files`.

Резюме

NGINX предлагает различные способы интеграции приложения с высокопроизводительным веб-сервером. Мы рассмотрели несколько таких возможностей как для унаследованных, так и для вновь разрабатываемых приложений. Кэширование играет важную роль в любом современном веб-приложении. В NGINX реализованы пассивные и активные механизмы кэширования с целью более быстрого возврата веб-страниц.

Мы также показали, как с помощью NGINX изменить ответ, добавив или заменив текст. NGINX поддерживает также технологию включения на стороне сервера – SSI. Мы видели, как команды SSI вставляются в обычную HTML-разметку. Затем мы рассмотрели мощные возможности встроенного в NGINX интерпретатора Perl. С помощью стандартных средств NGINX возможно также выполнять преобразование изображений. Мы обсудили установку уникальных куков для отслеживания посетителей веб-сайта. И завершили главу предостережением – как предотвратить непреднамеренное выполнение кода. В общем и целом, используя NGINX в качестве веб-сервера, разработчик получает немало полезных инструментов.

В следующей главе мы поговорим о технике поиска неполадок и объясним, как поступать, когда что-то идет вразрез с ожиданиями.



Глава 8. Техника устранения неполадок

Наш мир несовершенен. Вопреки самым лучшим намерениям и тщательному планированию иногда что-то идет не так, как мы рассчитывали. Тогда нужно на шаг отступить и выяснить, что случилось. Если с первого взгляда причина неясна, то придется обратиться к методикам исследования проблемы. Процедура, позволяющая понять, в чем состоит ошибка и как ее исправить, и называется устранением неполадок.

В этой главе мы рассмотрим различные приемы устранения неполадок при работе с NGINX:

- ❑ Анализ журналов.
- ❑ Настройка расширенного протоколирования.
- ❑ Типичные ошибки конфигурирования.
- ❑ Ограничения операционной системы.
- ❑ Проблемы с производительностью.
- ❑ Использование модуля Stub Status.

Анализ журналов

Прежде чем приступить к длительному сеансу отладки, пытаясь понять, в чем причина проблемы, полезно заглянуть в журналы. Часто в них можно найти ключ, который поможет проследить истоки ошибки и исправить ее. Но сообщения в файле `error_log` иногда выглядят загадочно, поэтому обсудим формат журнальных записей и на примерах покажем, как их интерпретировать.

Форматы записей в журнале ошибок

Для записи в журнал `error_log` в NGINX применяются две разные функции. Форматы в них устроены следующим образом:

<временная метка> [уровень-протоколирования] <pid главного/рабочего процесса>#0: сообщение

Например:

```
2012/10/14 18:56:41 [notice] 2761#0: using inherited sockets from «6;»
```

Это пример информационного сообщения (уровень протоколирования `notice`). В нем говорится, что процесс `nginx` заменил исполнявшийся ранее и успешно унаследовал сокеты от старого процесса.

В случае ошибки порождается сообщение такого вида:

```
2012/10/14 18:50:34 [error] 2632#0: *1 open() «/opt/nginx/html/blog» failed
(2: No such file or directory), client: 127.0.0.1, server: www.example.com,
request: «GET /blog HTTP/1.0», host: «www.example.com»
```

В зависимости от ошибки может быть выведено либо сообщение операционной системы (как в данном случае), либо сообщение самого NGINX. В этом сообщении можно выделить следующие компоненты:

- ❑ временная метка (2012/10/14 18:50:34);
- ❑ уровень протоколирования (`error`);
- ❑ идентификатор рабочего процесса (2632);
- ❑ номер соединения (1);
- ❑ системный вызов (`open`);
- ❑ аргумент системного вызова (`/opt/nginx/html/blog`);
- ❑ сообщение об ошибке, возвращенное системным вызовом (2: No such file or directory);
- ❑ IP-адрес клиента, который отправил запрос, приведший к ошибке (127.0.0.1);
- ❑ сервер, обслуживавший данный запрос (`www.example.com`);
- ❑ сам запрос (`GET /blog HTTP/1.0`);
- ❑ заголовок `Host` запроса (`www.example.com`).

Ниже приведен пример критической ошибки:

```
2012/10/14 19:11:50 [crit] 3142#0: the changing binary signal is ignored: you
should shutdown or terminate before either old or new binary's process
```

Критическая ошибка означает, что NGINX не смог выполнить запрошенного действия. Если в этот момент NGINX еще не работал, значит, он и не запустился.

Ниже приведен пример сообщения о чрезвычайной ошибке:

```
2012/10/14 19:12:05 [emerg] 3195#0: bind() to 0.0.0.0:80 failed (98: Address
already in use)
```

Чрезвычайная ошибка также означает, что NGINX не смог сделать то, о чем его просили. Если это произошло при попытке запуска, NGINX не запустился, а если во время работы в ответ на требование перечитать конфигурационный файл – значит, запрошенное изменение не выполнено.



Если вас интересует, почему после модификации конфигурационного файла ничего не изменилось, загляните в журнал ошибок. Скорее всего, NGINX обнаружил ошибку в файле и не применил изменения.

Примеры записей в журнале ошибок

Ниже приведены примеры сообщений об ошибках, взятые из реальных журналов. После каждого примера дается краткое пояснение. Отметим, что в ваших журналах текст сообщения может отличаться в связи с улучшениями в новых версиях NGINX.

```
2012/11/29 21:31:34 [error] 6338#0: *1 upstream prematurely closed connection
while reading response header from upstream, client: 127.0.0.1, server: ,
request: «GET / HTTP/1.1», upstream: «fastcgi://127.0.0.1:8080», host: «www.
example.com»
```

Это сообщение можно интерпретировать двояко. Возможно, в сервере, с которым мы взаимодействуем, некорректно реализован протокол FastCGI. Но, быть может, мы по ошибке направили трафик HTTP-серверу вместо FastCGI-сервера. В последнем случае проблему можно решить простым изменением конфигурации (включить директиву `fastcgi_pass` вместо `proxy_pass` или правильно задать адрес FastCGI-сервера).

Это сообщение может также означать, что проксируемый сервер просто слишком долго генерировал ответ. Причин может быть масса, но на стороне NGINX решение очевидно: увеличить тайм-аут. В зависимости от того, какой модуль устанавливал соединение, нужно будет изменить подразумеваемое по умолчанию значение 60s в директиве `proxy_read_timeout` или `fastcgi_read_timeout` (или еще в какой-то директиве вида `*_read_timeout`).

```
2012/11/29 06:31:42 [error] 2589#0: *6437 client intended to send too large
body: 13106010 bytes, client: 127.0.0.1, server: , request: «POST /upload_file.
php HTTP/1.1», host: «www.example.com», referrer: «http:// www.example.com/
file_upload.html»
```

Здесь все просто. NGINX сообщает, что не смог закачать слишком большой файл. Чтобы решить проблему, нужно увеличить значение в директиве `client_body_size`. Имейте в виду, что из-за кодирования объем закачиваемых данных может процентов на 30 превышать размер самого файла (поэтому, если вы собираетесь разрешить пользователям закачивать файлы размером до 12 МБ, задавайте значение 16m).

```
2012/10/14 19:51:22 [emerg] 3969#0: «proxy_pass» cannot have URI part in
location given by regular expression, or inside named location, or inside «if»
statement, or inside «limit_except» block in /opt/nginx/conf/nginx.conf:16
```

Здесь мы видим, что NGINX не запустился из-за ошибки в конфигурационном файле. В чем дело, описано очень подробно. Проблема в том, что в директиве `proxy_pass` присутствует URL, которого в этом месте быть не должно. NGINX даже сообщает, в какой строке (16) какого файла (`/opt/nginx/conf/nginx.conf`) произошла ошибка.

```
2012/10/14 18:46:26 [emerg] 2584#0: mkdir() «/home/www/tmp/proxy_temp» failed
(2: No such file or directory)
```

В этом примере NGINX не запустился, потому что не смог сделать то, о чем его просили. В директиве `proxy_temp_path` указано, где сохранять временные файлы при проксировании. Если NGINX не сможет создать такой каталог, то не запустится, поэтому лучше создайте требуемый каталог заранее.

```
2012/10/14 18:46:54 [emerg] 2593#0: unknown directive «client_body_temp_path»
in /opt/nginx/conf/nginx.conf:6
```

А вот это сообщение озадачивает. Мы знаем, что директива `client_body_temp_path` существует, но NGINX почему-то не распознает ее. Но, поразмыслив, как NGINX обрабатывает конфигурационный файл, мы понимаем, что сообщение имеет смысл. NGINX построен по модульному принципу. Каждый модуль отвечает за обработку своего конфигурационного контекста. Отсюда можно сделать вывод, что эта директива оказалась вне контекста модуля, который ее понимает.

```
2012/10/16 20:56:31 [emerg] 3039#0: «try_files» directive is not allowed here
in /opt/nginx/conf/nginx.conf:16
```

Иногда NGINX подсказывает, что не так. В примере выше NGINX распознал директиву `try_files`, но говорит, что она находится не там, где надо. И при этом любезно сообщает, в каком месте конфигурационного файла произошла ошибка, так что найти ее не составит труда.

```
2012/10/16 20:56:42 [emerg] 3043#0: host not found in upstream «tickets.
example.com» in /opt/nginx/conf/nginx.conf:22
```

Это чрезвычайное сообщение показывает, насколько сильно NGINX зависит от DNS, если в конфигурационном файле употребляются доменные имена. Если NGINX не может разрешить имена, встречающиеся в директивах `upstream`, `proxy_pass`, `fastcgi_pass` или иных директивах вида `*_pass`, то он не запускается. Поэтому так важен порядок инициализации различных подсистем при загрузке операционной системы. Позаботьтесь о том, чтобы к моменту запуска NGINX подсистема разрешения доменных имен уже была запущена.

```
2012/10/29 18:59:26 [emerg] 2287#0: unexpected «}» in /opt/nginx/conf/ nginx.conf:40
```

Это сообщение означает, что NGINX не смог закрыть конфигурационный контекст. Что-то предшествующее указанной строке помешало NGINX сформировать полный контекст, заключенный в фигурные скобки. Обычно так бывает, когда предыдущая строка не заканчивается точкой с запятой, поэтому NGINX считает прочитанный символ `}` частью незавершенной строки.

```
2012/10/28 21:38:34 [emerg] 2318#0: unexpected end of file, expecting «}» in /
opt/nginx/conf/nginx.conf:21
```

Это сообщение тесно связано с предыдущим и означает, что NGINX дошел до конца конфигурационного файла, но не нашел парную закрывающую скобку. Такая ошибка случается, когда скобки `{` и `}` не сбалансированы. Текстовый редактор, отслеживающий соответственные скобки, поможет понять, какая отсутствует. Внимательно следите, куда вставляете отсутствующую скобку; если допустить ошибку, то можно получить совершенно не то, что ожидалось.

```
2012/10/29 18:50:11 [emerg] 2116#0: unknown «exclusion» variable
```

Это пример использования необъявленной переменной; сообщение означает, что переменная `$exclusion` встретилась в конфигурационном файле до директивы `set`, `map` или `geo`, в которой определено ее значение. Возможно, все объясняется простой опечаткой, например была определена переменная `$exclusions`, но по ошибке в ссылке фигурирует имя `$exclusion`.

```
2012/11/29 21:26:51 [error] 3446#0: *2849 SSL3_GET_FINISHED:digest check failed
```

Это означает, что нужно запретить повторное использование сессий SSL. Сделать это можно, задав в директиве `proxy_ssl_session_reuse` значение `off`.

Настройка расширенного протоколирования

При нормальных обстоятельствах мы хотим, чтобы журналы были как можно меньше. Обычно нас интересует, к каким URI-адресам обращались клиенты, когда это происходило и – в случае ошибки – сообщение о ней. Если же требуется больше информации, то придется включить отладочное протоколирование.

Отладочное протоколирование

Для активации отладочного протоколирования на этапе конфигурирования следует задать флаг `--with-debug--with-debug`. Поскольку делать это в производственной системе, которая должна работать максимально быстро, не рекомендуется, то нужно подготовить два варианта двоичного файла `nginx`: один для производственной системы и другой – откомпилированный точно с такими же параметрами и флагом `--with-debug`, – который можно будет подставить вместо первого, когда возникает необходимость в отладке.

Переключение двоичного файла во время выполнения

NGINX может переключать двоичный файл во время выполнения. Заменяв один файл `nginx` другим – то ли для перехода на новую версию, то ли вследствие желания изменить состав включенных модулей, – мы можем приступить к процедуре переключения.

1. Послать работающему главному процессу NGINX сигнал `USR2`, по которому тот запускает новый главный процесс. В результате PID-файл процесса будет переименован с добавлением суффикса `.oldbin` (например, `/var/run/nginx.pid.oldbin`):

```
# kill -USR2 `cat /var/run/nginx.pid`
```

В этот момент у нас есть два главных процесса NGINX, каждый со своим набором рабочих процессов для обработки поступающих запросов:

```
root 1149 0.0 0.2 20900 11768 ?? Is Fri03PM 0:00.13 nginx: master
process /usr/local/sbin/nginx
www 36660 0.0 0.2 20900 11992 ?? S 12:52PM 0:00.19 nginx: worker process
(nginx)
www 36661 0.0 0.2 20900 11992 ?? S 12:52PM 0:00.19 nginx: worker process
(nginx)
www 36662 0.0 0.2 20900 12032 ?? I 12:52PM 0:00.01 nginx: worker process
(nginx)
www 36663 0.0 0.2 20900 11992 ?? S 12:52PM 0:00.18 nginx: worker process
(nginx)
root 50725 0.0 0.1 18844 8408 ?? I 3:49PM 0:00.05 nginx: master process
/usr/local/sbin/nginx
www 50726 0.0 0.1 18844 9240 ?? I 3:49PM 0:00.00 nginx: worker process
(nginx)
www 50727 0.0 0.1 18844 9240 ?? S 3:49PM 0:00.01 nginx: worker process
(nginx)
www 50728 0.0 0.1 18844 9240 ?? S 3:49PM 0:00.01 nginx: worker process
(nginx)
www 50729 0.0 0.1 18844 9240 ?? S 3:49PM 0:00.01 nginx: worker process
(nginx)
```

2. Послать старому главному процессу NGINX сигнал `WINCH`, по которому тот прекращает прием новых запросов и уничтожает свои рабочие процессы по мере того, как они заканчивают обработку начатых запросов.

```
# kill -WINCH `cat /var/run/nginx.pid.oldbin`
```

В ответ мы получаем такие сообщения:

```
root 1149 0.0 0.2 20900 11768 ?? Ss Fri03PM 0:00.14 nginx: master
process /usr/local/sbin/nginx
root 50725 0.0 0.1 18844 8408 ?? I 3:49PM 0:00.05 nginx: master process
/usr/local/sbin/nginx
www 50726 0.0 0.1 18844 9240 ?? I 3:49PM 0:00.00 nginx: worker process
(nginx)
www 50727 0.0 0.1 18844 9240 ?? S 3:49PM 0:00.01 nginx: worker process
(nginx)
```

```
www 50728 0.0 0.1 18844 9240 ?? S 3:49PM 0:00.01 nginx: worker process
(nginx)
www 50729 0.0 0.1 18844 9240 ?? S 3:49PM 0:00.01 nginx: worker process
(nginx)
```

3. Послать старому главному процессу NGINX сигнал QUIT, после того как все его рабочие процессы завершились. После этого остается только новый `nginx`, который обрабатывает все запросы:
`kill -QUIT `cat /var/run/nginx.pid.oldbin``

Если в новом двоичном файле обнаружится какая-то ошибка, то мы можем вернуться к старому, если еще не успели послать ему сигнал QUIT:

```
# kill -HUP `cat /var/run/nginx.pid.oldbin`
# kill -QUIT `cat /var/run/nginx.pid`
```

Если новый главный процесс все еще работает, ему можно послать сигнал TERM для принудительного завершения:

```
# kill -TERM `cat /var/run/nginx.pid`
```

Аналогично новые рабочие процессы, продолжающие работать, могут быть остановлены сигналом KILL.



Некоторые операционные системы автоматически выполняют процедуру замены исполняемого файла после модернизации пакета `nginx`.

Запустив `nginx` с поддержкой отладки, мы можем настроить отладочное протоколирование:

```
user www;

events {
    worker_connections 1024;
}

error_log logs/debug.log debug;

http {
    ...
}
```

Директиву `error_log` мы поместили в главный контекст конфигурации NGINX, чтобы ее действие распространялось на все дочерние

контексты, где эта директива не переопределена. Можно включить несколько директив `error_log` с разными уровнями протоколирования, указывающих на разные файлы. Помимо `debug`, в директиве `error_log` могут быть заданы следующие параметры:

- ☐ `debug_core`
- ☐ `debug_alloc`
- ☐ `debug_mutex`
- ☐ `debug_event`
- ☐ `debug_http`
- ☐ `debug_imap`

Каждый из них предназначен для отладки одного модуля NGINX.

Имеет также смысл сконфигурировать отдельный журнал ошибок для каждого виртуального сервера, тогда в него будут попадать ошибки, относящиеся только к этому серверу. Эту идею можно обобщить на модули `core` и `http`:

```
error_log logs/core_error.log;

events {
    worker_connections 1024;
}

http {
    error_log logs/http_error.log;

    server {
        server_name www.example.com;
        error_log logs/www.example.com_error.log;
    }

    server {
        server_name www.example.org;
        error_log logs/www.example.org_error.log;
    }
}
```

Следуя этому образцу, мы можем отлаживать конкретный виртуальный сервер, если задача состоит именно в этом.

```
server {
    server_name www.example.org;
    error_log logs/www.example.org_debug.log debug_http;
}
```

Ниже приведен пример отладочных сообщений для одного запроса, когда задан уровень `debug_http`. Распечатка перемежается комментариями.

```
<timestamp> [debug] <worker pid>#0: *<connection number> http cl:-1 max:1048576
```

На ранней стадии обработки запроса активируется модуль `rewrite`:

```
<timestamp> [debug] <worker pid>#0: *<connection number> rewrite phase: 3
<timestamp> [debug] <worker pid>#0: *<connection number> post rewrite phase: 4
<timestamp> [debug] <worker pid>#0: *<connection number> generic phase: 5
<timestamp> [debug] <worker pid>#0: *<connection number> generic phase: 6
<timestamp> [debug] <worker pid>#0: *<connection number> generic phase: 7
```

Проверяются ограничения доступа:

```
<timestamp> [debug] <worker pid>#0: *<connection number> access phase: 8
<timestamp> [debug] <worker pid>#0: *<connection number> access: 0100007F
FFFFFFFF 0100007F
```

Далее разбирается директива `try_files`. Путем конкатенации строки (`http script copy`) и переменной (`http script var`), указанных в параметрах этой директивы, конструируется относительный путь к файлу:

```
<timestamp> [debug] <worker pid>#0: *<connection number> try files phase: 11
<timestamp> [debug] <worker pid>#0: *<connection number> http script copy: "/"
<timestamp> [debug] <worker pid>#0: *<connection number> http script var:
"ImageFile.jpg"
```

Результат вычисления конкатенируется с путем, заданным в директиве `alias` или `root` для данного местоположения, в результате чего получается полный путь:

```
<timestamp> [debug] <worker pid>#0: *<connection number> trying to use file: «/
ImageFile.jpg» «/data/images/ImageFile.jpg»
<timestamp> [debug] <worker pid>#0: *<connection number> try file uri: «/
ImageFile.jpg»
```

Обрабатывается содержимое найденного файла:

```
<timestamp> [debug] <worker pid>#0: *<connection number> content phase: 12
<timestamp> [debug] <worker pid>#0: *<connection number> content phase: 13
<timestamp> [debug] <worker pid>#0: *<connection number> content phase: 14
<timestamp> [debug] <worker pid>#0: *<connection number> content phase: 15
<timestamp> [debug] <worker pid>#0: *<connection number> content phase: 16
```

http filename ниже — это полный путь к отправляемому файлу:

```
<timestamp> [debug] <worker pid>#0: *<connection number> http filename: "/data/images/ImageFile.jpg"
```

Модуль static принимает дескриптор этого файла:

```
<timestamp> [debug] <worker pid>#0: *<connection number> http static fd: 15
```

Временно сохраненное тело ответа больше не нужно:

```
<timestamp> [debug] <worker pid>#0: *<connection number> http set discard body
```

Имея всю информацию о файле, NGINX может сконструировать заголовки ответа:

```
<timestamp> [debug] <worker pid>#0: *<connection number> HTTP/1.1 200 OK
Server: nginx/<version>
Date: <Date header>
Content-Type: <MIME type>
Content-Length: <filesize>
Last-Modified: <Last-Modified header>
Connection: keep-alive
Accept-Ranges: bytes
```

На следующей фазе применяются активные фильтры, которые могут производить различные преобразования:

```
<timestamp> [debug] <worker pid>#0: *<connection number> http write filter: 1:0
f:0 s:219
<timestamp> [debug] <worker pid>#0: *<connection number> http output filter «/
ImageFile.jpg?file=ImageFile.jpg»
<timestamp> [debug] <worker pid>#0: *<connection number> http copy filter: «/
ImageFile.jpg?file=ImageFile.jpg»
<timestamp> [debug] <worker pid>#0: *<connection number> http postpone filter
«/ImageFile.jpg?file=ImageFile.jpg» 00007FFF30383040
<timestamp> [debug] <worker pid>#0: *<connection number> http write filter: 1:1
f:0 s:480317
<timestamp> [debug] <worker pid>#0: *<connection number> http write filter
limit 0
<timestamp> [debug] <worker pid>#0: *<connection number> http write filter
0000000001911050
<timestamp> [debug] <worker pid>#0: *<connection number> http copy filter: -2
«/ImageFile.jpg?file=ImageFile.jpg»
<timestamp> [debug] <worker pid>#0: *<connection number> http finalize request:
-2, «/ImageFile.jpg?file=ImageFile.jpg» a:1, c:1
<timestamp> [debug] <worker pid>#0: *<connection number> http run request: «/
ImageFile.jpg?file=ImageFile.jpg»
```

```

<timestamp> [debug] <worker pid>#0: *<connection number> http writer handler:
«/ImageFile.jpg?file=ImageFile.jpg»
<timestamp> [debug] <worker pid>#0: *<connection number> http output filter «/
ImageFile.jpg?file=ImageFile.jpg»
<timestamp> [debug] <worker pid>#0: *<connection number> http copy filter: «/
ImageFile.jpg?file=ImageFile.jpg»
<timestamp> [debug] <worker pid>#0: *<connection number> http postpone filter
«/ImageFile.jpg?file=ImageFile.jpg» 0000000000000000
<timestamp> [debug] <worker pid>#0: *<connection number> http write filter: 1:1
f:0 s:234338
<timestamp> [debug] <worker pid>#0: *<connection number> http write filter
limit 0
<timestamp> [debug] <worker pid>#0: *<connection number> http write filter
0000000000000000
<timestamp> [debug] <worker pid>#0: *<connection number> http copy filter: 0 «/
ImageFile.jpg?file=ImageFile.jpg»
<timestamp> [debug] <worker pid>#0: *<connection number> http writer output
filter: 0, «/ImageFile.jpg?file=ImageFile.jpg»
<timestamp> [debug] <worker pid>#0: *<connection number> http writer done: «/
ImageFile.jpg?file=ImageFile.jpg»

```

После того как все фильтры отработали, обработка запроса завершается:

```

<timestamp> [debug] <worker pid>#0: *<connection number> http finalize request:
0, «/ImageFile.jpg?file=ImageFile.jpg» a:1, c:1

```

Обработчик `keepalive` смотрит, оставить ли соединение открытым:

```

<timestamp> [debug] <worker pid>#0: *<connection number> set http keepalive
handler
<timestamp> [debug] <worker pid>#0: *<connection number> http close request

```

Запрос обработан, можно поместить запись о нем в журнал:

```

<timestamp> [debug] <worker pid>#0: *<connection number> http log handler
<timestamp> [debug] <worker pid>#0: *<connection number> hc free:
0000000000000000 0
<timestamp> [debug] <worker pid>#0: *<connection number> hc busy:
0000000000000000 0
<timestamp> [debug] <worker pid>#0: *<connection number> tcp_nodelay

```

Клиент закрыл соединение, и NGINX – тоже:

```

<timestamp> [debug] <worker pid>#0: *<connection number> http keepalive handler
<timestamp> [info] <worker pid>#0: *<connection number> client <IP address>
closed keepalive connection
<timestamp> [debug] <worker pid>#0: *<connection number> close http connection: 3

```

Как видите, информации немало. Если вы никак не можете понять, почему некоторая конфигурация не работает, изучение отладочного журнала может оказаться очень кстати. Из него видно, в каком порядке работают фильтры и какие обработчики вызывались для обслуживания запроса.

Использование журналов доступа для отладки

Когда я только учился программировать и не мог найти источник ошибки, мой друг советовал «понаставить printf'ы». Именно так ему удавалось быстрее всего разобраться в проблеме. А смысл совета заключался в том, чтобы печатать сообщения в каждой точке ветвления программы, тогда было бы видно, по какому пути шло выполнение и где логика дала сбой.

Ту же идею можно применить и к настройке NGINX. Только вместо printf() мы используем директивы log_format и access_log для визуализации и анализа порядка обработки запроса. Директива log_format позволяет увидеть значения переменных в различных точках конфигурации:

```
http {
    log_format sentlog '[$time_local] «$request» $status $body_bytes_sent ';
    log_format imagelog '[$time_local] $image_file $image_type '
'$body_bytes_sent $status';
    log_format authlog '[$time_local] $remote_addr $remote_user '
'«$request» $status';
}
```

Используя несколько директив access_log, мы можем посмотреть, какие местоположения вызываются в различные моменты времени. Настроив свой журнал доступа для каждого местоположения, легко понять, какие не используются вовсе. Любое изменение в таком местоположении не повлияет на обработку запроса. Первыми следует изучить местоположения, расположенные выше в иерархии.

```
http {
    log_format sentlog '[$time_local] «$request» $status $body_bytes_sent ';
    log_format imagelog '[$time_local] $image_file $image_type '
'$body_bytes_sent $status';
    log_format authlog '[$time_local] $remote_addr $remote_user '
'«$request» $status';

    server {
```

```
server_name .example.com;
root /home/www;

location / {
    access_log logs/example.com-access.log combined;
    access_log logs/example.com-root_access.log sentlog;

    rewrite ^/(.*)\.(png|jpg|gif)$ /images/$1.$2;
    set $image_file $1;
    set $image_type $2;
}

location /images {
    access_log logs/example.com-images_access.log imagelog;
}

location /auth {
    auth_basic «authorized area»;
    auth_basic_user_file conf/htpasswd;
    deny all;

    access_log logs/example.com-auth_access.log authlog;
}
}
```

В этом примере в каждое местоположение включена директива `access_log`, и для каждого журнала доступа задан свой формат в директиве `log_format`. Заглянув в журнал доступа, мы сможем определить, какие запросы в какие местоположения попадали. Если в файле `example.com-images_access.log` не оказалось ни одной записи, значит, ни один запрос не достиг местоположения `/images`. Сравнив содержимое различных файлов, нетрудно понять, правильные ли значения присвоены переменным. Например, если переменные `$image_file` и `$image_type` пусты, то соответствующие им в формате `imagelog` места окажутся незаполненными в журнале доступа.

Типичные ошибки конфигурирования

Следующий шаг при поиске неполадок – разобраться, достигает ли конфигурация поставленных перед ней целей. Интернет вот уже много лет пестрит конфигурациями для NGINX. Зачастую они были написаны для прежних версий и решали какую-то конкретную задачу. Увы, эти конфигурации копируют бездумно, не понимая,

для чего они предназначены. Иногда существует более правильный способ решить ту же задачу, воспользовавшись более современными средствами.

Использование `if` вместо `try_files`

Рассмотрим, к примеру, случай, когда требуется доставить пользователю статический файл, если он есть в файловой системе, а в противном случае передать запрос FastCGI-серверу:

```
server {
    root /var/www/html;

    location / {
        if (!-f $request_filename) {
            include fastcgi_params;
            fastcgi_pass 127.0.0.1:9000;
            break;
        }
    }
}
```

Так обычно решалась эта задача до появления директивы `try_files` в версии NGINX 0.7.27. Сейчас такая конфигурация считается ошибкой, потому что `if` используется внутри `location`. В разделе «Преобразование конфигурации с `'if'` в более современную форму» из главы 4 было объяснено, почему это может иметь неожиданные последствия и даже стать причиной аварийного завершения программы. Правильное решение выглядит так:

```
server {
    root /var/www/html;

    location / {
        try_files $uri $uri/ @fastcgi;
    }

    location @fastcgi {
        include fastcgi_params;
        fastcgi_pass 127.0.0.1:9000;
    }
}
```

Директива `try_files` смотрит, существует ли файл, и, если нет, передает запрос FastCGI-серверу. Никакие `if` при этом не нужны.

Использование *if* для ветвления по имени хоста

Не счесть примеров конфигураций, в которых `if` используется для переадресации запросов на основе HTTP-заголовка `Host`. В этом случае конфигурация работает как селектор, вычисляемый для каждого запроса:

```
server {
    server_name .example.com;
    root /var/www/html;
    if ($host ~* ^example\.com) {
        rewrite ^/(.*)$ http://www.example.com/$1 redirect;
    }
}
```

Вместо того чтобы тратить ресурсы на вычисление `if` для каждого запроса, не проще ли воспользоваться обычной процедурой NGINX маршрутизации запросов к подходящему виртуальному серверу? Тогда переадресацию можно поместить туда, где ей место, и при этом без всякого `rewrite`:

```
server {
    server_name example.com;
    return 301 $scheme://www.example.com;
}

server {
    server_name www.example.com;
    root /var/www/html;
    location / {
        ...
    }
}
```

Неоптимальное использование контекста *server*

Еще одно место, где некритичное копирование фрагментов часто приводит к некорректной конфигурации, — это контекст `server`. В нем описывается виртуальный сервер (все, что адресуется с помощью имени, указанного в директиве `server_name`). Но эта идея в копируемых фрагментах не находит должного отражения.

Часто приходится видеть директивы `root` и `index` в каждой секции `location`:

```
server {
    server_name www.example.com;

    location / {
        root /var/www/html;
        index index.php index.html index.htm;
    }

    location /ftp{
        root /var/www/html;
        index index.php index.html index.htm;
    }
}
```

Это может стать причиной ошибок при добавлении нового местоположения, если забыть скопировать туда эти директивы или скопировать их неправильно. Смысл директив `root` и `index` заключается соответственно в том, чтобы указать корень документов для виртуального сервера и файлы, которые следует поискать, когда в URI-адресе указан только каталог. Любая секция `location` наследует эти значения от контекста `server`.

```
server {
    server_name www.example.com;
    root /var/www/html;
    index index.php index.html index.htm;

    location / {
        ...
    }

    location /ftp{
        ...
    }
}
```

Здесь мы указали, что все файлы находятся в каталоге `/var/www/html` и его подкаталогах, и в любом местоположении следует искать файлы `index.php`, `index.html` и `index.htm` именно в таком порядке.

Ограничения операционной системы

Обычно операционная система – последнее место, которое мы подозреваем, сталкиваясь с ошибкой. Мы предполагаем, что человек, который устанавливал систему, оптимизировал ее под нашу рабочую нагрузку и протестировал в похожих условиях. Часто это не так. Бывает, что приходится копаться в настройках самой операционной системы, чтобы найти узкое место.

Что касается NGINX, то обращать внимание нужно прежде всего на два потенциальных источника проблем: **ограничение на количество файловых дескрипторов** и **сетевые лимиты**.

Ограничение на количество файловых дескрипторов

В NGINX файловые дескрипторы применяются несколькими способами. Прежде всего дескриптор необходим для каждого соединения с клиентом. Для каждого исходящего соединения (а они обязательно присутствуют, когда NGINX играет роль прокси-сервера) требуется уникальная пара (IP-адрес, номер TCP-порта), которая также представляется файловым дескриптором. Дескриптор нужен и тогда, когда NGINX возвращает статический файл или ответ из своего кэша. Как видите, количество потребных дескрипторов может быстро возрасть с увеличением количества одновременных пользователей. Однако общее число дескрипторов, доступных NGINX, ограничено операционной системой.

В типичной UNIX-системе к суперпользователю (с именем `root`) и к обычному пользователю применяются различные лимиты, поэтому приведенную ниже команду следует запускать от имени того непривилегированного пользователя, под которым работает NGINX (заданного либо с помощью параметра `--user` на этапе конфигурирования, либо в директиве `user` в конфигурационном файле).

```
ulimit -n
```

Эта команда показывает, сколько файловых дескрипторов доступно данному пользователю. Обычно их 1024 или того меньше. Поскольку мы знаем, что NGINX будет основным потребителем дескрипторов на данной машине, то можем существенно увеличить этот предел. Насколько именно, зависит от операционной системы. Делается это следующим образом:

❑ Linux

```
vi /etc/security/limits.conf
```

```
www-run hard nofile 65535
$ ulimit -n 65535
```

❑ FreeBSD

```
vi /etc/sysctl.conf
```

```
kern.maxfiles=65535
kern.maxfilesperproc=65535
kern.maxvnodes=65535
# /etc/rc.d/sysctl reload
```

❑ Solaris

```
# projadd -c «increased file descriptors»
-K «process.max-file-descriptor=(basic,65535,deny)» resource.file

# usermod -K project=resource.file www
```

Показанные команды увеличивают максимальное число файловых дескрипторов, доступных новому процессу, запущенному от имени пользователя `www`. Указанное значение будет действовать и после перезагрузки ОС.

Следующие две команды увеличивают число файловых дескрипторов, доступных работающему процессу NGINX:

```
# prctl -r -t privileged -n process.max-file-descriptor -v 65535 -i process
`pgrep nginx`
# prctl -x -t basic -n process.max-file-descriptor -i process `pgrep nginx`
```

Все вышеописанные методы изменяют лимиты операционной системы, но NGINX об этом ничего не узнает, пока мы не зададим количество файловых дескрипторов в директиве `worker_rlimit_nofile`:

```
worker_rlimit_nofile 65535;
worker_processes 8;

events {
    worker_connections 8192;
}
```

После этого нужно послать главному процессу `nginx` сигнал `HUP`:

```
# kill -HUP `cat /var/run/nginx.pid`
```

Теперь NGINX сможет обслуживать свыше 65 000 одновременных клиентов, соединений с проксируемыми серверами и локальных статических или кэшированных файлов. Столько рабочих процессов (директива `worker_processes`) имеет смысл запускать, только если машина действительно оснащена восемью процессорными ядрами или рабочая нагрузка подразумевает большой объем ввода-вывода. В противном случае уменьшите значение `worker_processes`, сделав его равным количеству ядер, и увеличьте `worker_connections`, так чтобы произведение обоих чисел было приблизительно равно 65 000.

Разумеется, можно довести общее количество файловых дескрипторов и `worker_connections` до величины, имеющей смысл для конкретного состава оборудования и решаемых задач. NGINX в состоянии обслужить миллионы одновременных соединений при условии правильного задания лимитов операционной системы и конфигурационных параметров.

Сетевые лимиты

Очувтившись в ситуации нехватки сетевых буферов, вы, скорее всего, сможете только зайти с консоли – и то если повезет. Это бывает, когда к NGINX поступает так много клиентских соединений, что все сетевые буферы оказываются заняты. Процедура увеличения количества сетевых буферов также зависит от операционной системы:

❑ FreeBSD

```
vi /boot/loader.conf

kern.ipc.nmbclusters=262144
```

❑ Solaris

```
# ndd -set /dev/tcp tcp_max_buf 16777216
```

Если NGINX работает как прокси-сервер – почтовый или HTTP, – то ему необходимо открывать много соединений с проксируемыми серверами. Чтобы увеличить число соединений, необходимо до максимума расширить диапазон эфемерных TCP-портов.

❑ Linux

```
vi /etc/sysctl.conf

net.ipv4.ip_local_port_range = 1024 65535
# sysctl -p /etc/sysctl.conf
```

❑ FreeBSD

```
vi /etc/sysctl.conf
net.inet.ip.portrange.first=1024
net.inet.ip.portrange.last=65535
# /etc/rc.d/sysctl reload
```

❑ Solaris

```
# ndd -set /dev/tcp tcp_smallest_anon_port 1024
# ndd -set /dev/tcp tcp_largest_anon_port 65535
```

Разобравшись с основными параметрами, можно переходить к настройке более специфичных параметров, относящихся к производительности. Они описаны в следующем разделе.

Проблемы с производительностью

Проектируя приложение и настраивая для него NGINX, мы, естественно, рассчитываем на высокую производительность. Если же в этом плане возникают какие-то проблемы, то необходимо выяснить, в чем причина. Возможно, дело в самом приложении. А возможно, в конфигурации NGINX. Мы должны научиться определять источник проблемы.

Работая в качестве прокси-сервера, NGINX задействует главным образом сеть. Если на сетевом уровне имеются какие-то ограничения, то NGINX не сможет работать оптимально. Процедура настройки сети зависит от операционной системы и от самой сети, в которой эксплуатируется NGINX, поэтому следует проводить ее с учетом конкретных условий.

Один из важных параметров, относящихся к производительности сети, – размер очереди TCP-соединений системного вызова `listen`. Чем больше это значение, тем больше можно обслужить клиентов. Каким должно быть оптимальное значение и как его задать, зависит от операционной системы.

❑ Linux

```
vi /etc/sysctl.conf

net.core.somaxconn = 3240000
# sysctl -p /etc/sysctl.conf
```

❑ FreeBSD

```
vi /etc/sysctl.conf

kern.ipc.somaxconn=4096
# /etc/rc.d/sysctl reload
```

❑ Solaris

```
# ndd -set /dev/tcp tcp_conn_req_max_q 1024
# ndd -set /dev/tcp tcp_conn_req_max_q0 4096
```

Следующий интересный параметр – размеры буферов приема и отправки. Приведенные ниже значения показаны только для примера – они могут стать причиной повышенного потребления памяти, так что применяйтесь к своей ситуации и тестируйте.

❑ Linux

```
vi /etc/sysctl.conf
```

```
net.ipv4.tcp_wmem = 8192 87380 1048576
net.ipv4.tcp_rmem = 8192 87380 1048576
# sysctl -p /etc/sysctl.conf
```

❑ FreeBSD

```
vi /etc/sysctl.conf
```

```
net.inet.tcp.sendspace=1048576
net.inet.tcp.recvspace=1048576
# /etc/rc.d/sysctl reload
```

❑ Solaris

```
# ndd -set /dev/tcp tcp_xmit_hiwat 1048576
# ndd -set /dev/tcp tcp_recv_hiwat 1048576
```

Можно также изменить размеры буферов непосредственно в конфигурационном файле NGINX, тогда они будут действовать только с NGINX. Это желательно, когда на машине работает несколько служб, но требуется, чтобы именно NGINX мог получить максимум отдачи от имеющихся сетевых возможностей:

```
server {
    listen 80 sndbuf=1m rcvbuf=1m;
}
```

Настройка сети может оказать существенное влияние на производительность. Однако рекомендуется изменять по одному параметру за раз и замерять результаты после изменения. Оптимизировать производительность можно на стольких разных уровнях, что краткое введение в этой главе не дает полного представления о предмете. Для интересующихся этой тематикой есть немало книг и сетевых ресурсов.



Как сделать измененные сетевые параметры постоянными в Solaris

Выше мы изменяли некоторые параметры TCP в командной строке. В случае Linux и FreeBSD эти изменения сохраняются и после перезагрузки ОС, потому что записываются в конфигурационный файл (например, `/etc/sysctl.conf`). Но в Solaris ситуация иная. Эти изменения никуда не записываются и поэтому не сохраняются при перезагрузке.

Начиная с версии Solaris 10, предоставляется уникальный каркас управления службами (**Service Management Framework – SMF**), позволяющий к тому же задавать порядок их запуска при загрузке ОС. (Это, конечно, чрезмерно упрощенное описание, каркас SMF умеет гораздо больше.) Чтобы настройки TCP сохранялись, необходимо написать манифест SMF и соответствующий скрипт.

Подробнее см. в приложении D «Сохранение сетевых настроек в Solaris».

Использование модуля Stub Status

В состав NGINX входит модуль самодиагностики, который выводит статистические данные о работе программы. Этот модуль называется **Stub Status** и активируется заданием параметра `--with-http_stub_status_module` на этапе конфигурирования.

Чтобы увидеть статистику, порождаемую этим модулем, необходимо включить директиву `stub_status`, задав в ней значение `on`. Для этого модуля следует создать отдельную секцию `location`, чтобы можно было применить список ACL:

```
location /nginx_status {
    stub_status on;
    access_log off;
    allow 127.0.0.1;
    deny all;
}
```

Обращение к этому URI с компьютера `localhost` (например, `curl http://localhost/nginx_status`) возвращает ответ такого вида:

```
Active connections: 2532
server accepts handled requests
 1476737983 1476737983 3553635810
Reading: 93 Writing: 13 Waiting: 2426
```

Мы видим, что открыто 2532 соединения, и в настоящий момент для 93 соединений NGINX читает заголовки запроса, а 13 находятся в состоянии, когда NGINX либо читает тело запроса, либо обрабатывает запрос, либо отправляет ответ клиенту. Остальные 2426 соединений имеют тип `keepalive` и находятся в состоянии ожидания. С момента запуска процесс `nginx` принял и обработал 1 476 737 983 соединения, то есть ни одно не было закрыто сразу после приема. Всего было обработано 3 553 635 810 запросов, то есть в среднем по 2,4 запроса в одном соединении.

Эти данные можно передать в какую-нибудь программу обработки метрик и представить в графическом виде. Существуют модули, подключаемые к системам Munin, Nagios, collectd и другим, которые для сбора статистики обращаются к модулю `stub_status`. Со временем, имея достаточно данных, вы сможете обнаружить какие-то тенденции и коррелировать их с теми или иными факторами. На графиках должны быть видны всплески количества запросов, а также изменения в настройках операционной системы.

Резюме

Когда вводится в эксплуатацию новая программа, проблемы возникают на самых разных уровнях. Некоторые ошибки можно «отловить» и устранить в тестовой среде, другие проявляются только в условиях реальной нагрузки. Для поиска причин ошибок NGINX предлагает очень подробные журналы с различными уровнями протоколирования. Некоторые сообщения допускают несколько интерпретаций, но общий принцип понятен. Экспериментируя с различными конфигурациями и наблюдая за появляющимися сообщениями, можно научиться интерпретировать записи в журнале. На работу NGINX оказывает влияние операционная система, которая накладывает определенные ограничения, когда параметры заданы по умолчанию в расчете на наличие многих пользователей. Для оптимизации параметров с учетом реальных условий полезно понимать, что происходит на уровне TCP. И в завершение краткого обзора поиска и устранения неполадок мы показали, какую информацию возвращает модуль `stub_status`. Эти данные полезны, когда требуется составить общее представление о том, как работает NGINX.

Далее идут приложения. В первом из них перечислены все конфигурационные директивы NGINX с указанием значений по умолчанию и контекста, в котором можно употреблять директиву.



Приложение А. Справочник директив

В этом приложении перечислены все конфигурационные директивы, встречавшиеся на страницах этой книги. Для полноты приведены также некоторые директивы, которые мы не рассматривали. Дополнительно для каждой директивы указывается контекст, в котором она может употребляться. Если у директивы есть значение по умолчанию, оно также указывается. Директивы описаны по состоянию в версии NGINX 1.3.9. Актуальный список можно найти по адресу <http://nginx.org/en/docs/dirindex.html>.

Справочник директив

Директива	Описание	Контекст/ Умолчание
accept_mutex	Сериализует обращения к методу <code>accept()</code> для новых соединений со стороны рабочих процессов	Допустимый контекст: <code>events</code> По умолчанию: <code>on</code>
accept_mutex_delay	Сколько времени рабочий процесс может ждать возможности принимать новые соединения, если в данный момент этим занимается другой рабочий процесс	Допустимый контекст: <code>events</code> По умолчанию: <code>500ms</code>

Директива	Описание	Контекст/ Умолчание
access_log	Определяет, куда и как записывать журналы доступа. Первый параметр – путь к файлу журнала. Путь может содержать переменные. Специальное значение <code>off</code> отключает протоколирование. Необязательный второй параметр определяет директиву <code>log_format</code> , описывающую формат журнала. Если этот параметр не задан, используется предопределенный формат. Необязательный третий параметр задает размер буфера в случае, если запись в журнал буферизуется. При использовании буферизации этот размер не должен превосходить длину операции атомарной записи на диск для конкретной файловой системы. Если третий параметр равен <code>gzip</code> , то журнал буферизуется и на лету сжимается при условии, что двоичный файл <code>nginx</code> собирався с библиотекой <code>zlib</code> . Последний параметр <code>flush</code> определяет, сколько времени данные могут оставаться в буфере в памяти перед сбросом на диск	Допустимые контексты: <code>http</code> , <code>server</code> , <code>location</code> , <code>if</code> в <code>location</code> , <code>limit_except</code> По умолчанию: <code>logs/access.log combined</code>
add_after_body	Поместить результат обработки подзапроса после тела ответа	Допустимый контекст: <code>location</code> По умолчанию: -
add_before_body	Поместить результат обработки подзапроса перед телом ответа	Допустимый контекст: <code>location</code> По умолчанию: -
add_header	Добавляет заголовок в ответ с кодом состояния 200, 204, 206, 301, 302, 303, 304 или 307	Допустимые контексты: <code>http</code> , <code>server</code> , <code>location</code> По умолчанию: -
addition_types	Указывается список MIME-типов ответа (в дополнение к <code>text/html</code>), для которых производится добавление. Звездочка (*) означает все MIME-типы	Допустимые контексты: <code>http</code> , <code>server</code> , <code>location</code> По умолчанию: <code>text/html</code>

Директива	Описание	Контекст/ Умолчание
aio	Разрешает использование асинхронного файлового ввода-вывода. Это возможно во всех современных версиях FreeBSD и дистрибутивах Linux. Во FreeBSD директиву aio можно использовать для предварительной загрузки данных для sendfile. В Linux требуется директива directio, которая автоматически отключает sendfile	Допустимые контексты: http, server, location По умолчанию: off
alias	Определяет путь в файловой системе, соответствующий имени местоположения. Если местоположение задано с помощью регулярного выражения, то значение alias должно ссылаться на запомненные подвыражения	Допустимый контекст: location По умолчанию: -
allow	Разрешает доступ с указанного IP-адреса, из указанной сети или отовсюду (all)	Допустимые контексты: http, server, location, limit_except По умолчанию: logs/access.log combined
ancient_browser	Подстроки, при обнаружении которых в заголовке User-Agent браузер считается устаревшим. В результате в переменную \$ancient_browser записывается значение, заданное в директиве ancient_browser_value	Допустимые контексты: http, server, location По умолчанию: -
ancient_browser_value	Значение, присваиваемое переменной \$ancient_browser	Допустимые контексты: http, server, location По умолчанию: 1
auth_basic	Разрешает аутентификацию по схеме HTTP Basic Authentication. Параметр задает имя области (realm). Специальное значение off означает, что значение auth_basic, заданное на родительском уровне, отменяется	Допустимые контексты: http, server, location, limit_except По умолчанию: off
auth_basic_user_file	Определяет, где находится файл, содержащий тройки username:password:comment, который используется для аутентификации клиентов. Поле password должно быть зашифровано алгоритмом crypt. Поле comment необязательно	Допустимые контексты: http, server, location, limit_except По умолчанию: -

Директивы	Описание	Контекст/ Умолчание
auth_http	Задаёт сервер, который служит для аутентификации пользователя в протоколах POP3 и IMAP	Допустимые контексты: mail, server По умолчанию: -
auth_http_header	Включает дополнительный заголовок (первый параметр) с указанным значением (второй параметр)	Допустимые контексты: mail, server По умолчанию: -
auth_http_timeout	Максимальное время ожидания ответа от сервера аутентификации	Допустимые контексты: mail, server По умолчанию: 60s
autoindex	Разрешает автоматическую генерацию страницы с содержимым каталога	Допустимые контексты: http, server, location По умолчанию: off
autoindex_exact_size	Определяет, следует ли указывать размеры файлов в каталоге в байтах или округлять до килобайт, мегабайт или гигабайт	Допустимые контексты: http, server, location По умолчанию: on
autoindex_localtime	Какое время последней модификации файла указывать на странице с содержимым каталога: местное (on) или UTC (off)	Допустимые контексты: http, server, location По умолчанию: off
break	Завершает обработку директив модуля rewrite, находящихся в том же контексте	Допустимые контексты: server, location, if По умолчанию: -
charset	Добавляет указанную кодировку в заголовок ответа Content-Type. Если кодировка отличается от указанной в директиве source_charset, то производится перекодирование	Допустимые контексты: http, server, location, if в location По умолчанию: off
charset_map	Задаёт таблицу перекодирования из одной кодировки в другую. Коды символов записываются в шестнадцатеричном виде. В файлах conf/koi-win, conf/koi-utf и conf/win-utf хранятся соответственно таблицы перекодирования из koi8-r в windows-1251, из koi8-r в utf-8 и из windows-1251 в utf-8	Допустимый контекст: http По умолчанию: -

Директивы	Описание	Контекст/ Умолчение
<code>charset_types</code>	Список MIME-типов ответа (помимо <code>text/html</code>), для которых будет производиться перекодирование	Допустимые контексты: <code>http, server, location</code> По умолчанию: <code>text/html, text/xml, text/plain, text/vnd.wap.wml, application/javascript, application/rss+xml</code>
<code>chunked_transfer_encoding</code>	Позволяет отключить специфицированный в стандарте HTTP/1.1 механизм поблочной передачи данных (<code>chunked transfer encoding</code>) в ответе клиенту	Допустимые контексты: <code>http, server, location</code> По умолчанию: <code>on</code>
<code>client_body_buffer_size</code>	Задаёт размер буфера для чтения тела запроса клиента. По умолчанию для буфера выделяются две страницы памяти. Увеличение размера позволяет предотвратить запись во временный файл на диске	Допустимые контексты: <code>http, server, location</code> По умолчанию: <code>8k 16k</code> (в зависимости от платформы)
<code>client_body_in_file_only</code>	Используется для отладки или последующей обработки тела запроса клиента. Если значение равно <code>on</code> , то тело запроса принудительно записывается в файл. Значение <code>clean</code> приводит к удалению файлов после завершения обработки запроса	Допустимые контексты: <code>http, server, location</code> По умолчанию: <code>off</code>
<code>client_body_in_single_buffer</code>	Заставляет NGINX сохранить все тело запроса клиента в одном буфере, чтобы уменьшить количество операций копирования	Допустимые контексты: <code>http, server, location</code> По умолчанию: <code>off</code>
<code>client_body_temp_path</code>	Определяет путь к каталогу для сохранения файлов с телами запросов клиентов. Второй, третий и четвёртый параметры (если заданы) определяют иерархию подкаталогов в виде количества знаков в имени подкаталога	Допустимые контексты: <code>http, server, location</code> По умолчанию: <code>client_body_temp</code>

Директивы	Описание	Контекст/ Умолчание
client_body_timeout	Задаёт время между последовательными операциями чтения тела запроса клиента. В случае превышения клиент получает сообщение об ошибке 408 (Request Timeout)	Допустимые контексты: http, server, location По умолчанию: 60s
client_header_buffer_size	Задаёт размер буфера для чтения заголовка запроса клиента, если он превышает подразумеваемую по умолчанию величину 1 КБ	Допустимые контексты: http, server По умолчанию: 1k
client_header_timeout	Время, отведенное на чтение всего заголовка запроса. В случае превышения клиент получает сообщение об ошибке 408 (Request Timeout)	Допустимые контексты: http, server По умолчанию: 60s
client_max_body_size	Максимальный размер тела запроса клиента. В случае превышения отправляется ответ 413 (Request Entity Too Large)	Допустимые контексты: http, server, location По умолчанию: 1m
connection_pool_size	Позволяет производить точную настройку выделения памяти под конкретные соединения	Допустимые контексты: http, server По умолчанию: 256
create_full_put_path	Разрешает создание промежуточных каталогов при работе с WebDAV	Допустимые контексты: http, server, location По умолчанию: off
daemon	Определяет, будет ли nginx запускаться в режиме демона	Допустимый контекст: main По умолчанию: on
dav_access	Устанавливает права доступа к создаваемым файлам и каталогам. Если задан параметр group или all, то user можно опустить	Допустимые контексты: http, server, location По умолчанию: user:rw
dav_methods	Разрешает указанные HTTP- и WebDAV-методы. При использовании метода PUT сначала создается временный файл, а затем он переименовывается. Поэтому рекомендуется указывать в директиве client_body_temp_path ту файловую систему, в которой должен находиться загруженный файл. Время модификации таких файлов можно задать в заголовке Date	Допустимые контексты: http, server, location По умолчанию: off

Директивы	Описание	Контекст/ Умолчение
<code>debug_connection</code>	Включает отладочное протоколирование для клиентов, соответствующих значению этой директивы. Может встречаться несколько раз. Для отладки сокетов в домене UNIX следует указать <code>unix</code> :	Допустимый контекст: <code>events</code> По умолчанию: -
<code>debug_points</code>	В случае обнаружения внутренней ошибки процесс либо создает <code>core</code> -файл (<code>abort</code>), либо останавливается (<code>stop</code>) с целью последующего подключения системного отладчика	Допустимые контексты: <code>main</code> По умолчанию: -
<code>default_type</code>	Определяет подразумеваемый по умолчанию MIME-тип ответа. Используется в случае, когда MIME-тип файла не удастся сопоставить ни с одним из определенных в директиве <code>types</code>	Допустимые контексты: <code>http, server, location</code> По умолчанию: <code>text/plain</code>
<code>deny</code>	Запрещает доступ с указанного IP-адреса, из указанной сети или отовсюду (<code>all</code>)	Допустимые контексты: <code>http, server, location, limit_except</code> По умолчанию: -
<code>directio</code>	Разрешает использовать зависящий от операционной системы флаг при чтении файлов, размер которых больше или равен указанному. Обязательна при использовании директивы <code>aio</code> в Linux	Допустимые контексты: <code>http, server, location</code> По умолчанию: <code>off</code>
<code>directio_alignment</code>	Устанавливает выравнивание для <code>directio</code> . Обычно подразумеваемого по умолчанию значения 512 достаточно, но при использовании XFS в Linux рекомендуется увеличить до 4K	Допустимые контексты: <code>http, server, location</code> По умолчанию: 512
<code>disable_symlinks</code>	См. таблицу «Директивы HTTP-сервера, относящиеся к путям в файловой системе» в разделе «Поиск файлов» главы 6 «NGINX как HTTP-сервер»	Допустимые контексты: <code>http, server, location</code> По умолчанию: <code>off</code>
<code>empty_gif</code>	Порождает прозрачный GIF размером 1×1 для данного местоположения	Допустимый контекст: <code>location</code> По умолчанию: -

Директивы	Описание	Контекст/ Умолчание
env	Определяет переменные окружения, которые используются в следующих случаях: • наследование во время обновления NGINX на лету; • в модуле perl; • предоставление в распоряжение рабочих процессов. Если задать только имя переменной, то будет взято значение из окружения nginx. Чтобы присвоить переменной значение, следует воспользоваться формой <code>var=value</code>. N.B. Переменная NGINX является внутренней, пользователь не должен ее устанавливать	Допустимый контекст: <code>main</code> По умолчанию: TZ
error_log	Это файл, в который записываются сообщения об ошибках. Если ни в каком другом контексте директивы <code>error_log</code> нет, то в этом файле будут регистрироваться вообще все ошибки. Второй параметр директивы обозначает уровень сообщений, попадающих в журнал (<code>debug</code>, <code>info</code>, <code>notice</code>, <code>warn</code>, <code>error</code>, <code>crit</code>, <code>alert</code>, <code>emerg</code>). Сообщения уровня <code>debug</code> выводятся, только если программа была сконфигурирована с параметром <code>--with-debug</code>	Допустимые контексты: <code>main</code>, <code>http</code>, <code>server</code>, <code>location</code> По умолчанию: <code>logs/error.log error</code>
error_page	Определяет URL-адрес страницы, которую нужно вернуть, если код ответа попадает в диапазон ошибок. Необязательный параметр, следующий за знаком <code>=</code>, позволяет изменить код ответа. Если после знака равенства не указан код ответа, то он берется из URI-адреса, при этом соответствующая страница должна возвращаться каким-то проксируемым сервером	Допустимые контексты: <code>http</code>, <code>server</code>, <code>location</code>, <code>if</code> в <code>location</code> По умолчанию: -

Директивы	Описание	Контекст/ Умолчание
etag	Отключает автоматическую генерацию заголовка ответа ETag для статических ресурсов	Допустимые контексты: http, server, location По умолчанию: on
events	Определяет новый контекст, в котором задаются директивы обработки соединений	Допустимый контекст: main По умолчанию: -
expires	См. таблицу «Директивы для модификации заголовков» в разделе «Кэширование в файловой системе» главы 7 «NGINX для раз-работчика»	Допустимые контексты: http, server, location По умолчанию: off
fastcgi_bind	Определяет адрес исходящих соединений с FastCGI-сервером	Допустимые контексты: http, server, location По умолчанию: -
fastcgi_buffer_size	Размер буфера для первой части ответа от FastCGI-сервера, в которой находятся заголовки	Допустимые контексты: http, server, location По умолчанию: 4k 8k (в зависимости от платформы)
fastcgi_buffers	Количество и размер буферов для получения ответа от FastCGI-сервера в расчете на одно соединение	Допустимые контексты: http, server, location По умолчанию: 4k 8k (в зависимости от платформы)
fastcgi_busy_buffers_size	Суммарный размер буферов, которые могут быть заняты для отправки ответа клиенту, пока ответ от FastCGI-сервера ещё не прочитан целиком. Обычно устанавливается вдвое большим, чем размер, указанный в директиве fastcgi_buffers	Допустимые контексты: http, server, location По умолчанию: 4k 8k (в зависимости от платформы)
fastcgi_cache	Определяет зону разделяемой памяти, используемую для кэширования	Допустимые контексты: http, server, location По умолчанию: off
fastcgi_cache_bypass	Одна или несколько строковых переменных. Если хотя бы одна из них непуста и не содержит нуль, то ответ будет запрошен у FastCGI-сервера, а не взят из кэша	Допустимые контексты: http, server, location По умолчанию: -
fastcgi_cache_key	Строка, которая используется как ключ для поиска значения в кэше	Допустимые контексты: http, server, location По умолчанию: -

Директивы	Описание	Контекст/ Умолчание
<code>fastcgi_cache_lock</code>	Если включено, то предотвращается отправка нескольких запросов FastCGI-серверу в случае отсутствия в кэше	Допустимые контексты: <code>http, server, location</code> По умолчанию: <code>off</code>
<code>fastcgi_cache_lock_timeout</code>	Сколько времени запрос может ждать появления записи в кэше или освобождения блокировки <code>fastcgi_cache_lock</code>	Допустимые контексты: <code>http, server, location</code> По умолчанию: <code>5s</code>
<code>fastcgi_cache_min_uses</code>	Сколько запросов с данным ключом должно поступить, прежде чем ответ будет помещен в кэш	Допустимые контексты: <code>http, server, location</code> По умолчанию: <code>1</code>
<code>fastcgi_cache_path</code>	См. таблицу «Директивы модуля FastCGI для управления потоковой передачей» в разделе «Использование NGINX совместно с PHP-FPM» главы 6 «NGINX как HTTP-сервер»	Допустимые контексты: <code>http, server, location</code> По умолчанию: <code>-</code>
<code>fastcgi_cache_use_stale</code>	В каких случаях допустимо использовать устаревшие кэшированные данные, если при доступе к FastCGI-серверу произошла ошибка. Параметр <code>updating</code> разрешает использовать кэшированный ответ, если как раз в данный момент загружаются более свежие данные	Допустимые контексты: <code>http, server, location</code> По умолчанию: <code>off</code>
<code>fastcgi_cache_valid</code>	Сколько времени считать действительным кэшированный ответ с кодом 200, 301 или 302. Если перед параметром <code>time</code> указан необязательный код ответа, то заданное время относится только к ответу с таким кодом. Специальный параметр <code>any</code> означает, что в течение заданного времени следует кэшировать ответ с любым кодом	Допустимые контексты: <code>http, server, location</code> По умолчанию: <code>-</code>
<code>fastcgi_connect_timeout</code>	Максимальное время, в течение которого NGINX ожидает установления соединения при отправке запроса FastCGI-серверу	Допустимые контексты: <code>http, server, location</code> По умолчанию: <code>60s</code>
<code>fastcgi_hide_header</code>	Список заголовков, которые не следует передавать клиенту	Допустимые контексты: <code>http, server, location</code> По умолчанию: <code>-</code>

Директивы	Описание	Контекст/ Умолчание
<code>fastcgi_ignore_client_abort</code>	Если значение равно <code>on</code> , то NGINX не станет разрывать соединение с FastCGI-сервером в случае, когда клиент разрывает свое соединение	Допустимые контексты: <code>http, server, location</code> По умолчанию: <code>off</code>
<code>fastcgi_ignore_headers</code>	Какие заголовки можно игнорировать при обработке ответа от FastCGI-сервера	Допустимые контексты: <code>http, server, location</code> По умолчанию: <code>-</code>
<code>fastcgi_index</code>	Задаёт имя файла, дописываемое в конце строки <code>\$fastcgi_script_name</code> после знака косой черты	Допустимые контексты: <code>http, server, location</code> По умолчанию: <code>-</code>
<code>fastcgi_intercept_errors</code>	Если значение равно <code>on</code> , то NGINX будет отображать страницу, заданную директивой <code>error_page</code> , вместо ответа, полученного от FastCGI-сервера	Допустимые контексты: <code>http, server, location</code> По умолчанию: <code>off</code>
<code>fastcgi_keep_conn</code>	Разрешает соединения типа <code>keepalive</code> с FastCGI-сервером, инструктируя его не закрывать соединение немедленно	Допустимые контексты: <code>http, server, location</code> По умолчанию: <code>off</code>
<code>fastcgi_max_temp_file_size</code>	Максимальный размер временного файла, в который записывается часть ответа в случае, когда он не умещается целиком в буферах памяти	Допустимые контексты: <code>http, server, location</code> По умолчанию: <code>1024m</code>
<code>fastcgi_next_upstream</code>	См. таблицу «Директивы модуля FastCGI для управления потоковой передачей» в разделе «Использование NGINX совместно с PHP-FPM» главы 6 «NGINX как HTTP-сервер»	Допустимые контексты: <code>http, server, location</code> По умолчанию: <code>error Timeout</code>
<code>fastcgi_no_cache</code>	Одна или несколько строковых переменных. Если хотя бы одна из них непуста и не содержит нуль, то NGINX не будет помещать в кэш ответ, полученный от FastCGI-сервера	Допустимые контексты: <code>http, server, location</code> По умолчанию: <code>-</code>
<code>fastcgi_param</code>	Задаёт имя и значение параметра, передаваемого FastCGI-серверу. Если следует передавать только параметры с непустыми значениями, то необходимо задать дополнительный параметр <code>if_not_empty</code>	Допустимые контексты: <code>http, server, location</code> По умолчанию: <code>-</code>

Директивы	Описание	Контекст/ Умолчание
<code>fastcgi_pass</code>	Определяет FastCGI-сервер, которому передается запрос, в виде пары <code>address:port</code> или <code>unix:path</code> для сокета в домене UNIX	Допустимые контексты: <code>http, server, location</code> По умолчанию: -
<code>fastcgi_pass_header</code>	Отменяет сокрытие заголовков, определенных в директиве <code>fastcgi_hide_header</code> , разрешая передавать их клиенту	Допустимые контексты: <code>http, server, location</code> По умолчанию: -
<code>fastcgi_read_timeout</code>	Сколько времени может пройти между двумя последовательными операциями чтения данных от FastCGI-сервера, прежде чем соединение будет закрыто	Допустимые контексты: <code>http, server, location</code> По умолчанию: 60s
<code>fastcgi_send_lowat</code>	Эта директива предназначена только для FreeBSD. Если значение отлично от нуля, то при взаимодействии с проксируемым сервером NGINX будет использовать либо флаг <code>NOTE_LOWAT</code> метода <code>kqueue</code> , либо параметр сокета <code>SO_SNDLOWAT</code> с указанным размером. В Linux, Solaris и Windows игнорируется	Допустимые контексты: <code>http, server, location</code> По умолчанию: 0
<code>fastcgi_send_timeout</code>	Сколько времени может пройти между двумя последовательными операциями записи данных на FastCGI-сервер, прежде чем соединение будет закрыто	Допустимые контексты: <code>http, server, location</code> По умолчанию: 60s
<code>fastcgi_split_path_info</code>	Задаёт регулярное выражение с двумя запоминаемыми подвыражениями. Первая запомненная строка становится значением переменной <code>\$fastcgi_script_name</code> , вторая – значением переменной <code>\$fastcgi_path_info</code>	Допустимые контексты: <code>location</code> По умолчанию: -

Директивы	Описание	Контекст/ Умолчание
<code>fastcgi_store</code>	Разрешает сохранение полученных от FastCGI-сервера ответов в файлах на диске. Если параметр равен <code>on</code> , то в качестве пути к каталогу с сохраняемыми файлами используется значение, заданное в директиве <code>alias</code> или <code>root</code> . Можно вместо этого задать строку, определяющую другой каталог для хранения файлов	Допустимые контексты: <code>http, server, location</code> По умолчанию: <code>off</code>
<code>fastcgi_store_access</code>	Какие права доступа следует задать для новых файлов, создаваемых в соответствии с директивой <code>fastcgi_store</code>	Допустимые контексты: <code>http, server, location</code> По умолчанию: <code>user:rw</code>
<code>fastcgi_temp_file_write_size</code>	Ограничивает размер данных в одной операции записи во временный файл, чтобы NGINX не слишком долго блокировала исполнение программы при обработке одного запроса	Допустимые контексты: <code>http, server, location</code> По умолчанию: <code>8k 16k</code> (в зависимости от платформы)
<code>fastcgi_temp_path</code>	Каталог для хранения временных файлов, получаемых от FastCGI-сервера. Может быть многоуровневым. Второй, третий и четвертый параметры (если заданы) определяют иерархию подкаталогов в виде количества знаков в имени подкаталога	Допустимые контексты: <code>http, server, location</code> По умолчанию: <code>fastcgi_temp</code>
<code>flv</code>	Активирует модуль <code>flv</code> для данного местоположения	Допустимый контекст: <code>location</code> По умолчанию: <code>-</code>

Директивы	Описание	Контекст/ Умолчание
geo	Определяет новый контекст, в котором переменной присваивается значение, зависящее от IP-адреса, находящегося в другой переменной. Если другая переменная не задана, берется IP-адрес из переменной <code>\$remote_addr</code>. Формат определения контекста следующий: <pre>geo [\$address-variable] \$variable_to-be-set { ... }</pre> Распознаются следующие параметры: • <code>delete</code>: удалить указанную сеть; • <code>default</code>: в переменную будет записано это значение, если нет подходящего IP-адреса; • <code>include</code>: включает файл с определениями соответствий между адресом и значением переменной; • <code>proxy</code>: определяет адреса (индивидуальные или целой подсети), при запросе с которых будет использоваться IP-адрес, взятый из заголовка запроса <code>X-Forwarded-For</code>; • <code>proxy-recursive</code>: используется вместе с параметром <code>proxy</code> и означает, что если заголовок <code>X-Forwarded-For</code> содержит несколько значений, то следует брать последнее; • <code>ranges</code>: если присутствует, то означает, что следующие далее адреса заданы в виде диапазонов	Допустимый контекст: <code>http</code> По умолчанию: -

Директивы	Описание	Контекст/ Умолчение
geoip_city	Путь к базе данных GeoIP, содержащей соотвествия между IP-адресами и городами. Становятся доступны следующие переменные: • \$geoip_city_country_code: двух-буквенный код страны; • \$geoip_city_country_code3: трех-буквенный код страны; • \$geoip_city_country_name: название страны; • \$geoip_region: название региона страны; • \$geoip_city: название города; • \$geoip_postal_code: почтовый индекс	Допустимый контекст: http По умолчанию: -
geoip_country	Путь к базе данных GeoIP, содержащей соотвествия между IP-адресами и странами. Становятся доступны следующие переменные: • \$geoip_city_country_code: двух-буквенный код страны; • \$geoip_city_country_code3: трех-буквенный код страны; • \$geoip_city_country_name: название страны	Допустимый контекст: http По умолчанию: -
geoip_org	Путь к базе данных GeoIP, содержащей соотвествия между IP-адресами и организациями. Становится доступна следующая переменная: \$geoip_org: название организации	Допустимый контекст: http По умолчанию: -
geoip_proxy	Определяет адреса (индивидуальные или целой подсети), при запросе с которых будет использоваться IP-адрес, взятый из заголовка запроса X-Forwarded-For	Допустимый контекст: http По умолчанию: -

Директивы	Описание	Контекст/ Умолчание
geoip_proxy_recursive	Используется вместе с директивой geoip_proxy и означает, что если заголовок X-Forwarded-For содержит несколько значений, то следует брать последнее	Допустимый контекст: http По умолчанию: off
gunzip	Разрешает или запрещает распаковку сжатых алгоритмом gzip файлов в случае, когда клиент не поддерживает gzip	Допустимые контексты: http, server, location По умолчанию: off
gunzip_buffers	Задаёт количество и размер буферов для распаковки ответа	Допустимые контексты: http, server, location По умолчанию: 32 4k 16 8k (в зависимости от платформы)
gzip	Разрешает или запрещает сжатие ответов	Допустимые контексты: http, server, location, if в location По умолчанию: off
gzip_buffers	Определяет количество и размеры буферов для сжатия ответа	Допустимые контексты: http, server, location По умолчанию: 32 4k 16 8k (в зависимости от платформы)
gzip_comp_level	Уровень сжатия gzip (1–9)	Допустимые контексты: http, server, location По умолчанию: 1
gzip_disable	Регулярное выражение, описывающее те пользовательские агенты, которые не должны получать сжатое содержимое. Специальное значение msie6 является сокращением выражения MSIE [4-6]\., которое исключает MSIE 6.0 ... SV1	Допустимые контексты: http, server, location По умолчанию: -
gzip_http_version	Минимальная версия протокола HTTP в запросе, до которой вопрос о сжатии вообще не рассматривается	Допустимые контексты: http, server, location По умолчанию: 1.1
gzip_min_length	Минимальная длина ответа (определяемая заголовком Content-Length), до которой вопрос о сжатии вообще не рассматривается	Допустимые контексты: http, server, location По умолчанию: 20

Директивы	Описание	Контекст/ Умолчание
gzip_proxied	См. таблицу «Директивы модуля gzip» в разделе «Сжатие» главы 5 «Обратное проксирование, дополнительные вопросы»	Допустимые контексты: http, server, location По умолчанию: off
gzip_static	Разрешает или запрещает проверку наличия предварительного сжатого файла, который можно было бы доставить клиенту, под-держивающему gzip	Допустимые контексты: http, server, location По умолчанию: off
gzip_types	Типы MIME (в дополнение к text/html), которые следует сжимать. Задание * разрешает все типы MIME	Допустимые контексты: http, server, location По умолчанию: text/html
gzip_vary	Разрешает или запрещает включение в ответ заголовка Vary: Accept-Encoding, если активна ди-ректива gzip или gzip_static	Допустимые контексты: http, server, location По умолчанию: off
http	Определяет конфигурационный контекст, в котором задаются директивы HTTP-сервера	Допустимый контекст: main По умолчанию: -
if	См. таблицу «Директивы модуля rewrite» в разделе «Введение в модуль rewrite» в приложении В «Руководство по правилам переписывания»	Допустимые контексты: server, location По умолчанию: -
if_modified_since	Управляет порядком сравнения времени модификации ответа со значением в заголовке запроса If-Modified-Since. - off: заголовок If-Modified-Since игнорируется. - exact: точное соответствие (по умолчанию). - before: время модификации ответа меньше или равно зна-чению в заголовке If-Modified-Since	Допустимые контексты: http, server, location По умолчанию: exact

Директивы	Описание	Контекст/ Умолчение
ignore_invalid_headers	Разрешает или запрещает игнорировать заголовки с недопустимыми именами (по умолчанию on). Допустимыми считаются имена, содержащие буквы в кодировке ASCII, цифры, знак минус и, возможно, знак подчеркивания (определяется директивой <code>underscores_in_headers</code>)	Допустимые контексты: http, server По умолчанию: on
image_filter	См. таблицу «Директивы модуля <code>image_filter</code> » в разделе «Генерация изображений» главы 7 «NGINX для разработчика»	Допустимый контекст: location По умолчанию: -
image_filter_buffer	Размер буфера для обработки изображений. Если потребуется больше памяти, сервер вернет ошибку 415 (Unsupported Media Type)	Допустимые контексты: http, server, location По умолчанию: 1M
image_filter_jpeg_quality	Качество результирующего изображения в формате JPEG. Не рекомендуется указывать значение больше 95	Допустимый контекст: http, server, location По умолчанию: 75
image_filter_sharpen	Повышает резкость результирующего изображения на указанный процент	Допустимые контексты: http, server, location По умолчанию: 0
image_filter_transparency	Определяет, сохранять ли прозрачность при обработке изображений в форматах PNG и GIF. Подразумеваемое по умолчанию значение on означает сохранение прозрачности	Допустимые контексты: http, server, location По умолчанию: on
imap_auth	Задаёт поддерживаемый механизм аутентификации. Допустимы значения (одно или несколько) из следующего списка: login, plain, cram-md5	Допустимые контексты: mail, server По умолчанию: plain
imap_capabilities	Определяет, какие возможности протокола IMAP4 поддерживает проксируемый сервер	Допустимые контексты: mail, server По умолчанию: IMAP4 IMAP4rev1 UIDPLUS
imap_client_buffer	Задаёт размер буфера для чтения команд IMAP	Допустимый контекст: mail, server По умолчанию: 4k 8k (в зависимости от платформы)

Директивы	Описание	Контекст/ Умолчание
include	Путь к файлу, содержащему дополнительные конфигурационные директивы. Может быть задан в виде маски, которой отвечает несколько файлов	Допустимый контекст: любой По умолчанию: -
index	Определяет, какой файл возвращать клиенту в ответ на URI, завершающийся знаком /. Можно указать несколько значений	Допустимые контексты: http, server, location По умолчанию: index.html
internal	Означает, что данное местоположение можно использовать только для внутренних запросов (переедресации, определенной в других директивах, переписывания, страниц ошибок и т. д.)	Допустимые контексты: location По умолчанию: -
ip_hash	Обеспечивает равномерное распределение клиентских соединений по всем серверам за счет хэширования IP-адреса по его сети класса C	Допустимый контекст: upstream По умолчанию: -
keepalive	Количество соединений с проксируемыми серверами, кэшированных в одном рабочем процессе. При использовании с HTTP-соединениями значение proxy_http_version должно быть равно 1.1, а значение proxy_set_header — Connection «»	Допустимый контекст: upstream По умолчанию: -
keepalive_disable	Запрещает соединения типа keep-alive для некоторых браузеров	Допустимые контексты: http, server, location По умолчанию: msie6
keepalive_requests	Определяет, сколько запросов можно принять по одному соединению типа keep-alive, прежде чем закрывать его	Допустимые контексты: http, server, location По умолчанию: 100
keepalive_timeout	Определяет, сколько времени соединение типа keep-alive может оставаться открытым. Можно задать второй параметр, используемый для формирования заголовка ответа «Keep-Alive»	Допустимые контексты: http, server, location По умолчанию: 75s
large_client_header_buffers	Задаёт максимальное число и размер буферов для чтения большого заголовка запроса клиента	Допустимые контексты: http, server По умолчанию: 4 8k

Директивы	Описание	Контекст/ Умолчание
least_conn	Активирует алгоритм балансировки нагрузки, согласно которому очередной запрос передается серверу с наименьшим числом активных соединений	Допустимый контекст: upstream По умолчанию: -
limit_conn	Определяет зону разделяемой памяти (настраиваемую с помощью директивы limit_conn_zone) и максимальное количество соединений с одинаковым значением ключа	Допустимые контексты: http, server, location По умолчанию: -
limit_conn_log_level	Если NGINX ограничивает соединения согласно директиве limit_conn, то эта директива определяет уровень протоколирования для сообщения о достижении пороговой величины	Допустимые контексты: http, server, location По умолчанию: error
limit_conn_zone	В первом параметре задается ключ, к которому относятся ограничения, указанные в директиве limit_conn. Второй параметр задает имя зоны разделяемой памяти, в которой хранится не более указанного числа соединений для каждого ключа, а также размер этой зоны (name:size)	Допустимый контекст: http По умолчанию: -
limit_except	Ограничивает HTTP-методы, доступные внутри секции location (GET включает также HEAD)	Допустимый контекст: location По умолчанию: -
limit_rate	Ограничивает скорость (в байтах/с) отдачи содержимого клиентам. Ограничение действует на уровне соединения, то есть один клиент может повысить свою пропускную способность, открыв несколько соединений	Допустимые контексты: http, server, location, if в location По умолчанию: 0
limit_rate_after	Начинает применять ограничение limit_rate после того, как передано указанное количество байтов	Допустимые контексты: http, server, location, if в location По умолчанию: 0

Директивы	Описание	Контекст/ Умолчание
limit_req	Задаёт ограничение при резком увеличении (всплеске) количества запросов для указанного ключа в зоне разделяемой памяти (заданной в директиве <code>limit_req_zone</code>). Всплеск описывается вторым параметром. Если до возникновения всплеска задерживать запросы не нужно, следует включить третий параметр <code>nodelay</code>	Допустимые контексты: <code>http, server, location</code> По умолчанию: -
limit_req_log_level	Если NGINX ограничивает количество запросов согласно директиве <code>limit_req</code> , то эта директива определяет уровень протоколирования для сообщения о достижении пороговой величины. Сообщение о применении задержки имеет уровень, на единицу меньший указанного в этой директиве	Допустимые контексты: <code>http, server, location</code> По умолчанию: -
limit_req_zone	В первом параметре задается ключ, к которому относятся ограничения, указанные в директиве <code>limit_req</code> . Второй параметр задает имя зоны разделяемой памяти, в которой хранится не более указанного числа запросов для каждого ключа, а также размер этой зоны (<code>name:size</code>). Третий параметр определяет количество запросов в секунду (<code>r/s</code>) или в минуту (<code>r/m</code>), при превышении которого начинает применяться ограничение	Допустимый контекст: <code>http</code> По умолчанию: -
limit_zone	Директива устарела. Использовать вместо нее <code>limit_conn_zone</code>	Допустимый контекст: <code>http</code> По умолчанию: -
lingering_close	Определяет, следует ли оставлять соединение открытым в ожидании дополнительных данных от клиента	Допустимые контексты: <code>http, server, location</code> По умолчанию: <code>on</code>
lingering_time	Связана с директивой <code>lingering_close</code> и определяет, сколько времени держать соединение открытым для обработки дополнительных данных	Допустимые контексты: <code>http, server, location</code> По умолчанию: <code>30s</code>

Директивы	Описание	Контекст/ Умолчание
lingering_timeout	Также связана с директивой <code>lingering_close</code> и определяет, сколько времени держать соединение открытым в ожидании дополнительных данных	Допустимые контексты: <code>http, server, location</code> По умолчанию: 5s
listen (http)	См. таблицу «Параметры директивы <code>listen</code> » в разделе «Секция с описанием виртуального сервера» главы 2 «Руководство по настройке»	Допустимый контекст: <code>server</code> По умолчанию: 80 *: 8000
listen (mail)	Директива <code>listen</code> в NGINX однозначно идентифицирует привязку к сокету и принимает следующий параметр: <code>bind</code>: выполнять отдельный вызов <code>bind()</code> для данной пары <code>address:port</code>	Допустимый контекст: <code>server</code> По умолчанию: -
location	Определяет новый контекст на основе URI запроса	Допустимые контексты: <code>server, location</code> По умолчанию: -
lock_file	Префикс имени файлов-блокировок. На некоторых платформах для реализации директивы <code>accept_mutex</code> и сериализации доступа к памяти может понадобиться файл-блокировка	Допустимый контекст: <code>main</code> По умолчанию: <code>logs/nginx.lock</code> .
log_format	Определяет состав и формат полей в журнале	Допустимый контекст: <code>http</code> По умолчанию: <code>combined</code> <code>\$remote_addr</code> <code>- \$remote_user</code> <code>[\$time_local],</code> <code>«\$request»</code> <code>\$status</code> <code>\$body_bytes_</code> <code>sent, «\$http_</code> <code>referer»»\$http_</code> <code>user_agent»'</code>
log_not_found	Подавляет запись в журнал сообщений об ошибке 404	Допустимые контексты: <code>http, server, location</code> По умолчанию: <code>on</code>

Директивы	Описание	Контекст/ Умолчание
log_subrequest	Разрешает или запрещает протоколирование подзапросов в журнале доступа	Допустимые контексты: http, server, location По умолчанию: off
mail	Определяет конфигурационный контекст, в котором задаются директивы почтового сервера	Допустимый контекст: main По умолчанию: -
map	Определяет новый контекст, в котором некоторой переменной присваивается значение, зависящее от значения исходной переменной. Формат определения контекста следующий: <pre>map \$source-variable \$variable-to-be-set {...}</pre> Сопоставляемая строка (или строки) может также быть регулярным выражением. В контексте распознаются следующие параметры: • default: задает значение переменной по умолчанию в случае, если значение исходной переменной не сопоставилось ни с одной из заданных строк или регулярных выражений; • hostnames: означает, что в качестве исходных значений можно использовать маску для первой или последней части имени хоста; • include: включает файл, содержащий соответствия между строками и значениями	Допустимый контекст: http По умолчанию: -
map_hash_bucket_size	Размер кластера в хэш-таблицах для директивы map	Допустимый контекст: http По умолчанию: 32 64 128
map_hash_max_size	Максимальный размер хэш-таблиц для директивы map	Допустимый контекст: http По умолчанию: 2048
master_process	Определяет, нужно ли запускать рабочие процессы	Допустимый контекст: main По умолчанию: on

Директивы	Описание	Контекст/ Умолчение
<code>max_ranges</code>	Задаёт максимальное количество диапазонов, допустимых в запросе с указанием диапазонов байтов. Если задано значение 0, то поддержка диапазонов байтов отключается	Допустимые контексты: <code>http, server, location</code> По умолчанию: -
<code>memcached_bind</code>	Определяет адрес исходящих соединений с сервером <code>memcached</code>	Допустимые контексты: <code>http, server, location</code> По умолчанию: -
<code>memcached_buffer_size</code>	Размер буфера для ответа от <code>memcached</code> . Этот ответ синхронно отправляется клиенту	Допустимые контексты: <code>http, server, location</code> По умолчанию: 4k 8k
<code>memcached_connect_timeout</code>	Максимальное время, в течение которого NGINX ожидает установления соединения при отправке запроса серверу <code>memcached</code>	Допустимые контексты: <code>http, server, location</code> По умолчанию: 60s
<code>memcached_gzip_flag</code>	Задаёт значение, которое, будучи найдено в ответе от сервера <code>memcached</code> , приведёт к записи значения <code>gzip</code> в заголовок <code>Content-Encoding</code>	Допустимые контексты: <code>http, server, location</code> По умолчанию: -
<code>memcached_next_upstream</code>	См. таблицу «Директивы модуля <code>memcached</code> » в разделе «Кэширование в базе данных» главы 7 «NGINX для разработчика»	Допустимые контексты: <code>http, server, location</code> По умолчанию: <code>error timeout</code>
<code>memcached_pass</code>	Определяет имя или адрес сервера <code>memcached</code> и его порт. Можно также указывать группу серверов, объявленную в контексте <code>upstream</code>	Допустимые контексты: <code>location, if</code> в <code>location</code> По умолчанию: -
<code>memcached_read_timeout</code>	Сколько времени может пройти между двумя последовательными операциями чтения данных от сервера <code>memcached</code> , прежде чем соединение будет закрыто	Допустимые контексты: <code>http, server, location</code> По умолчанию: 60s
<code>memcached_send_timeout</code>	Сколько времени может пройти между двумя последовательными операциями записи данных на сервер <code>memcached</code> , прежде чем соединение будет закрыто	Допустимые контексты: <code>http, server, location</code> По умолчанию: 60s

Директивы	Описание	Контекст/ Умолчание
<code>merge_slashes</code>	Разрешает или запрещает удаление идущих подряд знаков косой черты. Подразумеваемое по умолчанию значение <code>on</code> означает, что NGINX будет заменять несколько соседних знаков / одним	Допустимые контексты: <code>http, server</code> По умолчанию: <code>on</code>
<code>min_delete_depth</code>	Разрешает методу WebDAV DELETE удалять файлы при условии, что число элементов в пути запроса не меньше заданного	Допустимые контексты: <code>http, server, location</code> По умолчанию: <code>0</code>
<code>modern_browser</code>	Задаёт параметры <code>browser</code> и <code>version</code> , которые в совокупности определяют, что этот браузер следует считать современным, и в этом случае в переменную <code>\$modern_browser</code> записывается значение, указанное в директиве <code>modern_browser_value</code> . Параметр <code>browser</code> может принимать следующие значения: <code>msie</code> , <code>gecko</code> , <code>opera</code> , <code>safari</code> , <code>konqueror</code> . Альтернативно можно задать параметр <code>unlisted</code> , который означает, что браузер, не найденный ни в одном из списков <code>ancient_browser</code> и <code>modern_browser</code> или не приславший заголовок <code>User-Agent</code> , следует считать современным	Допустимые контексты: <code>http, server, location</code> По умолчанию: <code>-</code>
<code>modern_browser_value</code>	Значение, записываемое в переменную <code>\$modern_browser</code>	Допустимые контексты: <code>http, server, location</code> По умолчанию: <code>1</code>
<code>mp4</code>	Активирует модуль <code>mp4</code> в данной секции <code>location</code>	Допустимый контекст: <code>location</code> По умолчанию: <code>-</code>
<code>mp4_buffer_size</code>	Задаёт начальный размер буфера для доставки MP4-файлов	Допустимые контексты: <code>http, server, location</code> По умолчанию: <code>512K</code>
<code>mp4_max_buffer_size</code>	Задаёт максимальный размер буфера для обработки метаданных MP4	Допустимые контексты: <code>http, server, location</code> По умолчанию: <code>10M</code>

Директивы	Описание	Контекст/ Умолчение
<code>msie_padding</code>	Разрешает или запрещает добавлять комментарии в ответы со статусом больше 400 для увеличения размера ответа до 512 байт при работе с MSIE	Допустимые контексты: <code>http, server, location</code> По умолчанию: <code>on</code>
<code>msie_refresh</code>	Разрешает или запрещает отправлять MSIE-клиентам ответ <code>Refresh</code> вместо перенаправления	Допустимые контексты: <code>http, server, location</code> По умолчанию: <code>off</code>
<code>multi_accept</code>	Инструктирует рабочий процесс принимать сразу все новые соединения. Игнорируется в случае использования метода обработки соединений <code>kqueue</code> , так как данный метод сам сообщает число новых соединений, ожидающих приёма	Допустимый контекст: <code>events</code> По умолчанию: <code>off</code>
<code>open_file_cache</code>	Настраивает кэш, в котором могут храниться дескрипторы открытых файлов, информация о существовании каталогов и информация об ошибках поиска файлов	Допустимые контексты: <code>http, server, location</code> По умолчанию: <code>off</code>
<code>open_file_cache_errors</code>	Разрешает или запрещает кэширование ошибок поиска файлов в кэше <code>open_file_cache</code>	Допустимые контексты: <code>http, server, location</code> По умолчанию: <code>off</code>
<code>open_file_cache_min_uses</code>	Задаёт минимальное число обращений к файлу в течение времени, заданного параметром <code>inactive</code> директивы <code>open_file_cache</code> , необходимое для того, чтобы дескриптор файла оставался в кэше открытых дескрипторов	Допустимые контексты: <code>http, server, location</code> По умолчанию: <code>1</code>
<code>open_file_cache_valid</code>	Задаёт время между последовательными проверками актуальности данных, хранящихся в кэше <code>open_file_cache</code>	Допустимые контексты: <code>http, server, location</code> По умолчанию: <code>60s</code>
<code>open_log_file_cache</code>	См. таблицу «Директивы протоколирования из модуля HTTP» из раздела «Протоколирование» главы 6 «NGINX как HTTP-сервер»	Допустимые контексты: <code>http, server, location</code> По умолчанию: <code>off</code>
<code>optimize_server_names</code>	Директива устарела. Использовать вместо нее <code>server_name_in_redirect</code>	Допустимые контексты: <code>http, server</code> По умолчанию: <code>off</code>

Директивы	Описание	Контекст/ Умолчание
override_charset	Определяет, следует ли осуществлять перекодирование из кодировки, указанной в заголовке Content-Type ответа, который получен от сервера, заданного в директиве proxy_pass или fastcgi_pass. Если ответ является результатом подзапроса, то перекодирование в кодировку главного запроса производится безусловно	Допустимые контексты: http, server, location, if в location По умолчанию: off
pcre_jit	Разрешает или запрещает JIT-компиляцию совместимых с Perl регулярных выражений, известных на этапе конфигурирования. Чтобы воспользоваться этой оптимизацией, необходимо включить поддержку JIT-компиляции в библиотеке PCRE	Допустимый контекст: main По умолчанию: off
perl	Активирует обработчик Perl в данном местоположении. В аргументе указывается имя обработчика или строка, содержащая полный код подпрограммы	Допустимые контексты: location, limit_except По умолчанию: -
perl_modules	Определяет дополнительные пути поиска Perl-модулей	Допустимый контекст: http По умолчанию: -
perl_require	Задаёт имя Perl-модуля, который должен загружаться при каждом изменении конфигурационного файла NGINX. Может встречаться несколько раз, если требуется указать несколько модулей	Допустимый контекст: http По умолчанию: -
perl_set	Устанавливает Perl-обработчик, который присваивает значение переменной. В аргументе указывается имя обработчика или строка, содержащая полный код подпрограммы	Допустимый контекст: http По умолчанию: -
pid	Файл, в котором хранится идентификатор главного процесса. Переопределяет значение, заданное на этапе конфигурирования и компиляции	Допустимый контекст: main По умолчанию: nginx.pid
pop3_auth	Задаёт поддерживаемый механизм аутентификации. Допустимы значения (одно или несколько) из следующего списка: plain, apop, cram-md5	Допустимые контексты: mail, server По умолчанию: plain

Директивы	Описание	Контекст/ Умолчание
pop3_capabilities	Определяет, какие возможности протокола POP3 поддерживает проксируемый сервер	Допустимые контексты: mail, server По умолчанию: TOP USER UIDL
port_in_redirect	Определяет, нужно ли указывать данный порт при переадресации средствами NGINX	Допустимые контексты: http, server, location По умолчанию: on
postpone_output	Задаёт минимальный размер порции данных, отправляемых клиенту. Если возможно, данные не будут отправляться, пока не накопится указанное количество	Допустимые контексты: http, server, location По умолчанию: 1460
protocol	Определяет, какой протокол поддерживает данный контекст виртуального сервера. Может принимать значения imap, pop3, smtp	Допустимый контекст: server По умолчанию: -
proxy	Разрешает или запрещает проксирование почты	Допустимый контекст: server По умолчанию: -
proxy_bind	Определяет адрес исходящих соединений с проксируемым сервером	Допустимые контексты: http, server, location По умолчанию: -
proxy_buffer	Позволяет задать размер буфера, используемого для проксирования. По умолчанию размер буфера равен размеру страницы	Допустимые контексты: mail, server По умолчанию: 4k 8k (в зависимости от платформы)
proxy_buffer_size	Размер буфера для первой части ответа от проксируемого сервера, в которой находятся заголовки	Допустимые контексты: http, server, location По умолчанию: 4k 8k (в зависимости от платформы)
proxy_buffering	Разрешает или запрещает буферизацию проксированного содержимого. Если буферизация выключена, то ответы отправляются клиенту синхронно по мере получения	Допустимые контексты: http, server, location По умолчанию: on

Директивы	Описание	Контекст/ Умолчание
proxy_buffers	Количество и размер буферов для хранения ответов от проксируемых серверов	Допустимые контексты: http, server, location По умолчанию: 8 k 8k (в зависимости от платформы)
proxy_busy_buffers_size	Суммарный размер буферов, которые могут быть заняты для отправки ответа клиенту, пока ответ ещё не прочитан целиком. Обычно устанавливается в два раза больше, чем размер, указанный в директиве proxy_buffers	Допустимые контексты: http, server, location По умолчанию: 8k 16k (в зависимости от платформы)
proxy_cache	Определяет размер зоны разделяемой памяти, отведенной под кэш	Допустимые контексты: http, server, location По умолчанию: off
proxy_cache_bypass	Одна или несколько строковых переменных. Если хотя бы одна из них не пуста и не содержит нуль, то ответ будет запрошен у проксируемого сервера, а не взят из кэша	Допустимые контексты: http, server, location По умолчанию: -
proxy_cache_key	Строка, которая используется как ключ для поиска значения в кэше. Можно использовать переменные, но следует внимательно следить за тем, чтобы не кэшировалось несколько копий одного и того же содержимого	Допустимые контексты: http, server, location По умолчанию: \$scheme\$proxy_host\$request_uri
proxy_cache_lock	Если эта директива принимает значение on, то предотвращается отправка нескольких запросов проксируемому серверу (или серверам) в случае отсутствия в кэше	Допустимые контексты: http, server, location По умолчанию: off
proxy_cache_lock_timeout	Сколько времени запрос может ждать появления записи в кэше или освобождения блокировки proxy_cache_lock	Допустимые контексты: http, server, location По умолчанию: 5s

Директивы	Описание	Контекст/ Умолчение
proxy_cache_min_uses	Сколько запросов с данным ключом должно поступить, прежде чем ответ будет помещен в кэш	Допустимые контексты: http, server, location По умолчанию: 1
proxy_cache_path	См. таблицу «Директивы модуля проху для управления кэшированием» в разделе «Кэширование в файловой системе» главы 5 «Обратное проксирование, дополнительные вопросы»	Допустимый контекст: http По умолчанию: _
proxy_cache_use_stale	В каких случаях допустимо использовать устаревшие кэшированные данные, если при доступе к проксируемому серверу произошла ошибка. Параметр <code>updating</code> разрешает использовать кэшированный ответ, если как раз в данный момент загружаются более свежие данные	Допустимые контексты: http, server, location По умолчанию: off
proxy_cache_valid	Сколько времени считать действительным кэшированный ответ с кодом 200, 301 или 302. Если перед параметром <code>time</code> указан необязательный код ответа, то заданное время относится только к ответу с таким кодом. Специальный параметр <code>any</code> означает, что в течение заданного времени следует кэшировать ответ с любым кодом	Допустимые контексты: http, server, location По умолчанию: -
proxy_connect_timeout	Максимальное время ожидания соединения с проксируемым сервером	Допустимые контексты: http, server, location По умолчанию: 60s
proxy_cookie_domain	Подменяет атрибут <code>domain</code> в заголовке <code>Set-Cookie</code> от проксируемого сервера; можно указать строку, регулярное выражение или ссылку на переменную	Допустимые контексты: http, server, location По умолчанию: off

Директивы	Описание	Контекст/ Умолчание
proxy_cookie_path	Подменяет атрибут path в заголовке Set-Cookie от проксируемого сервера; можно указать строку, регулярное выражение или ссылку на переменную	Допустимые контексты: http, server, location По умолчанию: off
proxy_headers_hash_bucket_size	Максимальный размер имен заголовков (ни одно имя не может быть длиннее указанной в этой директиве величины)	Допустимые контексты: http, server, location По умолчанию: 64
proxy_headers_hash_max_size	Общий размер заголовков, полученных от проксируемого сервера	Допустимые контексты: http, server, location По умолчанию: 512
proxy_hide_header	Список заголовков, которые не следует передавать клиенту	Допустимые контексты: http, server, location По умолчанию: -
proxy_http_version	Версия протокола HTTP, которой следует придерживаться при взаимодействии с проксируемым сервером (для соединений типа keepalive должна быть 1.1)	Допустимые контексты: http, server, location По умолчанию: 1.0
proxy_ignore_client_abort	Если значение равно on, то NGINX не станет разрывать соединение с проксируемым сервером в случае, когда клиент разрывает свое соединение	Допустимые контексты: http, server, location По умолчанию: off
proxy_ignore_headers	Какие заголовки можно игнорировать при обработке ответа от проксируемого сервера	Допустимые контексты: http, server, location По умолчанию: -
proxy_intercept_errors	Если значение равно on, то NGINX будет отображать страницу, заданную директивой error_page, вместо ответа, полученного от проксируемого сервера	Допустимые контексты: http, server, location По умолчанию: off
proxy_max_temp_file_size	Максимальный размер временного файла, в который записывается часть ответа в случае, когда он не умещается целиком в буферах памяти	Допустимые контексты: http, server, location По умолчанию: 1024m



Директивы	Описание	Контекст/ Умолчение
proxy_next_upstream	Определяет условия, при которых для ответа будет выбран следующий проксируемый сервер. Это не происходит, если клиент уже что-то отправил. Условия задаются с помощью следующих параметров: • <code>error</code>: произошла ошибка при взаимодействии с проксируемым сервером; • <code>timeout</code>: произошел таймаут при взаимодействии с проксируемым сервером; • <code>invalid_header</code>: проксируемый сервер вернул пустой или недопустимый ответ; • <code>http_500</code>: проксируемый сервер вернул ответ с кодом ошибки 500; • <code>http_503</code>: проксируемый сервер вернул ответ с кодом ошибки 503; • <code>http_504</code>: проксируемый сервер вернул ответ с кодом ошибки 504; • <code>http_404</code>: проксируемый сервер вернул ответ с кодом ошибки 404; • <code>off</code>: запретить передачу запроса следующему проксируемому серверу в случае ошибки	Допустимые контексты: <code>http</code>, <code>server</code>, <code>location</code> По умолчанию: <code>error</code> <code>timeout</code>
proxy_no_cache	Определяет условия, при которых ответ не сохраняется в кэше. Одна или несколько строковых переменных. Если хотя бы одна из них непуста и не содержит нуль, то ответ не кэшируется	Допустимые контексты: <code>http</code>, <code>server</code>, <code>location</code> По умолчанию: -



Директивы	Описание	Контекст/ Умолчание
proxy_pass	Определяет проксируемый сервер, которому передается запрос в виде URL	Допустимые контексты: location, if в location, limit_except По умолчанию: -
proxy_pass_error_message	Полезна, если процедура аутентификации на проксируемом сервере возвращает осмысленное сообщение клиенту	Допустимые контексты: mail, server По умолчанию: off
proxy_pass_header	Отменяет сокрытие заголовков, определенных в директиве proxy_hide_header, разрешая передавать их клиенту	Допустимые контексты: http, server, location По умолчанию: -
proxy_pass_request_body	Если значение равно off, то тело запроса не передается проксируемому серверу	Допустимые контексты: http, server, location По умолчанию: on
proxy_pass_request_headers	Если значение равно off, то заголовки запроса не передаются проксируемому серверу	Допустимые контексты: http, server, location По умолчанию: on
proxy_read_timeout	Сколько времени может пройти между двумя последовательными операциями чтения данных от проксируемого сервера, прежде чем соединение будет закрыто. Значение следует увеличить, если проксируемый сервер обрабатывает запросы медленно	Допустимые контексты: http, server, location По умолчанию: 60s
proxy_redirect	Перезаписывает заголовки Location и Refresh, полученные от проксируемого сервера; полезно для обхода допущений, принятых каркасом разработки приложений	Допустимые контексты: http, server, location По умолчанию: default
proxy_send_lowat	Если значение отлично от нуля, NGINX попытается минимизировать количество операций записи по соединению с проксируемым сервером. В Linux, Solaris и Windows игнорируется	Допустимые контексты: http, server, location По умолчанию: 0
proxy_send_timeout	Сколько времени может пройти между двумя последовательными операциями записи данных на проксируемый сервер, прежде чем соединение будет закрыто	Допустимые контексты: http, server, location По умолчанию: 60s

Директивы	Описание	Контекст/ Умолчание
<code>proxy_set_body</code>	Эта директива позволяет изменить тело запроса, отправляемое проксируемому серверу	Допустимые контексты: <code>http, server, location</code> По умолчанию: -
<code>proxy_set_header</code>	Перезаписывает заголовки, отправляемые проксируемому серверу. Может также применяться для подавления некоторых заголовков (если в качестве значения указать пустую строку)	Допустимые контексты: <code>http, server, location</code> По умолчанию: <code>Host \$proxy_host, Connection close</code>
<code>proxy_ssl_session_reuse</code>	Определяет, можно ли повторно использовать SSL-сеансы при проксировании	Допустимые контексты: <code>http, server, location</code> По умолчанию: <code>on</code>
<code>proxy_store</code>	Разрешает сохранение полученных от проксируемого сервера ответов в файлах на диске. Если параметр равен <code>on</code> , то в качестве пути к каталогу с сохраняемыми файлами используется значение, заданное в директиве <code>alias</code> или <code>root</code> . Можно вместо этого задать строку, определяющую другой каталог для хранения файлов	Допустимые контексты: <code>http, server, location</code> По умолчанию: <code>off</code>
<code>proxy_store_access</code>	Какие права доступа следует задать для новых файлов, создаваемых в соответствии с директивой <code>proxy_store</code>	Допустимые контексты: <code>http, server, location</code> По умолчанию: <code>user:rw</code>
<code>proxy_temp_file_write_size</code>	Ограничивает размер данных в одной операции записи во временный файл, чтобы NGINX не слишком долго блокировал исполнение программы при обработке одного запроса	Допустимые контексты: <code>http, server, location</code> По умолчанию: <code>8k 16k</code> (в зависимости от платформы)
<code>proxy_temp_path</code>	Каталог для хранения временных файлов, получаемых от проксируемого сервера. Может быть многоуровневым. Второй, третий и четвертый параметры (если заданы) определяют иерархию подкаталогов в виде количества знаков в имени подкаталога	Допустимые контексты: <code>http, server, location</code> По умолчанию: <code>proxy_temp</code>

Директивы	Описание	Контекст/ Умолчание
proxy_timeout	Используется, если тайм-аут должен быть больше значения по умолчанию – 24 часа	Допустимые контексты: mail, server По умолчанию: 24h
random_index	Активирует выбор случайного файла для отправки пользователю, если URI заканчивается знаком /	Допустимый контекст: location По умолчанию: off
read_ahead	Если возможно, ядро будет сразу считывать из файла столько байтов, сколько указано в параметре size. Поддерживается в текущих версиях FreeBSD и Linux (в Linux параметр size игнорируется)	Допустимые контексты: http, server, location По умолчанию: 0
real_ip_header	Задаёт название заголовка, значение которого рассматривается как IP-адрес клиента, когда директива set_real_ip_from соответствует IP-адресу отправителя	Допустимые контексты: http, server, location По умолчанию: X-Real-IP
real_ip_recursive	Означает, что в случае, когда заголовков real_ip_header содержит несколько адресов, использовать нужно последний	Допустимые контексты: http, server, location По умолчанию: off
recursive_error_pages	Разрешает производить несколько переадресаций с помощью директивы error_page	Допустимые контексты: http, server, location По умолчанию: off
referer_hash_bucket_size	Размер кластера в хэш-таблице допустимых рефереров	Допустимые контексты: server, location По умолчанию: 64
referer_hash_max_size	Максимальный размер хэш-таблицы допустимых рефереров	Допустимые контексты: server, location По умолчанию: 2048
request_pool_size	Позволяет производить точную настройку выделения памяти под конкретные запросы	Допустимые контексты: http, server По умолчанию: 4k
reset_timeout_connection	Если значение этой директивы равно on, то сокеты, для которых истек тайм-аут, сбрасываются немедленно, в результате чего освобождается выделенная для них память. По умолчанию сокет остается в состоянии FIN_WAIT1. На соединения типа keep-alive эта директива не распространяется, они всегда закрываются обычным образом	Допустимые контексты: http, server, location По умолчанию: off

Директивы	Описание	Контекст/ Умолчение
resolver	Задаёт имена одного или нескольких серверов, используемых для получения IP-адресов по доменным именам. Необязательный параметр <code>valid</code> переопределяет время TTL, заданное в записи о доменном имени	Допустимые контексты: <code>http, server, location</code> По умолчанию: -
resolver_timeout	Задаёт тайм-аут разрешения имен	Допустимые контексты: <code>http, server, location</code> По умолчанию: 30s
return	Прекращает обработку и возвращает указанный код клиенту. При возврате нестандартного кода 444 соединение закрывается без отправки заголовков ответа. Если помимо кода указывается ещё и текст, то он помещается в тело ответа. Если же вместо текста указан URL-адрес, то он становится значением заголовка <code>Location</code> . Если указан URL-адрес без кода, то подразумевается код 302	Допустимые контексты: <code>server, location, if</code> По умолчанию: -
rewrite	См. таблицу «Директивы модуля <code>rewrite</code> » в разделе «Введение в модуль <code>rewrite</code> » в приложении В «Руководство по правилам переписывания»	Допустимые контексты: <code>server, location, if</code> По умолчанию: -
rewrite_log	Разрешает или запрещает записывать в <code>error_log</code> на уровне <code>notice</code> результаты перезаписи URL	Допустимые контексты: <code>http, server, if</code> в <code>server</code> , <code>location, if</code> в <code>location</code> По умолчанию: <code>off</code>
root	Задаёт путь к корню документов. Для поиска файлов строка, указанная в URL-адресе, дописывается в конец этого значения	Допустимые контексты: <code>http, server, location, if</code> в <code>location</code> По умолчанию: <code>html</code>
satisfy	Разрешает доступ, если все (<code>all</code>) или хотя бы одна (<code>any</code>) из директив <code>access</code> или <code>auth_basic</code> разрешает доступ. Подразумеваемое по умолчанию значение <code>all</code> говорит, что пользователь должен находиться в определенной сети и ввести правильный пароль	Допустимые контексты: <code>http, server, location</code> По умолчанию: <code>all</code>

Директивы	Описание	Контекст/ Умолчение
<code>satisfy_any</code>	Директива устарела. Использовать директиву <code>satisfy</code> с параметром <code>any</code>	Допустимые контексты: <code>http, server, location</code> По умолчанию: <code>off</code>
<code>secure_link_secret</code>	Затравка для вычисления MD5-свертки ссылки	Допустимый контекст: <code>location</code> По умолчанию: <code>-</code>
<code>send_lowat</code>	Если значение отлично от нуля, то NGINX пытается минимизировать количество операций отправки данных через клиентские сокет. В Linux, Solaris и Windows эта директива игнорируется	Допустимые контексты: <code>http, server, location</code> По умолчанию: <code>0</code>
<code>send_timeout</code>	Задаёт тайм-аут между двумя последовательными операциями записи при передаче ответа клиенту	Допустимые контексты: <code>http, server, location</code> По умолчанию: <code>60s</code>
<code>sendfile</code>	Разрешает использовать системный вызов <code>sendfile (2)</code> для прямого копирования из одного файлового дескриптора в другой	Допустимые контексты: <code>http, server, location, if</code> в <code>location</code> По умолчанию: <code>off</code>
<code>sendfile_max_chunk</code>	Задаёт максимальный размер данных, который можно скопировать за один вызов <code>sendfile (2)</code> . Без этого ограничения одно быстрое соединение может целиком захватить рабочий процесс	Допустимые контексты: <code>http, server, location</code> По умолчанию: <code>0</code>
<code>server (http)</code>	Создаёт новый конфигурационный контекст, в котором определяется виртуальный хост. Директива <code>listen</code> определяет IP-адрес(а) и порт(ы). В директиве <code>server_name</code> перечисляются значения заголовков <code>Host</code> , сопоставляемые с этим контекстом	Допустимый контекст: <code>http</code> По умолчанию: <code>-</code>
<code>server (upstream)</code>	См. таблицу «Директивы модуля <code>upstream</code> » в разделе «Модуль <code>upstream</code> » главы 4 «NGINX как обратный прокси-сервер»	Допустимый контекст: <code>upstream</code> По умолчанию: <code>-</code>
<code>server (mail)</code>	Создаёт новый конфигурационный контекст, в котором определяется почтовый сервер. Директива <code>listen</code> определяет IP-адрес(а) и порт(ы). В директиве <code>server_name</code> указывается имя сервера	Допустимый контекст: <code>mail</code> По умолчанию: <code>-</code>

Директивы	Описание	Контекст/ Умолчание
server_name (http)	Задаёт имя, на которое отвечает данный виртуальный хост	Допустимый контекст: server По умолчанию: «»
server_name (mail)	Задаёт имя сервера, используемое в следующих случаях: - приветствие в протоколе POP3/SMTP; - затравка для аутентификации по методу SASL CRAM-MD5; - имя в команде EHLO, когда для взаимодействия с проксируемым SMTP-сервером используется <code>xclient</code>	Допустимые контексты: mail, server По умолчанию: hostname
server_name_in_redirect	Разрешает использовать первое из указанных в директиве <code>server_name</code> значений при любой переадресации, произведённой NGINX в данном контексте	Допустимые контексты: http, server, location По умолчанию: off
server_names_hash_bucket_size	Размер кластера в хэш-таблице имен серверов	Допустимый контекст: http По умолчанию: 32 64 128 (в зависимости от процессора)
server_names_hash_max_size	Максимальный размер хэш-таблицы имен серверов	Допустимый контекст: http По умолчанию: 512
server_tokens	Разрешает или запрещает включение номера версии NGINX в отправляемые сообщения об ошибках и заголовок <code>Server</code>	Допустимые контексты: http, server, location По умолчанию: on
set	Присваивает значение указанной переменной	Допустимые контексты: server, location, if По умолчанию: -
set_real_ip_from	Задаёт один или несколько адресов, для которых IP-адрес клиента будет извлекаться из заголовка, указанного в директиве <code>real_ip_header</code> . Значение <code>unix</code> : говорит, что таким образом следует трактовать все сокеты в домене UNIX	Допустимые контексты: http, server, location По умолчанию: -
smtp_auth	Задаёт поддерживаемый механизм аутентификации SASL-клиента. Может принимать одно или несколько значений из списка <code>login</code> , <code>plain</code> и <code>cram-md5</code>	Допустимые контексты: mail, server По умолчанию: login, plain

Директивы	Описание	Контекст/ Умолчание
smtp_capabilities	Определяет, какие возможности поддерживает проксируемый SMTP-сервер	Допустимые контексты: mail, server По умолчанию: -
so_keepalive	Для соединения с проксируемым сервером задается режим TCP keepalive	Допустимые контексты: mail, server По умолчанию: off
source_charset	Определяет кодировку ответа. Если она отличается от указанной в директиве charset, то производится перекодирование	Допустимые контексты: http, server, location, if в location По умолчанию: -
split_clients	Создает контекст, в котором определены переменные для A/B-тестирования. Строка, заданная в первом параметре, хэшируется согласно алгоритму MurmurHash2. Затем переменной, заданной во втором параметре, присваивается значение, зависящее от того, в какой диапазон попадает хэш первой строки. Веса задаются либо в процентах, либо в виде символа *	Допустимый контекст: http По умолчанию: -
ssi	Разрешает обработку SSI-файлов	Допустимые контексты: http, server, location, if в location По умолчанию: off
ssi_min_file_chunk	Задаёт минимальный размер файла, начиная с которого имеет смысл посылать его с помощью системного вызова sendfile (2)	Допустимые контексты: http, server, location По умолчанию: 1k
ssi_silent_errors	Подавляет сообщение, которое обычно выводится в случае ошибки во время обработки SSI	Допустимые контексты: http, server, location По умолчанию: off
ssi_types	Указывается список MIME-типов ответа (в дополнение к text/html), для которых обрабатываются команды SSI. Звездочка (*) означает все MIME-типы	Допустимые контексты: http, server, location По умолчанию: text/html
ssi_value_length	Задаёт максимальную длину значений параметров в командах SSI	Допустимые контексты: http, server, location По умолчанию: 256

Директивы	Описание	Контекст/ Умолчение
ssl (http)	Включает или выключает поддержку SSL-транзакций в этом контексте	Допустимые контексты: http, server По умолчанию: off
ssl (mail)	Определяет, следует ли поддерживать в этом контексте транзакции SSL/TLS	Допустимые контексты: mail, server По умолчанию: off
ssl_certificate (http)	Путь к SSL-сертификату в формате PEM для данного виртуального сервера. Если требуются промежуточные сертификаты, то их необходимо поместить в тот же файл после основного сертификата в нужном порядке (корневой – последним)	Допустимые контексты: http, server По умолчанию: -
ssl_certificate (mail)	Путь к SSL-сертификату (или сертификатам) в формате PEM для данного виртуального сервера	Допустимые контексты: mail, server По умолчанию: -
ssl_certificate_key (http)	Путь к файлу, содержащему секретный ключ SSL-сертификата	Допустимые контексты: http, server По умолчанию: -
ssl_certificate_key (mail)	Путь к файлу, содержащему секретный ключ SSL в формате PEM для данного виртуального сервера	Допустимые контексты: mail, server По умолчанию: -
ssl_ciphers	Определяет, какие шифры нужно поддерживать в этом контексте (в формате OpenSSL)	Допустимые контексты: http, server По умолчанию: HIGH: !aNULL: !MD5
ssl_client_certificate	Путь к файлу, содержащему сертификат(ы) в формате PEM общеизвестных удостоверяющих центров, подписывающих сертификаты клиентов	Допустимые контексты: http, server По умолчанию: -
ssl_crl	Путь к файлу, содержащему список отозванных сертификатов (CRL) в формате PEM	Допустимые контексты: http, server По умолчанию: -
ssl_dhparam	Путь к файлу с параметрами для шифров с обменом EDH-ключами	Допустимые контексты: http, server По умолчанию: -

Директивы	Описание	Контекст/ Умолчание
ssl_engine	Задаёт аппаратный ускоритель SSL	Допустимый контекст: main По умолчанию: -
ssl_prefer_server_ciphers (http)	Указывает, что при использовании протоколов SSLv3 и TLSv1 серверные шифры более приоритетны, чем клиентские	Допустимые контексты: http, server По умолчанию: off
ssl_prefer_server_ciphers (mail)	Указывает, что при использовании протоколов SSLv3 и TLSv1 серверные шифры более приоритетны, чем клиентские	Допустимые контексты: mail, server По умолчанию: off
ssl_protocols (http)	Определяет, какие протоколы SSL разрешить	Допустимые контексты: http, server По умолчанию: SSLv3, TLSv1, TLSv1.1, TLSv1.2
ssl_protocols (mail)	Определяет, какие протоколы SSL разрешить	Допустимые контексты: mail, server По умолчанию: SSLv3, TLSv1, TLSv1.1, TLSv1.2
ssl_session_cache (http)	Определяет тип и размер кэша параметров сеансов SSL. Допустимы следующие типы кэша: - off: клиентам сообщается, что сеансы повторно не используются; - none: клиентам сообщается, что сеансы используются повторно, но на самом деле это не так; - builtin: встроенный в OpenSSL кэш используется только одним рабочим процессом; размер кэша задается в сессиях; - shared: кэш разделяется всеми рабочими процессами, задаются имя кэша и размер сессии в мегабайтах	Допустимые контексты: http, server По умолчанию: none

Директивы	Описание	Контекст/ Умолчение
ssl_session_cache (mail)	Определяет тип и размер кэша параметров сеансов SSL. Допустимы следующие типы кэша: • off: клиентам сообщается, что сеансы повторно не используются; • none: клиентам сообщается, что сеансы используются повторно, но на самом деле это не так; • builtin: встроенный в OpenSSL кэш используется только одним рабочим процессом; размер кэша задается в сессиях; • shared: кэш разделяется всеми рабочими процессами, задаются имя кэша и размер сессии в мегабайтах	Допустимые контексты: mail, server По умолчанию: none
ssl_session_timeout (http)	Определяет, сколько времени клиент может пользоваться одними и теми же параметрами SSL при условии, что они хранятся в кэше	Допустимые контексты: http, server По умолчанию: 5m
ssl_session_timeout (mail)	Определяет, сколько времени клиент может пользоваться одними и теми же параметрами SSL при условии, что они хранятся в кэше	Допустимые контексты: mail, server По умолчанию: 5m
ssl_stapling	Разрешает подшивку (stapling) ответов по протоколу OCSP. Сертификат издателя сертификата сервера должен находиться в файле, который указан в директиве ssl_trusted_certificate. Необходимо также указать DNS-сервер, который может разрешить доменное имя OCSP-сервера	Допустимые контексты: http, server По умолчанию: off
ssl_stapling_file	Путь к файлу в формате DER, содержащему подшиваемый ответ OCSP-сервера	Допустимые контексты: http, server По умолчанию: -
ssl_stapling_responder	URL-адрес OCSP-сервера. В настоящее время поддерживаются только URL со схемой http://	Допустимые контексты: http, server По умолчанию: -

Директивы	Описание	Контекст/ Умолчание
ssl_stapling_verify	Разрешает верификацию ответов OCSP-сервера	Допустимые контексты: http, server По умолчанию: -
ssl_trusted_certificate	Если включён режим ssl_stapling, то задает путь к файлу с SSL-сертификатами в формате PEM удостоверяющих центров, которые подписывают клиентские сертификаты и ответы OCSP	Допустимые контексты: http, server По умолчанию: -
ssl_verify_client	Включает проверку клиентских SSL-сертификатов. Если задан параметр optional, то клиентский сертификат запрашивается и при его наличии проверяется. Если задан параметр optional_no_ca, то клиентский сертификат запрашивается, но не требуется, чтобы он был подписан доверенным УЦ	Допустимые контексты: http, server По умолчанию: off
ssl_verify_depth	Какое максимальное число подписавших сторон следует проверить, прежде чем объявить сертификат недействительным	Допустимые контексты: http, server По умолчанию: 1
starttls	Определяет, поддерживаются ли команды STLS/STARTTLS и (или) требуются ли они для взаимодействия с данным сервером	Допустимые контексты: mail, server По умолчанию: off
sub_filter	Задаёт заменяемую строку (без учета регистра) и заменяющую строку. Заменяющая строка может содержать переменные	Допустимые контексты: http, server, location По умолчанию: -
sub_filter_once	Если параметр равен off, то директива sub_filter применяется ко всем вхождениям заменяемой строки	Допустимые контексты: http, server, location По умолчанию: on
sub_filter_types	Указывается список MIME-типов ответа (в дополнение к text/html), для которых производится замена. Звездочка (*) означает все MIME-типы	Допустимые контексты: http, server, location По умолчанию: text/html
tcp_nodelay	Разрешает или запрещает использование параметра TCP_NODELAY для соединений типа keep-alive	Допустимые контексты: http, server, location По умолчанию: on



Директивы	Описание	Контекст/ Умолчание
tcp_nopush	Учитывается только при использовании директивы <code>sendfile</code> . Разрешает NGINX отправлять заголовки ответа одним пакетом, а также передавать файл полными пакетами	Допустимые контексты: <code>http, server, location</code> По умолчанию: <code>off</code>
timeout	Сколько времени NGINX будет ждать установления соединения с проксируемым сервером	Допустимые контексты: <code>mail, server</code> По умолчанию: <code>60s</code>
timer_resolution	Определяет, как часто вызывать функцию <code>gettimeofday()</code> . По умолчанию она вызывается при каждом получении события от ядра	Допустимый контекст: <code>main</code> По умолчанию: <code>-</code>
try_files	Проверяет существование файлов, указанных в параметрах. Если ни один файл, кроме последнего, не найден, то последний параметр считается «последней надеждой», поэтому позаботьтесь о том, чтобы такой файл или именованное местоположение существовали	Допустимые контексты: <code>server, location</code> По умолчанию: <code>-</code>
types	Определяет соответствие между MIME-типами и расширениями имен файлов. В дистрибутив NGINX входит файл <code>conf/mime.types</code> , содержащий большинство соответствий. Как правило, достаточно просто включить этот файл директивой <code>include</code>	Допустимые контексты: <code>http, server, location</code> По умолчанию: <code>text/html html;</code> <code>image/gif gif;</code> <code>image/jpeg jpg;</code>
types_hash_bucket_size	Размер кластера хэш-таблицы MIME-типов	Допустимые контексты: <code>http, server, location</code> По умолчанию: <code>32 64 128</code> (в зависимости от процессора)

Директивы	Описание	Контекст/ Умолчение
<code>types_hash_max_size</code>	Максимальный размер хэш-таблицы MIME-типов	Допустимые контексты: <code>http, server, location</code> По умолчанию: 1024
<code>underscores_in_headers</code>	Разрешает или запрещает использование символа подчеркивания в заголовках запросов от клиентов. Если оставлено подразумеваемое по умолчанию значение <code>off</code> , то анализ таких заголовков производится согласно режиму, заданному в директиве <code>ignore_invalid_headers</code>	Допустимые контексты: <code>http, server</code> По умолчанию: <code>off</code>
<code>uninitialized_variable_warn</code>	Следует ли помещать в журнал сообщения о неинициализированных переменных	Допустимые контексты: <code>http, server, location, if</code> По умолчанию: <code>on</code>
<code>upstream</code>	Открывает именованный контекст, в котором определена группа серверов	Допустимый контекст: <code>http</code> По умолчанию: -
<code>use</code>	Задаёт способ обработки соединений. Эта директива переопределяет режим, заданный при компиляции, и если используется, то должна находиться в контексте <code>events</code> . Она особенно полезна, если обнаруживается, что со временем режим по умолчанию начинает приводить к ошибкам	Допустимый контекст: <code>events</code> По умолчанию: -
<code>user</code>	Пользователь и группа, от имени которых исполняются рабочие процессы. Если группа опущена, то подразумевается группа, имя которой совпадает с именем пользователя	Допустимый контекст: <code>main</code> По умолчанию: <code>nobody</code> <code>nobody</code>

Директивы	Описание	Контекст/ Умолчение
userid	Активирует модуль в соответствии со следующими параметрами: • <code>on</code>: устанавливает куки версии 2 и записывает в журнал полученные куки; • <code>v1</code>: устанавливает куки версии 1 и записывает в журнал полученные куки; • <code>log</code>: отключает установку куков, но продолжает записывать их в журнал; • <code>off</code>: отключает как установку, так и протоколирование куков	Допустимые контексты: <code>http, server, location</code> По умолчанию: <code>off</code>
userid_domain	Настраивает записываемый в куки домен	Допустимые контексты: <code>http, server, location</code> По умолчанию: <code>none</code>
userid_expires	Задаёт срок хранения куки. Если указано значение <code>max</code>, то устанавливается дата 31 Dec 2037 23:55:55 GMT	Допустимые контексты: <code>http, server, location</code> По умолчанию: <code>-</code>
userid_mark	Задаёт первый символ окончания base64-представления куки с именем <code>userid_name</code>	Допустимые контексты: <code>http, server, location</code> По умолчанию: <code>off</code>
userid_name	Задаёт имя куки	Допустимые контексты: <code>http, server, location</code> По умолчанию: <code>uid</code>
userid_p3p	Настраивает заголовок P3P для сайтов, которые объявляют свою политику конфиденциальности, следуя протоколу Platform for Privacy Preferences Project	Допустимые контексты: <code>http, server, location</code> По умолчанию: <code>-</code>
userid_path	Задаёт путь, указываемый в куки	Допустимые контексты: <code>http, server, location</code> По умолчанию: <code>/</code>
userid_service	Идентификатор службы, которая устанавливает куки. Например, для куки версии 2 по умолчанию подразумевается IP-адрес сервера, установившего куки	Допустимые контексты: <code>http, server, location</code> По умолчанию: IP-адрес сервера

Директивы	Описание	Контекст/ Умолчание
valid_referers	Определяет, при каких значениях заголовка <code>Referer</code> переменная <code>\$invalid_referer</code> будет содержать пустую строку. В остальных случаях в эту переменную будет записана 1. Параметр может принимать одно или несколько значений из следующего списка: • <code>none</code>: заголовок <code>Referer</code> нет; • <code>blocked</code>: заголовок <code>Referer</code> присутствует, но пуст, или содержит URL без схемы; • <code>server_names</code>: в заголовке <code>Referer</code> указано одно из имен, перечисленных в директиве <code>server_name</code>; • произвольная строка: задает имя сервера в заголовке <code>Referer</code>, возможно с префиксом URI и знаком <code>*</code> в начале или в конце; • регулярное выражение: сопоставляется с частью указанного в заголовке <code>Referer</code> значения после схемы	Допустимые контексты: <code>server</code>, <code>location</code> По умолчанию: -
variables_hash_bucket_size	Размер кластера хэш-таблицы переменных	Допустимый контекст: <code>http</code> По умолчанию: 64
variables_hash_max_size	Максимальный размер хэш-таблицы переменных	Допустимые контексты: <code>http</code>, <code>server</code>, <code>location</code> По умолчанию: 512
worker_aio_requests	Количество открытых асинхронных операций ввода-вывода в одном рабочем процессе при использовании <code>aio</code> в режиме <code>epoll</code>	Допустимый контекст: <code>events</code> По умолчанию: 32
worker_connections	Задаёт максимальное число соединений, одновременно открытых в одном рабочем процессе. Сюда входят, в частности, соединения с клиентами и с проксируемыми серверами (но не только)	Допустимый контекст: <code>events</code> По умолчанию: 512

Директивы	Описание	Контекст/ Умолчание
worker_cpu_affinity	Привязывает рабочие процессы к группам процессоров, как описано в битовой маске. Доступна только в ОС FreeBSD и Linux	Допустимый контекст: main По умолчанию: -
worker_priority	Задаёт приоритет планирования для рабочих процессов. Работает как команда nice – отрицательные значения означают более высокий приоритет	Допустимый контекст: main По умолчанию: 0
worker_processes	Количество рабочих процессов, создаваемых сразу после запуска. Эти процессы обрабатывают запросы на соединения со стороны клиентов. Правильный выбор значения – сложная задача, но для начала можно взять количество процессорных ядер	Допустимый контекст: main По умолчанию: 1
worker_rlimit_core	Изменяет ограничение на размер файла core для рабочих процессов	Допустимый контекст: main По умолчанию: -
worker_rlimit_nofile	Изменяет ограничение на количество файлов, открытых в рабочем процессе	Допустимый контекст: main По умолчанию: -
worker_rlimit_sigpending	Изменяет ограничение на размер очереди сигналов для рабочего процесса при использовании метода обработки соединений rtsig	Допустимый контекст: main По умолчанию: -
working_directory	Текущий каталог для рабочих процессов. Чтобы рабочие процессы могли создавать файлы core, в этот каталог должна быть разрешена запись	Допустимый контекст: main По умолчанию: -
xclient	В протоколе SMTP предусмотрена проверка на основе параметров IP/HELO/LOGIN, которые передаются в команде XCLIENT. Данная директива позволяет NGINX передать эту информацию серверу	Допустимые контексты: mail, server По умолчанию: on
xml_entities	Путь к файлу DTD-схемы, где объявлены знаковые сущности, на которые есть ссылки в обрабатываемом XML-файле	Допустимые контексты: http, server, location По умолчанию: -

Директивы	Описание	Контекст/ Умолчание
xslt_param	Параметры, передаваемые в таблицы стилей; значениями являются выражения XPath	Допустимые контексты: http, server, location По умолчанию: -
xslt_string_param	Параметры, передаваемые в таблицы стилей; значениями являются строки	Допустимые контексты: http, server, location По умолчанию: -
xslt_stylesheet	Путь к таблице стилей XSLT, применяемой для преобразования XML-ответа. Можно передать параметры в виде списка пар ключ-значение	Допустимый контекст: location По умолчанию: -
xslt_types	Указывается список MIME-типов ответа (в дополнение к text/xml), для которых производится замена. Звездочка (*) означает все MIME-типы. Если результатом преобразования является HTML-документ, то его MIME-тип будет заменен на text/html	Допустимые контексты: http, server, location По умолчанию: text/xml



Приложение В. Руководство по правилам переписывания

Это приложение является введением в модуль NGINX `rewrite` и может использоваться как руководство по созданию новых правил и преобразованию правил переписывания из формата Apache в формат NGINX. Мы обсудим следующие вопросы.

- ☐ Введение в модуль `rewrite`.
- ☐ Создание новых правил переписывания.
- ☐ Преобразование правил из формата Apache.

Введение в модуль `rewrite`

Модуль `rewrite` в NGINX представляет собой простой сопоставитель с регулярными выражениями в сочетании с виртуальной стековой машиной. Первая часть любого правила переписывания – это регулярное выражение. Некоторые его части можно заключить в круглые скобки, это будут «запоминаемые подвыражения», на которые впоследствии можно сослаться с помощью позиционных переменных. Значение позиционной переменной зависит от порядка следования соответствующего ей запоминаемого подвыражения. Позиционные переменные обозначаются числами, так что `$1` ссылается на значение первого запомненного подвыражения, `$2` – на значение второго и т. д. Рассмотрим, например, такое регулярное выражение:

```
~/images/([a-z]{2})/([a-z0-9]{5})/(.*)\.(png|jpg|gif)$
```

Первая позиционная переменная `$1` ссылается на строку из двух букв, которая следует сразу за строкой `/images/` в начале URI-адреса. Вторая позиционная переменная `$2` ссылается на строку из пяти символов – строчных букв и цифр от 0 до 9. Третья позиционная

переменная \$3 предположительно является именем файла. И последняя переменная, выделяемая этим регулярным выражением, \$4, содержит одну из строк png, jpg или gif, находящуюся в самом конце URI.

Вторая часть правила переписывания – это переписанный URI-адрес запроса. Он может содержать позиционные переменные, выделенные регулярным выражением, а также любые другие переменные, допустимые на данном уровне конфигурационного файла NGINX:

```
/data?file=$3.$4
```

Если этот URI не соответствует ни одному местоположению, определенному в конфигурации NGINX, то он возвращается клиенту в заголовке Location с кодом состояния 301 (Moved Permanently) или 302 (Found), означающим, что требуется переадресация (постоянная или временная). Код состояния можно указать и явно, если третий параметр директивы равен permanent или redirect.

Третьим параметром правила переписывания может быть также ключевое слово last или break, означающее, что никаких других директив модуля rewrite обрабатывать не надо. Флаг last инструктирует NGINX искать местоположение, соответствующее переписанному URI.

```
rewrite '~/images/([a-z]{2})/([a-z0-9]{5})/(.*)\.(png|jpg|gif)$'
/data?file=$3.$4 last;
```

Флаг break может употребляться и в качестве самостоятельной директивы, чтобы прекратить обработку директивы модуля rewrite внутри блока if или в другом контексте, где модуль rewrite активен. Во фрагменте кода ниже предполагается, что какой-то внешний метод записывает в переменную \$bwhog непустое и ненулевое значение в случае, когда клиент потребляет слишком большую часть полосы пропускания. Тогда директива limit_rate принудительно понизит скорость передачи. Директива break в данном случае включена потому, что блок if находится внутри модуля rewrite, и мы не хотим обрабатывать другие директивы этого модуля.

```
if ($bwhog) {
    limit_rate 300k;
    break;
}
```

Еще один способ прекратить обработку директив модуля `rewrite` состоит в том, чтобы с помощью директивы `return` вернуть управление главному модулю `http`, обрабатывающему запрос. Это может означать, что NGINX возвращает информацию непосредственно клиенту, но чаще `return` употребляется в сочетании с `error_page`, чтобы либо представить клиенту отформатированную HTML-страницу, либо активировать другой модуль, который завершит обработку запроса. В директиве `return` можно указать только код состояния, код состояния с текстом или код состояния с URI-адресом. Если в качестве единственного параметра указан URI-адрес, подразумевается код состояния 302. Если за кодом состояния следует текст, то он становится телом ответа. Если же после кода состояния указан URI-адрес, то он становится значением заголовка `Location`, то есть адресом, на который переадресуется клиент.

Пусть, например, мы хотим задать короткий текст сообщения о том, что в некотором местоположении не найден файл. Местоположение опишем со знаком равенства, то есть будем сравнивать на точное совпадение:

```
location = /image404.html {  
    return 404 «image not found\n»;  
}
```

На любое обращение к этому URI-адресу будет возвращен ответ с кодом 404 и текстом **image not found\n**. Таким образом, мы могли бы использовать местоположение `/image404.html` в конце директивы `try_files` или как страницу ошибки для графических файлов.

Помимо директив, относящихся собственно к переписыванию URI, модуль `rewrite` включает также директиву `set`, которая создает переменные и присваивает им значения. Это полезно для разных целей: от создания флагов при выполнении некоторых условий до передачи именованных аргументов в другое местоположение и протоколирования произведенных действий.

В примере ниже демонстрируются некоторые из этих идей и применение соответствующих директив.

```
http {  
  
    # специальный формат журнала, в котором используются переменные,  
    # определяемые ниже  
    log_format imagelog '[time_local] ' $image_file ' ' $image_type ' ' $body_  
bytes_sent ' ' $status;
```



```
# мы хотим включить отладку правил переписывания, чтобы убедиться,
# что правило работает, как задумано
rewrite_log on;

server {
    root /home/www;

    location / {
        # определяем, в какой журнал должны записываться отладочные
        # сообщения, касающиеся правил переписывания
        error_log logs/rewrite.log notice;

        # Наше правило, в котором используются запоминаемые подвыражения
        # и позиционные переменные. Обратите внимание на кавычки вокруг
        # регулярного выражения – они необходимы, потому что фигурные
        # скобки {} встречаются в самом выражении.
        rewrite '~^/images/([a-z]{2})/([a-z0-9]{5})/(.*)\.(png|jpg|gif)$' /
data?file=$3.$4;

        # Обратите внимание на отсутствие параметра 'last' в выражении выше.
        # Если бы мы включили его, то переменные не были бы созданы, потому
        # NGINX прекратила бы обработку директив модуля rewrite.

        # Здесь устанавливаются переменные, используемые в формате
        # журнала 'imagemlog'
        set $image_file $3;
        set $image_type $4;
    }

    location /data {
        # мы хотим протоколировать обращения ко всем изображениям в
        # этом журнале со специальным форматом, чтобы потом было проще
        # выделить тип и размер
        access_log logs/images.log imagemlog;
        root /data/images;

        # можно было бы воспользоваться определенными ранее переменными
        # $image-*, но ссылка на аргумент выглядит понятнее
        try_files /$arg_file /image404.html;
    }

    location = /image404.html {
        # специальное сообщение об ошибке для несуществующих графических
        # файлов
        return 404 «image not found\n»;
    }
}
```

В таблице ниже перечислены все директивы модуля `rewrite`, рассматриваемые в этом разделе.

Директивы модуля `rewrite`

Директива	Описание
<code>break</code>	Завершает обработку директив модуля <code>rewrite</code> , находящихся в текущем контексте
<code>if</code>	Вычисляет условие, и если оно истинно, то выполняет следующие далее директивы модуля <code>rewrite</code>: <pre>if (условие) { ... }</pre> Допустимые условия: - имя переменной: <code>false</code>, если значение пусто или является строкой, которая начинается символом 0; - сравнение строк: с использованием операторов <code>=</code> и <code>!=</code>; - сопоставление с регулярным выражением: с использованием операторов <code>~</code> (с учетом регистра), <code>~*</code> (без учета регистра) и соответствующих операторов отрицания <code>!</code> и <code>!~*</code>; - существование файла: с использованием операторов <code>-f</code> и <code>!-f</code>; - существование каталога: с использованием операторов <code>-d</code> и <code>!-d</code>; - существование файла, каталога или символической ссылки: с использованием операторов <code>-e</code> и <code>!-e</code>; - исполнимость файла: с использованием операторов <code>-x</code> и <code>!-x</code>
<code>return</code>	Прекращает обработку и возвращает клиенту указанный код. Нестандартный код 444 означает закрытие соединения без отправки заголовков ответа. Если с кодом дополнительно ассоциирован текст, то он помещается в тело ответа. Если же за кодом следует URL-адрес, то он становится значением заголовка <code>Location</code> . При наличии одного лишь URL-адреса, без кода, подразумевается код 302
<code>rewrite</code>	Подменяет URI-адрес, сопоставившийся с регулярным выражением, заданным в первом параметре, строкой из второго параметра. В качестве необязательного третьего параметра может быть задан один из следующих флагов: <code>last</code>: прекращает обработку директив модуля <code>rewrite</code> и ищет местоположение, соответствующее измененному URI-адресу; <code>break</code>: прекращает обработку директив модуля <code>rewrite</code>; <code>redirect</code>: возвращает код временной переадресации (302); используется, если URI не начинается схемой; <code>permanent</code>: возвращает код постоянной переадресации (301)
<code>rewrite_log</code>	Разрешает или запрещает записывать в <code>error_log</code> на уровне <code>notice</code> результаты обработки директив модуля <code>rewrite</code>
<code>set</code>	Присваивает значение указанной переменной
<code>uninitialized_variable_warn</code>	Определяет, нужно ли записывать в журнал предупреждения о неинициализированных переменных

Создание новых правил переписывания

Создавая новое правило с нуля, подумайте, чего точно вы хотите добиться. Задайте себе следующие вопросы.

- ☐ Как устроены мои URL-адреса?
- ☐ Можно ли попасть на некоторую страницу несколькими способами?
- ☐ Хочу ли я запоминать части URL в переменных?
- ☐ Буду ли я осуществлять переадресацию на сайт, находящийся на другом сервере, или могу снова наткнуться на свое же правило?
- ☐ Хочу ли я заменять аргументы в строке запроса?

Уяснить, как устроены URL-адреса, можно путем анализа структуры сайта или веб-приложения. Если на некоторую страницу можно попасть несколькими способами, то создайте правило переписывания, которое отправит клиенту код постоянной переадресации. Имея эту информацию, вы сможете построить каноническое представление сайта или приложения. Это не только поможет навести порядок в URL, но и позволит проще находить ваш сайт в Интернете.

Например, пусть имеется контроллер `home`, обрабатывающий адреса по умолчанию, и пусть попасть на тот же контроллер можно через индексную страницу. Таким образом, при переходе по любому из следующих URL пользователь получит одну и ту же информацию:

```
/
/home
/home/
/home/index
/home/index/
/index
/index.php
/index.php/
```

Было бы эффективнее переадресовать запросы, содержащие имя контроллера и (или) индексную страницу, на корень:

```
rewrite ~(home(/index)?|index(\.php)?)/?$ $scheme://$host/ permanent;
```

Мы воспользовались переменными `$scheme` и `$host`, потому что производим постоянную переадресацию (код 301) и хотим, чтобы NGINX построила URL с теми же параметрами, которые были в исходном URL.

Чтобы поместить в журнал отдельные части URL, можно было бы включить в регулярное выражение выделяющие их запоминаемые подвыражения, затем скопировать позиционные переменные в именованные, на которые уже сослаться в определении формата журнала. Подобный пример мы видели в предыдущем разделе. Компоненты выделяются следующим образом:

```
log_format imagelog '[$time_local] ' $image_file ' ' $image_type ' ' $body_
bytes_sent ' ' $status;

rewrite '^/images/([a-z]{2})/([a-z0-9]{5})/(.*)\.(png|jpg|gif)$' /
data?file=$3.$4;

set $image_file $3;
set $image_type $4;

access_log logs/images.log imagelog;
```

Если правило переписывания завершается внутренней переадресацией или инструктирует клиента обратиться к местоположению, в котором определено само это правило, то следует позаботиться о том, чтобы не заикнуться. Например, в контексте `server` правило можно определить с флагом `last`, но при определении в местоположении, на которое оно ссылается, необходимо использовать флаг `break`.

```
server {
    rewrite ^(/images)/(.*)\.(png|jpg|gif)$ $1/$3/$2.$3 last;

    location /images/ {
        rewrite ^(/images)/(.*)\.(png|jpg|gif)$ $1/$3/$2.$3 break;
    }
}
```

Передача дополнительных аргументов в командной строке – одна из целей, преследуемых при создании правил переписывания. Однако если исходные аргументы следует отбросить, а использовать только те, что определены в правиле, то в конце списка новых аргументов нужно поставить знак `?`.

```
rewrite ^/images/(.*)_(\d+)\(\d+)\.(png|jpg|gif)$ /resizer/$1.$4?width
=$2&height=$3? last;
```

Преобразование правил из формата Apache

Созданию правил переписывания для наделенного развитой функциональностью модуля Apache `mod_rewrite` посвящено немало ресурсов в Интернете. Для преобразования правил, записанных в формате Apache, в формат, понятный NGINX, нужно придерживаться нескольких простых рекомендаций.

Рекомендация 1: заменить проверки существования каталогов и файлов директивой `try_files`

Встретив правило переписывания для Apache такого вида:

```
RewriteCond %{REQUEST_FILENAME} !-f
RewriteCond %{REQUEST_FILENAME} !-d
RewriteRule ^(.*)$ index.php?q=$1 [L]
```

преобразуйте его в следующий фрагмент конфигурации NGINX:

```
try_files $uri $uri/ /index.php?q=$uri;
```

Эти правила говорят, что в случае, когда заданному в URI имени не соответствует ни файл, ни каталог на диске, запрос следует переадресовать файлу `index.php` в корне текущего контекста, передав в аргументе `q` исходный URI-адрес.

Пока в NGINX не появилась директива `try_files`, не было никакого другого выбора, кроме как проверять существование файла или каталога:

```
if (!-e $request_filename) {
    rewrite ^/(.*)$ /index.php?q=$1 last;
}
```

Не делайте этого. В Интернете можно встретить советы поступать именно таким образом, но они либо были даны очень давно, либо скопированы из устаревших конфигурационных файлов. Хотя директива `try_files` принадлежит базовому модулю `http` и, строго говоря, не может считаться правилом переписывания, указанную задачу она решает гораздо эффективнее, так как именно для этого и создавалась.

Рекомендация 2: заменить сравнение с `REQUEST_URI` секцией `location`

В Apache правила переписывания часто помещают в файлы `.htaccess`, так как исторически сложилось, что пользователи должны были иметь доступ к этим файлам. Типичный администратор коллективного хостинга вряд ли даст пользователю прямой доступ к конфигурационному контексту виртуального хоста, описывающего его сайт, но зато позволит помещать практически любую информацию в файл `.htaccess`. Это и привело к нынешней ситуации, когда файлы `.htaccess` забиты правилами переписывания.

Хотя в Apache тоже имеется директива `Location`, она редко используется для сопоставления с URI, потому что может встречаться либо в конфигурации главного сервера, либо в конфигурации виртуального хоста и нигде более. Поэтому-то мы то и дело натыкаемся на правила переписывания, в которых производится сравнение с `REQUEST_URI`:

```
RewriteCond %{REQUEST_URI} ^/niceurl
RewriteRule ^(.*)$ /index.php?q=$1 [L]
```

В NGINX для этой цели лучше использовать местоположение:

```
location /niceurl {
    include fastcgi_params;
    fastcgi_index index.php;
    fastcgi_pass 127.0.0.1:9000;
}
```

Разумеется, директивы в контексте `location` всецело зависят от вашего сайта, но принцип остается в силе – сопоставление с URI лучше производить с помощью местоположений.

То же относится и к директивам `RewriteRule` с неявным `REQUEST_URI`. Как правило, они просто преобразуют URI из старого формата в новый. Так, в примере ниже `show.do` больше не нужен:

```
RewriteRule ^/controller/show.do$ http://example.com/controller [L,R=301]
```

Такое правило транслируется в следующий фрагмент конфигурации NGINX:

```
location = /controller/show.do {
    rewrite ^ http://example.com/controller permanent;
}
```

Чтобы не создавать слишком много местоположений при преобразовании директив `RewriteRule`, следует помнить, что регулярные выражения переносятся без изменений.

Рекомендация 3: заменить сравнение с HTTP_HOST секцией server

Эта рекомендация тесно связана с предыдущей; мы рассматриваем конфигурации, в которых нужно добавить либо удалить из доменного имени часть `www`. Такие правила переписывания нередко встречаются в файлах `.htaccess` или в виртуальных хостах с перегруженными `ServerAlias`:

```
RewriteCond %{HTTP_HOST} !^www
RewriteRule ^(.*)$ http://www.example.com/$1 [L,R=301]
```

Здесь мы транслируем URL без `www` в начале доменного имени в URL, содержащий `www`:

```
server {
    server_name example.com;
    rewrite ^ http://www.example.com$request_uri permanent;
}
```

В противоположном случае, когда требуется убрать `www`, мы встречаемся с таким правилом:

```
RewriteCond %{HTTP_HOST} ^www
RewriteRule ^(.*)$ http://example.com/$1 [L,R=301]
```

Оно транслируется в следующую конфигурацию NGINX:

```
server {
    server_name www.example.com;
    rewrite ^ http://example.com$request_uri permanent;
}
```

Контекст `server` для переадресованного варианта не показан, потому что к переписыванию он отношения не имеет.

Тот же принцип применим и в случаях, когда нужно не просто распознать наличие или отсутствие `www`. Его можно использовать всегда, когда в условии `RewriteCond` встречается `%{HTTP_HOST}`. В NGINX для этого лучше заводить контексты `server`, по одному для каждого проверяемого условия.

Пусть, например, имеется такая конфигурация Apache с несколькими сайтами:

```
RewriteCond %{HTTP_HOST} ^site1
RewriteRule ^(.*)$ /site1/$1 [L]
RewriteCond %{HTTP_HOST} ^site2
RewriteRule ^(.*)$ /site2/$1 [L]
RewriteCond %{HTTP_HOST} ^site3
RewriteRule ^(.*)$ /site3/$1 [L]
```

Она транслируется в конфигурацию, где производится сравнение с именем хоста и для каждого хоста задается своя директива `root`.

```
server {
    server_name site1.example.com;
    root /home/www/site1;
}

server {
    server_name site2.example.com;
    root /home/www/site2;
}

server {
    server_name site3.example.com;
    root /home/www/site3;
}
```

По существу, это виртуальные хосты, так что и описывать их в конфигурационном файле следует соответственно.

Рекомендация 4: заменить RewriteCond проверкой переменной в директиве if

Эта рекомендация применяется после рекомендаций 1–3. Если после их применения остались еще какие-то условия, то для проверки переменных можно использовать директиву `if`. Любой HTTP-переменной соответствует переменная NGINX, имя которой образовано путем перевода имени исходной переменной в нижний регистр и добавления префикса `$http_`. Дефисы (-) при этом следует заменять знаками подчеркивания (_).

Следующий пример (взятый из документации Apache по модулю `mod_rewrite` на странице http://httpd.apache.org/docs/2.2/mod/mod_rewrite.html) позволяет вернуть пользователю ту или иную страницу в зависимости от заголовка `User-Agent`:

```
RewriteCond %{HTTP_USER_AGENT} ^Mozilla
RewriteRule ^/$ /homepage.max.html [L]
RewriteCond %{HTTP_USER_AGENT} ^Lynx
RewriteRule ^/$ /homepage.min.html [L]
RewriteRule ^/$ /homepage.std.html [L]
```

Эквивалентная конфигурация NGINX выглядит следующим образом:

```
if ($http_user_agent ~* ^Mozilla) {
    rewrite ^/$ /homepage.max.html break;
}
if ($http_user_agent ~* ^Lynx) {
    rewrite ^/$ /homepage.min.html break;
}
index homepage.std.html;
```

Конечно, специальные переменные, доступные только в модуле Apache `mod_rewrite`, в NGINX проверить невозможно.

Резюме

В этом приложении мы рассмотрели модуль NGINX `rewrite`. С ним ассоциировано лишь несколько директив, но это не мешает создавать достаточно сложные конфигурации. Хочется надеяться, что пошаговое рассмотрение процедуры создания новых правил убедило вас, что это не так уж трудно. Но для написания сколько-нибудь сложных правил необходимо уверенное владение регулярными выражениями. В конце мы обсудили, как преобразовать правила переписывания из формата Apache в конфигурацию, понятную NGINX. Попутно мы обнаружили, что многие задачи, которые в Apache решаются с помощью правил переписывания, в NGINX можно решить по-другому.



Приложение С. Сообщество NGINX

Ныне NGINX поддерживается не только полным жизни сообществом, но и компанией. Игорь Сысоев, автор NGINX, в 2011 году стал одним из учредителей компании NGINX, Inc., которая предлагает профессиональную поддержку организациям, использующим NGINX. Но и он сам, и другие разработчики NGINX не отделились от сообщества. В этом приложении приводится краткий обзор общедоступных сетевых ресурсов. Рассматриваются следующие вопросы:

- ☐ Список рассылки.
- ☐ IRC-канал.
- ☐ Веб-ресурсы.
- ☐ Как правильно составить отчет об ошибке.

Список рассылки

Список рассылки по адресу nginx@nginx.org поддерживается с 2005 года. Чтобы понять, как эффективнее всего поставить этот список себе на пользу, подпишитесь на него и посмотрите, какие в нем задают вопросы и как на них отвечают. Прежде чем задавать вопрос, поищите ответ в Сети. Имеется также список ЧАВО по адресу <http://wiki.nginx.org/Faq>. Покопайтесь в архивах на сайте <http://mailman.nginx.org/pipermail/nginx/> — быть может, кто-то недавно задавал аналогичный вопрос. Повторять недавно рассматривавшийся вопрос зазорно, к тому же это раздражает читателей списка рассылки.

IRC-канал

IRC-канал `#nginx` на сайте irc.freenode.net — ресурс для тех, кто хочет познакомиться с разработчиками и немедленно получить полезный ответ на короткий вопрос. Но при посещении канала соблю-

дайте этикет IRC. Длинные куски текста – конфигурационные файлы или вывод компилятора – следует размещать на каком-нибудь стороннем сайте, а в канале оставлять только URL. Дополнительные сведения об этом канале можно найти по адресу <http://wiki.nginx.org/IRC>.

Веб-ресурсы

Вики по адресу <http://wiki.nginx.org> уже много лет является полезным ресурсом. Здесь вы найдете полный справочник по директивам, список модулей и много примеров конфигураций. Однако помните, что это вики, поэтому информация не обязательно точна, актуальна или отвечает вашей конкретной ситуации. В этой книге мы неоднократно видели, что, прежде чем предлагать решение, очень важно ясно понять, чего мы хотим достичь.

Компания NGINX, Inc. публикует официальную справочную документацию по адресу <http://nginx.org/en/docs/>. Там же имеются краткое введение в NGINX, различные пособия и описания всех модулей и директив.

Как правильно составить отчет об ошибке

Обращаясь за помощью, нужно знать, как правильно составить отчет об ошибке. Ответ придет гораздо быстрее, если вы сможете описать проблему ясно и так, чтобы ее можно было воспроизвести. В этом разделе мы покажем, как это сделать.

Самое трудное в отчете об ошибке – определить, в чем состоит проблема. Сначала подумайте, чего вы добиваетесь. Сформулируйте задачу четко и кратко, например:

Я хочу, чтобы все запросы к имени `subdomain.example.com` обслуживались сервером `server1`.

Не пишите в таком духе:

Когда я обращаюсь к `subdomain.example.com`, все запросы обслуживаются из локальной файловой системы, а не проксируются на `server1`.

Улавливаете разницу? В первом случае вы четко формулируете цель, а во втором описываете результат, а не исходную задачу.



Определив, в чем проблема, опишите, как ее воспроизвести:

Обращение к <http://subdomain.example.com/serverstatus> дает «404 File Not Found».

Теперь всякий, кто займется вашей проблемой, будет знать, на что смотреть. И как доказать, что проблема решена, – неработавший пример должен заработать.

Далее полезно описать, в какой среде наблюдалась проблема. Некоторые ошибки проявляются только в определенных операционных системах и при компоновке с конкретной версией той или иной библиотеки.

В отчет следует также включить все конфигурационные файлы, необходимые для воспроизведения проблемы. Если файл имеется в каком-нибудь архиве ПО, достаточно ссылки на него.

Прежде чем отправлять отчет об ошибке, перечитайте его. Часто обнаруживается, что какой-то информации не хватает. А иногда оказывается, что для решения проблемы достаточно было всего лишь четко сформулировать ее.

Резюме

В этом приложении мы рассказали о сообществе, сложившемся вокруг NGINX. Мы познакомились с основными игроками и с доступными сетевыми ресурсами. Мы также узнали, как правильно составить отчет об ошибке, который поможет найти решение проблемы.



Приложение D. Сохранение сетевых настроек в Solaris

В главе 8 «Техника устранения неполадок» мы видели, как изменить различные настройки сети в разных операционных системах. В этом приложении описываются детали, имеющие отношение к ОС Solaris версии 10 и выше.

Следующий скрипт запускается каркасом **Service Management Framework (SMF)**, чтобы установить сетевые параметры с помощью программы `ndd`. Сохраните его в файле `/lib/svc/method/network-tuning.sh` и сделайте исполняемым.

```
# vi /lib/svc/method/network-tuning.sh
```

Ниже приведено содержимое файла `/lib/svc/method/network-tuning.sh`:

```
#!/sbin/sh
# Set the following values as desired
ndd -set /dev/tcp tcp_max_buf 16777216
ndd -set /dev/tcp tcp_smallest_anon_port 1024
ndd -set /dev/tcp tcp_largest_anon_port 65535
ndd -set /dev/tcp tcp_conn_req_max_q 1024
ndd -set /dev/tcp tcp_conn_req_max_q0 4096
ndd -set /dev/tcp tcp_xmit_hiwat 1048576
ndd -set /dev/tcp tcp_recv_hiwat 1048576
# chmod 755 /lib/svc/method/network-tuning.sh
```

В приведенном ниже манифесте определена служба настройки сети и сказано, что скрипт нужно выполнить на этапе загрузки. Обратите внимание на атрибут *transient*, который говорит SMF, что это запускаемый однократно скрипт, а не постоянно работающий демон.

```

<?xml version="1.0"?>
<!DOCTYPE service_bundle SYSTEM «/usr/share/lib/xml/dtd/service_ bundle.dtd.1»>
<service_bundle type='manifest' name='SUNW:network_tuning'>
  <service
    name='site/network_tuning'
    type='service'
    version='1'>
    <create_default_instance enabled='true' />
    <single_instance />
    <dependency
      name='usr'
      type='service'
      grouping='require_all'
      restart_on='none'>
      <service_fmri value='svc:/system/filesystem/minimal' />
    </dependency>

    <!-- Run ndd commands after network/physical is plumbed. -->
    <dependency
      name='network-physical'
      grouping='require_all'
      restart_on='none'
      type='service'>
      <service_fmri value='svc:/network/physical' />
    </dependency>

    <!-- but run the commands before network/initial -->
    <dependent
      name='ndd_network-initial'
      grouping='optional_all'
      restart_on='none'>
      <service_fmri value='svc:/network/initial' />
    </dependent>

    <exec_method
      type='method'
      name='start'
      exec='/lib/svc/method/network-tuning.sh'
      timeout_seconds='60' />
    <exec_method
      type='method'
      name='stop'
      exec=':true'
      timeout_seconds='60' />
    <property_group name='startd' type='framework'>
      <propval name='duration' type='astring' value='transient' />
    </property_group>
    <stability value='Unstable' />
  </template>

```

```
<common_name>
  <loctext xml:lang='C'>
    Network Tunings
  </loctext>
</common_name>
</template>
</service>
</service_bundle>
```

Сохраните этот код в файле `/var/svc/manifest/site/network-tuning.xml` и импортируйте его такой командой:

```
# svccfg import /var/svc/manifest/site/network-tuning.xml
```

Служба намеренно сделана простой, так как предназначена только для демонстрации. Интересующийся читатель может узнать о SMF из страниц руководства в Solaris и из сетевых ресурсов.

Предметный указатель

СИМВОЛЫ

- @STRENGTH, строка, 100
- \$arg_name, переменная, 141
- \$args, переменная, 141
- \$binary_remote_addr, переменная, 135, 141
- \$body_bytes_sent, переменная, 127
- \$bwhog, переменная, 255
- \$bytes_sent, переменная, 127
- \$connection_requests, переменная, 127
- \$connection, переменная, 127
- \$content_length, переменная, 141
- \$content_type, переменная, 141
- \$cookie_name, переменная, 142
- \$document_root, переменная, 142
- \$document_uri, переменная, 142
- \$hostname, переменная, 142
- \$host, переменная, 142
- \$http_name, переменная, 142
- \$https, переменная, 142
- \$is_args, переменная, 142
- \$limit_rate, переменная, 142
- \$memcached_key, переменная, 159
- \$msec, переменная, 127
- \$nginx_version, переменная, 142
- \$pid, переменная, 142
- \$pipe *, переменная, 127
- \$query_string, переменная, 142
- \$realpath_root, переменная, 142
- \$remote_addr, переменная, 142
- \$remote_port, переменная, 142
- \$remote_user, переменная, 142
- \$request_body_file, переменная, 142
- \$request_body, переменная, 142
- \$request_completion, переменная, 142
- \$request_filename, переменная, 142
- \$request_length *, переменная, 127
- \$request_method, переменная, 142
- \$request_time, переменная, 127
- \$request_uri, переменная, 142
- \$request, переменная, 142
- \$scheme, переменная, 142
- \$sent_http_name, переменная, 142
- \$server_addr, переменная, 143
- \$server_name, переменная, 143
- \$server_port, переменная, 143
- \$server_protocol, переменная, 143
- \$ssl_client_cert, переменная, 100
- \$status, переменная, 127, 143
- \$tcpinfo_rcv_space, переменная, 143
- \$tcpinfo_rttvar, переменная, 143
- \$tcpinfo_rtt, переменная, 143
- \$tcpinfo_snd_cwnd, переменная, 143
- \$time_iso8601 *, переменная, 127
- \$time_local *, переменная, 127
- \$uri, переменная, 143
- 403 Forbidden, ошибка, 174
- 504 Gateway Timeout Error, ошибка, 108
- conf-path=<path>, параметр, 23
- error-log-path=<path>, параметр, 23
- group=<group>, параметр, 23
- .htaccess, файл, 262
- http-client-body-temp-path=<path>, пап, 26
- http-fastcgi-temp-path=<path>,

параметр, 26
--http-log-path=<path>, параметр, 26
--http-proxy-temp-path=<path>, параметр, 26
--http-scgi-temp-path=<path>, параметр, 26
--http-uwsgi-temp-path=<path>, параметр, 26
--lock-path=<path>, параметр, 23
--pid-path=<path>, параметр, 23
--prefix=<path>, параметр, 23
--sbin-path=<path>, параметр, 23
--user=<user>, параметр, 23
--with-cc-opt=<options>, параметр, 24
--with-cc=<path>, параметр, 24
--with-cpp=<path>, параметр, 24
--with-cpu-opt=<cpu>, параметр, 24
--with-debug, параметр, 23, 186
--with-file-aio, параметр, 23
--with-http_addition_module, параметр, 27
--with-http_dav_module, параметр, 27
--with-http_flv_module, параметр, 27
--with-http_geoip_module, параметр, 27
--with-http_gunzip_static_module, параметр, 27
--with-http_gzip_static_module, параметр, 27
--with-http_image_filter_module, параметр, 27
--with-http_mp4_module, параметр, 27
--with-http_perl_module, параметр, 25
--with-http_random_index_module, параметр, 27
--with-http_realip_module, параметр, 26
--with-http_secure_link_module, параметр, 27
--with-http_ssl_module, параметр, 26
--with-http_stub_status_module, параметр, 27

--with-http_sub_module, параметр, 27
--with-ld-opt=<options>, параметр, 24
--with-mail_ssl_module, параметр, 24
--with-mail, параметр, 24
--without-http_access_module, параметр, 29
--without-http_auth_basic_module, параметр, 29
--without-http_autoindex_module, параметр, 29
--without-http_browser_module, параметр, 30
--without-http-cache, параметр, 25
--without-http_charset_module, параметр, 28
--without-http_empty_gif_module, параметр, 30
--without-http_fastcgi_module, параметр, 29
--without-http_geo_module, параметр, 29
--without-http_gzip_module, параметр, 28
--without-http_limit_conn_module, параметр, 29
--without-http_limit_req_module, параметр, 29
--without-http_map_module, параметр, 29
--without-http_memcached_module, параметр, 29
--without-http_proxy_module, параметр, 29
--without-http_referer_module, параметр, 29
--without-http_rewrite_module, параметр, 29
--without-http_scgi_module, параметр, 29
--without-http_split_clients_module, параметр, 29
--without-http_ssi_module, параметр, 28

--without-http_upstream_ip_hash_module, параметр, 30
 --without-http_userid_module, параметр, 29
 --without-http_uwsgi_module, параметр, 29
 --without-http, параметр, 25
 --without-mail_imap_module, параметр, 25
 --without-mail_pop3_module, параметр, 25
 --without-mail_smtp_module, параметр, 25
 --with-perl_modules_path=<path>, параметр, 26
 --with-perl=<path>, параметр, 26

A

accept_filter, параметр, 42
 accept_mutex_delay, директива, 205
 accept_mutex, директива, 205
 access_log, директива, 125, 193, 206
 add_after_body, директива, 165, 206
 add_before_body, директива, 165, 206
 add_header, директива, 161, 206
 addition_types, директива, 165, 206
 addition, модуль, 164
 aio, директива, 38, 207
 alias, директива, 46, 207
 allow, директива, 137, 207
 ancient_browser_value, директива, 207
 ancient_browser, директива, 207
 Apache, документация по модулю mod_rewrite, 264
 Apache, правила переписывания
 замена RewriteCond проверкой переменной в директиве if, 264
 замена проверки существования директивой try_files, 261
 замена сравнения с HTTP_HOST секцией server, 263

 замена сравнения с REQUEST_URI секцией location, 262
 общие сведения, 261
 APOP, метод аутентификации, 54
 Atmail, 52
 atomic_ops, библиотека, 22
 auth_basic_user_file, директива, 137, 139, 207
 auth_basic, директива, 137, 207
 auth_http_header, директива, 208
 auth_http_timeout, директива, 208
 auth_http, директива, 48, 208
 auth, метод, 66
 autoindex_exact_size, директива, 208
 autoindex_localtime, директива, 208
 autoindex, директива, 208
 autoindex, модуль, 29

B

backlog, параметр, 41
 bind, параметр, 42
 block, команда, 168
 break, директива, 208, 258
 break, флаг, 77, 255, 258

C

CACHE, 114
 charset_map, директива, 208
 charset_types, директива, 209
 charset, директива, 208
 charset, модуль, 28
 chunked_transfer_encoding, директива, 36, 209
 client_body_buffer_size, директива, 37, 209, 210
 client_body_in_file_only, директива, 37, 209
 client_body_in_single_buffer, директива, 37, 209
 client_body_max_body_size, директива, 210
 client_body_temp_path, директива, 37, 209

client_body_timeout,
директива, 37, 210
client_header_buffer_size,
директива, 37
client_header_timeout, директива, 37
client_max_body_size, директива, 37
collectd, 204
config, команда, 168
connection_pool_size, директива, 210
create_full_put_path, директива, 210
crypt(), функция, 138

D

daemon, директива, 210
dav_access, директива, 210
dav_methods, директива, 210
DDOS-атака, 133
debug_connection, директива, 211
debug_points, директива, 211
default_server, параметр, 41
default_type, директива, 131, 211
deferred, параметр, 42
deny, директива, 137, 211
directio_alignment,
директива, 38, 211
directio, директива, 38, 211
disable_symlinks, директива, 129, 211
Django, 153
Drupal, 147
 пример конфигурации, 147

E

echo, команда, 168
empty_gif, директива, 175, 211
env, директива, 212
error_log, директива, 34, 70, 181, 212
error_page, директива, 102, 131,
132, 160, 212, 256
etag, директива, 131, 213
events, директива, 213
expires, директива, 161, 213

F

fastcgi_bind, директива, 213

fastcgi_buffer_size,
директива, 143, 213
fastcgi_buffers, директива, 143, 213
fastcgi_busy_buffers_size,
директива, 144, 213
fastcgi_cache_bypass,
директива, 144, 213
fastcgi_cache_key,
директива, 144, 213
fastcgi_cache_lock timeout,
директива, 144, 214
fastcgi_cache_lock,
директива, 144, 214
fastcgi_cache_min_uses,
директива, 144, 214
fastcgi_cache_path,
директива, 144, 214
fastcgi_cache_use_stale,
директива, 144, 214
fastcgi_cache_valid,
директива, 145, 214
fastcgi_cache, директива, 144, 213
fastcgi_connect_timeout,
директива, 145, 214
fastcgi_hide_header,
директива, 145, 214
fastcgi_ignore_client_abort,
директива, 145, 215
fastcgi_ignore_headers,
директива, 145, 215
fastcgi_index, директива, 145, 215
fastcgi_intercept_errors,
директива, 145, 215
fastcgi_keep_conn,
директива, 145, 215
fastcgi_max_temp_file_size,
директива, 145, 215
fastcgi_next_upstream,
директива, 145, 215
fastcgi_no_cache, директива, 146, 215
fastcgi_param, директива, 146, 215
fastcgi_pass_header,
директива, 146, 216
fastcgi_pass, директива, 146, 216
fastcgi_read_timeout,
директива, 146, 216

fastcgi_send_lowat, директива, 216
fastcgi_send_timeout,
директива, 146, 216
fastcgi_split_path_info,
директива, 146, 216
fastcgi_store_access,
директива, 146, 217
fastcgi_store, директива, 146, 217
fastcgi_temp_file_write_size,
директива, 146, 217
fastcgi_temp_path,
директива, 146, 217
FastCGI, проксируемые серверы, 88
FastMail, 51
flv, директива, 140, 217
FreeBSD, 203
менеджер пакетов, 20

G

GD, библиотека, 175
geoip_city, директива, 219
geoip_country, директива, 219
geoip_org, директива, 219
geoip_proxy_recursive,
директива, 220
geoip_proxy, директива, 219
geo, директива, 218
gunzip_buffers, директива, 220
gunzip, директива, 220
gzip_buffers, директива, 118, 220
gzip_comp_level, директива, 118, 220
gzip_disable, директива, 118, 220
gzip_http_version, директива, 220
gzip_min_length, директива, 118, 220
gzip_proxied, директива, 118, 221
gzip_static, директива, 221
gzip_types, директива, 118, 221
gzip_vary, директива, 118, 221
gzip, директива, 118, 220
gzip, модуль, 28, 117
директивы, 118

H

Host, заголовок, 79
http_auth, директива, 72

http, директива, 221
http, модуль, 68, 122
взаимодействие с клиентами, 131
директива server, 123
модель протоколирования, 124
параметры configure, 25
поиск файлов, 127
разрешение имен, 129
HTTP-сервер, 121

I

if_modified_since, директива, 131, 221
if, директива, 89, 141, 221, 258
ветвление по имени хоста, 196
использование вместо
try_files, 195
if, команда, 168
ignore_invalid_headers,
директива, 131, 222
image_filter_buffer,
директива, 175, 222
image_filter_jpeg_quality,
директива, 176, 222
image_filter_sharpen,
директива, 176, 222
image_filter_transparency,
директива, 176, 222
image_filter, директива, 175, 222
IMAP, 52
imap_auth, директива, 55, 222
imap_capabilities,
директива, 48, 55, 222
imap_client_buffer, директива, 222
include, директива, 223
include, команда, 168
index, директива, 223
internal, директива, 46, 223
ip_hash, директива, 82, 84, 223
ipv6only, параметр, 42
IRC-канал, 266

K

keepalive_disable, директива, 37, 223
keepalive_requests,
директива, 37, 223

keepalive_timeout, директива, 37, 223
keepalive, директива, 82, 83, 223
keepalive-соединения, 83, 204
KILL, сигнал, 188

L

large_client_header_buffers,
директива, 37, 223
large_client_header_buffers,
директива, 37
last, флаг, 255, 258
least_conn, директива, 82, 84, 224
libatomic, библиотека, 22
limit_conn_log_level,
директива, 133, 224
limit_conn_zone, директива, 133, 224
limit_conn, директива, 133, 224
limit_except, директива, 46, 224
limit_rate_after, директива, 134, 224
limit_rate_log_level, директива, 134
limit_rate_zone, директива, 134
limit_rate, директива, 134, 224
limit_req_log_level, директива, 225
limit_req_zone, директива, 225
limit_req, директива, 134, 225
limit_zone, директива, 225
lingering_close, директива, 40, 225
lingering_timeout, директива, 40, 226
lingering_time, директива, 40, 225
Linux, 203
Linux, менеджер пакетов, 19
listen (http), директива, 226
listen (mail), директива, 226
listen, директива, 41, 99, 123
location, директива, 45, 226
lock_file, директива, 226
log_format, директива, 124, 125,
126, 193, 226
log_not_found, директива, 126, 226
log_subrequest, директива, 126, 227

M

mail, директива, 227
map_hash_bucket_size,
директива, 227

map_hash_max_size, директива, 227
map, директива, 227
map, модуль, 29
master_process, директива, 227
max_ranges, директива, 134, 228
MD5, 22
memcached
 интеграция, 68
 параметры, 69
memcached_bind, директива, 228
memcached_buffer_size,
директива, 160, 228
memcached_connect_timeout,
директива, 160, 228
memcached_gzip_flag, директива, 228
memcached_next_upstream,
директива, 160, 228
memcached_pass,
директива, 88, 160, 228
memcached_read_timeout,
директива, 161, 228
memcached_send_timeout,
директива, 161, 228
memcached, модуль директивы, 160
merge_slashes, директива, 131, 229
min_delete_depth, директива, 229
modern_browser_value,
директива, 229
modern_browser, директива, 229
mod_rewrite, модуль, 261
mp4_buffer_size, директива, 140, 229
mp4_max_buffer_size,
директива, 140, 229
mp4, директива, 140, 229
msie_padding, директива, 37, 230
msie_refresh, директива, 37, 230
multi_accept, директива, 230
Munin, 204

N

Nagios, 204
NGINX
 актуальный список директив, 205
 архитектура, 121
 включение модулей, 26

- глобальные конфигурационные параметры, 34
- документация в Сети, 267
- интеграция с uWSGI, 152
- использование совместно с PHP-FPM, 143
- ключ подписания, 21
- конфигурационный файл, 33
- модуль rewrite, 254
- общие сведения, 19, 51
- отключение неиспользуемых модулей, 28
- параметры configure
 - модуль http, 25
 - модуль mail, 24
 - общие, 23
- предопределенные переменные, 141
- принятие решений, 170
- сборка из исходного кода, 21
- сообщество, 266
- сторонние модули, поиск и установка, 30
- установка с помощью менеджера пакетов, 19
- формат конфигурационного файла, 33
- ЧАВО, 266
- nginx.conf, конфигурационный файл, 35
- ngx_lua, сторонний модуль, 30

О

- open_file_cache_errors, директива, 38, 230
- open_file_cache_valid, директива, 38, 230
- open_file_cache, директива, 38, 230
- open_file_min_uses, директива, 38, 230
- open_log_file_cache, директива, 126, 230
- OpenSSL, генерация SSL-сертификата, 57
- optimize_server_names, директива, 230

- override_charset, директива, 231

Р

- pcre_jit, директива, 231
- PCRE (Perl Compatible Regular Expressions), библиотека, 22
- PEP-3333, 152
- perl_modules, директива, 170, 231
- perl_require, директива, 170, 231
- perl_set, директива, 170, 231
- perl, директива, 170, 231
- permanent, флаг, 258
- PHP-FPM, технология, 143
- pid, директива, 34, 231
- Platform for Privacy Preferences Project, 179
- POP3, 52, 53
- pop3_auth, директива, 231
- pop3_capabilities, директива, 48, 232
- port_in_redirect, директива, 124, 232
- postpone_output, директива, 38, 232
- printf(), функция, 193
- protocol, директива, 48, 232
- proxy_bind, директива, 232
- proxy_buffering, директива, 109, 232
- proxy_buffer_size, директива, 80, 109, 232
- proxy_buffers, директива, 80, 109, 233
- proxy_buffer, директива, 48, 232
- proxy_busy_buffers_size, директива, 80, 109, 233
- proxy_cache_bypass, директива, 112, 233
- proxy_cache_key, директива, 112, 233
- proxy_cache_lock_timeout, директива, 113, 233
- proxy_cache_lock, директива, 113, 233
- proxy_cache_min_uses, директива, 113, 234
- proxy_cache_path, директива, 113, 234
- proxy_cache_use_stale, директива, 113, 234
- proxy_cache_valid, директива, 113, 234

proxy_cache, директива, 112, 233
proxy_connect_timeout,
директива, 77, 80, 234
proxy_cookie_domain,
директива, 77, 234
proxy_cookie_path,
директива, 77, 235
proxy_headers_hash_bucket_size,
директива, 77, 235
proxy_headers_hash_max_size,
директива, 77, 235
proxy_hide_header,
директива, 77, 235
proxy_http_version,
директива, 78, 235
proxy_ignore_client_abort,
директива, 78, 235
proxy_ignore_headers,
директива, 78, 235
proxy_intercept_errors,
директива, 78, 235
proxy_max_temp_file_size,
директива, 78, 235
proxy_next_upstream, директива, 236
proxy_no_cache, директива, 236
proxy_pass_error_message,
директива, 48, 237
proxy_pass_header, директива, 78, 237
proxy_pass_request_body,
директива, 78, 237
proxy_pass_request_headers,
директива, 78, 237
proxy_pass, директива, 78, 237
proxy_read_timeout,
директива, 78, 80, 237
proxy_redirect, директива, 78, 237
proxy_send_lowat, директива, 80, 237
proxy_send_timeout,
директива, 78, 80, 237
proxy_set_body, директива, 78, 238
proxy_set_header,
директива, 79, 94, 238
proxy_ssl_session_reuse,
директива, 238
proxy_store_access, директива, 238
proxy_store, директива, 117, 238

proxy_temp_file_write_size,
директива, 79, 238
proxy_temp_path,
директива, 79, 184, 238
proxy_timeout, директива, 48, 239
proxy, директива, 48, 232
proxy, модуль
 директивы, 77
 общие сведения, 29, 51
 унаследованные серверы
 с куками, 81

R

random_index, директива, 239
RBAC (Role-based access control), 20
rcvbuf, параметр, 41
read_ahead, директива, 38, 239
real_ip_header, директива, 239
real_ip_recursive, директива, 239
recursive_error_pages,
директива, 131, 239
redirect, флаг, 258
referer_hash_bucket_size,
директива, 239
referer_hash_max_size,
директива, 239
request_pool_size, директива, 239
reset_timedout_connection,
директива, 40, 239
resolver_timeout, директива, 240
resolver, директива, 130, 240
return, директива, 240, 256, 258
rewrite_log, директива, 240, 258
RewriteRule, 262
rewrite, директива, 240, 258
rewrite, модуль, 29, 190, 254
 директивы, 258
root, директива, 129, 240
Ruby, 61

S

satisfy_any, директива, 241
satisfy, директива, 137, 240
SCGI модуль, 29, 89
secure_link_secret,
директива, 173, 241

- secure_link, модуль, 173
- sendfile_max_chunk,
 - директива, 39, 241
- sendfile, директива, 39, 241
- send_lowat, директива, 40, 241
- send_timeout, директива, 40, 241
- server (http), директива, 241
- server (mail), директива, 241
- server_name (http), директива, 242
- server_name_in_redirect,
 - директива, 124, 242
- server_name (mail), директива, 242
- server_names_hash_bucket_size,
 - директива, 39, 242
- server_names_hash_max_size,
 - директива, 39, 242
- server_name, директива, 42, 124
- server_tokens, директива, 124, 242
- server (upstream), директива, 241
- server, директива, 82, 123, 124
- setfib, параметр, 41
- set_real_ip_from, директива, 242
- set, директива, 242, 258
- set, команда, 169
- SHA-1, 22
- SMTP, 52, 55
- smtp_auth, директива, 56, 242
- smtp_capabilities, директива, 243
- sndbuf, параметр, 42
- so_keepalive, директива, 243
- so_keepalive, параметр, 42
- Solaris, сохранение сетевых настроек, 269
- source_charset, директива, 243
- split_clients, директива, 243
- ssi_min_file_chunk, директива, 243
- SSI (Server Side Includes), 167
- ssi_silent_errors, директива, 167, 243
- ssi_types, директива, 167, 243
- ssi_value_length, директива, 243
- ssi, директива, 167, 243
- ssl, модуль директивы, 167
- SSL
 - аутентификация клиентов, 100
 - общие сведения, 56
 - шифрование трафика, 98
- ssl_certificate (http), директива, 244
- ssl_certificate_key (http),
 - директива, 244
- ssl_certificate_key (mail),
 - директива, 244
- ssl_certificate_key, директива, 49
- ssl_certificate (mail), директива, 244
- ssl_certificate, директива, 49
- ssl_ciphers, директива, 49, 244
- ssl_client_certificate, директива, 244
- ssl_crl, директива, 101, 244
- ssl_dhparam, директива, 244
- ssl_engine, директива, 245
- ssl (http), директива, 244
- ssl (mail), директива, 244
- ssl_prefer_server_ciphers (http),
 - директива, 245
- ssl_prefer_server_ciphers (mail),
 - директива, 245
- ssl_prefer_server_ciphers,
 - директива, 49
- ssl_protocols (http), директива, 245
- ssl_protocols (mail), директива, 245
- ssl_protocols, директива, 49
- ssl_session_cache (http)
 - директива, 245
- ssl_session_cache (mail)
 - директива, 246
- ssl_session_cache, директива, 49
- ssl_session_timeout (http),
 - директива, 246
- ssl_session_timeout (mail),
 - директива, 246
- ssl_session_timeout, директива, 49
- ssl_stapling_file, директива, 246
- ssl_stapling_responder,
 - директива, 246
- ssl_stapling_verify, директива, 247
- ssl_stapling, директива, 246
- ssl_trusted_certificate, директива, 247
- ssl_verify_client, директива, 101, 247
- ssl_verify_depth, директива, 102, 247
- ssl, директива, 49
- ssl, модуль, 99
- ssl, параметр, 42
- SSL-сертификат, генерация с помощью SSL, 57
- starttls, директива, 247

Stub Status, модуль, 203
sub_filter_once, директива, 165, 166, 247
sub_filter_types, директива, 166, 247
sub_filter, директива, 166, 247
sub, модуль, 165
 директивы, 166
sudo, команда, 20

T

tcp_nodelay, директива, 40, 247
tcp_nopush, директива, 40, 248
timeout, директива, 60, 248
timer_resolution, директива, 248
TLS, 56
try_files, директива, 47, 86, 128, 129, 185, 190, 195, 248, 261
types_hash_bucket_size, директива, 39, 248
types_hash_max_size, директива, 39, 249
types, директива, 132, 248

U

underscores_in_headers, директива, 132, 249
uninitialized_variable_warn, директива, 249
upstream, директива, 249
upstream, модуль, 82
 алгоритмы балансировки нагрузки, 84
 директивы, 82
 соединения типа keepalive, 83
userid_domain, директива, 179, 250
userid_expires, директива, 179, 250
userid_mark, директива, 250
userid_name, директива, 179, 250
userid_p3p, директива, 179, 250
userid_path, директива, 179, 250
userid_service, директива, 179, 250
userid, директива, 178, 250
userid, модуль, 29
 директивы, 178
user, директива, 34, 249

use, директива, 34, 249
uWSGI, интеграция с NGINX, 152

V

valid_referers, директива, 251
variables_hash_bucket_size, директива, 39, 251
variables_hash_max_size, директива, 39, 251

W

worker_aio_requests, директива, 251
worker_connections, директива, 35, 73, 251
worker_cpu_affinity, директива, 252
worker_priority, директива, 252
worker_processes, директива, 34, 252
worker_rlimit_core, директива, 252
worker_rlimit_nofile, директива, 73, 199, 252
worker_rlimit_sigpending, директива, 252
working_directory, директива, 252
WSGI (Web Server Gateway Interface), 152

X

X-Accel-Expires, заголовок, 157
xclient, директива, 48, 252
XCLIENT, расширение SMTP, 53
xml_entities, директива, 167, 252
xslt_param, директива, 167, 253
xslt_string_param, директива, 167, 253
xslt_stylesheet, директива, 167, 253
xslt_types, директива, 167, 253
xslt, модуль, 166
 директивы, 167

Y

yum, добавление репозитория
NGINX, 20

Z

Zimbra, 52, 68
zlib, библиотека, 22

A

Аутентификация клиентов
по протоколу SSL, 100

B

Безопасная ссылка, создание, 173
Безопасность за счет разделения, 98
Буферизация, 108

B

Веб-ресурсы, 267

Г

Главный процесс, 121
Глобальные конфигурационные
параметры NGINX
 error_log, 34
 pid, 34
 use, 34
 user, 34
 worker_connections, 35
 worker_processes, 34

Д

Двоичный файл, переключение
во время выполнения, 186
Директива listen, параметры
 accept_filter, 42
 backlog, 41
 bind, 42
 default_server, 41
 deferred, 42
 ipv6only, 42
 rcvbuf, 41
 setfib, 41
 sndbuf, 42
 so_keepalive, 42
 ssl, 42

Директивы HTTP-сервера
 port_in_redirect, 124
 server, 124
 server_name, 124
 server_name_in_redirect, 124
 server_tokens, 124

Директивы для задания
пределных значений, 133
 limit_conn, 133
 limit_conn_log_level, 133
 limit_conn_zone, 133
 limit_rate, 134
 limit_rate_after, 134
 limit_req, 134
 limit_req_log_level, 134
 limit_req_log_zone, 134
 max_ranges, 134

Директивы модуля addition
 add_after_body, 165
 add_before_body, 165
 addition_types, 165

Директивы модуля FastCGI
 fastcgi_buffers, 143
 fastcgi_buffer_size, 143
 fastcgi_busy_buffers_size, 144
 fastcgi_cache, 144
 fastcgi_cache_bypass, 144
 fastcgi_cache_key, 144
 fastcgi_cache_lock, 144
 fastcgi_cache_lock_timeout, 144
 fastcgi_cache_min_uses, 144
 fastcgi_cache_path, 144
 fastcgi_cache_use_stale, 144
 fastcgi_cache_valid, 145
 fastcgi_connect_timeout, 145
 fastcgi_hide_header, 145
 fastcgi_ignore_client_abort, 145
 fastcgi_ignore_headers, 145
 fastcgi_index, 145
 fastcgi_intercept_errors, 145
 fastcgi_keep_conn, 145
 fastcgi_max_temp_file_size, 145
 fastcgi_next_upstream, 145
 fastcgi_no_cache, 146
 fastcgi_param, 146
 fastcgi_pass, 146

- fastcgi_pass_header, 146
- fastcgi_read_timeout, 146
- fastcgi_send_timeout, 146
- fastcgi_split_path_info, 146
- fastcgi_store, 146
- fastcgi_store_access, 146
- fastcgi_temp_file_write_size, 146
- fastcgi_temp_path, 146
- Директивы модуля gzip, 118
 - gzip, 118
 - gzip_buffers, 118
 - gzip_comp_level, 118
 - gzip_disable, 118
 - gzip_min_length, 118
 - gzip_proxied, 118
 - gzip_types, 118
 - gzip_vary, 118
- Директивы модуля image_filter, 175
 - empty_gif, 175
 - image_filter, 175
 - image_filter_buffer, 175
 - image_filter_jpeg_quality, 176
 - image_filter_sharpen, 176
 - image_filter_transparency, 176
- Директивы модуля memcached
 - memcached_buffer_size, 160
 - memcached_connect_timeout, 160
 - memcached_next_upstream, 160
 - memcached_pass, 160
 - memcached_read_timeout, 161
 - memcached_send_timeout, 161
- Директивы модуля perl
 - perl, 170
 - perl_modules, 170
 - perl_require, 170
 - perl_set, 170
- Директивы модуля proxy
 - proxy_connect_timeout, 77
 - proxy_cookie_domain, 77
 - proxy_cookie_path, 77
 - proxy_headers_hash_bucket_size, 77
 - proxy_headers_hash_max_size, 77
 - proxy_hide_header, 77
 - proxy_http_version, 78
 - proxy_ignore_client_abort, 78
 - proxy_ignore_headers, 78
 - proxy_intercept_errors, 78
 - proxy_max_temp_file_size, 78
 - proxy_pass, 78
 - proxy_pass_header, 78
 - proxy_pass_request_body, 78
 - proxy_pass_request_headers, 78
 - proxy_read_timeout, 78
 - proxy_redirect, 78
 - proxy_send_timeout, 78
 - proxy_set_body, 78
 - proxy_set_header, 79
 - proxy_temp_file_write_size, 79
 - proxy_temp_path, 79
- Директивы модуля rewrite
 - break, 258
 - if, 258
 - return, 258
 - rewrite, 258
 - rewrite_log, 258
 - set, 258
 - uninitialized_variable_warn, 258
- Директивы модуля ssi
 - ssi, 167
 - ssi_silent_errors, 167
 - ssi_types, 167
- Директивы модуля sub
 - sub_filter, 166
 - sub_filter_once, 166
 - sub_filter_types, 166
- Директивы модуля upstream
 - ip_hash, 82
 - keepalive, 82
 - least_conn, 82
 - server, 82
- Директивы модуля userid, 179
 - userid, 178
 - userid_expires, 179
 - userid_name, 179
 - userid_p3p, 179
 - userid_path, 179
 - userid_service, 179
- Директивы модуля xslt
 - xml_entities, 167
 - xslt_param, 167
 - xslt_string_param, 167

- xslt_stylesheet, 167
- xslt_types, 167
- Директивы, относящиеся
 - к вводу-выводу, 38
 - aio, 38
 - directio, 38
 - directio_alignment, 38
 - open_file_cache, 38
 - open_file_cache_errors, 38
 - open_file_cache_valid, 38
 - postpone_output, 38
 - read_ahead, 38
 - sendfile, 39
 - sendfile_max_chunk, 39
- Директивы, относящиеся
 - к сокетам
 - lingering_close, 40
 - lingering_time, 40
 - lingering_timeout, 40
 - reset_timedout_connection, 40
 - send_lowat, 40
 - send_timeout, 40
 - tcp_nodelay, 40
 - tcp_nopush, 40
- Директивы, относящиеся
 - к хэш-таблицам
 - server_names_hash_bucket_size, 39
 - server_names_hash_max_size, 39
 - types_hash_bucket_size, 39
 - types_names_hash_max_size, 39
 - variables_hash_bucket_size, 39
 - variables_names_hash_max_size, 39
- Директивы протоколирования
 - access_log, 125
 - log_format, 125
 - log_not_found, 126
 - log_subrequest, 126
 - open_log_file_cache, 126
- Диспетчер кэша, 122
- Документы с описанием ошибок,
 - использование для обработки
 - ошибок проксирования, 93
- Доступа ограничение, 136
- Дэниэл Кегель, 19

Ж

- Журнал ошибок
 - примеры записей, 183
 - форматы, 181
- Журналы
 - анализ, 181
 - интерпретация, 70
 - общие сведения, 70
- Журналы доступа, использование
 - для отладки, 193

З

- Загрузчик кэша, 122

И

- Изображения, генерация, 174

К

- Клиент, определение IP-адреса, 94
- Клиентские директивы, секция
 - http, 36
 - chunked_transfer_encoding, 36
 - client_body_buffer_size, 37
 - client_body_in_file_only, 37
 - client_body_in_single_buffer, 37
 - client_body_temp_path, 37
 - client_body_timeout, 37
 - client_header_buffer_size, 37
 - client_header_timeout, 37
 - client_max_body_size, 37
 - keepalive_disable, 37
 - keepalive_requests, 37
 - keepalive_timeout, 37
 - large_client_header_buffers, 37
 - msie_padding, 37
 - msie_refresh, 37
- Кэширование
 - в базе данных, 158
 - в файловой системе, 161
 - интеграция с, 156
 - концепция сохранения, 116
 - общие сведения, 111, 156

M

Масштабируемость, 97

изоляция компонентов
приложения, 105

Менеджер пакетов, установка
NGINX, 19

Модули, отключение
неиспользуемых, 29

--without-http_auth_module, 29
--without-http_autoindex_
module, 29
--without-http_browser_
module, 30
--without-http_charset_module, 28
--without-http_empty_gif_
module, 30
--without-http_fastcgi_module, 29
--without-http_geo_module, 29
--without-http_gzip_module, 28
--without-http_limit_conn_
module, 29
--without-http_limit_req_
module, 29
--without-http_map_module, 29
--without-http_memcached_
module, 29
--without-http_proxy_module, 29
--without-http_referer_module, 29
--without-http_rewrite_module, 29
--without-http_scgi_module, 29
--without-http_split_clients_
module, 29
--without-http_ssi_module, 28
--without-http_upstream_ip_
hash_module, 30
--without-http_userid_module, 29
--without-http_uwsgi_module, 29

Мультимедийные файлы
потоковая передача, 140

O

Обратный прокси-сервер, 75, 76
оптимизация
производительности, 108
буферизация, 108

кэширование, 111
сжатие, 117

Ограничения на количество
файловых дескрипторов, 198
Операционная система,
ограничения, 72
на количество файловых
дескрипторов, 198
сетевые лимиты, 200

Отладочное протоколирование, 186
Отслеживание посетителей
сайта, 178
Отчет об ошибке, составление, 267
Ошибки конфигурирования, 194
использование if вместо
try_files, 195
использование if для ветвления
по имени хоста, 196

P

Параметры configure, модуль http
--http-client-body-temp-
path=<path>, 26
--http-fastcgi-temp-path=
<path>, 26
--http-log_path=<path>, 26
--http-proxy-temp-path=
<path>, 26
--http-scgi-temp-path=<path>, 26
--http-uwsgi-temp-path=
<path>, 26
--with-http_perl_module, 25
--without-http-cache, 25
--with-perl_modules_
path=<path>, 26
--with-perl=<path>, 26

Параметры configure, модуль mail
--with-mail, 24
--with-mail_imap_module, 25
--with-mail_pop3_module, 25
--with-mail_smtp_module, 25
--with-mail_ssl_module, 24
--without-http, 25

Параметры configure, прочие
модули

--with-http_addition_module, 27

- with-http_dav_module, 27
 - with-http_flv_module, 27
 - with-http_geoip_module, 27
 - with-http_gunzip_module, 27
 - with-http_image_module, 27
 - with-http_mp4_module, 27
 - with-http_secure_link_module, 27
 - with-http_ssl_module, 26
 - with-http_stub_status_module, 27
 - with-http_sub_module, 27
 - with-realip_module, 26
 - Подзапросы, 122
 - Поиск и устранение неполадок
 - анализ журналов, 181
 - использование модуля Stub Status, 203
 - ограничения операционной системы, 198
 - ошибки конфигурирования, 194
 - проблемы
 - с производительностью, 201
 - расширенное протоколирование, 186
 - Поиск файлов, 127
 - Правила переписывания, создание, 259
 - Предопределенные переменные
 - NGINX
 - \$arg_name, 141
 - \$args, 141
 - \$binary_remote_addr, 141
 - \$content_length, 141
 - \$content_type, 141
 - \$cookie_name, 142
 - \$document_root, 142
 - \$document_uri, 142
 - \$host, 142
 - \$hostname, 142
 - \$http_name, 142
 - \$https, 142
 - \$is_args, 142
 - \$limit_rate, 142
 - \$nginx_version, 142
 - \$pid, 142
 - \$query_string, 142
 - \$realpath_root, 142
 - \$remote_addr, 142
 - \$remote_port, 142
 - \$remote_user, 142
 - \$request, 142
 - \$request_body, 142
 - \$request_body_file, 142
 - \$request_completion, 142
 - \$request_filename, 142
 - \$request_method, 142
 - \$request_uri, 142
 - \$scheme, 142
 - \$sent_http_name, 142
 - \$server_addr, 143
 - \$server_name, 143
 - \$server_port, 143
 - \$server_protocol, 143
 - \$status, 143
 - \$tcpinfo_rcv_space, 143
 - \$tcpinfo_rttvar, 143
 - \$uri, 143
 - Принятие решений в NGINX, 170
 - Производительность, проблемы, 201
 - Проксируемые серверы
 - единственный, 85
 - не-HTTP, 87
 - несколько, 86
 - общие сведения, 75
 - типы, 85
 - Псевдопоточковая передача, 140
- P**
- Рабочий процесс, 122
 - Расширенное протоколирование, настройка, 186
- C**
- Секция с описанием HTTP-сервера
 - директивы, относящиеся к вводу-выводу, 38
 - директивы, относящиеся к сокетам, 40

- директивы, относящиеся к хэш-таблицам, 39
- клиентские директивы, 36
- общие сведения, 36
- пример конфигурации, 40
- Секция с описанием виртуального сервера, 41
- Секция с описанием почтового сервера, 48
- Сервер по умолчанию, 123
- Сетевые лимиты, 198, 200
- Сжатие, 117
- Случайное выполнение кода, предотвращение, 179
- С наименьшим количеством соединений, алгоритм балансировки, 84
- Список рассылки, 266

Т

- Трафик
 - блокирование по IP-адресу отправителя, 103
 - шифрование, 98

У

- Унаследованные серверы, с куками, 81

Х

- Хэширования, алгоритм, 22
- IP-адреса, 84

Ц

- Циклический алгоритм балансировки нагрузки, 84

Ш

- Шифры SSL, 100

Книги издательства «ДМК Пресс» можно заказать
в торгово-издательском холдинге «Планета Альянс» наложенным платежом,
выслав открытку или письмо по почтовому адресу:

115487, г. Москва, 2-й Нагатинский пр-д, д. 6А.

При оформлении заказа следует указать адрес (полностью),
по которому должны быть высланы книги;
фамилию, имя и отчество получателя.

Желательно также указать свой телефон и электронный адрес.

Эти книги вы можете заказать и в интернет-магазине: **www.aliants-kniga.ru**.

Оптовые закупки: тел. **(499) 782-38-89**.

Электронный адрес: **books@aliants-kniga.ru**.

Димитрий Айвалиотис

Администрирование сервера NGINX

Главный редактор *Мовчан Д. А.*
dmkpress@gmail.com

Верстка *Чаннова А. А.*

Корректор *Синяева Г. И.*

Дизайн обложки *Мовчан А. Г.*

Формат 60×90 1/16.

Гарнитура «Петербург».

Усл. печ. л. 18. Тираж 100 экз.

Веб-сайт издательства: www.dmk.ru